

UNI!

Federica Serra, Alessandro Oliva, Rocco Notarnicola 13/06/2026

13/06/2026

Sommario

- [Analisi](#)
 - [Descrizione del problema](#)
 - [Funzionalità principali](#)
 - [Architettura generale](#)
 - [Strutture dati principali](#)
 - [Principali variabili e stato applicativo](#)
 - [File principali del progetto](#)
 - [Librerie utilizzate](#)
 - [Requisiti funzionali](#)
 - [Casi d'uso](#)
- [Test](#)
 - [Esempi di esecuzione](#)
 - [Risultati dei test](#)
- [Conclusioni](#)
- [Documentazione Doxygen + Frontend](#) (refman generato da Doxygen, allegato in coda)
- [Documentazione Frontend \(TSDoc\)](#) (API del frontend generate da TypeDoc, allegato in coda)

Analisi

Descrizione del problema

Uni! è un clone multigiocatore del celebre gioco di carte UNO, pensato per essere giocato in tempo reale da più utenti tramite browser. Il progetto nasce dall'esigenza di sperimentare un'architettura full-stack moderna che unisca:

- un **backend** scritto in C++23, basato sulla libreria `uWebSockets`, responsabile della logica di gioco, dell'autenticazione e della persistenza dei dati;
- un **frontend** realizzato con Svelte 5 (runes) e TypeScript, che offre un'interfaccia reattiva, animata e gradevole, in stile "pixel-art".

Gli utenti possono registrarsi, creare o unirsi a lobby (anche tramite codice di invito a 6 caratteri), configurare le regole della partita (numero di carte iniziali, mod opzionali, presenza di bot, ecc.) e disputare partite di UNO con amici o con intelligenze artificiali. Al termine delle partite vengono aggiornate statistiche personali e una classifica globale.

Il sistema è pensato per gestire correttamente anche scenari "di confine" tipici di un gioco multiplayer in tempo reale: disconnessioni temporanee con periodo di tolleranza, sostituzione automatica dei giocatori con bot, salvataggio e ripresa di partite interrotte, e sincronizzazione dello stato di gioco tra tutti i client connessi.

Funzionalità principali

- **Autenticazione:** registrazione e login con email/password, sessione gestita tramite cookie HttpOnly e token JWT firmato (HS256).
- **Lobby:**
 - creazione di lobby pubbliche o private, con nome personalizzabile;
 - elenco delle lobby pubbliche disponibili, con possibilità di refresh;
 - ingresso tramite codice invito a 6 caratteri alfanumerici;
 - gestione dei membri da parte dell'host (promozione, espulsione);

- configurazione dettagliata delle regole di partita (timer, numero di carte iniziali, composizione del mazzo, mod opzionali come "Seven-Zero", "Jump In", "No Bluffing", "Force Play", "Progressive", "Draw Stacking");
- gestione dei bot (numero, modalità di turno, takeover/replacement);
- riconnessione automatica ("rejoin") alla lobby dopo una disconnessione.

- **Partita:**

- motore di gioco basato su una coda di "effetti" risolti sequenzialmente (architettura *Effect-based*), che permette di gestire in modo estendibile le carte speciali (Salta, Inverti, Pesca Due/Quattro, Jolly, ecc.);
- turni con timer configurabile e gestione automatica dell'AFK tramite bot;
- animazioni di "volo" delle carte tra mazzo, mano e scarti, gestite dal `CardBus` lato frontend;
- interazione drag-and-drop per il riordino delle carte in mano;
- azioni interattive (scelta colore, scelta target per lo scambio mani, decisione su carta pescata, dichiarazione "UNO");
- salvataggio dello stato di una partita interrotta e successivamente ripresa.

- **Statistiche:**

- classifica globale (vittorie/sconfitte, percentuale di vittorie);
- statistiche personali per tipologia di carta giocata.

Architettura generale

Il backend espone sia rotte HTTP REST (per autenticazione e statistiche) sia un endpoint WebSocket unico, su cui viaggiano tutti i messaggi relativi a lobby e partite. I messaggi sono pacchetti JSON con un campo `action` che ne identifica il tipo; un `ActionRouter` smista ciascun messaggio al controller competente (`LobbyController` , `GameController`). Il frontend comunica tramite un client WebSocket centralizzato (`WebSocketClient`), che supporta sia un canale "fire-and-forget" (`emit`) sia un canale di richiesta/risposta correlato tramite `request_id` (`emitAndWait`), con riconnessione automatica e backoff esponenziale.

Lo stato applicativo del frontend è organizzato in store singleton reattivi (`storeAuth` , `storeLobby` , `storeGame` , `storeNavigation` , `storeToast` , `storeStats`), ciascuno responsabile di un dominio specifico e sincronizzato con gli eventi provenienti dal server.

Strutture dati principali

Il dominio di gioco è modellato da poche strutture dati centrali, condivise tra i moduli del backend e serializzate in JSON verso il frontend.

Struttura	Dove	Descrizione
<code>CompactCard</code>	<code>common/game/card_types.hpp</code>	Rappresentazione compatta di una carta: un intero a 32 bit che impacchetta colore , valore e ID univoco (accessibili via <code>GetColor</code> , <code>GetValue</code> , <code>GetId</code> , <code>MakeCard</code>). Riduce drasticamente l'occupazione di memoria dei mazzi.

GameState	game/game_state.hpp	"Fotografia" completa della partita: status, lista players, current_player_index, play_direction, pending_draws (catena +2/+4), draw_pile / discard_pile, active_color, la coda di effetti effect_queue, i campi di input pendente (pending_player, pending_input_type, ...), last_play (provenienza dell'ultima carta giocata, per le animazioni) e turn_end_time.
Player	game/game_state.hpp	Un giocatore (umano o bot): username, hand (vettore di CompactCard), is_bot, has_called_uno.
MatchInstance	game/match_instance.hpp	Il motore di gioco vero e proprio: incapsula lo state_ (GameState), le regole attive active_rules_, le settings_ e le statistiche di sessione. Espone PlayCard, DrawCard, Tick, ProvideInput, SerializePlayerState, ecc.
Effect / GameRule	common/game/effect.hpp, common/game/gamerule.hpp	Astrazioni dell'architettura <i>Effect-based</i> : gli Effect (es. AdvanceTurnEffect, ChooseColorEffect) sono unità di logica risolte in sequenza dalla effect_queue; le GameRule (es. StandardRule, DrawStackingRule) validano e reagiscono alle giocate in modo estendibile.
Lobby / LobbyMember	common/lobby.hpp	Lobby aggrega id, invite_code, host, members, name, settings e l'eventuale match. LobbyMember descrive un partecipante: username, socket, is_connected, is_bot, disconnected_at (per il grace period).
LobbySettings	common/lobby.hpp	Configurazione della partita (vedi sezione successiva).
Card / GamePlayer / GameState (TS)	frontend/src/lib/stores/game.svelte.ts	Controparti frontend delle strutture di gioco, decodificate dal JSON del server (es. colore e valore testuali invece che numerici).

CardBus / CardFlightEvent	frontend/src/lib/components/game/card- bus.svelte.ts	Bus centralizzato delle animazioni: registra gli elementi DOM (pile e slot delle mani) e orchestra i "voli" delle carte tra mazzo, mano e scarti.
------------------------------	---	--

Principali variabili e stato applicativo

Variabile / parametro	Ambito	Ruolo
state_ (GameState)	MatchInstance (backend)	Stato autorevole della partita; ogni mossa lo modifica e lo ri-serializza per ciascun client.
pending_draws	GameState	Carte di penalità accumulate dalla catena +2/+4 (mod <i>Draw Stacking</i>).
current_player_index / play_direction	GameState	Indice del giocatore di turno e verso di gioco (1 orario, -1 antiorario).
active_color	GameState	Colore obbligatorio corrente (cambia con i Jolly).
turn_time_limit_ms	LobbySettings	Durata del timer di turno (default 15 000 ms); il client riceve turn_time_remaining_ms .
active_mods	LobbySettings	Elenco delle mod attive (draw_stacking , progressive , force_play , jump_in , no_bluffing , seven_zero , ...).
starting_cards , count_*	LobbySettings	Numero di carte iniziali e composizione del mazzo (zeri, numeri, salta, inverti, pesca due, jolly, jolly pesca quattro).
bot_count , bot_mode , allow_bot_*	LobbySettings	Numero di bot e politiche di sostituzione/takeover.
PASSWORD_PEPPER , segreto JWT	variabili d'ambiente (.env , common/env .hpp)	Materiale crittografico per l'hashing delle password e la firma dei token.
storeAuth , storeLobby , storeGame , storeNavigation , storeToast , storeStats , ws	store singleton (frontend)	Stato reattivo dell'applicazione, un singolo store per ciascun dominio, sincronizzato con gli eventi WebSocket.

File principali del progetto

Backend (C++23)

File	Responsabilità
src/main.cpp	Entry point: inizializza database, webserver e router.
src/webserver.cpp	Configurazione del server uWebSockets (rotte HTTP e endpoint WebSocket).
src/action_router.cpp	Smista i messaggi WebSocket (action) verso il controller competente.

src/http_router.cpp	Routing delle rotte HTTP REST.
src/database.cpp	Wrapper su SQLite (query, transazioni).
src/controllers/auth_controller.cpp	Registrazione, login, logout, sessione (/auth/*).
src/controllers/lobby_controller.cpp	Gestione lobby, membri, impostazioni, partite salvate.
src/controllers/game_controller.cpp	Flusso di gioco, broadcast dello stato, timer di turno.
src/controllers/stats_controller.cpp	Classifica globale e statistiche personali (/stats/*).
src/game/match_instance.cpp	Motore di gioco: giocate, pesche, risoluzione effetti, IA dei bot.
src/game/effects/*.cpp	Effetti concreti (avanzamento turno, scelta colore, scambio mani, ...).
src/game/rules/*.cpp	Regole standard e mod opzionali (draw_stacking , progressive , force_play , jump_in , no_bluffing , 70).

Frontend (Svelte 5 + TypeScript)

File / cartella	Responsabilità
src/lib/stores/	Store reattivi (auth , lobby , game , navigation , toast , stats , ws).
src/lib/components/game/	Tavolo di gioco: GameBoard , PlayerHand , OpponentHand , GamePiles , GameCard , FlyingCardsOverlay , DrawStackIndicator , card-bus .
src/lib/components/lobby/	Schermate e controlli di lobby e impostazioni.
src/app.css	Stili globali e definizioni dei font pixel-art.

Librerie utilizzate

Backend — gestite tramite Conan e compilate con CMake + Ninja:

Libreria	Uso
uWebSockets 20.48	Server HTTP e WebSocket ad alte prestazioni.
libuv 1.46	Event loop asincrono (dipendenza di uWebSockets).
OpenSSL 3.2	TLS e primitive crittografiche (hashing password).
zlib 1.3	Compressione (permessaggi WebSocket).
SQLite3 3.45	Persistenza di utenti, partite, statistiche e salvataggi.
jwt-cpp 0.7	Emissione e verifica dei token di sessione JWT (HS256).
nlohmann/json 3.11	Serializzazione/deserializzazione di tutti i messaggi JSON.

Frontend — gestito tramite npm:

Libreria	Uso
Svelte 5 (runes)	Framework UI reattivo.

Vite 6	Bundler e dev server.
TypeScript	Tipizzazione statica del codice client.
@dnd-kit/svelte, svelte-dnd-action	Drag-and-drop per il riordino delle carte in mano.
esbuild	Transpilazione veloce.
TypeDoc + typedoc-plugin-markdown	Generazione della documentazione delle API frontend (TSDoc).
Prettier (+ plugin Svelte)	Formattazione del codice.
md-to-pdf	Conversione della documentazione Markdown in PDF.

La documentazione del backend è generata con **Doxygen** (output LaTeX compilato con **XeLaTeX**).

Requisiti Funzionali

Codice	Nome	Descrizione
R01	Registrazione	Il sistema deve permettere a un nuovo utente di creare un account fornendo username, email e password, validati sia client-side che server-side.
R02	Login	Il sistema deve permettere l'accesso tramite email e password, restituendo una sessione persistente tramite cookie.
R03	Logout	Il sistema deve permettere all'utente di terminare la propria sessione.
R04	Creazione lobby	Il sistema deve permettere di creare una nuova lobby specificandone nome e visibilità (pubblica/privata).
R05	Elenco lobby pubbliche	Il sistema deve mostrare l'elenco delle lobby pubbliche disponibili, con numero di membri e bot.
R06	Ingresso in lobby	Il sistema deve permettere di entrare in una lobby tramite codice invito a 6 caratteri.
R07	Riconnessione alla lobby	Il sistema deve permettere a un utente disconnesso di rientrare automaticamente nella lobby (o nella partita) precedente entro un periodo di tolleranza.
R08	Gestione membri	L'host della lobby deve poter promuovere un altro membro a host o espellere un membro.
R09	Configurazione regole	L'host deve poter modificare le impostazioni della lobby (timer turno, numero carte iniziali, composizione mazzo, mod attive, numero e comportamento dei bot).
R10	Avvio partita	L'host deve poter avviare la partita quando sono presenti almeno due giocatori.
R11	Visualizzazione tavolo di gioco	Il sistema deve mostrare in tempo reale lo stato della partita: mano del giocatore, mani degli avversari (carte coperte), pila degli scarti, colore attivo, direzione di gioco e turno corrente.
R12	Gioco di una carta	Il giocatore di turno deve poter giocare una carta valida dalla propria mano, rispettando le regole di colore/valore/jolly.

R13	Pescare una carta	Il giocatore di turno, se non può o non vuole giocare, deve poter pescare una carta dal mazzo.
R14	Decisione su carta pescata	Se la carta pescata è giocabile, il sistema deve chiedere al giocatore se intende giocarla immediatamente o tenerla.
R15	Scelta del colore	Dopo la giocata di un Jolly, il sistema deve chiedere al giocatore di scegliere il nuovo colore attivo.
R16	Scelta del target (mod)	Con le mod opportune, il sistema deve permettere al giocatore di scegliere con quale avversario scambiare la mano.
R17	Dichiarazione "UNO"	Quando un giocatore ha due carte in mano e può giocarne almeno una, il sistema deve mostrare il pulsante per dichiarare "UNO" e applicare le eventuali penalità in caso di mancata dichiarazione.
R18	Timer di turno e AFK	Il sistema deve applicare un timer per ogni turno; allo scadere, deve eseguire automaticamente un'azione di default o far giocare il bot al posto del giocatore.
R19	Fine partita	Il sistema deve rilevare la fine della partita (un giocatore svuota la mano o la partita viene interrotta), notificarlo a tutti i client e aggiornare le statistiche.
R20	Salvataggio e ripresa partita	Se configurato, il sistema deve salvare lo stato della partita interrotta e permettere ai membri della lobby di riprenderla in seguito, oppure eliminarla.
R21	Visualizzazione classifica	Il sistema deve mostrare la classifica globale dei giocatori per vittorie/sconfitte.
R22	Visualizzazione statistiche personali	Il sistema deve mostrare all'utente le statistiche dettagliate sulle tipologie di carte giocate.

Casi d'uso

Caso d'uso del requisito R01 – Registrazione

Campo	Descrizione
Codice	R01
Nome	Registrazione di un nuovo account
Descrizione	Un visitatore non autenticato crea un nuovo account fornendo username, email e password.
Pre-condizioni	L'utente non è autenticato e si trova sul form di registrazione (RegisterForm). Username compreso tra 3 e 32 caratteri, password di almeno 8 caratteri.
Post-condizioni (Successo)	Viene creato un record nella tabella users con password salata e hashata; il server risponde 200 OK ; l'utente può ora effettuare il login.
Post-condizioni (Fallimento)	Nessun account creato; viene mostrato un errore di campo specifico (es. username/email già in uso, password troppo corta, username riservato ai bot).
Evento innescante	L'utente compila e invia il form di registrazione.
Scenario di base	Il client valida i campi (username 3-32, password ≥ 8) e invia POST /auth/register con {username, email, password} . Il server riconvalida, verifica l'assenza di duplicati e che lo username non inizi con il prefisso riservato bot_ , calcola l'hash della password (salt + pepper), inserisce il record e risponde 200 OK .
Scenario alternativo	Username o email già presenti → 409 Conflict ("Username or email already taken"); validazione fallita → 400 Bad Request ("Username must be 3-32 characters" / "Password must be at least 8 characters"); username che inizia con bot_ → errore "This username is reserved for AI bots".

Caso d'uso del requisito R02 – Login

Campo	Descrizione
Codice	R02
Nome	Autenticazione tramite email e password
Descrizione	Un utente registrato effettua l'accesso fornendo email e password, ottenendo una sessione persistente.
Pre-condizioni	L'utente non è autenticato e dispone di credenziali valide.
Post-condizioni (Successo)	Il server emette un token JWT (HS256) e lo imposta come cookie HttpOnly; il client salva lo username e naviga alla schermata di elenco lobby (lobbies).
Post-condizioni (Fallimento)	Nessuna sessione creata; viene mostrato un errore ("Invalid credentials").
Evento innescante	L'utente compila e invia il form di login (LoginForm).
Scenario di base	Il client invia POST /auth/login con {email, password} . Il server verifica la password (VerifyPassword), emette il token (IssueToken), imposta il cookie di sessione e risponde con lo username; il client aggiorna storeAuth e naviga a lobbies .
Scenario alternativo	Campi mancanti → 400 Bad Request ("email and password are required"); credenziali errate → 401 Unauthorized ("Invalid credentials").

Caso d'uso del requisito R03 – Logout

Campo	Descrizione
Codice	R03
Nome	Terminazione della sessione
Descrizione	Un utente autenticato termina la propria sessione, invalidando il cookie di autenticazione.
Pre-condizioni	L'utente è autenticato.
Post-condizioni (Successo)	Il cookie di sessione viene invalidato lato server; lo stato locale (<code>storeAuth</code>) viene azzerato; l'utente viene reindirizzato alla schermata di autenticazione (<code>auth</code>).
Post-condizioni (Fallimento)	In caso di errore di rete, lo stato locale viene comunque ripulito (logout best-effort).
Evento innescante	L'utente clicca il pulsante di logout.
Scenario di base	Il client invia <code>POST /auth/logout</code> ; il server rimuove/invalida il cookie; <code>storeAuth.logout()</code> azzerava <code>username</code> e richiama <code>storeNavigation.goto("auth")</code> .
Scenario alternativo	Se la richiesta fallisce per problemi di rete, il client procede comunque con la pulizia dello stato locale e il redirect.

Caso d'uso del requisito R04 – Creazione lobby

Campo	Descrizione
Codice	R04
Nome	Creazione di una nuova lobby
Descrizione	Un utente autenticato crea una nuova lobby, diventandone l'host.
Pre-condizioni	L'utente è autenticato e connesso al WebSocket.
Post-condizioni (Successo)	Viene creata una nuova Lobby con un invite_code generato; il creatore ne diventa host e primo membro; il client riceve lobby_joined e naviga alla schermata di lobby.
Post-condizioni (Fallimento)	La lobby non viene creata; viene mostrato un toast di errore.
Evento innescante	L'utente compila il form di creazione (nome, visibilità pubblica/privata) e conferma.
Scenario di base	Il client invia lobby_create con {name, is_public} (canale richiesta/risposta emitAndWait); LobbyController::HandleCreate istanzia la lobby, aggiunge il membro host e risponde lobby_joined con i dati della lobby; il client salva l'invite_code in localStorage e naviga a lobby.
Scenario alternativo	In caso di errore di validazione o di connessione, il server risponde con error e il client mostra un toast, restando sulla schermata corrente.

Caso d'uso del requisito R05 – Elenco lobby pubbliche

Campo	Descrizione
Codice	R05
Nome	Visualizzazione delle lobby pubbliche
Descrizione	Un utente autenticato consulta l'elenco delle lobby pubbliche disponibili.
Pre-condizioni	L'utente è autenticato e connesso al WebSocket.
Post-condizioni (Successo)	Il client mostra la lista delle lobby pubbliche con nome, numero di membri/bot e codice di accesso.
Post-condizioni (Fallimento)	In assenza di lobby, viene mostrata una lista vuota; in caso di errore viene mostrato un toast.
Evento innescante	L'utente apre la schermata <code>lobbies</code> o preme il pulsante di refresh.
Scenario di base	Il client invia <code>lobby_list; LobbyController::HandleList</code> raccoglie le lobby pubbliche e risponde con l'azione <code>lobby_list</code> contenente l'array delle lobby; lo store popola la lista.
Scenario alternativo	Nessuna lobby pubblica disponibile → lista vuota; errore di rete → toast di errore.

Caso d'uso del requisito R06 – Ingresso in lobby tramite codice invito

Campo	Descrizione
Codice	R06
Nome	Ingresso in lobby tramite codice invito
Descrizione	Un utente autenticato inserisce un codice invito di 6 caratteri per entrare in una lobby esistente.
Pre-condizioni	L'utente deve essere autenticato e connesso al WebSocket. Il codice invito deve corrispondere a una lobby esistente e non piena (massimo 4 membri).
Post-condizioni (Successo)	L'utente viene aggiunto alla lista dei membri della lobby; tutti i membri ricevono l'evento <code>Lobby_updated</code> ; il client dell'utente passa alla schermata di lobby (<code>LobbyScreen</code>).
Post-condizioni (Fallimento)	Il sistema mostra un messaggio di errore (es. "codice non valido" o "lobby piena") tramite toast, l'utente resta nella schermata di elenco lobby.
Evento innescante	L'utente digita il codice nel form <code>LobbyJoinForm</code> e conferma.
Scenario di base	Il client valida localmente il formato del codice (6 caratteri alfanumerici), invia l'azione <code>Lobby_join</code> con il codice; il server verifica l'esistenza della lobby e lo spazio disponibile, aggiunge il membro e risponde con <code>Lobby_joined</code> . Il client salva il codice in <code>localStorage</code> per future riconessioni e naviga alla schermata di lobby.
Scenario alternativo	Se il codice non corrisponde a nessuna lobby, oppure la lobby ha già 4 membri, il server risponde con un'azione <code>error</code> e una <code>reason</code> testuale, mostrata tramite <code>storeToast.error</code> .

Caso d'uso del requisito R07 – Riconnessione alla lobby

Campo	Descrizione
Codice	R07
Nome	Riconnessione automatica alla lobby o alla partita
Descrizione	Un utente disconnesso (es. refresh della pagina, caduta temporanea di rete) rientra automaticamente nella lobby — e, se presente, nella partita — in cui si trovava.
Pre-condizioni	Il client ha un <code>invite_code</code> salvato in <code>localStorage</code> ; la lobby (e l'eventuale partita) esiste ancora entro il periodo di tolleranza, durante il quale il membro non viene rimosso ma marcato come disconnesso.
Post-condizioni (Successo)	Il socket viene ri-associato al membro esistente (<code>is_connected = true</code>); tutti i membri ricevono <code>lobby_updated</code> ; se era in corso una partita, il client riceve <code>game_state_updated</code> e viene portato alla schermata di gioco.
Post-condizioni (Fallimento)	Se la lobby è scaduta o non esiste più, il codice salvato viene rimosso da <code>localStorage</code> e l'utente resta sulla schermata di elenco lobby.
Evento innescante	All'avvio dell'app o alla riconnessione del WebSocket, <code>storeLobby</code> esegue <code>#tryRejoin</code> .
Scenario di base	Il client invia <code>lobby_rejoin</code> con il codice salvato; <code>LobbyController::HandleRejoin</code> ritrova il membro, ne aggiorna il socket e lo stato di connessione, quindi diffonde lo stato aggiornato (lobby e/o partita).
Scenario alternativo	Se la lobby non esiste più, il server risponde con una <code>reason</code> ; il client mostra un toast informativo e rimuove il codice non più valido.

Caso d'uso del requisito R08 – Gestione dei membri (promozione/espulsione)

Campo	Descrizione
Codice	R08
Nome	Promozione a host ed espulsione di un membro
Descrizione	L'host della lobby promuove un altro membro al ruolo di host oppure espelle un membro dalla lobby.
Pre-condizioni	Il richiedente è l'host della lobby; il membro target è presente nella lobby. Per la promozione il target non deve essere un bot; per l'espulsione l'host non può espellere se stesso.
Post-condizioni (Successo)	Promozione: il ruolo di host viene trasferito al membro indicato. Espulsione: il membro viene rimosso dalla lobby. In entrambi i casi tutti i membri ricevono <code>lobby_updated</code> .
Post-condizioni (Fallimento)	Il server risponde con un errore ("You must be the host of this lobby!", "No such user is in the lobby", "You cannot promote a bot to host", "You cannot kick yourself") e lo stato della lobby resta invariato.
Evento innescante	L'host clicca l'azione di promozione o espulsione su un membro nell'elenco dei membri della lobby.
Scenario di base	Il client invia <code>lobby_promote</code> o <code>lobby_kick</code> con lo username del target; <code>HandlePromote</code> / <code>HandleKick</code> verificano che il richiedente sia host e che il target sia valido, applicano la modifica e diffondono il nuovo stato.
Scenario alternativo	Se il richiedente non è l'host o il target non è valido, il server rifiuta la richiesta con <code>error</code> ; lato client i controlli sono comunque disabilitati per i non-host.

Caso d'uso del requisito R09 – Configurazione delle regole della lobby

Campo	Descrizione
Codice	R09
Nome	Configurazione delle regole della lobby
Descrizione	L'host di una lobby modifica una o più impostazioni della partita (es. timer del turno, numero di carte iniziali, mod attive, numero di bot).
Pre-condizioni	L'utente che effettua la richiesta deve essere l'host della lobby corrente. La partita non deve ancora essere iniziata (oppure, per alcune impostazioni, può essere modificata anche a partita in corso, a seconda del campo).
Post-condizioni (Successo)	Le nuove impostazioni vengono memorizzate nello stato della lobby; tutti i membri ricevono l'evento <code>lobby_updated</code> con le impostazioni aggiornate; viene mostrato un toast di conferma all'host.
Post-condizioni (Fallimento)	Viene mostrato un messaggio di errore (es. "operazione non permessa") e le impostazioni non vengono modificate.
Evento innescante	L'host interagisce con i controlli del pannello <code>LobbySettings</code> (toggle, slider, selettori, griglia delle regole).
Scenario di base	Il client invia l'azione <code>lobby_update_settings</code> con un sottoinsieme dei campi di <code>LobbySettings</code> da aggiornare; il server verifica che il richiedente sia l'host, applica le modifiche all'oggetto <code>Lobby::settings</code> e diffonde il nuovo stato a tutti i membri tramite <code>lobby_updated</code> .
Scenario alternativo	Se il richiedente non è l'host, il server rifiuta la richiesta con un'azione <code>error</code> ; i controlli risultano comunque disabilitati (<code>disabled={!isHost}</code>) lato client per i non-host, per cui questo scenario si verifica solo in caso di richieste dirette anomale.

Caso d'uso del requisito R10 – Avvio della partita

Campo	Descrizione
Codice	R10
Nome	Avvio della partita da parte dell'host
Descrizione	L'host avvia la partita quando nella lobby sono presenti almeno due giocatori (umani o bot).
Pre-condizioni	Il richiedente è l'host; la lobby contiene almeno 2 membri; nessuna partita è già in corso.
Post-condizioni (Successo)	Viene creata una <code>MatchInstance</code> con le impostazioni della lobby, generato e mescolato il mazzo, distribuite le carte iniziali e impostata la carta di partenza; lo stato viene diffuso a tutti con <code>game_state_updated</code> e i client navigano alla schermata di gioco.
Post-condizioni (Fallimento)	Il server risponde con un errore ("Only the host can start the game.", "Not enough players to start.", "Match already started!") e nessuna partita viene creata.
Evento innescante	L'host clicca il pulsante "START!".
Scenario di base	Il client invia <code>lobby_start_match</code> ; <code>LobbyController::HandleStartGame</code> valida host e numero di giocatori, costruisce la lista <code>players_info</code> , istanzia la <code>MatchInstance</code> , richiama <code>Start()</code> e diffonde lo stato iniziale a tutti i membri.
Scenario alternativo	Con meno di 2 membri il pulsante "START!" è disabilitato lato client; un'eventuale richiesta diretta viene rifiutata con "Not enough players to start."

Caso d’uso del requisito R11 – Visualizzazione del tavolo di gioco

Campo	Descrizione
Codice	R11
Nome	Visualizzazione in tempo reale dello stato della partita
Descrizione	Il client mostra al giocatore lo stato corrente del tavolo, aggiornandolo a ogni evento ricevuto dal server.
Pre-condizioni	È in corso una partita; il client si trova nella schermata di gioco e riceve gli eventi <code>game_state_updated</code> .
Post-condizioni (Successo)	Il <code>GameBoard</code> mostra: la mano del giocatore locale, le mani degli avversari (carte coperte con relativo conteggio), la pila degli scarti con la <code>top_card</code> , il colore attivo, la direzione di gioco, il giocatore di turno, il timer del turno e l'eventuale contatore <code>+N</code> delle penalità accumulate.
Post-condizioni (Fallimento)	N/A — si tratta di un rendering passivo; in assenza di stato il tabellone resta in attesa.
Evento innescante	Ricezione di un messaggio <code>game_state_updated</code> dal server.
Scenario di base	<code>storeGame</code> decodifica <code>game_state</code> e aggiorna lo stato reattivo; i componenti <code>PlayerHand</code> , <code>OpponentHand</code> , <code>GamePiles</code> e <code>GameHud</code> si ridisegnano; <code>FlyingCardsOverlay</code> confronta lo stato precedente con quello nuovo per animare i movimenti di carta (pesca, giocata, penalità).
Scenario alternativo	Se il payload è incompleto o assente, il tabellone mostra lo stato parziale disponibile in attesa del successivo aggiornamento.

Caso d'uso del requisito R12 – Gioco di una carta

Campo	Descrizione
Codice	R12
Nome	Gioco di una carta dalla mano
Descrizione	Il giocatore a cui spetta il turno seleziona una carta dalla propria mano e la gioca sulla pila degli scarti.
Pre-condizioni	Deve essere il turno del giocatore (<code>current_turn == username</code>). La carta selezionata deve essere effettivamente presente nella mano del giocatore. Non deve essere in corso un'azione che richiede <code>input (actionRequired nullo)</code> .
Post-condizioni (Successo)	La carta viene rimossa dalla mano del giocatore e posta in cima alla pila degli scarti; vengono applicati gli eventuali effetti associati (Salta, Inverti, Pesca, Cambio colore, ecc.); il turno passa al giocatore successivo (salvo effetti che lo modificano); lo stato aggiornato viene diffuso a tutti i client tramite <code>game_state_updated</code> ; viene riprodotta l'animazione di volo della carta verso la pila degli scarti.
Post-condizioni (Fallimento)	La giocata viene rifiutata (es. la carta non corrisponde per colore/valore/jolly al <code>top_card</code> , oppure non è il turno del giocatore); lo stato di gioco resta inalterato e viene eventualmente notificato un errore.
Evento innescante	Il giocatore clicca (o preme Invio su) una carta nella propria mano (<code>PlayerHand</code> → <code>GameCard</code> → <code>onCardClick</code>).
Scenario di base	Il client invia l'azione <code>game_play_card</code> con l' <code>id</code> della carta. Il server, tramite <code>MatchInstance::PlayCard</code> , valida la giocata tramite <code>StandardRule::ValidatePlay</code> ; se valida, accoda gli <code>Effect</code> corrispondenti (es. <code>AdvanceTurnEffect</code> , <code>DrawEffect</code> , <code>SkipEffect</code> , <code>ReverseEffect</code> , <code>ChooseColorEffect</code>) e li risolve in sequenza tramite il <code>GameController</code> . Il nuovo stato viene serializzato per ciascun giocatore (<code>SerializePlayerState</code> , che nasconde le mani avversarie) e inviato in broadcast.
Scenario alternativo 1 — giocata fuori turno o carta non valida	Se non è il turno del giocatore o la carta non è giocabile, il server non modifica lo stato; può essere inviato un messaggio <code>error</code> al client con motivazione.
Scenario alternativo 2 — carta speciale con effetto interattivo	Se la carta giocata è un Jolly, il flusso prosegue verso il caso d'uso R15 (Scelta del colore); se la carta attiva una mod di scambio mani, prosegue verso R16 (Scelta del target).

Caso d'uso del requisito R13 – Pescare una carta

Campo	Descrizione
Codice	R13
Nome	Pesca di una carta dal mazzo
Descrizione	Il giocatore di turno, non potendo o non volendo giocare, pesca una carta dal mazzo centrale.
Pre-condizioni	È il turno del giocatore (<code>current_turn == username</code>); non è in corso un'azione che richiede input.
Post-condizioni (Successo)	Una o più carte (a seconda delle mod attive) vengono aggiunte alla mano del giocatore; se la carta pescata è giocabile e la mod "Force Play" non è attiva, viene richiesto un input (<code>action_required: play_drawn_card</code>); altrimenti il turno avanza. Lo stato aggiornato viene diffuso a tutti.
Post-condizioni (Fallimento)	Se non è il turno del giocatore, l'azione viene ignorata e lo stato resta invariato.
Evento innescante	Il giocatore clicca sul mazzo di pesca (<code>GamePiles → draw_pile</code>).
Scenario di base	Il client invia <code>game_draw_card</code> ; <code>MatchInstance::DrawCard</code> aggiunge la carta in cima al mazzo alla mano, ne valuta la giocabilità e, in base alle regole, accoda gli effetti opportuni; lo stato viene serializzato e diffuso.
Scenario alternativo	Se <code>pending_draws > 0</code> (catena +2/+4) il giocatore pesca le carte di penalità accumulate e il turno avanza; con la mod "Progressive" il motore continua a pescare finché non trova una carta giocabile; con la mod "Force Play" la carta giocabile pescata viene giocata immediatamente.

Caso d'uso del requisito R14 – Decisione sulla carta pescata

Campo	Descrizione
Codice	R14
Nome	Decisione di giocare o tenere la carta pescata
Descrizione	Quando la carta pescata è giocabile, il giocatore decide se giocarla immediatamente o tenerla in mano.
Pre-condizioni	L'ultima pesca ha prodotto una carta giocabile e la mod "Force Play" non è attiva; un <code>DecideDrawnCardEffect</code> è in attesa (<code>action_required: play_drawn_card</code>).
Post-condizioni (Successo)	Scelta "Play": la carta pescata viene giocata sulla pila degli scarti (proseguendo con i relativi effetti). Scelta "Keep": la carta resta in mano e il turno avanza.
Post-condizioni (Fallimento)	In assenza di un input valido, il gioco resta in attesa e nessun altro giocatore può agire.
Evento innescante	Il client riceve <code>action_required: play_drawn_card</code> e mostra il prompt di decisione; l'utente sceglie.
Scenario di base	Il client invia <code>game_submit_input</code> con value pari a "PLAY" o "KEEP"; <code>MatchInstance::ProvideInput</code> memorizza l'input e <code>DecideDrawnCardEffect::Resolve</code> lo interpreta, giocando la carta o lasciandola in mano e avanzando il turno.
Scenario alternativo	Un valore non riconosciuto viene ignorato e l'effetto resta in stato <code>kNeedsInput</code> in attesa di un input valido.

Caso d'uso del requisito R15 – Scelta del colore dopo un Jolly

Campo	Descrizione
Codice	R15
Nome	Scelta del colore attivo dopo la giocata di un Jolly
Descrizione	Dopo che un giocatore ha giocato una carta Jolly (o Jolly Pesca Quattro), il sistema richiede a quel giocatore di scegliere il nuovo colore attivo tra i quattro disponibili.
Pre-condizioni	È stata appena giocata una carta di colore "wild"; il ChooseColorEffect è in testa alla coda degli effetti e attende input dal giocatore (pending_player).
Post-condizioni (Successo)	Il colore attivo (active_color) dello stato di gioco viene aggiornato al colore scelto; l'effetto si risolve e il flusso degli effetti successivi (tipicamente AdvanceTurnEffect) prosegue; lo stato aggiornato viene diffuso a tutti i client.
Post-condizioni (Fallimento)	In assenza di un input valido, il gioco resta in attesa (actionRequired === "choose_color") e nessun altro giocatore può agire.
Evento innescante	Il client del giocatore in attesa riceve actionRequired === "choose_color" e mostra il componente ActionPickColor , in cui l'utente clicca uno dei quattro pulsanti colore.
Scenario di base	Il client invia l'azione game_submit_input con value pari al colore scelto (in maiuscolo, es. "RED"). Il server, tramite MatchInstance::ProvideInput , memorizza l'input in provided_input ; il ChooseColorEffect::Resolve legge il valore, aggiorna active_color e restituisce EffectStatus::kResolved , permettendo al motore di proseguire con gli effetti successivi nella coda.
Scenario alternativo	Se il colore inviato non è uno dei quattro valori validi, il server può ignorare l'input o richiederlo nuovamente, mantenendo l'effetto in stato kNeedsInput .

Caso d'uso del requisito R16 – Scelta del target per lo scambio mani (mod)

Campo	Descrizione
Codice	R16
Nome	Scelta dell'avversario con cui scambiare la mano
Descrizione	Con la mod "Seven-Zero" attiva, dopo aver giocato la carta che innesca lo scambio (il "7"), il giocatore sceglie con quale avversario scambiare la propria mano.
Pre-condizioni	È stata giocata la carta che innesca lo scambio mani; un <code>DecideSwapTargetEffect</code> è in testa alla coda ed è in attesa di input (<code>action_required: choose_target</code>), con un <code>action_context</code> che elenca gli username selezionabili.
Post-condizioni (Successo)	La mano del giocatore viene scambiata con quella dell'avversario scelto; l'effetto si risolve e il flusso prosegue (tipicamente con l'avanzamento del turno); lo stato aggiornato viene diffuso a tutti.
Post-condizioni (Fallimento)	In assenza di un target valido, il gioco resta in attesa e nessun altro giocatore può agire.
Evento innescante	Il client riceve <code>action_required: choose_target</code> e mostra il componente <code>ActionChooseTarget</code> , evidenziando gli avversari selezionabili; l'utente clicca uno di essi.
Scenario di base	Il client invia <code>game_submit_input</code> con <code>value</code> pari allo username del target; <code>MatchInstance::ProvideInput</code> memorizza l'input e <code>DecideSwapTargetEffect::Resolve</code> esegue lo scambio delle mani tra i due giocatori.
Scenario alternativo	Se lo username inviato non è tra quelli validi presenti in <code>action_context</code> , l'input viene ignorato e l'effetto resta in attesa.

Caso d'uso del requisito R17 – Dichiarazione "UNO"

Campo	Descrizione
Codice	R17
Nome	Dichiarazione "UNO"
Descrizione	Un giocatore con due carte in mano, di cui almeno una giocabile, dichiara "UNO" prima del proprio turno successivo per evitare la penalità.
Pre-condizioni	Il giocatore ha esattamente 2 carte in mano; almeno una di esse è giocabile rispetto alla <code>top_card</code> ; il giocatore non ha già dichiarato UNO per questa mano (<code>has_called_uno == false</code>).
Post-condizioni (Successo)	Il flag <code>has_called_uno</code> del giocatore viene impostato a <code>true</code> ; il pulsante fluttuante "UNO!" viene sostituito da un badge "☑ UNO Dichiarato"; nessuna penalità viene applicata quando il giocatore scende a una carta.
Post-condizioni (Fallimento)	Se il giocatore gioca una carta scendendo a una sola carta senza aver dichiarato UNO, il sistema applica la penalità prevista (pesca di carte aggiuntive).
Evento innescante	Il client mostra il pulsante "UNO!" (<code>GameUnoButton</code>) quando è il turno del giocatore, ha due carte ed esiste almeno una giocata valida; l'utente clicca il pulsante.
Scenario di base	Il client invia l'azione <code>game_call_uno</code> ; il server marca <code>has_called_uno = true</code> per il giocatore nello stato di gioco e diffonde l'aggiornamento.
Scenario alternativo	Se il giocatore gioca la propria penultima carta senza prima dichiarare UNO, alla successiva sincronizzazione dello stato il sistema applica la penalità configurata (pesca carte) prima di concludere il turno.

Caso d'uso del requisito R18 – Timer di turno e gestione AFK

Campo	Descrizione
Codice	R18
Nome	Timer di turno e azione automatica in caso di inattività
Descrizione	Ogni turno è soggetto a un timer configurabile; allo scadere, il sistema agisce automaticamente al posto del giocatore inattivo.
Pre-condizioni	È in corso una partita; al giocatore di turno è associato un limite di tempo (<code>turn_time_limit_ms</code>).
Post-condizioni (Successo)	Allo scadere del timer il server esegue un'azione di default o fa giocare un bot al posto del giocatore (<code>TakeBotTurn</code>), quindi diffonde lo stato aggiornato con <code>game_state_updated</code> .
Post-condizioni (Fallimento)	N/A — la scadenza del timer è un evento di sistema.
Evento innescante	Lo scadere del timer di turno gestito dal <code>GameController</code> .
Scenario di base	All'inizio di ogni turno il <code>GameController</code> arma un timer; alla scadenza esegue la mossa automatica e ri-diffonde lo stato. Il client mostra il countdown sincronizzato sul campo <code>turn_time_remaining_ms</code> ricevuto dal server.
Scenario alternativo	Se il giocatore compie una mossa valida prima della scadenza, il timer corrente viene annullato e riarmato per il turno del giocatore successivo.

Caso d'uso del requisito R19 – Fine partita

Campo	Descrizione
Codice	R19
Nome	Rilevamento e gestione della fine partita
Descrizione	La partita termina quando un giocatore esaurisce le carte in mano (vittoria) oppure quando si verifica un'interruzione che rende impossibile continuare (es. tutti i giocatori umani disconnessi).
Pre-condizioni	La partita è in stato <code>kPlaying</code> .
Post-condizioni (Successo — Vittoria)	Lo stato passa a <code>kFinished</code> , viene impostato <code>winner</code> ; tutti i client ricevono l'evento <code>game_over</code> con il nome del vincitore; il frontend mostra <code>GameVictoryPopup</code> (con audio di vittoria/sconfitta in base al destinatario) e aggiorna le statistiche su database.
Post-condizioni (Successo — Interruzione)	Lo stato passa a <code>kFinished</code> senza <code>winner</code> ; il frontend mostra <code>GameInterruptedPopup</code> , informando che la partita è stata annullata.
Post-condizioni (Fallimento)	N/A — il rilevamento della fine partita è un evento di sistema, non un'azione utente che possa "fallire" dal punto di vista applicativo.
Evento innescante	Risoluzione di un effetto di gioco che porta la mano di un giocatore a zero carte, oppure notifica di disconnessione di tutti gli umani da parte del <code>LobbyController</code> .
Scenario di base	Il <code>GameController</code> rileva la condizione di vittoria dopo la risoluzione degli effetti, imposta <code>state_.status = kFinished</code> e <code>state_.winner</code> , quindi invia in broadcast l'azione <code>game_over</code> . Il frontend (<code>storeGame</code>) imposta <code>state.is_over = true</code> e <code>state.winner</code> , interrompe il timer del turno e mostra il popup di vittoria/sconfitta. Le statistiche dei giocatori (vittorie, sconfitte, carte giocate per tipo) vengono aggiornate sul database.
Scenario alternativo	Se la partita termina per interruzione (nessun vincitore), il frontend mostra <code>GameInterruptedPopup</code> e, cliccando "Back to Lobby", l'utente torna alla schermata di lobby (<code>storeGame.returnToLobby()</code>), che a sua volta richiama <code>storeLobby.leave()</code> se necessario.

Caso d'uso del requisito R20 – Salvataggio e ripresa di una partita

Campo	Descrizione
Codice	R20
Nome	Salvataggio e ripresa di una partita interrotta
Descrizione	Se la lobby ha l'opzione "Save Match" attiva e una partita viene interrotta (es. un giocatore abbandona e "Quitting stops the Match" è attivo), lo stato viene salvato; i membri della lobby possono successivamente riprenderla o eliminarla.
Pre-condizioni	quit_deletes_match e save_state sono attivi nelle impostazioni della lobby; una partita era in corso al momento dell'interruzione.
Post-condizioni (Successo — Salvataggio)	Lo stato serializzato della partita (<code>ExportState</code>) viene scritto su database, associato a un <code>match_id</code> e alla lista dei giocatori coinvolti; la partita compare nella lista "Saved Matches" della lobby.
Post-condizioni (Successo — Ripresa)	Una nuova <code>MatchInstance</code> viene creata a partire dal JSON salvato; la partita riprende dallo stesso punto (mani, mazzo, turno corrente, ecc.); tutti i membri vengono reindirizzati alla schermata di gioco.
Post-condizioni (Successo — Eliminazione)	Il record della partita salvata viene rimosso dal database e dalla lista <code>savedMatches</code> .
Post-condizioni (Fallimento)	In caso di errore (es. partita non trovata), viene mostrato un toast di errore ("Saved games list error" / "Failed to delete match") e lo stato locale non viene modificato.
Evento innescante	Interruzione della partita con le impostazioni sopra descritte (salvataggio); apertura del menu a tendina "Saved Matches" e clic su "Start" o sull'icona del cestino (ripresa/eliminazione).
Scenario di base	Salvataggio: alla distruzione/interruzione del match, <code>LobbyController::SaveMatchStateToDB</code> persiste lo stato. Elenco: all'apertura della lobby, il client invia <code>lobby_list_saved_matches</code> e popola <code>storeLobby.savedMatches</code> . Ripresa: il client invia <code>lobby_resume_saved_match</code> con <code>match_id</code> ; il server ricarica lo stato e avvia il <code>GameController</code> , inviando <code>game_state_updated</code> a tutti i membri. Eliminazione: il client invia <code>lobby_delete_saved_match</code> con <code>match_id</code> ; il server elimina il record e risponde con successo, mostrando il toast "Saved match deleted".
Scenario alternativo	Se il <code>match_id</code> richiesto per ripresa o eliminazione non esiste più (es. eliminato da un altro membro nel frattempo), il server risponde con <code>ok: false</code> e una <code>reason</code> ; il client mostra il messaggio di errore corrispondente senza modificare la lista locale.

Caso d'uso del requisito R21 – Visualizzazione della classifica

Campo	Descrizione
Codice	R21
Nome	Consultazione della classifica globale
Descrizione	Un utente consulta la classifica globale dei giocatori, ordinata per vittorie.
Pre-condizioni	L'utente accede alla sezione statistiche/classifica.
Post-condizioni (Successo)	Viene mostrata la classifica dei migliori giocatori (fino a 50) con vittorie, sconfitte e posizione (rank).
Post-condizioni (Fallimento)	In caso di errore viene mostrato un toast ("Failed to load leaderboard.").
Evento innescante	L'utente apre la schermata delle statistiche/classifica.
Scenario di base	Il client invia GET /stats/leaderboard ; il server esegue una query ordinata per vittorie con calcolo del rank e risponde con l'array leaderboard ; storeStats popola lo stato e l'UI mostra la tabella.
Scenario alternativo	In caso di errore di rete, lo store non aggiorna la classifica e viene mostrato un toast di errore.

Caso d'uso del requisito R22 – Visualizzazione delle statistiche personali

Campo	Descrizione
Codice	R22
Nome	Consultazione delle statistiche personali
Descrizione	Un utente autenticato consulta le proprie statistiche dettagliate sulle carte giocate.
Pre-condizioni	L'utente è autenticato (richiesta con cookie di sessione).
Post-condizioni (Successo)	Viene mostrato un riepilogo con vittorie, sconfitte e i conteggi delle carte giocate per colore e per valore/tipologia.
Post-condizioni (Fallimento)	In caso di errore viene mostrato un toast di errore.
Evento innescante	L'utente apre la schermata delle statistiche personali.
Scenario di base	Il client invia <code>GET /stats/me</code> includendo le credenziali; il server interroga la tabella <code>player_stats</code> per l'utente e restituisce i conteggi aggregati; <code>storeStats.fetchMe</code> popola lo stato e l'UI mostra i dati.
Scenario alternativo	Per un utente senza partite giocate, i conteggi risultano pari a zero.

Test

Esempi di esecuzione

Di seguito alcuni esempi rappresentativi di interazione client/server tramite i messaggi WebSocket scambiati, utili a documentare il comportamento atteso del sistema in scenari concreti.

Esempio 1 — Login con credenziali corrette

```
POST /auth/login
{ "email": "mario@example.com", "password": "password123" }

→ 200 OK
{ "username": "Mario", "avatar": "" }
```

Esempio 2 — Creazione lobby e ricezione del codice invito

```
→ client invia:
{ "action": "lobby_create", "request_id": "1", "name": "Stanza di Mario",
  "is_public": true }

← server risponde:
{
  "action": "lobby_joined",
  "request_id": "1",
  "lobby": {
    "invite_code": "AB12CD",
    "host": "Mario",
    "name": "Stanza di Mario",
    "member_count": 1,
    "members": [ { "username": "Mario", "is_connected": true, "is_host":
true, "is_bot": false } ],
    "settings": { "...": "..." }
  }
}
```

Esempio 3 — Tentativo di giocata non valida

```
→ client invia:
{ "action": "game_play_card", "card_id": 42 }

← server risponde (giocata rifiutata):
{ "action": "error", "reason": "Card does not match the current color, value
or wild." }
```

(Lo stato di gioco resta invariato e nessun `game_state_updated` viene inviato.)

Esempio 4 — Carta Jolly e scelta colore

→ client gioca un Jolly:

```
{ "action": "game_play_card", "card_id": 17 }
```

← server invia lo stato aggiornato con richiesta di input:

```
{  
  "action": "game_state_updated",  
  "game_state": { "...": "..." },  
  "action_required": "choose_color",  
  "action_context": null  
}
```

→ client risponde con il colore scelto:

```
{ "action": "game_submit_input", "value": "BLUE" }
```

← server invia il nuovo stato con active_color = BLUE e action_required = null

Risultati dei test

Codice requisito	Codice test	Nome	Descrizione test	Input	Risultato
R01	1.1	Registrazione	Username già esistente	username duplicato	409, errore "User taken."
R01	1.2	Registrazione	Password troppo corta	password < 8 caratteri	Errore di validazione richiesta al server
R02	2.1	Login	Credenziali corrette	email / password validi	200 OK, cookie creato a "lobbies"
R02	2.2	Login	Credenziali errate	password errata	401, messaggio di errore
R03	3.1	Logout	Terminazione della sessione	clic sul pulsante di logout con sessione attiva	POST /auth/logout stato locale azzerato
R04	4.1	Creazione lobby	Creazione lobby pubblica con nome valido	lobby_create con {name, is_public: true}	lobby_joined generato, navigazione
R05	5.1	Elenco lobby	Recupero delle lobby pubbliche	lobby_list	risposta lobby_storeLobby
R06	6.1	Ingresso lobby	Codice invito valido	codice a 6 caratteri di una lobby esistente, non piena	lobby_joined
R06	6.2	Ingresso lobby	Codice invito inesistente	codice random non assegnato	error , toast "Invito non valido"
R06	6.3	Ingresso lobby	Lobby piena	codice di lobby con 4/4 membri	error , toast "Lobby piena"
R07	7.1	Riconnessione	Codice salvato di lobby ancora esistente	lobby_rejoin con codice in localStorage	rientro nella lobby true , stato differenziale (lobby_update)
R07	7.2	Riconnessione	Codice di lobby scaduta/inesistente	lobby_rejoin con codice non più valido	reason di errore in localStorage
R08	8.1	Gestione membri	Promozione di un membro a host	lobby_promote inviato dall'host	ruolo host trasferito e diffuso
R08	8.2	Gestione membri	Espulsione di un membro	lobby_kick inviato dall'host	membro rimosso e lobby_update
R08	8.3	Gestione membri	Promozione tentata da non-host	lobby_promote inviato da un membro non host	error "You must be a host to promote a member in this lobby!", stato invariato
R09	9.1	Impostazioni lobby	Modifica timer da non-host	lobby_update_settings da un membro non host	error , impostazioni non modificate
R09	9.2	Impostazioni lobby	Modifica numero carte iniziali	host imposta starting_cards = 5	lobby_update "Impostazioni lobby modificate"

R10	10.1	Avvio partita	Meno di 2 giocatori	host con 1 solo membro nella lobby	pulsante "START" diretta → "Not enough players"
R10	10.2	Avvio partita	Almeno 2 giocatori	host con 2+ membri	partita avviata, messaggio inviato a tutti, n
R11	11.1	Visualizzazione tavolo	Aggiornamento dello stato di gioco	ricezione di <code>game_state_updated</code>	board aggiornato, avversari coperti, attivo, turno, tir
R12	12.1	Gioco carta	Carta valida (stesso colore)	carta dello stesso colore della <code>top_card</code>	carta spostata s
R12	12.2	Gioco carta	Carta non valida	carta di colore/valore diverso e non jolly	<code>error</code> , mano i
R12	12.3	Gioco carta	Fuori turno	giocata inviata quando <code>current_turn != username</code>	<code>error</code> , stato in
R13	13.1	Pescare carta	Pesca durante il proprio turno	<code>game_draw_card</code> da giocatore di turno	una carta aggiu <code>action_requirement</code> "play_drawn_c
R14	14.1	Decisione carta pescata	Carta pescata giocabile, scelta "Play It"	<code>game_submit_input</code> con "PLAY"	la carta pescata immediatamente
R14	14.2	Decisione carta pescata	Carta pescata giocabile, scelta "Keep It"	<code>game_submit_input</code> con "KEEP"	la carta resta in
R15	15.1	Scelta colore	Jolly giocato, colore scelto valido	<code>game_submit_input</code> con "GREEN"	<code>active_color</code>
R16	16.1	Scelta target	Mod Seven-Zero, target valido scelto	<code>game_submit_input</code> con lo <code>username</code> di un avversario in <code>action_context</code>	mani scambiate risolto
R17	17.1	Dichiarazione UNO	2 carte in mano, almeno una giocabile	<code>game_call_uno</code>	<code>has_called_uno</code> mostrato
R17	17.2	Penalità UNO mancata	Giocatore scende a 1 carta senza dichiarazione	giocata di una carta con <code>has_called_uno = false</code> e 2 carte iniziali	penalità applica
R18	18.1	Timer turno	Scadenza timer senza azione del giocatore	nessuna azione entro <code>turn_time_limit_ms</code>	turno gestito al di default), <code>game</code> aggiornato
R19	19.1	Fine partita	Vittoria	un giocatore gioca la sua ultima carta	<code>game_over</code> con popup di vittoria
R19	19.2	Fine partita	Interruzione	tutti gli umani si disconnettono durante la partita	<code>game_over</code> se "Interrupted" m

R20	20.1	Partite salvate	Elenco partite salvate	<code>lobby_list_saved_matches</code>	lista popolata in <code>storeLobby.s</code>
R20	20.2	Partite salvate	Ripresa partita	<code>lobby_resume_saved_match</code> con <code>match_id</code> valido	partita ripresa, 1 "game"
R20	20.3	Partite salvate	Eliminazione partita	<code>lobby_delete_saved_match</code> con <code>match_id</code> valido	record eliminato "deleted"
R20	20.4	Partite salvate	Eliminazione <code>match_id</code> inesistente	<code>match_id</code> non presente nel database	<code>ok: false</code> , to
R21	21.1	Classifica	Recupero classifica globale	<code>GET /stats/leaderboard</code>	lista top 50 gioc
R22	22.1	Statistiche personali	Recupero statistiche utente	<code>GET /stats/me</code>	oggetto con cor giocata

Conclusioni

Il progetto Uni-Uno ha permesso di affrontare in modo concreto le problematiche tipiche di un'applicazione multiplayer in tempo reale: sincronizzazione dello stato tra più client, gestione di disconnessioni e riconnessioni, validazione coerente delle regole sia lato client che lato server, ed estensibilità del motore di gioco tramite un'architettura a effetti e regole disaccoppiate.

Punti di forza

- L'architettura *Effect-based* del motore di gioco rende relativamente semplice aggiungere nuove carte speciali o mod opzionali senza modificare il core del motore, limitandosi a implementare nuovi `Effect` e `GameRule`.
- La separazione netta tra store reattivi nel frontend (uno per dominio) ha facilitato la gestione dello stato e la sua sincronizzazione con gli eventi WebSocket, anche dopo refactoring importanti (es. migrazione a classi singleton con prefisso `store*`).
- Il sistema di correlazione richiesta/risposta tramite `request_id` (`emitAndWait`) ha semplificato notevolmente la gestione degli errori lato frontend, fornendo un'interfaccia simile alle Fetch API.
- Il `CardBus` introdotto nel refactoring del tabellone di gioco ha permesso di centralizzare la logica delle animazioni di volo delle carte, risolvendo problemi di layout (passaggio da posizionamento assoluto a flexbox con gap negativo per l'effetto "ventaglio" delle carte in mano) e bug di reattività (loop infiniti causati da `$state` nel registro degli elementi).

Punti di debolezza e criticità riscontrate

- La revisione del codice backend ha evidenziato alcune criticità poi corrette, tra cui: una direzione errata nell'inserimento delle carte di penalità UNO (`push_front` invece di `push_back`), una possibile ricorsione non limitata in `TakeBotTurn`, un posizionamento non sicuro dei timer di turno, l'assenza del broadcast `game_over` in alcuni percorsi di fine partita e un bot che selezionava sempre il colore rosso per le carte Jolly.
- Alcune incongruenze tra frontend e backend (es. validazione delle password più stringente nel client rispetto al server, mancata corrispondenza tra `lobby_id` e `invite_code`, messaggi di errore non mappati correttamente perché il backend restituisce stringhe in inglese semplice) sono state individuate e risolte nel corso dello sviluppo, ma evidenziano l'importanza di mantenere sincronizzate le regole di validazione tra i due strati applicativi.

- L'introduzione di un indice della carta giocata per l'innesco delle animazioni lato client è stata identificata come miglioria utile ma non risulta ancora completamente integrata in tutti i flussi di gioco.

Sviluppi futuri

- Completare l'integrazione dell'indice di carta giocata per rendere le animazioni di scarto più robuste in tutti i casi (incluse le giocate dei bot).
- Estendere la suite di test automatici (attualmente verificata principalmente in modo manuale/funzionale) con test unitari sul motore di gioco (`Effect` , `GameRule`) e test end-to-end sui flussi WebSocket principali.
- Valutare l'aggiunta di mod aggiuntive (es. "Draw Stacking", "Seven-Zero") già previste nell'interfaccia di configurazione ma non ancora collegate al motore di gioco.
- Migliorare la gestione degli errori di rete lato frontend, uniformando i messaggi mostrati all'utente indipendentemente dal canale (HTTP REST o WebSocket) da cui provengono.

In conclusione, il piano di test eseguito ha confermato il corretto funzionamento delle funzionalità principali del sistema, e le criticità individuate durante la revisione del codice sono state affrontate in modo iterativo, contribuendo a un prodotto finale stabile e coerente con i requisiti definiti in fase di analisi.

Documentazione Doxygen + Frontend

Manuale di riferimento del backend (C++) generato da Doxygen, seguito dalla documentazione delle API del frontend (TypeDoc).

1	Gerarchia delle directory	1
1.1	Directory	1
2	Indice dei namespace	5
2.1	Lista dei namespace	5
3	Indice della gerarchia	7
3.1	Gerarchia delle classi	7
4	Indice dei tipi composti	9
4.1	Elenco dei tipi composti	9
5	Indice dei file	13
5.1	Elenco dei file	13
6	Documentazione delle directory	15
6.1	Riferimenti per la directory include/common	15
6.2	Riferimenti per la directory include/controllers	16
6.3	Riferimenti per la directory src/controllers	17
6.4	Riferimenti per la directory include/game/effects	18
6.5	Riferimenti per la directory src/game/effects	19
6.6	Riferimenti per la directory include/common/game	19
6.7	Riferimenti per la directory include/game	20
6.8	Riferimenti per la directory src/game	21
6.9	Riferimenti per la directory include	22
6.10	Riferimenti per la directory include/game/rules	23
6.11	Riferimenti per la directory src/game/rules	23
6.12	Riferimenti per la directory src	24
7	Documentazione dei namespace	25
7.1	Riferimenti per il namespace Base64	25
7.1.1	Documentazione delle funzioni	25
7.1.1.1	Decode()	25
7.1.1.2	Encode()	26
7.1.2	Documentazione delle variabili	26
7.1.2.1	kAlphabet	26
7.1.2.2	kDecodeTable	26
7.1.2.3	kInvalid	26
7.2	Riferimenti per il namespace Env	26
7.2.1	Documentazione delle funzioni	27
7.2.1.1	Get()	27
7.2.1.2	Load()	27
7.2.1.3	Require()	28
7.2.1.4	SetEnv()	29
7.2.1.5	Trim()	30

7.3 Riferimenti per il namespace game	30
7.3.1 Documentazione delle ridefinizioni di tipo (typedef)	33
7.3.1.1 CompactCard	33
7.3.1.2 RuleFactory	33
7.3.2 Documentazione dei tipi enumerati	33
7.3.2.1 Color	33
7.3.2.2 EffectStatus	34
7.3.2.3 EffectType	34
7.3.2.4 MatchStatus	34
7.3.2.5 Value	35
7.3.3 Documentazione delle funzioni	35
7.3.3.1 GetColor()	35
7.3.3.2 GetId()	36
7.3.3.3 GetValue()	37
7.3.3.4 MakeCard()	37
7.3.3.5 ReshuffleDiscardIntoDraw()	38
7.3.4 Documentazione delle variabili	38
7.3.4.1 kReservedBotNames	38
7.3.4.2 registrar [1/6]	39
7.3.4.3 registrar [2/6]	39
7.3.4.4 registrar [3/6]	39
7.3.4.5 registrar [4/6]	39
7.3.4.6 registrar [5/6]	39
7.3.4.7 registrar [6/6]	39
7.4 Riferimenti per il namespace http	39
7.4.1 Documentazione delle funzioni	40
7.4.1.1 GetCookieValue()	40
7.4.1.2 ReadBody()	41
7.4.1.3 TrimWhitespace()	41
7.5 Riferimenti per il namespace ws	42
7.5.1 Documentazione dei tipi enumerati	43
7.5.1.1 ClientAction	43
7.5.1.2 ServerAction	44
7.5.2 Documentazione delle funzioni	44
7.5.2.1 Get()	44
7.5.2.2 GetOr()	45
7.5.2.3 MakeResponse()	47
7.5.2.4 SendError()	48
7.5.3 Documentazione delle variabili	50
7.5.3.1 kClientActionStr	50
7.5.3.2 kServerActionStr	50
7.5.3.3 kStrClientAction	50

8 Documentazione delle classi	51
8.1 Riferimenti per la classe <code>ActionRouter</code>	51
8.1.1 Descrizione dettagliata	53
8.1.2 Documentazione dei costruttori e dei distruttori	53
8.1.2.1 <code>ActionRouter()</code>	53
8.1.3 Documentazione delle funzioni membro	54
8.1.3.1 <code>Dispatch()</code>	54
8.1.3.2 <code>On()</code>	54
8.1.3.3 <code>OnAny()</code>	55
8.1.4 Documentazione dei membri dato	56
8.1.4.1 <code>handlers_</code>	56
8.1.4.2 <code>wildcards_</code>	56
8.2 Riferimenti per la classe <code>game::AdvanceTurnEffect</code>	56
8.2.1 Descrizione dettagliata	57
8.2.2 Documentazione delle funzioni membro	57
8.2.2.1 <code>GetType()</code>	57
8.2.2.2 <code>Resolve()</code>	58
8.3 Riferimenti per la classe <code>AuthController</code>	58
8.3.1 Descrizione dettagliata	59
8.3.2 Documentazione dei costruttori e dei distruttori	60
8.3.2.1 <code>AuthController()</code>	60
8.3.3 Documentazione delle funzioni membro	61
8.3.3.1 <code>HandleLogin()</code>	61
8.3.3.2 <code>HandleLogout()</code>	62
8.3.3.3 <code>HandleMe()</code>	62
8.3.3.4 <code>HandleRegister()</code>	62
8.3.3.5 <code>HashPassword()</code>	63
8.3.3.6 <code>IssueToken()</code>	64
8.3.3.7 <code>VerifyPassword()</code>	65
8.3.3.8 <code>VerifyToken()</code>	66
8.3.4 Documentazione dei membri dato	67
8.3.4.1 <code>kHashBytes</code>	67
8.3.4.2 <code>kIterations</code>	67
8.3.4.3 <code>kMaxBodyBytes</code>	68
8.3.4.4 <code>kMaxUsernameLen</code>	68
8.3.4.5 <code>kMinPasswordLen</code>	68
8.3.4.6 <code>kMinUsernameLen</code>	68
8.3.4.7 <code>kSaltBytes</code>	68
8.4 Riferimenti per la struct <code>game::CardDrawnEvent</code>	69
8.4.1 Documentazione dei membri dato	69
8.4.1.1 <code>drawn_card</code>	69
8.4.1.2 <code>force_play</code>	69
8.4.1.3 <code>is_handled</code>	69

8.4.1.4	keep_drawing	69
8.4.1.5	player_username	69
8.5	Riferimenti per la struct game::CardPlayedEvent	70
8.5.1	Descrizione dettagliata	70
8.5.2	Documentazione dei membri dato	70
8.5.2.1	is_handled	70
8.5.2.2	is_out_of_turn	70
8.5.2.3	is_valid_play	70
8.5.2.4	played_card	70
8.5.2.5	player_username	71
8.6	Riferimenti per la classe game::ChooseColorEffect	71
8.6.1	Descrizione dettagliata	72
8.6.2	Documentazione dei costruttori e dei distruttori	72
8.6.2.1	ChooseColorEffect()	72
8.6.3	Documentazione delle funzioni membro	73
8.6.3.1	GetType()	73
8.6.3.2	Resolve()	73
8.6.4	Documentazione dei membri dato	73
8.6.4.1	target_username_	73
8.7	Riferimenti per la classe Database	74
8.7.1	Descrizione dettagliata	74
8.7.2	Documentazione dei costruttori e dei distruttori	75
8.7.2.1	Database() [1/2]	75
8.7.2.2	~Database()	75
8.7.2.3	Database() [2/2]	75
8.7.3	Documentazione delle funzioni membro	76
8.7.3.1	ApplySchema()	76
8.7.3.2	Close()	77
8.7.3.3	Exec()	77
8.7.3.4	Get()	78
8.7.3.5	IsOpen()	79
8.7.3.6	Open()	79
8.7.3.7	operator=()	80
8.7.3.8	Prepare()	81
8.7.3.9	Query()	81
8.7.3.10	QueryOne()	82
8.7.3.11	ReadRow()	83
8.7.4	Documentazione dei membri dato	83
8.7.4.1	db_	83
8.8	Riferimenti per la classe DataRow	83
8.8.1	Descrizione dettagliata	84
8.8.2	Documentazione delle funzioni membro	84
8.8.2.1	Get()	84

8.8.2.2 GetOr()	85
8.8.2.3 Has()	86
8.8.2.4 Set()	87
8.8.3 Documentazione dei membri dato	87
8.8.3.1 data_	87
8.9 Riferimenti per la classe game::DecideDrawnCardEffect	88
8.9.1 Descrizione dettagliata	89
8.9.2 Documentazione dei costruttori e dei distruttori	89
8.9.2.1 DecideDrawnCardEffect()	89
8.9.3 Documentazione delle funzioni membro	89
8.9.3.1 GetType()	89
8.9.3.2 Resolve()	90
8.9.4 Documentazione dei membri dato	90
8.9.4.1 card_id_	90
8.9.4.2 username_	91
8.10 Riferimenti per la classe game::DecideSwapTargetEffect	91
8.10.1 Descrizione dettagliata	92
8.10.2 Documentazione dei costruttori e dei distruttori	92
8.10.2.1 DecideSwapTargetEffect()	92
8.10.3 Documentazione delle funzioni membro	93
8.10.3.1 GetType()	93
8.10.3.2 Resolve()	93
8.10.4 Documentazione dei membri dato	94
8.10.4.1 username_	94
8.11 Riferimenti per la classe game::DrawEffect	94
8.11.1 Descrizione dettagliata	95
8.11.2 Documentazione dei costruttori e dei distruttori	95
8.11.2.1 DrawEffect()	95
8.11.3 Documentazione delle funzioni membro	96
8.11.3.1 GetType()	96
8.11.3.2 Resolve()	96
8.11.4 Documentazione dei membri dato	97
8.11.4.1 count_	97
8.11.4.2 target_username_	97
8.12 Riferimenti per la classe game::DrawStackingRule	97
8.12.1 Documentazione delle funzioni membro	98
8.12.1.1 OnCardPlayed()	98
8.12.1.2 ValidatePlay()	99
8.13 Riferimenti per la classe game::Effect	99
8.13.1 Descrizione dettagliata	100
8.13.2 Documentazione dei costruttori e dei distruttori	100
8.13.2.1 ~Effect()	100
8.13.3 Documentazione delle funzioni membro	101

8.13.3.1	GetType()	101
8.13.3.2	Resolve()	101
8.14	Riferimenti per la struct <code>game::EffectResult</code>	101
8.14.1	Descrizione dettagliata	102
8.14.2	Documentazione dei membri dato	102
8.14.2.1	input_context	102
8.14.2.2	input_type	102
8.14.2.3	status	102
8.14.2.4	target_player	102
8.15	Riferimenti per la struct <code>Error</code>	102
8.15.1	Descrizione dettagliata	103
8.15.2	Documentazione dei tipi enumerati (enum)	104
8.15.2.1	Code	104
8.15.3	Documentazione delle funzioni membro	104
8.15.3.1	BadRequest()	104
8.15.3.2	Conflict()	104
8.15.3.3	DatabaseFail()	105
8.15.3.4	HttpStatus()	105
8.15.3.5	Internal()	105
8.15.3.6	InvalidInput()	106
8.15.3.7	NotFound()	106
8.15.3.8	Unauthorised()	107
8.15.4	Documentazione dei membri dato	107
8.15.4.1	code	107
8.15.4.2	message	107
8.16	Riferimenti per la classe <code>game::ForcePlayRule</code>	108
8.16.1	Documentazione delle funzioni membro	109
8.16.1.1	OnCardDrawn()	109
8.17	Riferimenti per la classe <code>GameController</code>	109
8.17.1	Descrizione dettagliata	110
8.17.2	Documentazione dei costruttori e dei distruttori	111
8.17.2.1	GameController()	111
8.17.3	Documentazione delle funzioni membro	111
8.17.3.1	BroadcastGameState()	111
8.17.3.2	ClearTurnTimer()	112
8.17.3.3	HandleCallUno()	113
8.17.3.4	HandleDrawCard()	114
8.17.3.5	HandlePlayCard()	115
8.17.3.6	HandleProvideInput()	116
8.17.3.7	OnTurnStarted()	117
8.17.3.8	SetTurnTimer()	118
8.17.4	Documentazione dei membri dato	119
8.17.4.1	action_router__	119

8.17.4.2	active_turn_timers_	119
8.17.4.3	lobby_controller_	119
8.18	Riferimenti per la classe <code>game::GameRule</code>	119
8.18.1	Descrizione dettagliata	120
8.18.2	Documentazione dei costruttori e dei distruttori	120
8.18.2.1	~GameRule()	120
8.18.3	Documentazione delle funzioni membro	121
8.18.3.1	OnCardDrawn()	121
8.18.3.2	OnCardPlayed()	121
8.18.3.3	ValidatePlay()	121
8.19	Riferimenti per la struct <code>game::GameState</code>	122
8.19.1	Descrizione dettagliata	122
8.19.2	Documentazione dei membri dato	123
8.19.2.1	active_color	123
8.19.2.2	current_player_index	123
8.19.2.3	discard_pile	123
8.19.2.4	draw_pile	123
8.19.2.5	effect_queue	123
8.19.2.6	last_play	123
8.19.2.7	pending_draws	123
8.19.2.8	pending_input_context	124
8.19.2.9	pending_input_type	124
8.19.2.10	pending_player	124
8.19.2.11	play_direction	124
8.19.2.12	players	124
8.19.2.13	provided_input	124
8.19.2.14	status	124
8.19.2.15	turn_end_time	124
8.19.2.16	winner	125
8.20	Riferimenti per la classe <code>HttpRouter</code>	125
8.20.1	Descrizione dettagliata	126
8.20.2	Documentazione dei costruttori e dei distruttori	127
8.20.2.1	HttpRouter()	127
8.20.3	Documentazione delle funzioni membro	127
8.20.3.1	Attach()	127
8.20.3.2	Get()	128
8.20.3.3	OnAny()	129
8.20.3.4	Post()	130
8.20.3.5	WrapWithWildcards()	131
8.20.4	Documentazione dei membri dato	131
8.20.4.1	routes_	131
8.20.4.2	wildcards_	132
8.21	Riferimenti per la classe <code>game::JumpInRule</code>	132

8.21.1 Documentazione delle funzioni membro	133
8.21.1.1 OnCardPlayed()	133
8.21.1.2 ValidatePlay()	133
8.22 Riferimenti per la struct JwtPayload	134
8.22.1 Descrizione dettagliata	134
8.22.2 Documentazione dei membri dato	135
8.22.2.1 username	135
8.23 Riferimenti per la struct game::LastPlay	135
8.23.1 Descrizione dettagliata	135
8.23.2 Documentazione dei membri dato	135
8.23.2.1 card	135
8.23.2.2 hand_index	136
8.23.2.3 player	136
8.23.2.4 valid	136
8.24 Riferimenti per la struct Lobby	136
8.24.1 Descrizione dettagliata	137
8.24.2 Documentazione dei membri dato	137
8.24.2.1 host	137
8.24.2.2 id	137
8.24.2.3 invite_code	137
8.24.2.4 match	137
8.24.2.5 members	138
8.24.2.6 name	138
8.24.2.7 settings	138
8.25 Riferimenti per la classe LobbyController	138
8.25.1 Descrizione dettagliata	140
8.25.2 Documentazione dei costruttori e dei distruttori	141
8.25.2.1 LobbyController()	141
8.25.2.2 ~LobbyController()	142
8.25.3 Documentazione delle funzioni membro	142
8.25.3.1 BroadcastUpdate()	142
8.25.3.2 CheckMatchIntegrity()	143
8.25.3.3 FindLobbyForUser()	144
8.25.3.4 GenerateInviteCode()	144
8.25.3.5 GetLobbyByCode()	145
8.25.3.6 GetLobbyById()	146
8.25.3.7 HandleCreate()	146
8.25.3.8 HandleDeleteSavedMatch()	147
8.25.3.9 HandleGetSavedMatchesList()	148
8.25.3.10 HandleJoin()	149
8.25.3.11 HandleKick()	150
8.25.3.12 HandleLeave()	151
8.25.3.13 HandleList()	152

8.25.3.14	HandlePromote()	153
8.25.3.15	HandleRejoin()	154
8.25.3.16	HandleResumeSavedMatch()	155
8.25.3.17	HandleStartGame()	155
8.25.3.18	HandleUpdateSettings()	156
8.25.3.19	MemberListJson()	157
8.25.3.20	OnClose()	158
8.25.3.21	OnGameStarted()	159
8.25.3.22	OnOpen()	159
8.25.3.23	OnPlayerReplaced()	160
8.25.3.24	RemoveMember()	161
8.25.3.25	SaveMatchStateToDB()	162
8.25.3.26	SyncBots()	163
8.25.4	Documentazione dei membri dato	163
8.25.4.1	action_router__	163
8.25.4.2	app__	163
8.25.4.3	code_to_id__	164
8.25.4.4	eviction_timer__	164
8.25.4.5	game_started_callback__	164
8.25.4.6	kMaxMembers	164
8.25.4.7	kReconnectGraceMs	164
8.25.4.8	lobbies__	164
8.25.4.9	next_id__	165
8.25.4.10	player_replaced_callback__	165
8.26	Riferimenti per la struct LobbyMember	165
8.26.1	Descrizione dettagliata	165
8.27	Riferimenti per la struct LobbySettings	165
8.27.1	Descrizione dettagliata	166
8.28	Riferimenti per la struct Logger	166
8.28.1	Descrizione dettagliata	167
8.28.2	Documentazione delle funzioni membro	167
8.28.2.1	ColorReset()	167
8.28.2.2	ColorWhite()	167
8.28.2.3	Error()	168
8.28.2.4	HTTP()	169
8.28.2.5	Info()	169
8.28.2.6	Init()	170
8.28.2.7	Lobby()	171
8.28.2.8	Log()	172
8.28.2.9	Print()	173
8.28.2.10	timestamp()	174
8.28.2.11	Warn()	175
8.28.2.12	WS()	176

8.28.3 Documentazione dei membri dato	176
8.28.3.1 Blue	176
8.28.3.2 Bold	177
8.28.3.3 Cyan	177
8.28.3.4 Gray	177
8.28.3.5 Green	177
8.28.3.6 is_initialized	177
8.28.3.7 Magenta	177
8.28.3.8 Red	177
8.28.3.9 Reset	177
8.28.3.10 White	177
8.28.3.11 Yellow	177
8.29 Riferimenti per la classe game::MatchInstance	178
8.29.1 Descrizione dettagliata	180
8.29.2 Documentazione dei costruttori e dei distruttori	180
8.29.2.1 MatchInstance() [1/2]	180
8.29.2.2 MatchInstance() [2/2]	181
8.29.3 Documentazione delle funzioni membro	181
8.29.3.1 AddPlayerMidGame()	181
8.29.3.2 CallUno()	182
8.29.3.3 DrawCard()	182
8.29.3.4 ExportState()	183
8.29.3.5 GenerateDeck()	184
8.29.3.6 GetCurrentPlayerUsername()	184
8.29.3.7 GetMatchId()	185
8.29.3.8 GetPendingInputContext()	185
8.29.3.9 GetPendingInputType()	185
8.29.3.10 GetPendingPlayer()	186
8.29.3.11 GetPlayer()	186
8.29.3.12 GetWinner()	187
8.29.3.13 HasMod()	187
8.29.3.14 IsGameOver()	187
8.29.3.15 IsWaitingForInput()	188
8.29.3.16 PlayCard()	188
8.29.3.17 ProvideInput()	189
8.29.3.18 RemovePlayerMidGame()	190
8.29.3.19 SerializePlayerState()	190
8.29.3.20 SetMatchId()	191
8.29.3.21 SetTurnEndTime()	191
8.29.3.22 Start()	191
8.29.3.23 TakeBotTurn()	192
8.29.3.24 Tick()	192
8.29.4 Documentazione dei membri dato	192

8.29.4.1	<code>active_rules__</code>	192
8.29.4.2	<code>match_id__</code>	193
8.29.4.3	<code>session_stats__</code>	193
8.29.4.4	<code>settings__</code>	193
8.29.4.5	<code>state__</code>	193
8.30	Riferimenti per la classe <code>game::NoBluffingRule</code>	193
8.30.1	Documentazione delle funzioni membro	194
8.30.1.1	<code>ValidatePlay()</code>	194
8.31	Riferimenti per la classe <code>game::PassHandsEffect</code>	195
8.31.1	Descrizione dettagliata	196
8.31.2	Documentazione delle funzioni membro	196
8.31.2.1	<code>Resolve()</code>	196
8.32	Riferimenti per la struct <code>PerSocketData</code>	197
8.32.1	Descrizione dettagliata	197
8.32.2	Documentazione dei membri dato	198
8.32.2.1	<code>lobby_code</code>	198
8.32.2.2	<code>room</code>	198
8.32.2.3	<code>username</code>	198
8.33	Riferimenti per la struct <code>game::Player</code>	198
8.33.1	Descrizione dettagliata	199
8.33.2	Documentazione dei costruttori e dei distruttori	199
8.33.2.1	<code>Player()</code> [1/2]	199
8.33.2.2	<code>Player()</code> [2/2]	199
8.33.3	Documentazione dei membri dato	199
8.33.3.1	<code>hand</code>	199
8.33.3.2	<code>has_called_uno</code>	200
8.33.3.3	<code>is_bot</code>	200
8.33.3.4	<code>username</code>	200
8.34	Riferimenti per la struct <code>game::PlayerSessionStats</code>	200
8.34.1	Descrizione dettagliata	200
8.34.2	Documentazione dei membri dato	201
8.34.2.1	<code>color_counts</code>	201
8.34.2.2	<code>value_counts</code>	201
8.35	Riferimenti per la classe <code>game::ProgressiveRule</code>	201
8.35.1	Documentazione delle funzioni membro	202
8.35.1.1	<code>OnCardDrawn()</code>	202
8.36	Riferimenti per la classe <code>game::ReverseEffect</code>	202
8.36.1	Descrizione dettagliata	203
8.36.2	Documentazione delle funzioni membro	203
8.36.2.1	<code>GetType()</code>	203
8.36.2.2	<code>Resolve()</code>	204
8.37	Riferimenti per la struct <code>HttpRouter::Route</code>	204
8.37.1	Descrizione dettagliata	205

8.37.2 Documentazione dei costruttori e dei distruttori	205
8.37.2.1 Route()	205
8.37.3 Documentazione dei membri dato	205
8.37.3.1 handler	205
8.37.3.2 method	205
8.37.3.3 path	205
8.38 Riferimenti per la classe Router	206
8.38.1 Descrizione dettagliata	206
8.38.2 Documentazione dei costruttori e dei distruttori	207
8.38.2.1 ~Router()	207
8.38.2.2 Router() [1/2]	207
8.38.2.3 Router() [2/2]	207
8.38.3 Documentazione delle funzioni membro	208
8.38.3.1 operator=()	208
8.39 Riferimenti per la struct game::RuleRegistrar	208
8.39.1 Descrizione dettagliata	208
8.39.2 Documentazione dei costruttori e dei distruttori	209
8.39.2.1 RuleRegistrar()	209
8.40 Riferimenti per la classe game::RuleRegistry	209
8.40.1 Descrizione dettagliata	210
8.40.2 Documentazione delle funzioni membro	210
8.40.2.1 Create()	210
8.40.2.2 GetMap()	211
8.41 Riferimenti per la classe game::SevenZeroRule	212
8.41.1 Documentazione dei costruttori e dei distruttori	213
8.41.1.1 ~SevenZeroRule()	213
8.41.2 Documentazione delle funzioni membro	213
8.41.2.1 OnCardPlayed()	213
8.41.2.2 ValidatePlay()	214
8.42 Riferimenti per la classe game::SkipEffect	214
8.42.1 Descrizione dettagliata	215
8.42.2 Documentazione delle funzioni membro	215
8.42.2.1 GetType()	215
8.42.2.2 Resolve()	216
8.43 Riferimenti per la classe game::StandardRule	216
8.43.1 Descrizione dettagliata	217
8.43.2 Documentazione dei costruttori e dei distruttori	218
8.43.2.1 ~StandardRule()	218
8.43.3 Documentazione delle funzioni membro	218
8.43.3.1 GetNextPlayer()	218
8.43.3.2 OnCardPlayed()	219
8.43.3.3 ValidatePlay()	219
8.44 Riferimenti per la classe StatsController	220

8.44.1	Descrizione dettagliata	221
8.44.2	Documentazione dei costruttori e dei distruttori	221
8.44.2.1	StatsController()	221
8.44.3	Documentazione delle funzioni membro	222
8.44.3.1	HandleGetLeaderboard()	222
8.44.3.2	HandleGetMe()	222
8.45	Riferimenti per la classe TransactionGuard	223
8.45.1	Descrizione dettagliata	224
8.45.2	Documentazione dei costruttori e dei distruttori	225
8.45.2.1	TransactionGuard() [1/2]	225
8.45.2.2	~TransactionGuard()	225
8.45.2.3	TransactionGuard() [2/2]	226
8.45.3	Documentazione delle funzioni membro	226
8.45.3.1	Commit()	226
8.45.3.2	operator=()	226
8.45.4	Documentazione dei membri dato	227
8.45.4.1	committed_	227
8.45.4.2	db_	227
8.46	Riferimenti per la struct TurnTimerData	227
8.46.1	Documentazione dei membri dato	228
8.46.1.1	callback	228
8.46.1.2	controller	228
8.46.1.3	lobby_id	228
8.47	Riferimenti per la classe WebServer	228
8.47.1	Descrizione dettagliata	230
8.47.2	Documentazione delle ridefinizioni dei tipi (typedef)	230
8.47.2.1	ConnectionHandler	230
8.47.3	Documentazione dei costruttori e dei distruttori	230
8.47.3.1	WebServer() [1/2]	230
8.47.3.2	~WebServer()	231
8.47.3.3	WebServer() [2/2]	232
8.47.4	Documentazione delle funzioni membro	232
8.47.4.1	GetActionRouter()	232
8.47.4.2	GetApp()	233
8.47.4.3	GetHTTPRouter()	233
8.47.4.4	GetMimeType()	234
8.47.4.5	HandleGet()	234
8.47.4.6	HandlePost()	234
8.47.4.7	InitDB()	235
8.47.4.8	OnConnectionClose()	235
8.47.4.9	OnConnectionOpen()	236
8.47.4.10	OnSocketClosed()	236
8.47.4.11	OnSocketMessage()	236

8.47.4.12 OnSocketOpen()	237
8.47.4.13 operator=()	237
8.47.4.14 ReadFile()	237
8.47.4.15 RegisterRoutes()	238
8.47.4.16 Run()	238
8.47.5 Documentazione dei membri dato	238
8.47.5.1 app_	238
8.47.5.2 connections_	239
8.47.5.3 db_file_	239
8.47.5.4 http_router_	239
8.47.5.5 on_close_hooks_	239
8.47.5.6 on_open_hooks_	239
8.47.5.7 port_	239
8.47.5.8 ws_router_	239
8.48 Riferimenti per la struct WsContext	240
8.48.1 Descrizione dettagliata	240
8.48.2 Documentazione dei membri dato	240
8.48.2.1 op_code	240
8.48.2.2 socket	241
8.48.2.3 socket_data	241
9 Documentazione dei file	243
9.1 Riferimenti per il file include/action_router.hpp	243
9.1.1 Descrizione dettagliata	244
9.1.2 Documentazione delle ridefinizioni di tipo (typedef)	244
9.1.2.1 ActionHandler	244
9.1.2.2 json	244
9.2 action_router.hpp	245
9.3 Riferimenti per il file include/common/base64.hpp	245
9.4 base64.hpp	246
9.5 Riferimenti per il file include/common/bot_names.hpp	247
9.5.1 Descrizione dettagliata	248
9.6 bot_names.hpp	249
9.7 Riferimenti per il file include/common/env.hpp	250
9.7.1 Descrizione dettagliata	251
9.8 env.hpp	252
9.9 Riferimenti per il file include/common/game/card_types.hpp	253
9.9.1 Descrizione dettagliata	254
9.10 card_types.hpp	254
9.11 Riferimenti per il file include/common/game/effect.hpp	255
9.11.1 Descrizione dettagliata	256
9.12 effect.hpp	256
9.13 Riferimenti per il file include/common/game/gamerule.hpp	257

9.13.1 Descrizione dettagliata	258
9.14 gamerule.hpp	258
9.15 Riferimenti per il file include/common/http.hpp	258
9.15.1 Descrizione dettagliata	260
9.16 http.hpp	260
9.17 Riferimenti per il file include/common/lobby.hpp	261
9.17.1 Descrizione dettagliata	262
9.17.2 Documentazione dei tipi enumerati	262
9.17.2.1 BotTakeoverMode	262
9.18 lobby.hpp	262
9.19 Riferimenti per il file include/common/ws.hpp	263
9.19.1 Descrizione dettagliata	265
9.20 ws.hpp	265
9.21 Riferimenti per il file include/controllers/auth_controller.hpp	267
9.21.1 Descrizione dettagliata	268
9.22 auth_controller.hpp	269
9.23 Riferimenti per il file include/controllers/game_controller.hpp	269
9.23.1 Descrizione dettagliata	270
9.24 game_controller.hpp	271
9.25 Riferimenti per il file include/controllers/lobby_controller.hpp	271
9.25.1 Descrizione dettagliata	272
9.25.2 Documentazione delle ridefinizioni di tipo (typedef)	272
9.25.2.1 GameStartedCallback	272
9.25.2.2 PlayerReplacedCallback	273
9.26 lobby_controller.hpp	273
9.27 Riferimenti per il file include/controllers/stats_controller.hpp	274
9.27.1 Descrizione dettagliata	275
9.28 stats_controller.hpp	276
9.29 Riferimenti per il file include/database.hpp	276
9.29.1 Descrizione dettagliata	277
9.29.2 Documentazione delle ridefinizioni di tipo (typedef)	277
9.29.2.1 DbValue	277
9.30 database.hpp	278
9.31 Riferimenti per il file include/error.hpp	279
9.31.1 Descrizione dettagliata	280
9.32 error.hpp	280
9.33 Riferimenti per il file include/game/effects/decide_swap_target.hpp	280
9.33.1 Descrizione dettagliata	282
9.34 decide_swap_target.hpp	282
9.35 Riferimenti per il file include/game/effects/pass_hands.hpp	282
9.35.1 Descrizione dettagliata	283
9.36 pass_hands.hpp	283
9.37 Riferimenti per il file include/game/game_state.hpp	284

9.37.1 Descrizione dettagliata	285
9.38 game_state.hpp	285
9.39 Riferimenti per il file include/game/match_instance.hpp	286
9.39.1 Descrizione dettagliata	287
9.40 match_instance.hpp	287
9.41 Riferimenti per il file include/game/rule_registry.hpp	288
9.41.1 Descrizione dettagliata	289
9.42 rule_registry.hpp	289
9.43 Riferimenti per il file include/game/effects/standard.hpp	290
9.43.1 Descrizione dettagliata	291
9.44 standard.hpp	291
9.45 Riferimenti per il file include/game/rules/standard.hpp	292
9.45.1 Descrizione dettagliata	293
9.46 standard.hpp	293
9.47 Riferimenti per il file include/http_router.hpp	293
9.47.1 Descrizione dettagliata	294
9.47.2 Documentazione delle ridefinizioni di tipo (typedef)	295
9.47.2.1 HttpHandler	295
9.48 http_router.hpp	295
9.49 Riferimenti per il file include/logger.hpp	295
9.49.1 Descrizione dettagliata	296
9.50 logger.hpp	296
9.51 Riferimenti per il file include/result.hpp	298
9.51.1 Descrizione dettagliata	299
9.51.2 Documentazione delle ridefinizioni di tipo (typedef)	299
9.51.2.1 Result	299
9.51.2.2 VoidResult	299
9.52 result.hpp	299
9.53 Riferimenti per il file include/router.hpp	300
9.53.1 Descrizione dettagliata	300
9.54 router.hpp	300
9.55 Riferimenti per il file include/webserver.hpp	301
9.55.1 Descrizione dettagliata	301
9.56 webserver.hpp	302
9.57 Riferimenti per il file include/websocket_context.hpp	302
9.57.1 Descrizione dettagliata	303
9.57.2 Documentazione delle ridefinizioni di tipo (typedef)	304
9.57.2.1 AppRequest	304
9.57.2.2 AppResponse	304
9.57.2.3 AppWebSocket	304
9.58 websocket_context.hpp	304
9.59 Riferimenti per il file src/action_router.cpp	305
9.60 Riferimenti per il file src/controllers/auth_controller.cpp	305

9.61	Riferimenti per il file <code>src/controllers/game_controller.cpp</code>	305
9.61.1	Descrizione dettagliata	306
9.62	Riferimenti per il file <code>src/controllers/lobby_controller.cpp</code>	306
9.62.1	Descrizione dettagliata	307
9.62.2	Documentazione delle funzioni	307
9.62.2.1	<code>GetRandomBotName()</code>	307
9.62.3	Documentazione delle variabili	308
9.62.3.1	<code>kAlphabetLen</code>	308
9.62.3.2	<code>kCodeAlphabet</code>	308
9.62.3.3	<code>kCodeLen</code>	308
9.63	Riferimenti per il file <code>src/controllers/stats_controller.cpp</code>	308
9.64	Riferimenti per il file <code>src/database.cpp</code>	309
9.65	Riferimenti per il file <code>src/game/effects/decide_swap_target.cpp</code>	309
9.66	Riferimenti per il file <code>src/game/effects/pass_hands.cpp</code>	310
9.67	Riferimenti per il file <code>src/game/match_instance.cpp</code>	310
9.68	Riferimenti per il file <code>src/game/rules/70.cpp</code>	311
9.69	Riferimenti per il file <code>src/game/rules/draw_stacking.cpp</code>	312
9.70	Riferimenti per il file <code>src/game/rules/force_play.cpp</code>	313
9.71	Riferimenti per il file <code>src/game/rules/jump_in.cpp</code>	313
9.72	Riferimenti per il file <code>src/game/rules/no_bluffing.cpp</code>	314
9.73	Riferimenti per il file <code>src/game/rules/progressive.cpp</code>	315
9.74	Riferimenti per il file <code>src/game/effects/standard.cpp</code>	315
9.75	Riferimenti per il file <code>src/game/rules/standard.cpp</code>	316
9.76	Riferimenti per il file <code>src/http_router.cpp</code>	316
9.77	Riferimenti per il file <code>src/main.cpp</code>	317
9.77.1	Documentazione delle funzioni	317
9.77.1.1	<code>main()</code>	317
9.78	Riferimenti per il file <code>src/webserver.cpp</code>	318
	Indice analitico	319

Capitolo 1

Gerarchia delle directory

1.1 Directory

common	15
game	19
card_types.hpp	253
effect.hpp	255
gamerule.hpp	257
base64.hpp	245
bot_names.hpp	247
env.hpp	250
http.hpp	258
lobby.hpp	261
ws.hpp	263
controllers	16
auth_controller.hpp	267
game_controller.hpp	269
lobby_controller.hpp	271
stats_controller.hpp	274
controllers	17
auth_controller.cpp	305
game_controller.cpp	305
lobby_controller.cpp	306
stats_controller.cpp	308
effects	18
decide_swap_target.hpp	280
pass_hands.hpp	282
standard.hpp	290
effects	19
decide_swap_target.cpp	309
pass_hands.cpp	310
standard.cpp	315
game	19
card_types.hpp	253
effect.hpp	255
gamerule.hpp	257

game	20
effects	18
decide_swap_target.hpp	280
pass_hands.hpp	282
standard.hpp	290
rules	23
standard.hpp	292
game_state.hpp	284
match_instance.hpp	286
rule_registry.hpp	288
game	21
effects	19
decide_swap_target.cpp	309
pass_hands.cpp	310
standard.cpp	315
rules	23
70.cpp	311
draw_stacking.cpp	312
force_play.cpp	313
jump_in.cpp	313
no_bluffing.cpp	314
progressive.cpp	315
standard.cpp	316
match_instance.cpp	310
include	22
common	15
game	19
card_types.hpp	253
effect.hpp	255
gamerule.hpp	257
base64.hpp	245
bot_names.hpp	247
env.hpp	250
http.hpp	258
lobby.hpp	261
ws.hpp	263
controllers	16
auth_controller.hpp	267
game_controller.hpp	269
lobby_controller.hpp	271
stats_controller.hpp	274
game	20
effects	18
decide_swap_target.hpp	280
pass_hands.hpp	282
standard.hpp	290
rules	23
standard.hpp	292
game_state.hpp	284
match_instance.hpp	286
rule_registry.hpp	288
action_router.hpp	243
database.hpp	276
error.hpp	279
http_router.hpp	293
logger.hpp	295

result.hpp	298
router.hpp	300
webserver.hpp	301
websocket_context.hpp	302
rules	23
standard.hpp	292
rules	23
70.cpp	311
draw_stacking.cpp	312
force_play.cpp	313
jump_in.cpp	313
no_bluffing.cpp	314
progressive.cpp	315
standard.cpp	316
src	24
controllers	17
auth_controller.cpp	305
game_controller.cpp	305
lobby_controller.cpp	306
stats_controller.cpp	308
game	21
effects	19
decide_swap_target.cpp	309
pass_hands.cpp	310
standard.cpp	315
rules	23
70.cpp	311
draw_stacking.cpp	312
force_play.cpp	313
jump_in.cpp	313
no_bluffing.cpp	314
progressive.cpp	315
standard.cpp	316
match_instance.cpp	310
action_router.cpp	305
database.cpp	309
http_router.cpp	316
main.cpp	317
webserver.cpp	318

Capitolo 2

Indice dei namespace

2.1 Lista dei namespace

Questa è l'elenco di tutti i namespace con una loro breve descrizione:

Base64	25
Env	26
game	30
http	39
ws	42

Capitolo 3

Indice della gerarchia

3.1 Gerarchia delle classi

Questo elenco di ereditarietà è ordinato approssimativamente, ma non completamente, in ordine alfabetico:

AuthController	58
game::CardDrawnEvent	69
game::CardPlayedEvent	70
Database	74
DbRow	83
game::Effect	99
game::AdvanceTurnEffect	56
game::ChooseColorEffect	71
game::DecideDrawnCardEffect	88
game::DecideSwapTargetEffect	91
game::DrawEffect	94
game::PassHandsEffect	195
game::ReverseEffect	202
game::SkipEffect	214
game::EffectResult	101
Error	102
GameController	109
game::GameRule	119
game::DrawStackingRule	97
game::ForcePlayRule	108
game::JumpInRule	132
game::NoBluffingRule	193
game::ProgressiveRule	201
game::SevenZeroRule	212
game::StandardRule	216
game::GameState	122
JwtPayload	134
game::LastPlay	135
Lobby	136
LobbyController	138
LobbyMember	165
LobbySettings	165
Logger	166

game::MatchInstance	178
PerSocketData	197
game::Player	198
game::PlayerSessionStats	200
HttpRouter::Route	204
Router	206
ActionRouter	51
HttpRouter	125
game::RuleRegistrar	208
game::RuleRegistry	209
StatsController	220
TransactionGuard	223
TurnTimerData	227
WebServer	228
WsContext	240

Capitolo 4

Indice dei tipi composti

4.1 Elenco dei tipi composti

Queste sono le classi, le struct, le union e le interfacce con una loro breve descrizione:

ActionRouter	
Smista i messaggi WebSocket ai vari controller in base al tipo di azione	51
game::AdvanceTurnEffect	
Effetto base che avanza il turno al giocatore successivo. Viene solitamente accodato alla fine di una giocata standard o dopo la risoluzione degli altri effetti	56
AuthController	
Gestisce le rotte HTTP per il login/registrazione e la sicurezza crittografica	58
game::CardDrawnEvent	69
game::CardPlayedEvent	
Struttura che incapsula i dati riguardanti l'evento di una carta messa in gioco	70
game::ChooseColorEffect	
Effetto interattivo che impone la selezione del nuovo colore attivo (Carte Jolly)	71
Database	
Pattern Singleton che incapsula la connessione univoca e condivisa al DB SQLite	74
DbRow	
Rappresenta una singola riga restituita da una query SQL. Memorizza i dati usando le chiavi di colonna come indici della mappa interna	83
game::DecideDrawnCardEffect	
Effetto interattivo che chiede al giocatore se vuole giocare la carta appena pescata	88
game::DecideSwapTargetEffect	
Sospende il gioco per chiedere a un utente con chi desidera scambiare le carte	91
game::DrawEffect	
Effetto penalità/obbligo che fa pescare un numero di carte a un giocatore. Usato per l'inizio del turno, le carte +2, +4, o per penalità da mod	94
game::DrawStackingRule	97
game::Effect	
Classe base astratta per tutti gli effetti applicabili nel gioco	99
game::EffectResult	
Struttura restituita dopo il tentativo di risoluzione di un effetto	101
Error	
Incapsula le informazioni riguardanti un errore verificatosi nel sistema	102
game::ForcePlayRule	108
GameController	
Riceve e processa l'input dei giocatori durante una partita attiva	109

game::GameRule	Interfaccia base che definisce il ciclo di vita per l'interpretazione di una giocata	119
game::GameState	Struttura dati monolitica che contiene l'intera "fotografia" della partita ad un dato istante. Viene manipolata dalle classi Effect e MatchInstance e serializzata in JSON .	122
HttpRouter	Gestisce la registrazione e il routing delle chiamate HTTP (GET, POST e middleware)	125
game::JumpInRule		132
JwtPayload	Rappresenta i dati (payload) estratti da un token JWT verificato con successo	134
game::LastPlay	Descrive l'ultima carta giocata e la sua provenienza, così che il client possa animare la carta dalla specifica posizione (slot) della mano di chi l'ha giocata	135
Lobby	Aggrega l'intero stato strutturale di una stanza di gioco	136
LobbyController	Gestisce lo stato in memoria di tutte le lobby attive	138
LobbyMember	Rappresenta un singolo partecipante all'interno di una lobby	165
LobbySettings	Contiene la configurazione dettagliata e le regole di una specifica lobby. Queste impostazioni sono serializzabili in JSON per essere trasmesse ai client	165
Logger	Fornisce metodi statici per stampare messaggi di log formattati sullo standard output	166
game::MatchInstance	Rappresenta un'istanza attiva di una partita (Match)	178
game::NoBluffingRule		193
game::PassHandsEffect	Effetto che esegue lo scambio delle carte in mano ai giocatori	195
PerSocketData	Dati di stato mantenuti all'interno di ciascuna connessione uWS	197
game::Player	Rappresenta un giocatore reale o Bot all'interno della partita in corso	198
game::PlayerSessionStats	Traccia le statistiche delle carte giocate/pescate in una singola sessione	200
game::ProgressiveRule		201
game::ReverseEffect	Effetto che inverte il senso di rotazione dei turni (Carta Inverti)	202
HttpRouter::Route	Struttura interna che memorizza temporaneamente le rotte prima del caricamento . .	204
Router	Interfaccia base non istanziabile per tutti i router dell'applicazione	206
game::RuleRegistrar	Struttura di supporto per l'auto-registrazione delle regole	208
game::RuleRegistry	Registro globale (Singleton) delle Factory delle regole di gioco	209
game::SevenZeroRule		212
game::SkipEffect	Effetto che salta il turno del giocatore successivo (Carta Divieto)	214
game::StandardRule	Validatore e processore principale delle meccaniche standard del gioco	216
StatsController	Esponde gli endpoint REST necessari al frontend per il pannello Profilo e Leaderboard	220
TransactionGuard	Implementa l'esecuzione sicura di transazioni database tramite il paradigma RAII . .	223
TurnTimerData		227
WebServer	Motore principale dell'applicazione backend	228

[WsContext](#)

Contesto di esecuzione passato ad ogni gestore del [ActionRouter](#) 240

Capitolo 5

Indice dei file

5.1 Elenco dei file

Questo è un elenco di tutti i file con una loro breve descrizione:

include/ action_router.hpp	Implementazione del router per lo smistamento dei messaggi WebSocket	243
include/ database.hpp	Astrazione leggera e sicura (Type-Safe) attorno alla libreria SQLite3	276
include/ error.hpp	Definizione degli errori standard di sistema e protocollo	279
include/ http_router.hpp	Implementazione del router per le richieste HTTP/REST	293
include/ logger.hpp	Sistema di logging thread-safe e formattato a colori per la console	295
include/ result.hpp	Definisce alias per la gestione funzionale degli errori tramite std::expected	298
include/ router.hpp	Classe base per l'implementazione dei router (HTTP e WebSocket)	300
include/ webserver.hpp	Definizione della classe principale WebServer che gestisce l'intero ciclo di vita dell'applicazione di rete	301
include/ websocket_context.hpp	Definisce i tipi e le strutture di contesto associati alle connessioni WebSocket	302
include/common/ base64.hpp		245
include/common/ bot_names.hpp	Raccoglitore di costanti per l'inizializzazione del motore AI (Bot)	247
include/common/ env.hpp	Gestione delle variabili d'ambiente e caricamento dal file di configurazione locale .env	250
include/common/ http.hpp	Funzioni di utilità per la gestione del protocollo HTTP	258
include/common/ lobby.hpp	Definisce le strutture dati base per la rappresentazione delle lobby	261
include/common/ ws.hpp	Definizione del protocollo di messaggistica WebSocket bidirezionale	263
include/common/game/ card_types.hpp	Definisce le strutture dati fondamentali e i tipi per la rappresentazione delle carte nel gioco	253
include/common/game/ effect.hpp	Sistema di base per la definizione e la risoluzione degli effetti di gioco	255

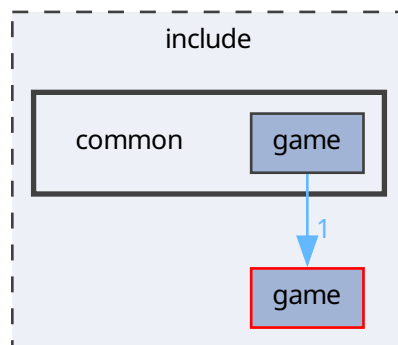
include/common/game/ gamerule.hpp	Definisce il motore base per l'applicazione delle regole del gioco	257
include/controllers/ auth_controller.hpp	Controller per l'autenticazione, registrazione e gestione dei token JWT	267
include/controllers/ game_controller.hpp	Controller per la gestione degli eventi in-game e del flusso dei turni	269
include/controllers/ lobby_controller.hpp	Definizione del controller per la gestione delle Lobby . Gestisce l'intero I/O tra Client e Server per quanto riguarda l'inizializzazione di Lobby , e l'inizio di Partite di Gioco, a cui successivamente passa il controllo	271
include/controllers/ stats_controller.hpp	Controller HTTP per la gestione delle statistiche utenti e classifiche	274
include/game/ game_state.hpp	Strutture dati fondamentali per la rappresentazione dello stato del gioco "UNO" . . .	284
include/game/ match_instance.hpp	Definizione della classe principale che gestisce l'intero ciclo di vita di una partita di gioco: una state machine che applica gli effetti di ogni carta giocata in base a delle regole che ne definiscono le varie proprietà	286
include/game/ rule_registry.hpp	Gestione della registrazione e istanziazione dinamica delle regole di gioco	288
include/game/effects/ decide_swap_target.hpp	Effetto interattivo per la scelta del bersaglio per lo scambio delle carte	280
include/game/effects/ pass_hands.hpp	Implementazione dell'effetto di rotazione o scambio delle mani di carte	282
include/game/effects/ standard.hpp	Implementazione degli effetti standard associati alle carte di base	290
include/game/rules/ standard.hpp	Implementazione del set di regole classiche di base del gioco UNO	292
src/ action_router.cpp		305
src/ database.cpp		309
src/ http_router.cpp		316
src/ main.cpp		317
src/ webserver.cpp		318
src/controllers/ auth_controller.cpp		305
src/controllers/ game_controller.cpp	Implementation of the GameController routing active transaction frames to running match engines	305
src/controllers/ lobby_controller.cpp	Implementation of the LobbyController class managing websocket signaling and session lifecycles for game rooms	306
src/controllers/ stats_controller.cpp		308
src/game/ match_instance.cpp		310
src/game/effects/ decide_swap_target.cpp		309
src/game/effects/ pass_hands.cpp		310
src/game/effects/ standard.cpp		315
src/game/rules/ 70.cpp		311
src/game/rules/ draw_stacking.cpp		312
src/game/rules/ force_play.cpp		313
src/game/rules/ jump_in.cpp		313
src/game/rules/ no_bluffing.cpp		314
src/game/rules/ progressive.cpp		315
src/game/rules/ standard.cpp		316

Capitolo 6

Documentazione delle directory

6.1 Riferimenti per la directory include/common

Grafo di dipendenza delle directory per common:



Directory

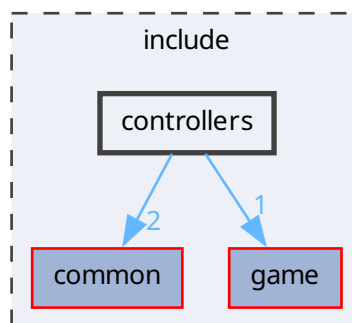
- directory [game](#)

File

- file [base64.hpp](#)
- file [bot_names.hpp](#)
Raccoglitore di costanti per l'inizializzazione del motore AI (Bot).
- file [env.hpp](#)
Gestione delle variabili d'ambiente e caricamento dal file di configurazione locale .env.
- file [http.hpp](#)
Funzioni di utilità per la gestione del protocollo HTTP.
- file [lobby.hpp](#)
Definisce le strutture dati base per la rappresentazione delle lobby.
- file [ws.hpp](#)
Definizione del protocollo di messaggistica WebSocket bidirezionale.

6.2 Riferimenti per la directory include/controllers

Grafo di dipendenza delle directory per controllers:

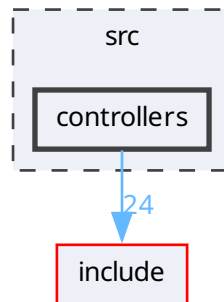


File

- file [auth_controller.hpp](#)
Controller per l'autenticazione, registrazione e gestione dei token JWT.
- file [game_controller.hpp](#)
Controller per la gestione degli eventi in-game e del flusso dei turni.
- file [lobby_controller.hpp](#)
Definizione del controller per la gestione delle [Lobby](#). Gestisce l'intero I/O tra Client e Server per quanto riguarda l'inizializzazione di [Lobby](#), e l'inizio di Partite di Gioco, a cui successivamente passa il controllo.
- file [stats_controller.hpp](#)
Controller HTTP per la gestione delle statistiche utenti e classifiche.

6.3 Riferimenti per la directory src/controllers

Grafo di dipendenza delle directory per controllers:

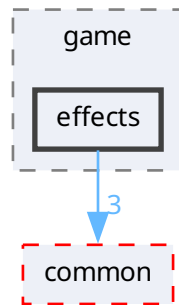


File

- file [auth_controller.cpp](#)
- file [game_controller.cpp](#)
Implementation of the [GameController](#) routing active transaction frames to running match engines.
- file [lobby_controller.cpp](#)
Implementation of the [LobbyController](#) class managing websocket signaling and session lifecycles for game rooms.
- file [stats_controller.cpp](#)

6.4 Riferimenti per la directory `include/game/effects`

Grafo di dipendenza delle directory per effects:



File

- file [decide_swap_target.hpp](#)
Effetto interattivo per la scelta del bersaglio per lo scambio delle carte.
- file [pass_hands.hpp](#)
Implementazione dell'effetto di rotazione o scambio delle mani di carte.
- file [standard.hpp](#)
Implementazione degli effetti standard associati alle carte di base.

6.5 Riferimenti per la directory src/game/effects

Grafo di dipendenza delle directory per effects:

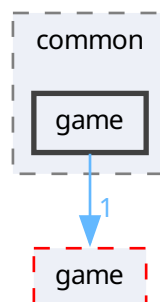


File

- file [decide_swap_target.cpp](#)
- file [pass_hands.cpp](#)
- file [standard.cpp](#)

6.6 Riferimenti per la directory include/common/game

Grafo di dipendenza delle directory per game:

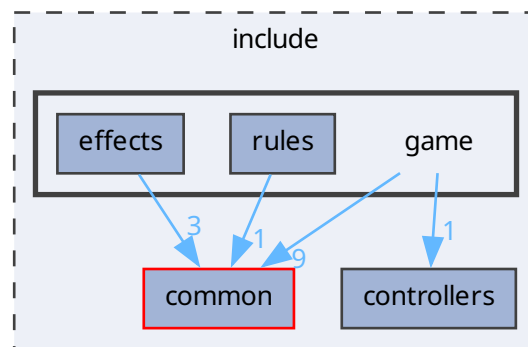


File

- file [card_types.hpp](#)
Definisce le strutture dati fondamentali e i tipi per la rappresentazione delle carte nel gioco.
- file [effect.hpp](#)
Sistema di base per la definizione e la risoluzione degli effetti di gioco.
- file [gamerule.hpp](#)
Definisce il motore base per l'applicazione delle regole del gioco.

6.7 Riferimenti per la directory include/game

Grafo di dipendenza delle directory per game:



Directory

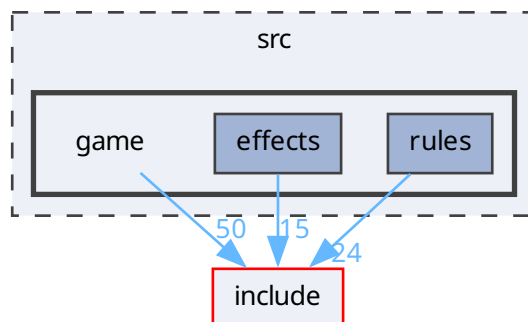
- directory [effects](#)
- directory [rules](#)

File

- file [game_state.hpp](#)
Strutture dati fondamentali per la rappresentazione dello stato del gioco "UNO".
- file [match_instance.hpp](#)
Definizione della classe principale che gestisce l'intero ciclo di vita di una partita di gioco: una state machine che applica gli effetti di ogni carta giocata in base a delle regole che ne definiscono le varie proprietà.
- file [rule_registry.hpp](#)
Gestione della registrazione e istanziazione dinamica delle regole di gioco.

6.8 Riferimenti per la directory src/game

Grafo di dipendenza delle directory per game:



Directory

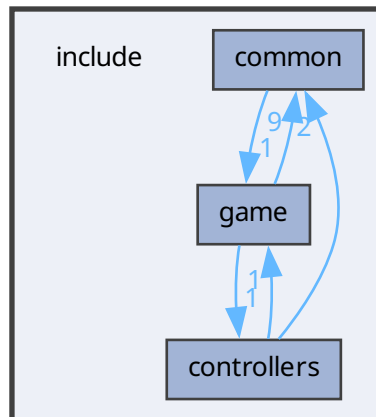
- directory [effects](#)
- directory [rules](#)

File

- file [match_instance.cpp](#)

6.9 Riferimenti per la directory include

Grafo di dipendenza delle directory per include:



Directory

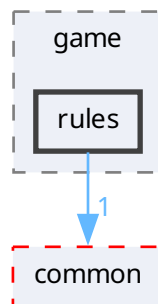
- directory [common](#)
- directory [controllers](#)
- directory [game](#)

File

- file [action_router.hpp](#)
Implementazione del router per lo smistamento dei messaggi WebSocket.
- file [database.hpp](#)
Astrazione leggera e sicura (Type-Safe) attorno alla libreria SQLite3.
- file [error.hpp](#)
Definizione degli errori standard di sistema e protocollo.
- file [http_router.hpp](#)
Implementazione del router per le richieste HTTP/REST.
- file [logger.hpp](#)
Sistema di logging thread-safe e formattato a colori per la console.
- file [result.hpp](#)
Definisce alias per la gestione funzionale degli errori tramite std::expected.
- file [router.hpp](#)
Classe base per l'implementazione dei router (HTTP e WebSocket).
- file [webserver.hpp](#)
Definizione della classe principale [WebServer](#) che gestisce l'intero ciclo di vita dell'applicazione di rete.
- file [websocket_context.hpp](#)
Definisce i tipi e le strutture di contesto associati alle connessioni WebSocket.

6.10 Riferimenti per la directory include/game/rules

Grafo di dipendenza delle directory per rules:

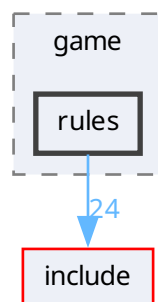


File

- file [standard.hpp](#)
Implementazione del set di regole classiche di base del gioco UNO.

6.11 Riferimenti per la directory src/game/rules

Grafo di dipendenza delle directory per rules:

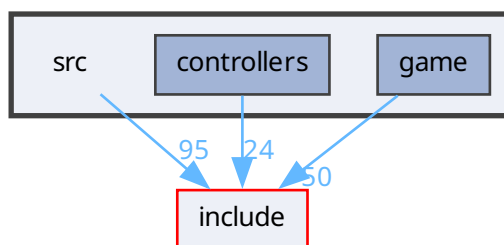


File

- file [70.cpp](#)
- file [draw_stacking.cpp](#)
- file [force_play.cpp](#)
- file [jump_in.cpp](#)
- file [no_bluffing.cpp](#)
- file [progressive.cpp](#)
- file [standard.cpp](#)

6.12 Riferimenti per la directory src

Grafo di dipendenza delle directory per src:



Directory

- directory [controllers](#)
- directory [game](#)

File

- file [action_router.cpp](#)
- file [database.cpp](#)
- file [http_router.cpp](#)
- file [main.cpp](#)
- file [webserver.cpp](#)

Capitolo 7

Documentazione dei namespace

7.1 Riferimenti per il namespace Base64

Funzioni

- `std::string` [Encode](#) (`const std::vector< uint8_t > &data`)
- `std::vector< uint8_t >` [Decode](#) (`const std::string &encoded`)

Variabili

- `constexpr char` [kAlphabet](#) [] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789"
- `constexpr uint8_t` [kInvalid](#) = 0xFF
- `constexpr std::array< uint8_t, 256 >` [kDecodeTable](#)

7.1.1 Documentazione delle funzioni

7.1.1.1 Decode()

```
std::vector< uint8_t > Base64::Decode (  
    const std::string & encoded) [inline]
```

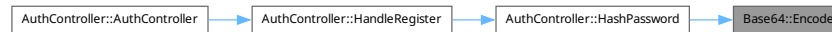
Questo è il grafo dei chiamanti di questa funzione:



7.1.1.2 Encode()

```
std::string Base64::Encode (
    const std::vector< uint8_t > & data) [inline]
```

Questo è il grafo dei chiamanti di questa funzione:



7.1.2 Documentazione delle variabili

7.1.2.1 kAlphabet

```
char Base64::kAlphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/" [inline], [constexpr]
```

7.1.2.2 kDecodeTable

```
std::array<uint8_t, 256> Base64::kDecodeTable [inline], [constexpr]
```

Valore iniziale:

```

= []() {
    std::array<uint8_t, 256> table{};
    table.fill(kInvalid);

    for (uint8_t i = 0; i < 64; ++i) {
        table[static_cast<uint8_t>(kAlphabet[i])] = i;
    }

    return table;
}()
```

7.1.2.3 kInvalid

```
uint8_t Base64::kInvalid = 0xFF [inline], [constexpr]
```

7.2 Riferimenti per il namespace Env

Funzioni

- std::string [Require](#) (const std::string &key)
Recupera il valore di una variabile d'ambiente obbligatoria.
- std::string [Get](#) (const std::string &key, const std::string &fallback="")
Recupera il valore di una variabile d'ambiente, fornendo un fallback opzionale.
- void [SetEnv](#) (const std::string &key, const std::string &value)
Funzione interna helper per impostare una variabile d'ambiente in modo cross-platform.
- std::string [Trim](#) (const std::string &s)
Funzione interna helper per rimuovere gli spazi bianchi ASCII all'inizio e alla fine di una stringa. Rimuove i caratteri ' ', '\t', '\r', '\n'. Previene comportamenti indefiniti (UB) restituendo una stringa vuota qualora la stringa d'ingresso sia composta interamente da spazi.
- int [Load](#) (std::string_view path=".env")
Legge il file riga per riga e inietta ogni coppia CHIAVE=VALORE nell'ambiente di processo.

7.2.1 Documentazione delle funzioni

7.2.1.1 Get()

```
std::string Env::Get (
    const std::string & key,
    const std::string & fallback = "") [inline]
```

Recupera il valore di una variabile d'ambiente, fornendo un fallback opzionale.

- Sicuro da chiamare anche prima dell'esecuzione di [Load\(\)](#), poiché interagisce direttamente con le variabili di sistema fornite dalla shell o dal SO.

- Parametri

key	Il nome della variabile d'ambiente da leggere.
fallback	Il valore di default da restituire se la variabile non esiste.

Restituisce

std::string Il valore estratto o la stringa di fallback.

Tag Identificativo: CMN-ENV-MTH-002

Questo è il grafo dei chiamanti di questa funzione:



7.2.1.2 Load()

```
int Env::Load (
    std::string_view path = ".env") [inline]
```

Legge il file riga per riga e inietta ogni coppia CHIAVE=VALORE nell'ambiente di processo.

- Non genera errori fatali se il file non esiste: gli ambienti di produzione moderni (Docker, systemd, CI) iniettano spesso le variabili d'ambiente in modo diretto senza creare alcun file .env sul disco.

- Parametri

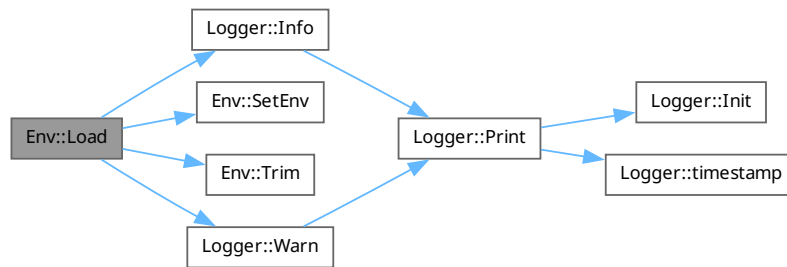
path	Il percorso del file da leggere (default: ".env").
------	--

Restituisce

int Il numero esatto di variabili d'ambiente lette ed iniettate con successo.

Tag Identificativo: CMN-ENV-MTH-005

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.2.1.3 Require()

```
std::string Env::Require (
    const std::string & key) [inline]
```

Recupera il valore di una variabile d'ambiente obbligatoria.

- Da utilizzare per valori fondamentali senza i quali il server non può funzionare correttamente (es. `JWT_SECRET`). Fallire rapidamente (fail-fast) all'avvio è preferibile rispetto a un errore criptico generato in profondità nello stack di chiamate.

- Parametri

key	Il nome della variabile d'ambiente da leggere.
-----	--

Restituisce

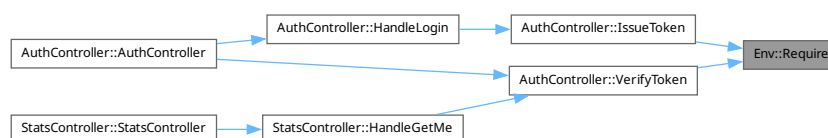
`std::string` Il valore estratto dall'ambiente operativo.

Eccezioni

<code>std::runtime_error</code>	Se la variabile richiesta non è definita nell'ambiente.
---------------------------------	---

Tag Identificativo: CMN-ENV-MTH-001

Questo è il grafo dei chiamanti di questa funzione:



7.2.1.4 SetEnv()

```
void Env::SetEnv (
    const std::string & key,
    const std::string & value) [inline]
```

Funzione interna helper per impostare una variabile d'ambiente in modo cross-platform.

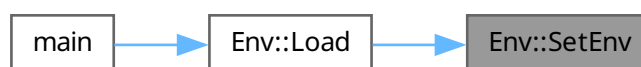
- POSIX: Utilizza `setenv(key, value, 1)` dove '1' forza la sovrascrittura se esiste già. Windows: Utilizza `_putenv_s` che sovrascrive per comportamento predefinito.

- Parametri

key	Il nome della variabile da impostare.
value	Il valore da assegnare alla variabile.

Tag Identificativo: CMN-ENV-MTH-003

Questo è il grafo dei chiamanti di questa funzione:



7.2.1.5 Trim()

```
std::string Env::Trim (
    const std::string & s)  [inline]
```

Funzione interna helper per rimuovere gli spazi bianchi ASCII all'inizio e alla fine di una stringa. Rimuove i caratteri ' ', '\t', '\r', '\n'. Previene comportamenti indefiniti (UB) restituendo una stringa vuota qualora la stringa d'ingresso sia composta interamente da spazi.

- Parametri

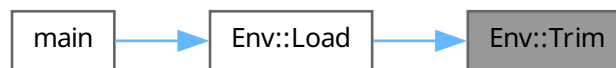
s	La stringa da ripulire.
---	-------------------------

Restituisce

std::string La stringa ripulita ("trimmata").

Tag Identificativo: CMN-ENV-MTH-004

Questo è il grafo dei chiamanti di questa funzione:



7.3 Riferimenti per il namespace game

Composti

- struct [EffectResult](#)
Struttura restituita dopo il tentativo di risoluzione di un effetto.
- class [Effect](#)
Classe base astratta per tutti gli effetti applicabili nel gioco.
- struct [CardPlayedEvent](#)
Struttura che incapsula i dati riguardanti l'evento di una carta messa in gioco.
- struct [CardDrawnEvent](#)
- class [GameRule](#)
Interfaccia base che definisce il ciclo di vita per l'interpretazione di una giocata.
- class [DecideSwapTargetEffect](#)
Sospende il gioco per chiedere a un utente con chi desidera scambiare le carte.
- class [PassHandsEffect](#)

- Effetto che esegue lo scambio delle carte in mano ai giocatori.
- class [AdvanceTurnEffect](#)
 - Effetto base che avanza il turno al giocatore successivo. Viene solitamente accodato alla fine di una giocata standard o dopo la risoluzione degli altri effetti.
- class [DecideDrawnCardEffect](#)
 - Effetto interattivo che chiede al giocatore se vuole giocare la carta appena pescata.
- class [DrawEffect](#)
 - Effetto penalità/obbligo che fa pescare un numero di carte a un giocatore. Usato per l'inizio del turno, le carte +2, +4, o per penalità da mod.
- class [SkipEffect](#)
 - Effetto che salta il turno del giocatore successivo (Carta Divieto).
- class [ReverseEffect](#)
 - Effetto che inverte il senso di rotazione dei turni (Carta Inverti).
- class [ChooseColorEffect](#)
 - Effetto interattivo che impone la selezione del nuovo colore attivo (Carte Jolly).
- struct [PlayerSessionStats](#)
 - Traccia le statistiche delle carte giocate/pescate in una singola sessione.
- struct [Player](#)
 - Rappresenta un giocatore reale o Bot all'interno della partita in corso.
- struct [LastPlay](#)
 - Descrive l'ultima carta giocata e la sua provenienza, così che il client possa animare la carta dalla specifica posizione (slot) della mano di chi l'ha giocata.
- struct [GameState](#)
 - Struttura dati monolitica che contiene l'intera "fotografia" della partita ad un dato istante. Viene manipolata dalle classi [Effect](#) e [MatchInstance](#) e serializzata in JSON.
- class [MatchInstance](#)
 - Rappresenta un'istanza attiva di una partita (Match).
- class [RuleRegistry](#)
 - Registro globale (Singleton) delle Factory delle regole di gioco.
- struct [RuleRegistrar](#)
 - Struttura di supporto per l'auto-registrazione delle regole.
- class [StandardRule](#)
 - Validatore e processore principale delle meccaniche standard del gioco.
- class [SevenZeroRule](#)
- class [DrawStackingRule](#)
- class [ForcePlayRule](#)
- class [JumpInRule](#)
- class [NoBluffingRule](#)
- class [ProgressiveRule](#)

Ridefinizioni di tipo (typedef)

- using [CompactCard](#) = uint32_t
 - Definizione del tipo [CompactCard](#). Impacchetta le informazioni di una carta all'interno di un intero a 32 bit. Formato binario: cccccccvvvvvvvvvvuuuuuuuuuuuuuuuuuu.
- using [RuleFactory](#) = std::function<std::unique_ptr<[GameRule](#)>()>
 - Definisce il tipo di funzione capace di generare una nuova istanza di una regola.

Tipi enumerati (enum)

- enum class `Color` : `uint8_t` {
`kRed` = 0 , `kBlue` = 1 , `kGreen` = 2 , `kYellow` = 3 ,
`kWild` = 4 }
 Enumera i possibili colori che una carta può assumere.
- enum class `Value` : `uint8_t` {
`k0` = 0 , `k1` , `k2` , `k3` ,
`k4` , `k5` , `k6` , `k7` ,
`k8` , `k9` , `kSkip` = 10 , `kReverse` = 11 ,
`kDraw2` = 12 , `kWild` = 13 , `kWildDraw4` = 14 }
 Enumera i possibili valori o azioni associati a una carta.
- enum class `EffectStatus` { `kResolved` , `kNeedsInput` , `kError` }
 Enumera i possibili esiti della risoluzione di un effetto.
- enum class `EffectType` {
`kAdvanceTurn` , `kDraw` , `kSkip` , `kPlayCard` ,
`kDecideDrawnCard` , `kChooseColor` , `kReverse` , `kDecideSwapTarget` ,
`kPassHands` , `kCustom` }
 Codici per l'identificazione a runtime del tipo di effetto in corso.
- enum class `MatchStatus` { `kWaitingToStart` , `kPlaying` , `kFinished` }
 Fasi del ciclo di vita globale di una partita.

Funzioni

- `Color GetColor (CompactCard c)`
 Estrae il colore da una carta compatta (Primi 8 bit).
- `Value GetValue (CompactCard c)`
 Estrae il valore o l'azione da una carta compatta (Secondi 8 bit).
- `uint16_t GetId (CompactCard c)`
 Estrae l'identificativo univoco a 16-bit della carta (Ultimi 16 bit).
- `CompactCard MakeCard (Color c, Value v, uint16_t id)`
 Crea una nuova `CompactCard` assemblando colore, valore e ID.
- `void ReshuffleDiscardIntoDraw (GameState *game_state)`
 Utilità di sistema che sposta e rimescola le carte dagli scarti al mazzo principale. Viene invocata automaticamente dal motore di gioco quando `draw_pile` si esaurisce.

Variabili

- `const std::vector< std::string > kReservedBotNames`
 Lista globale statica contenente nomi predefiniti per i bot guidati dalla CPU. Contiene references alla cultura pop, sci-fi e nomi casuali. Usata durante il subentro automatico dei bot o l'inizializzazione della lobby.
- static `RuleRegistrar registrar ("seven_zero", []) { return std::make_unique< SevenZeroRule >();}`
- static `RuleRegistrar registrar ("draw_stacking", []) { return std::make_unique< DrawStackingRule >();}`
- static `RuleRegistrar registrar ("force_play", []) { return std::make_unique< ForcePlayRule >();}`
- static `RuleRegistrar registrar ("jump_in", []) { return std::make_unique< JumpInRule >();}`
- static `RuleRegistrar registrar ("no_bluffing", []) { return std::make_unique< NoBluffingRule >();}`
- static `RuleRegistrar registrar ("progressive", []) { return std::make_unique< ProgressiveRule >();}`

7.3.1 Documentazione delle ridefinizioni di tipo (typedef)

7.3.1.1 CompactCard

```
using game::CompactCard = uint32_t
```

Definizione del tipo `CompactCard`. Impacchetta le informazioni di una carta all'interno di un intero a 32 bit. Formato binario: cccccccv vvvvvv uuuuuuuuuuuuuuuu.

- c (8 bit): Colore
- v (8 bit): Valore
- u (16 bit): ID univoco della carta

7.3.1.2 RuleFactory

```
using game::RuleFactory = std::function<std::unique_ptr<GameRule>()>
```

Definisce il tipo di funzione capace di generare una nuova istanza di una regola.

Tag Identificativo: RULE-REG-TYP-001

7.3.2 Documentazione dei tipi enumerati

7.3.2.1 Color

```
enum class game::Color : uint8_t [strong]
```

Enumera i possibili colori che una carta può assumere.

Valori del tipo enumerato

kRed	Colore Rosso
kBlue	Colore Blu
kGreen	Colore Verde
kYellow	Colore Giallo
kWild	Colore Jolly (Nessun colore base)

7.3.2.2 EffectStatus

enum class `game::EffectStatus` [strong]

Enumera i possibili esiti della risoluzione di un effetto.

Valori del tipo enumerato

kResolved	L'effetto è stato applicato con successo e completato.
kNeedsInput	L'effetto è sospeso in attesa di un input da un giocatore (es. scelta colore).
kError	Si è verificato un errore durante la risoluzione.

7.3.2.3 EffectType

enum class `game::EffectType` [strong]

Codici per l'identificazione a runtime del tipo di effetto in corso.

Valori del tipo enumerato

kAdvanceTurn	Passaggio del turno al giocatore successivo
kDraw	Obbligo di pescare carte
kSkip	Salto del turno
kPlayCard	Giocata di una carta
kDecideDrawnCard	Decisione su una carta appena pescata (giocare o tenere)
kChooseColor	Scelta di un nuovo colore attivo (es. dopo un Jolly)
kReverse	Inversione dell'ordine di gioco
kDecideSwapTarget	Scelta del target con cui scambiare le mani
kPassHands	Effetto per il passaggio o scambio delle mani tra i giocatori
kCustom	Effetto personalizzato (per mod o estensioni)

7.3.2.4 MatchStatus

enum class `game::MatchStatus` [strong]

Fasi del ciclo di vita globale di una partita.

Tag Identificativo: GAME-ENUM-001

Valori del tipo enumerato

kWaitingToStart	La partita sta inizializzando i deck o è in pre-game.
-----------------	---

kPlaying	Partita attivamente in corso (i turni scorrono).
kFinished	Partita terminata. C'è un vincitore stabilito.

7.3.2.5 Value

```
enum class game::Value : uint8_t [strong]
```

Enumera i possibili valori o azioni associati a una carta.

Valori del tipo enumerato

k0	
k1	
k2	
k3	
k4	
k5	
k6	
k7	
k8	
k9	Valori numerici standard da 0 a 9
kSkip	Carta Salta Turno
kReverse	Carta Inverti Giro
kDraw2	Carta Pesca Due
kWild	Carta Jolly Cambia Colore
kWildDraw4	Carta Jolly Pesca Quattro

7.3.3 Documentazione delle funzioni

7.3.3.1 GetColor()

```
Color game::GetColor (  
    CompactCard c) [inline]
```

Estrae il colore da una carta compatta (Primi 8 bit).

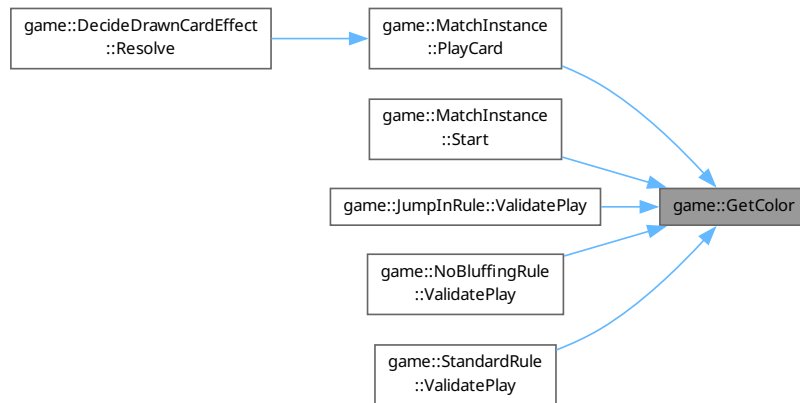
Parametri

c	La carta compatta da analizzare.
---	----------------------------------

Restituisce

[Color](#) Il colore decodificato dalla carta.

Questo è il grafo dei chiamanti di questa funzione:



7.3.3.2 GetId()

```
uint16_t game::GetId (
    CompactCard c) [inline]
```

Estrae l'identificativo univoco a 16-bit della carta (Ultimi 16 bit).

Parametri

c	La carta compatta da analizzare.
---	----------------------------------

Restituisce

uint16_t L'ID univoco.

Questo è il grafo dei chiamanti di questa funzione:



7.3.3.3 GetValue()

```
Value game::GetValue (
    CompactCard c) [inline]
```

Estrae il valore o l'azione da una carta compatta (Secondi 8 bit).

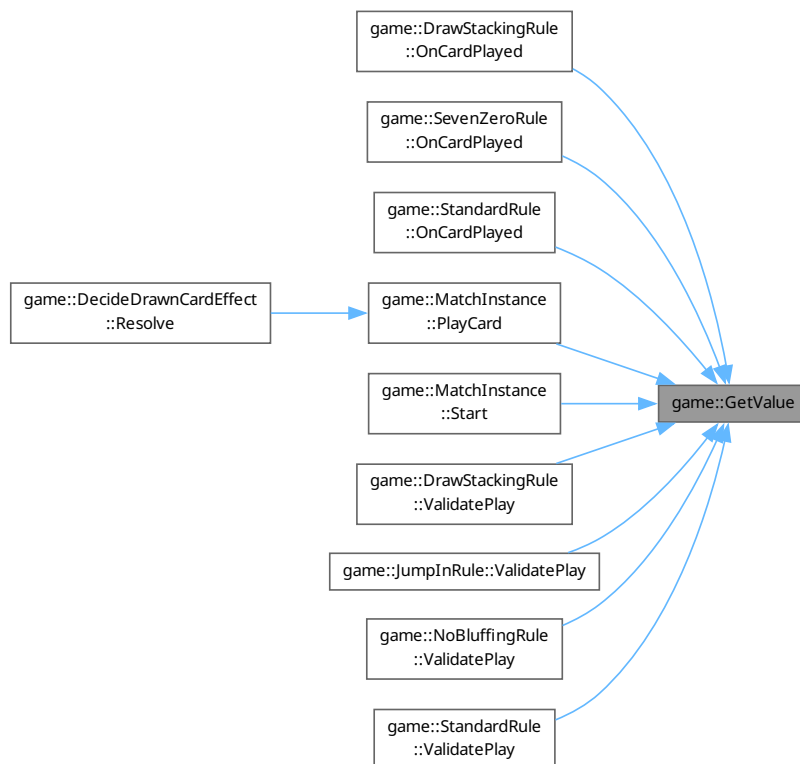
Parametri

c	La carta compatta da analizzare.
---	----------------------------------

Restituisce

Value Il valore decodificato dalla carta.

Questo è il grafo dei chiamanti di questa funzione:



7.3.3.4 MakeCard()

```
CompactCard game::MakeCard (
    Color c,
    Value v,
    uint16_t id) [inline]
```

Crea una nuova [CompactCard](#) assemblando colore, valore e ID.

Parametri

c	Il colore della carta.
v	Il valore della carta.
id	L'identificativo univoco.

Restituisce

[CompactCard](#) La carta impacchettata in un intero a 32-bit.

7.3.3.5 ReshuffleDiscardIntoDraw()

```
void game::ReshuffleDiscardIntoDraw (
    GameState * game_state)
```

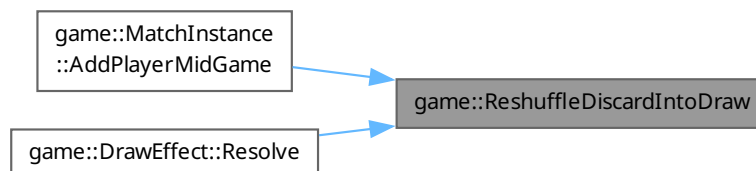
Utilità di sistema che sposta e rimescola le carte dagli scarti al mazzo principale. Viene invocata automaticamente dal motore di gioco quando draw_pile si esaurisce.

Parametri

game_state	Puntatore allo stato di gioco da manipolare.
------------	--

Tag Identificativo: GAME-UTIL-001

Questo è il grafo dei chiamanti di questa funzione:



7.3.4 Documentazione delle variabili

7.3.4.1 kReservedBotNames

```
const std::vector<std::string> game::kReservedBotNames [inline]
```

Lista globale statica contenente nomi predefiniti per i bot guidati dalla CPU. Contiene references alla cultura pop, sci-fi e nomi casuali. Usata durante il subentro automatico dei bot o l'inizializzazione della lobby.

Tag Identificativo: CMN-BOT-001

7.3.4.2 registrar [1/6]

```
RuleRegistrar game::registrar("draw_stacking",[]() { return std::make_unique< DrawStackingRule >();}) (
    "draw_stacking" ,
    [] () { return std::make_unique< DrawStackingRule >();} ) [static]
```

7.3.4.3 registrar [2/6]

```
RuleRegistrar game::registrar("force_play",[]() { return std::make_unique< ForcePlayRule >();}) (
    "force_play" ,
    [] () { return std::make_unique< ForcePlayRule >();} ) [static]
```

7.3.4.4 registrar [3/6]

```
RuleRegistrar game::registrar("jump_in",[]() { return std::make_unique< JumpInRule >();}) (
    "jump_in" ,
    [] () { return std::make_unique< JumpInRule >();} ) [static]
```

7.3.4.5 registrar [4/6]

```
RuleRegistrar game::registrar("no_bluffing",[]() { return std::make_unique< NoBluffingRule >();}) (
    "no_bluffing" ,
    [] () { return std::make_unique< NoBluffingRule >();} ) [static]
```

7.3.4.6 registrar [5/6]

```
RuleRegistrar game::registrar("progressive",[]() { return std::make_unique< ProgressiveRule >();}) (
    "progressive" ,
    [] () { return std::make_unique< ProgressiveRule >();} ) [static]
```

7.3.4.7 registrar [6/6]

```
RuleRegistrar game::registrar("seven_zero",[]() { return std::make_unique< SevenZeroRule >();}) (
    "seven_zero" ,
    [] () { return std::make_unique< SevenZeroRule >();} ) [static]
```

7.4 Riferimenti per il namespace http

Funzioni

- void [ReadBody](#) ([AppResponse](#) *res, size_t max_bytes, std::function< void(const std::string &)> callback)
Accumula asincronamente un intero body HTTP frammentato (chunked) da uWebSockets.
- constexpr std::string_view [TrimWhitespace](#) (std::string_view str)
Rimuove gli spazi bianchi ASCII (spazi, tabulazioni, a capo) all'inizio e alla fine di una stringa. Essendo constexpr, se valutata a tempo di compilazione non genera overhead.
- std::optional< std::string > [GetCookieValue](#) (std::string_view cookie_header, std::string_view target_key)
Estrae in modo sicuro il valore di un singolo cookie dalla stringa raw dell'header.

7.4.1 Documentazione delle funzioni

7.4.1.1 GetCookieValue()

```
std::optional< std::string > http::GetCookieValue (
    std::string_view cookie_header,
    std::string_view target_key) [inline]
```

Estrae in modo sicuro il valore di un singolo cookie dalla stringa raw dell'header.

- Scansiona iterativamente la stringa dell'header Cookie (es. session=123; user=abc), separando le coppie al punto e virgola e cercando la corrispondenza esatta della chiave, ripulendo eventuali spazi generati dal browser.

- Parametri

cookie_header	La stringa nativa dell'header HTTP contenente i cookie.
target_key	La chiave identificativa del cookie da ricercare.

Restituisce

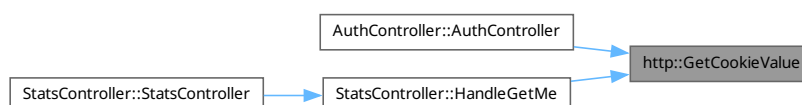
`std::optional<std::string>` Il valore del cookie decodificato, oppure una stringa vuota se la chiave non esiste.

Tag Identificativo: CMN-HTTP-MTH-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.4.1.2 ReadBody()

```
void http::ReadBody (
    AppResponse * res,
    size_t max_bytes,
    std::function< void(const std::string &)> callback) [inline]
```

Accumula asincronamente un intero body HTTP frammentato (chunked) da uWebSockets.

- Poiché uWebSockets non è bloccante, i body arrivano in uno o più frammenti `string_view` tramite la callback `onData`. Il parametro booleano `isLast` segnala l'ultimo frammento. Questa funzione bufferizza i frammenti autonomamente e agisce solo a trasferimento completato.

-

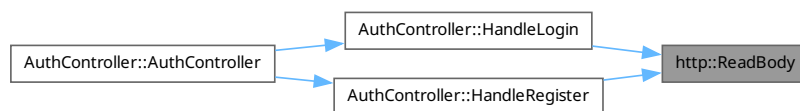
Implementa una guardia di memoria `is_alive` (`std::shared_ptr<bool>`) per gestire le disconnessioni improvvise. Se il client interrompe il trasferimento, uWS lancia `onAborted`. La guardia diventa `false` evitando che successivi trigger di `onData` accedano al puntatore `res` divenuto pendente (dangling).

- Parametri

res	Puntatore all'oggetto Response della richiesta HTTP corrente. Deve restare valido fino al termine.
max_bytes	Limite massimo rigido in byte (Hard cap) per evitare attacchi DDoS. Se superato, restituisce 413.
callback	La funzione da invocare, una volta sola, con la stringa completa del body (se non abortita).

Tag Identificativo: CMN-HTTP-MTH-001

Questo è il grafo dei chiamanti di questa funzione:



7.4.1.3 TrimWhitespace()

```
std::string_view http::TrimWhitespace (
    std::string_view str) [constexpr]
```

Rimuove gli spazi bianchi ASCII (spazi, tabulazioni, a capo) all'inizio e alla fine di una stringa. Essendo `constexpr`, se valutata a tempo di compilazione non genera overhead.

- Parametri

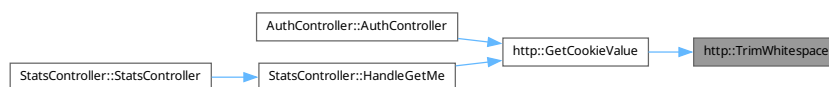
str	La vista della stringa originale.
-----	-----------------------------------

Restituisce

std::string_view Una porzione centrata della stringa priva degli spazi esterni. Restituisce una vista vuota se composta solo da spazi.

Tag Identificativo: CMN-HTTP-MTH-002

Questo è il grafo dei chiamanti di questa funzione:



7.5 Riferimenti per il namespace ws

Tipi enumerati (enum)

- enum class [ServerAction](#) {
[kSuccess](#) , [kError](#) , [kLobbyList](#) , [kLobbyUpdated](#) ,
[kLobbyJoined](#) , [kLobbyLeft](#) , [kLobbyEvicted](#) , [kGameStateUpdated](#) ,
[kGameOver](#) , [kChatMessage](#) }
 Azioni inviate dal Server ai Client connessi.
- enum class [ClientAction](#) {
[kLobbyStartMatch](#) , [kLobbyList](#) , [kLobbyCreate](#) , [kLobbyRejoin](#) ,
[kLobbyJoin](#) , [kLobbyLeave](#) , [kLobbyUpdateSettings](#) , [kLobbyDeleteSavedMatch](#) ,
[kLobbyListSavedMatches](#) , [kLobbyResumeSavedMatch](#) , [kLobbyPromote](#) , [kLobbyKick](#) ,
[kGameSubmitInput](#) , [kGamePlayCard](#) , [kGameDrawCard](#) , [kGameCallUno](#) ,
[kGameExit](#) , [kChatSend](#) }
 Azioni (richieste) inviate dai Client verso il Server.

Funzioni

- template<typename T>
[Result](#)< T > [Get](#) (const nlohmann::json &[json](#), std::string_view key)
 Estrae un campo obbligatorio da un payload JSON, ritornando un [Result](#) monadico.
- template<typename T>
T [GetOr](#) (const nlohmann::json &[json](#), std::string_view key, const T &default_value)
 Estrae un campo opzionale da un JSON, applicando un fallback in caso di assenza/errore.
- nlohmann::json [MakeResponse](#) ([ws::ServerAction](#) action, const std::string &request_id="")
 Costruisce l'intelaiatura di base per un messaggio JSON di risposta del server.
- void [SendError](#) ([AppWebSocket](#) *ws, uWS::OpCode op, const std::string &reason, const std::string &request_id)
 Invia un payload JSON formattato come errore sul socket specifico.

Variabili

- `const std::unordered_map< ServerAction, std::string > kServerActionStr`
- `const std::unordered_map< ClientAction, std::string > kClientActionStr`
- `const std::unordered_map< std::string, ClientAction > kStrClientAction`

7.5.1 Documentazione dei tipi enumerati

7.5.1.1 ClientAction

enum class [ws::ClientAction](#) [strong]

Azioni (richieste) inviate dai Client verso il Server.

Tag Identificativo: WS-ENUM-002

Valori del tipo enumerato

<code>kLobbyStartMatch</code>	Richiesta: Avvia la partita nella lobby corrente.
<code>kLobbyList</code>	Richiesta: Ottieni le lobby pubbliche.
<code>kLobbyCreate</code>	Richiesta: Crea una nuova lobby con impostazioni.
<code>kLobbyRejoin</code>	Richiesta: Riconnettiti a una lobby post-disconnessione.
<code>kLobbyJoin</code>	Richiesta: Unisciti a una lobby tramite codice.
<code>kLobbyLeave</code>	Richiesta: Abbandona la lobby corrente.
<code>kLobbyUpdateSettings</code>	Richiesta: Aggiorna i settings (solo Host).
<code>kLobbyDeleteSavedMatch</code>	Richiesta: Elimina partita salvata.
<code>kLobbyListSavedMatches</code>	Richiesta: Ottieni partite salvate per il gruppo attuale.
<code>kLobbyResumeSavedMatch</code>	Richiesta: Riprendi una partita salvata.
<code>kLobbyPromote</code>	Richiesta: Cedi il ruolo di Host a un altro giocatore.
<code>kLobbyKick</code>	Richiesta: Espelli un giocatore (solo Host).
<code>kGameSubmitInput</code>	Richiesta: Invia un input obbligatorio (es. scelta colore).
<code>kGamePlayCard</code>	Richiesta: Gioca una carta specifica.
<code>kGameDrawCard</code>	Richiesta: Pesca una carta dal mazzo.
<code>kGameCallUno</code>	Richiesta: Dichiarare "UNO".
<code>kGameExit</code>	Richiesta: Esci dalla partita in corso.
<code>kChatSend</code>	Richiesta: Invia un messaggio nella chat della lobby.

7.5.1.2 ServerAction

enum class [ws::ServerAction](#) [strong]

Azioni inviate dal Server ai Client connessi.

Tag Identificativo: WS-ENUM-001

Valori del tipo enumerato

kSuccess	Successo generico con dati arbitrari allegati.
kError	Errore generico contenente una motivazione (reason).
kLobbyList	Invio della lista delle lobby pubbliche disponibili.
kLobbyUpdated	Broadcast: lo stato della lobby è cambiato.
kLobbyJoined	Successo: il destinatario è entrato nella lobby.
kLobbyLeft	Successo: il destinatario è uscito dalla lobby.
kLobbyEvicted	Evizione: lobby chiusa perché senza giocatori umani.
kGameStateUpdated	Broadcast: lo stato della partita in corso è cambiato.
kGameOver	Broadcast: la partita è terminata.
kChatMessage	Broadcast: un nuovo messaggio in chat.

7.5.2 Documentazione delle funzioni

7.5.2.1 Get()

```
template<typename T>
Result< T > ws::Get (
    const nlohmann::json & json,
    std::string_view key) [inline]
```

Estrae un campo obbligatorio da un payload JSON, ritornando un [Result](#) monadico.

- Esegue validazione del tipo. Se manca il campo o il tipo è errato, restituisce un errore di tipo [Error::InvalidInput](#).

- Parametri dei template

T	Tipo di dato atteso (es. int, std::string).
---	---

Parametri

json	Payload JSON in ingresso (non fidato).
key	Chiave dell'attributo da cercare.

Restituisce

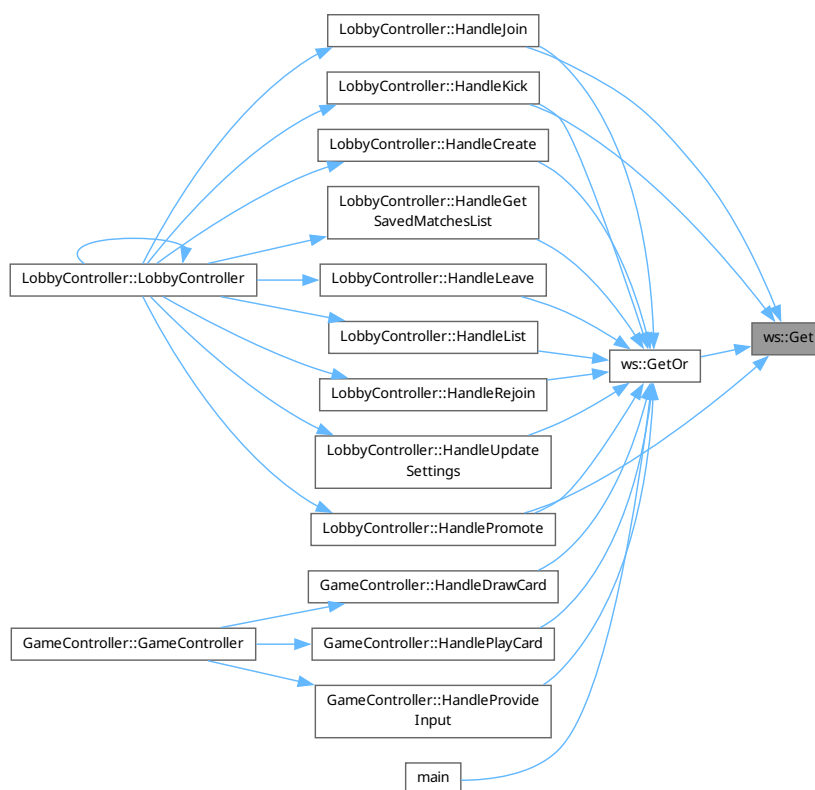
`Result<T>` Il valore estratto oppure uno stato di errore incapsulato.

Tag Identificativo: WS-UTIL-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.5.2.2 GetOr()

```

template<typename T>
T ws::GetOr (

```

```
const nlohmann::json & json,  
std::string_view key,  
const T & default_value) [inline]
```

Estrae un campo opzionale da un JSON, applicando un fallback in caso di assenza/errore.

- Parametri dei template

T	Tipo di dato atteso.
---	----------------------

Parametri

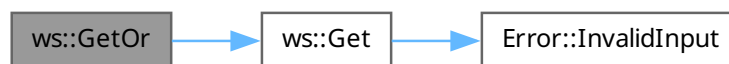
json	Payload JSON.
key	Chiave dell'attributo da cercare.
default_value	Valore di fallback restituito se il dato è assente/non valido.

Restituisce

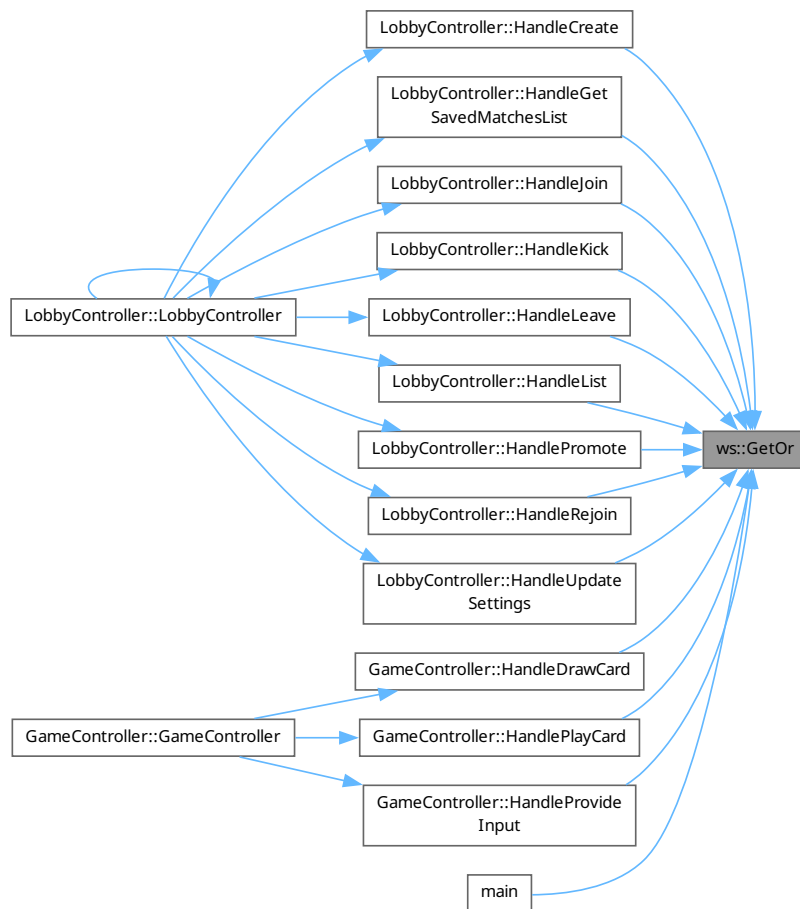
T Il valore estratto o il fallback fornito.

Tag Identificativo: WS-UTIL-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.5.2.3 MakeResponse()

```
nlohmann::json ws::MakeResponse (
    ws::ServerAction action,
    const std::string & request_id = "") [inline]
```

Costruisce l'intelaiatura di base per un messaggio JSON di risposta del server.

Parametri

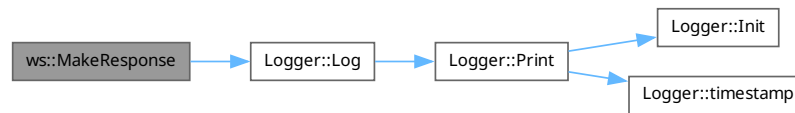
action	Il tipo di azione (ServerAction) da inserire nel payload.
request_id	(Opzionale) ID della richiesta originale per il tracciamento frontend.

Restituisce

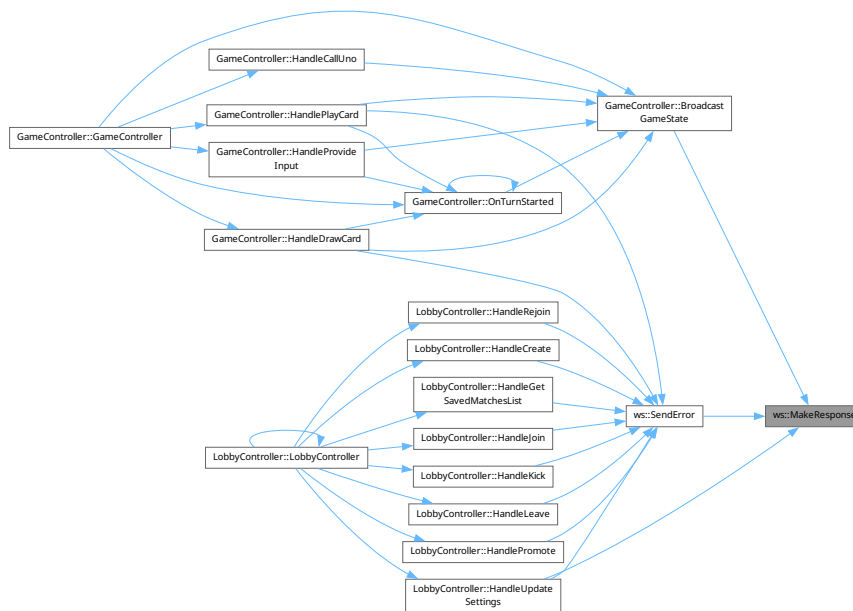
`nlohmann::json` Struttura JSON pronta per essere popolata con altri campi.

Tag Identificativo: WS-UTIL-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.5.2.4 SendError()

```

void ws::SendError (
    AppWebSocket * ws,
    uWS::OpCode op,
    const std::string & reason,
    const std::string & request_id) [inline]
  
```

Invia un payload JSON formattato come errore sul socket specifico.

Parametri

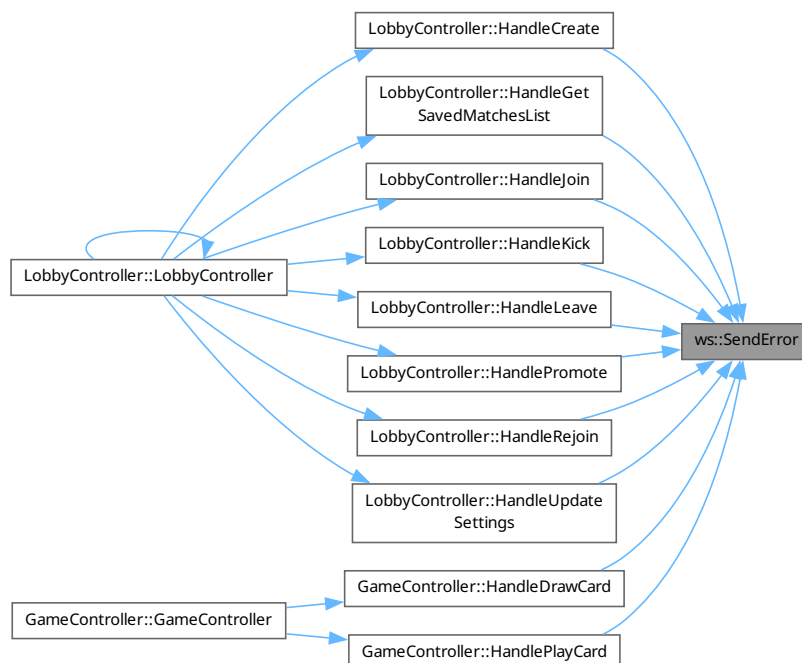
ws	Puntatore al socket destinatario.
op	L'OpCode di uWebSockets (es. testo).
reason	La stringa descrittiva dell'errore (motivo).
request_id	(Opzionale) L'ID del pacchetto che ha scatenato l'errore.

Tag Identificativo: WS-UTIL-004

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



7.5.3 Documentazione delle variabili

7.5.3.1 kClientActionStr

const std::unordered_map<ClientAction, std::string> ws::kClientActionStr [inline]

Valore iniziale:

```
{
    { ClientAction::kGamePlayCard,      "game_play_card"      },
    { ClientAction::kGameDrawCard,      "game_draw_card"      },
    { ClientAction::kGameCallUno,       "game_call_uno"       },

    { ClientAction::kChatSend,          "chat_send"           },

    { ClientAction::kLobbyList,         "lobby_list"          },
    { ClientAction::kLobbyStartMatch,   "lobby_start_match"   },
    { ClientAction::kLobbyCreate,       "lobby_create"        },
    { ClientAction::kLobbyRejoin,       "lobby_rejoin"        },
    { ClientAction::kLobbyJoin,         "lobby_join"          },
    { ClientAction::kLobbyLeave,         "lobby_leave"         },
    { ClientAction::kLobbyUpdateSettings, "lobby_update_settings" },
    { ClientAction::kLobbyDeleteSavedMatch, "lobby_delete_saved_match" },
    { ClientAction::kLobbyListSavedMatches, "lobby_list_saved_matches" },
    { ClientAction::kLobbyResumeSavedMatch, "lobby_resume_saved_match" },
    { ClientAction::kLobbyPromote,      "lobby_promote"       },
    { ClientAction::kLobbyKick,         "lobby_kick"          },

    { ClientAction::kGameSubmitInput,   "game_submit_input"   },
    { ClientAction::kGamePlayCard,      "game_play_card"      },
    { ClientAction::kGameDrawCard,      "game_draw_card"      },
    { ClientAction::kGameCallUno,       "game_call_uno"       },
    { ClientAction::kGameExit,          "game_exit"           },

    { ClientAction::kChatSend,          "chat_send"           },
}
```

7.5.3.2 kServerActionStr

const std::unordered_map<ServerAction, std::string> ws::kServerActionStr [inline]

Valore iniziale:

```
{
    { ServerAction::kSuccess,           "success"              },
    { ServerAction::kError,             "error"                },
    { ServerAction::kLobbyList,         "lobby_list"           },
    { ServerAction::kLobbyEvicted,      "lobby_evicted"        },
    { ServerAction::kLobbyUpdated,      "lobby_updated"        },
    { ServerAction::kLobbyJoined,       "lobby_joined"         },
    { ServerAction::kLobbyLeft,         "lobby_left"           },

    { ServerAction::kGameStateUpdated,  "game_state_updated"   },
    { ServerAction::kGameOver,          "game_over"            },

    { ServerAction::kChatMessage,       "chat_message"         },
}
```

7.5.3.3 kStrClientAction

const std::unordered_map<std::string, ClientAction> ws::kStrClientAction [inline]

Capitolo 8

Documentazione delle classi

8.1 Riferimenti per la classe ActionRouter

Smista i messaggi WebSocket ai vari controller in base al tipo di azione.

```
#include <action_router.hpp>
```

Diagramma delle classi per ActionRouter

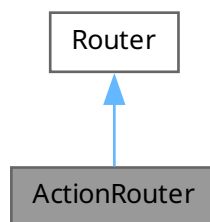
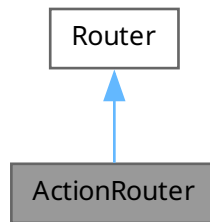


Diagramma di collaborazione per `ActionRouter`:



Membri pubblici

- `ActionRouter` `()=default`
- `ActionRouter` & `On` (`const std::string &action`, `ActionHandler` handler)
Registra un gestore per una specifica azione.
- `ActionRouter` & `OnAny` (`ActionHandler` handler)
Registra un gestore "wildcard" (middleware). I middleware vengono eseguiti nell'ordine di registrazione prima di ogni handler specifico.
- `bool Dispatch` (`WsContext` ctx, `const json &msg`) `const`
Avvia lo smistamento di un messaggio in arrivo.

Membri pubblici ereditati da `Router`

- `virtual ~Router` `()=default`
- `Router` (`const Router &`)=`delete`
Elimina il costruttore di copia.
- `Router & operator=` (`const Router &`)=`delete`
Elimina l'operatore di assegnamento.

Attributi privati

- `std::unordered_map< std::string, ActionHandler > handlers__`
- `std::vector< ActionHandler > wildcards__`

Altri membri ereditati

Membri protetti ereditati da `Router`

- `Router` `()=default`
Costruttore protetto, istanziabile solo dalle classi derivate.

8.1.1 Descrizione dettagliata

Smista i messaggi WebSocket ai vari controller in base al tipo di azione.

- Supporta l'uso di "wildcard" (middleware) che vengono eseguiti prima di ogni azione specifica. Se una wildcard restituisce false, il gestore specifico non viene mai chiamato. Eredita da [Router](#) per disabilitare la copia.

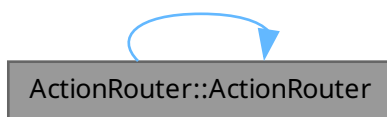
Tag Identificativo: ACT-RTR-CLS-001

8.1.2 Documentazione dei costruttori e dei distruttori

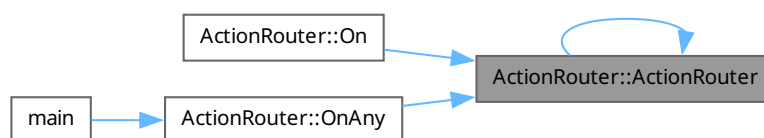
8.1.2.1 ActionRouter()

ActionRouter::ActionRouter () [default]

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.1.3 Documentazione delle funzioni membro

8.1.3.1 Dispatch()

```
bool ActionRouter::Dispatch (  
    WsContext ctx,  
    const json & msg) const
```

Avvia lo smistamento di un messaggio in arrivo.

- Viene chiamato internamente dal [WebServer](#) su ogni frame WebSocket ricevuto. Esegue prima le wildcard; se tutte restituiscono true, cerca ed esegue il gestore specifico.

Parametri

ctx	Il contesto della connessione WebSocket.
msg	Il payload JSON ricevuto dal client.

Restituisce

true se un gestore specifico è stato trovato ed eseguito, false se non esiste alcun gestore (dopo aver eseguito le wildcard).

Tag Identificativo: ACT-RTR-MTH-003

8.1.3.2 On()

```
ActionRouter & ActionRouter::On (  
    const std::string & action,  
    ActionHandler handler)
```

Registra un gestore per una specifica azione.

Parametri

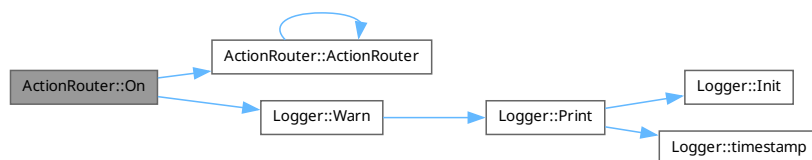
action	Il nome dell'azione (es. "play_card").
handler	La funzione da eseguire quando si riceve l'azione.

Restituisce

[ActionRouter](#)& Riferimento a se stesso per il concatenamento (chaining).

Tag Identificativo: ACT-RTR-MTH-001

Questo è il grafo delle chiamate per questa funzione:



8.1.3.3 OnAny()

`ActionRouter` & `ActionRouter::OnAny` (
 `ActionHandler` handler)

Registra un gestore "wildcard" (middleware). I middleware vengono eseguiti nell'ordine di registrazione prima di ogni handler specifico.

Parametri

handler	La funzione da eseguire. Se restituisce false, blocca la catena.
---------	--

Restituisce

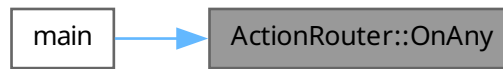
`ActionRouter`& Riferimento a se stesso.

Tag Identificativo: ACT-RTR-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.1.4 Documentazione dei membri dato

8.1.4.1 handlers_

`std::unordered_map<std::string, ActionHandler> ActionRouter::handlers_` [private]

Mappa delle azioni ai relativi handler specifici.

8.1.4.2 wildcards_

`std::vector<ActionHandler> ActionRouter::wildcards_` [private]

Lista dei middleware eseguiti prima di ogni azione specifica.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- `include/action_router.hpp`
- `src/action_router.cpp`

8.2 Riferimenti per la classe `game::AdvanceTurnEffect`

Effetto base che avanza il turno al giocatore successivo. Viene solitamente accodato alla fine di una giocata standard o dopo la risoluzione degli altri effetti.

```
#include <standard.hpp>
```

Diagramma delle classi per `game::AdvanceTurnEffect`

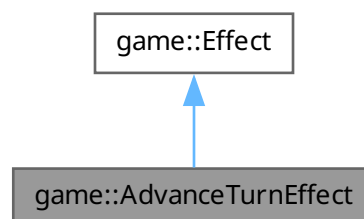
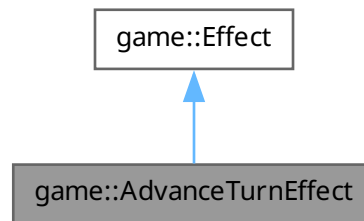


Diagramma di collaborazione per game::AdvanceTurnEffect:



Membri pubblici

- `EffectResult Resolve (GameState *game_state, MatchInstance *match_instance)` override
Risolve l'effetto aggiornando l'indice del giocatore corrente nello stato di gioco.
- `EffectType GetType ()` const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da `game::Effect`

- virtual `~Effect ()`=default

8.2.1 Descrizione dettagliata

Effetto base che avanza il turno al giocatore successivo. Viene solitamente accodato alla fine di una giocata standard o dopo la risoluzione degli altri effetti.

Tag Identificativo: EFFECT-STD-001

8.2.2 Documentazione delle funzioni membro

8.2.2.1 GetType()

`EffectType game::AdvanceTurnEffect::GetType ()` const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kAdvanceTurn.

Tag Identificativo: EFFECT-STD-MTH-002

Reimplementa `game::Effect`.

8.2.2.2 Resolve()

```
EffectResult game::AdvanceTurnEffect::Resolve (
    GameState * game_state,
    MatchInstance * match_instance)  [override], [virtual]
```

Risolve l'effetto aggiornando l'indice del giocatore corrente nello stato di gioco.

Parametri

game_state	Lo stato attuale del gioco.
match_instance	Istanza della partita.

Restituisce

[EffectResult](#) L'esito della risoluzione (sempre kResolved in assenza di errori).

Tag Identificativo: EFFECT-STD-MTH-001

Implementa [game::Effect](#).

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/game/effects/[standard.hpp](#)
- src/game/effects/[standard.cpp](#)

8.3 Riferimenti per la classe AuthController

Gestisce le rotte HTTP per il login/registrazione e la sicurezza crittografica.

```
#include <auth_controller.hpp>
```

Memberi pubblici

- [AuthController](#) ([HttpRouter](#) &router)
Costruttore dell'[AuthController](#). Registra in automatico le rotte POST e GET sul router HTTP fornito (/auth/register, /auth/login, ecc.).

Memberi pubblici statici

- static [Result](#)< [JwtPayload](#) > [VerifyToken](#) (const std::string &token)
Verifica matematicamente un token JWT e ne estrae il payload. Metodo statico richiamato dal [WebServer](#) all'inizio di ogni tentativo di upgrade verso il protocollo WebSocket, per bloccare client non autorizzati.

Membrî privati

- void `HandleRegister` (`ApiResponse` *res, `AppRequest` *req)
Gestisce la rotta POST /auth/register. Valida gli input (lunghezza e caratteri speciali), calcola l'hash sicuro della password e memorizza il nuovo utente nel database.
- void `HandleLogin` (`ApiResponse` *res, `AppRequest` *req)
Gestisce la rotta POST /auth/login. Cerca l'utente nel database, ricalcola l'hash per verificare la password fornita e, in caso di successo, emette e restituisce un JWT firmato.
- void `HandleLogout` (`ApiResponse` *res, `AppRequest` *req)
Gestisce la rotta POST /auth/logout. Invalida la sessione lato client (es. richiedendo l'eliminazione del cookie JWT).
- void `HandleMe` (`ApiResponse` *res, `AppRequest` *req)
Gestisce la rotta GET /auth/me. Restituisce i dettagli dell'utente attualmente autenticato valutandone il token.

Membrî privati statici

- static std::string `HashPassword` (const std::string &password)
Genera un salt crittografico e calcola l'hash della password.
- static bool `VerifyPassword` (const std::string &password, const std::string &stored)
Verifica se una password in chiaro corrisponde a quella memorizzata sul DB.
- static std::string `IssueToken` (const std::string &username)
Genera un nuovo token JWT firmato usando la chiave segreta (pepper).

Attributi privati statici

- static constexpr int `kMinUsernameLen` = 3
Lunghezza minima dell'username.
- static constexpr int `kMaxUsernameLen` = 32
Lunghezza massima dell'username.
- static constexpr int `kMinPasswordLen` = 8
Lunghezza minima della password.
- static constexpr int `kMaxBodyBytes` = 4096
Limite in bytes per il payload HTTP (Anti-DDoS).
- static constexpr int `kSaltBytes` = 16
Dimensione in bytes del Salt crittografico.
- static constexpr int `kHashBytes` = 32
Dimensione in bytes dell'Hash risultante.
- static constexpr int `kIterations` = 200'000
Numero di iterazioni dell'algoritmo PBKDF2 (minimo NIST SP 800-132).

8.3.1 Descrizione dettagliata

Gestisce le rotte HTTP per il login/registrazione e la sicurezza crittografica.

- Strategia di archiviazione Password: Utilizza l'algoritmo PBKDF2-HMAC-SHA256 (tramite OpenSSL) con un "salt" casuale di 16-byte per ogni utente e un numero configurabile di iterazioni (200.000, come da standard NIST). L'hash risultante (32 byte) è salvato codificato in base64 insieme al salt. Inoltre, un "pepper" globale (letto dalla variabile d'ambiente JWT_SECRET) viene appeso alla password prima dell'hashing, rendendo un eventuale furto del solo database insufficiente per craccare le password.
- Strategia JWT: I token sono firmati in HS256 utilizzando jwt-cpp. Il payload trasporta: sub (username), iat (issued-at), exp (scadenza impostata a 24 ore).

Tag Identificativo: CTRL-AUTH-CLS-001

8.3.2 Documentazione dei costruttori e dei distruttori

8.3.2.1 AuthController()

AuthController::AuthController (
[HttpRouter](#) & router) [explicit]

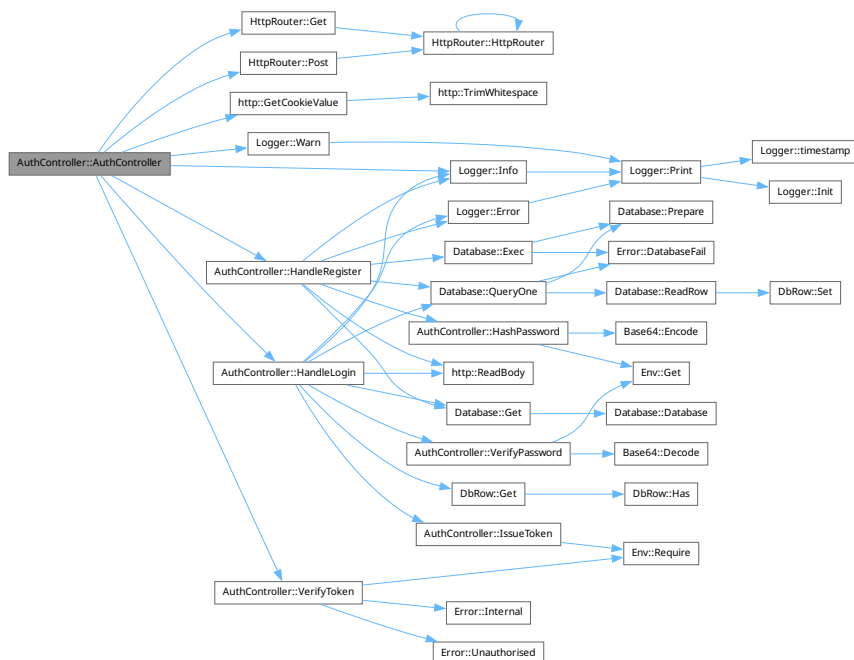
Costruttore dell'[AuthController](#). Registra in automatico le rotte POST e GET sul router HTTP fornito (/auth/register, /auth/login, ecc.).

- Parametri

router	Riferimento al router HTTP centrale. Deve sopravvivere a questa classe.
--------	---

Tag Identificativo: CTRL-AUTH-MTH-001

Questo è il grafo delle chiamate per questa funzione:



8.3.3 Documentazione delle funzioni membro

8.3.3.1 HandleLogin()

```
void AuthController::HandleLogin (
    AppResponse * res,
    AppRequest * req) [private]
```

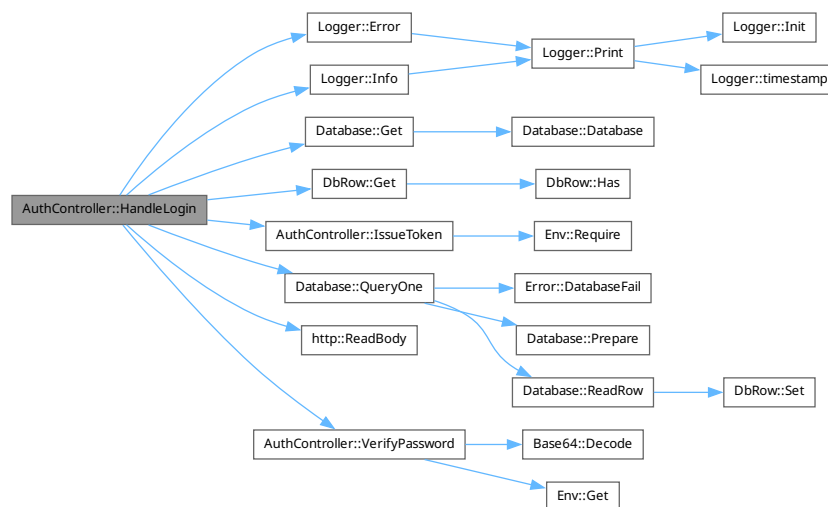
Gestisce la rotta POST /auth/login. Cerca l'utente nel database, ricalcola l'hash per verificare la password fornita e, in caso di successo, emette e restituisce un JWT firmato.

Parametri

res	Puntatore alla risposta HTTP.
req	Puntatore alla richiesta HTTP.

Tag Identificativo: CTRL-AUTH-ACT-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.3.2 HandleLogout()

```
void AuthController::HandleLogout (  
    AppResponse * res,  
    AppRequest * req) [private]
```

Gestisce la rotta POST /auth/logout. Invalida la sessione lato client (es. richiedendo l'eliminazione del cookie JWT).

Parametri

res	Puntatore alla risposta HTTP.
req	Puntatore alla richiesta HTTP.

Tag Identificativo: CTRL-AUTH-ACT-003

8.3.3.3 HandleMe()

```
void AuthController::HandleMe (  
    AppResponse * res,  
    AppRequest * req) [private]
```

Gestisce la rotta GET /auth/me. Restituisce i dettagli dell'utente attualmente autenticato valutandone il token.

Parametri

res	Puntatore alla risposta HTTP.
req	Puntatore alla richiesta HTTP.

Tag Identificativo: CTRL-AUTH-ACT-004

8.3.3.4 HandleRegister()

```
void AuthController::HandleRegister (  
    AppResponse * res,  
    AppRequest * req) [private]
```

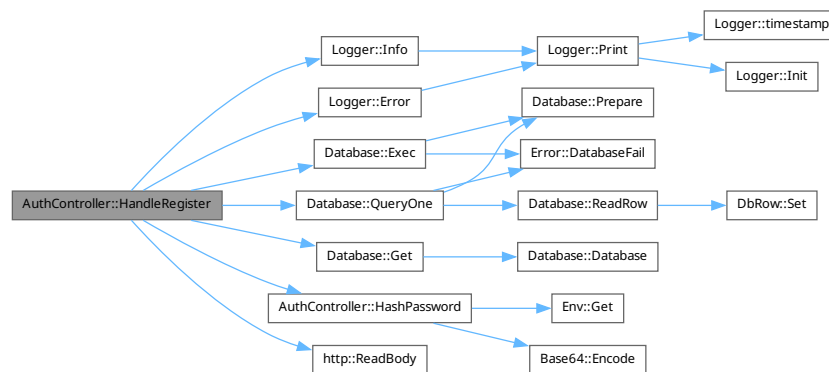
Gestisce la rotta POST /auth/register. Valida gli input (lunghezza e caratteri speciali), calcola l'hash sicuro della password e memorizza il nuovo utente nel database.

Parametri

res	Puntatore alla risposta HTTP.
req	Puntatore alla richiesta HTTP.

Tag Identificativo: CTRL-AUTH-ACT-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.3.5 HashPassword()

```
std::string AuthController::HashPassword (
    const std::string & password) [static], [private]
```

Genera un salt crittografico e calcola l'hash della password.

Parametri

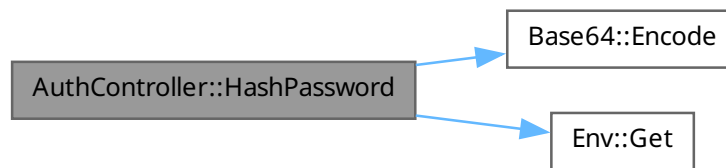
password	La password in chiaro inserita dall'utente.
----------	---

Restituisce

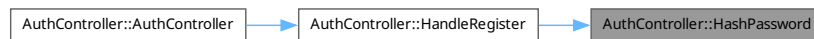
`std::string` Formato composto salvato su DB: `<base64_salt>:<base64_hash>`.

Tag Identificativo: CTRL-AUTH-CRYP-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.3.6 IssueToken()

`std::string AuthController::IssueToken (`
`const std::string & username) [static], [private]`

Genera un nuovo token JWT firmato usando la chiave segreta (pepper).

Parametri

username	Il nome utente da inserire nel payload (campo 'sub').
----------	---

Restituisce

`std::string` La stringa del token completa.

Tag Identificativo: CTRL-AUTH-CRYP-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.3.7 VerifyPassword()

```
bool AuthController::VerifyPassword (  
    const std::string & password,  
    const std::string & stored) [static], [private]
```

Verifica se una password in chiaro corrisponde a quella memorizzata sul DB.

Parametri

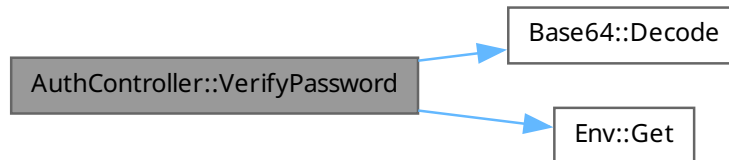
password	La password in chiaro inserita nel login.
stored	La stringa dal DB in formato <salt>:<hash>.

Restituisce

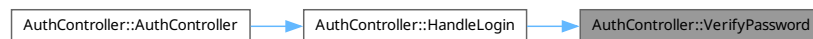
true se le credenziali combaciano, false altrimenti.

Tag Identificativo: CTRL-AUTH-CRYP-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.3.8 VerifyToken()

```
Result< JwpPayload > AuthController::VerifyToken (
    const std::string & token) [static]
```

Verifica matematicamente un token JWT e ne estrae il payload. Metodo statico richiamato dal [WebServer](#) all'inizio di ogni tentativo di upgrade verso il protocollo WebSocket, per bloccare client non autorizzati.

- Parametri

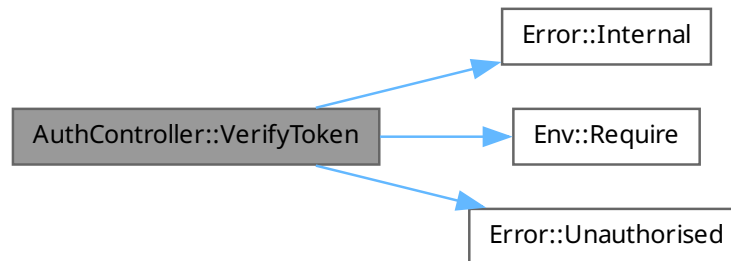
token	La stringa JWT grezza passata dal client (solitamente estratta dai cookie o dall'header).
-------	---

Restituisce

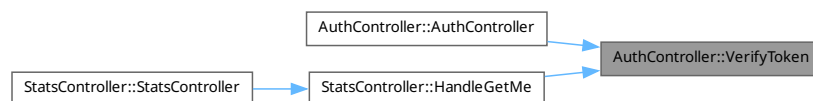
[Result<JwpPayload>](#) Il payload estratto in caso di successo, oppure un Errore.

Tag Identificativo: CTRL-AUTH-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.3.4 Documentazione dei membri dato

8.3.4.1 kHashBytes

```
int AuthController::kHashBytes = 32 [static], [constexpr], [private]
```

Dimensione in bytes dell'Hash risultante.

Tag Identificativo: CTRL-AUTH-CFG-006

8.3.4.2 kIterations

```
int AuthController::kIterations = 200'000 [static], [constexpr], [private]
```

Numero di iterazioni dell'algoritmo PBKDF2 (minimo NIST SP 800-132).

Tag Identificativo: CTRL-AUTH-CFG-007

8.3.4.3 kMaxBodyBytes

```
int AuthController::kMaxBodyBytes = 4096 [static], [constexpr], [private]
```

Limite in bytes per il payload HTTP (Anti-DDoS).

Tag Identificativo: CTRL-AUTH-CFG-004

8.3.4.4 kMaxUsernameLen

```
int AuthController::kMaxUsernameLen = 32 [static], [constexpr], [private]
```

Lunghezza massima dell'username.

Tag Identificativo: CTRL-AUTH-CFG-002

8.3.4.5 kMinPasswordLen

```
int AuthController::kMinPasswordLen = 8 [static], [constexpr], [private]
```

Lunghezza minima della password.

Tag Identificativo: CTRL-AUTH-CFG-003

8.3.4.6 kMinUsernameLen

```
int AuthController::kMinUsernameLen = 3 [static], [constexpr], [private]
```

Lunghezza minima dell'username.

Tag Identificativo: CTRL-AUTH-CFG-001

8.3.4.7 kSaltBytes

```
int AuthController::kSaltBytes = 16 [static], [constexpr], [private]
```

Dimensione in bytes del Salt crittografico.

Tag Identificativo: CTRL-AUTH-CFG-005

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/controllers/[auth_controller.hpp](#)
- src/controllers/[auth_controller.cpp](#)

8.4 Riferimenti per la struct `game::CardDrawnEvent`

```
#include <gamerule.hpp>
```

Attributi pubblici

- `std::string` `player_username`
- `CompactCard` `drawn_card`
- `bool` `is_handled` = false
- `bool` `keep_drawing` = false
- `bool` `force_play` = false

8.4.1 Documentazione dei membri dato

8.4.1.1 `drawn_card`

`CompactCard` `game::CardDrawnEvent::drawn_card`

8.4.1.2 `force_play`

`bool` `game::CardDrawnEvent::force_play` = false

8.4.1.3 `is_handled`

`bool` `game::CardDrawnEvent::is_handled` = false

8.4.1.4 `keep_drawing`

`bool` `game::CardDrawnEvent::keep_drawing` = false

8.4.1.5 `player_username`

`std::string` `game::CardDrawnEvent::player_username`

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/common/game/gamerule.hpp`

8.5 Riferimenti per la struct `game::CardPlayedEvent`

Struttura che incapsula i dati riguardanti l'evento di una carta messa in gioco.

```
#include <gamerule.hpp>
```

Attributi pubblici

- `std::string player_username`
- `CompactCard played_card`
- `bool is_valid_play = true`
- `bool is_handled = false`
- `bool is_out_of_turn = false`

8.5.1 Descrizione dettagliata

Struttura che incapsula i dati riguardanti l'evento di una carta messa in gioco.

8.5.2 Documentazione dei membri dato

8.5.2.1 `is_handled`

```
bool game::CardPlayedEvent::is_handled = false
```

Flag che indica se l'evento è già stato completamente processato da una regola.

8.5.2.2 `is_out_of_turn`

```
bool game::CardPlayedEvent::is_out_of_turn = false
```

Flag che indica se la giocata è avvenuta fuori turno (utile per regole speciali o penalità).

8.5.2.3 `is_valid_play`

```
bool game::CardPlayedEvent::is_valid_play = true
```

Flag che indica se la giocata è valida o va rigettata (impostabile dai validatori).

8.5.2.4 `played_card`

```
CompactCard game::CardPlayedEvent::played_card
```

La carta che viene giocata.

8.5.2.5 player_username

std::string game::CardPlayedEvent::player_username

Il nome utente del giocatore che sta tentando la giocata.

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/common/game/gamerule.hpp`

8.6 Riferimenti per la classe game::ChooseColorEffect

Effetto interattivo che impone la selezione del nuovo colore attivo (Carte Jolly).

```
#include <standard.hpp>
```

Diagramma delle classi per game::ChooseColorEffect

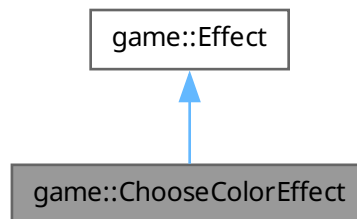
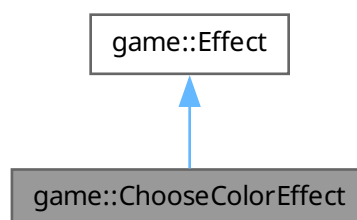


Diagramma di collaborazione per game::ChooseColorEffect:



Membri pubblici

- [ChooseColorEffect](#) (const std::string &target_username)
Costruttore dell'effetto.
- [EffectResult Resolve](#) ([GameState](#) *state, [MatchInstance](#) *match_instance) override
Sospende l'esecuzione in attesa dell'input dal target. Se l'input è già presente, aggiorna il colore globale.
- [EffectType GetType](#) () const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da [game::Effect](#)

- virtual [~Effect](#) ()=default

Attributi privati

- std::string [target_username_](#)

8.6.1 Descrizione dettagliata

Effetto interattivo che impone la selezione del nuovo colore attivo (Carte Jolly).

Tag Identificativo: EFFECT-STD-006

8.6.2 Documentazione dei costruttori e dei distruttori

8.6.2.1 ChooseColorEffect()

```
game::ChooseColorEffect::ChooseColorEffect (
    const std::string & target_username)  [inline], [explicit]
```

Costruttore dell'effetto.

Parametri

target_username	Il giocatore che ha giocato il Jolly e deve scegliere il colore.
-----------------	--

8.6.3 Documentazione delle funzioni membro

8.6.3.1 GetType()

`EffectType` game::ChooseColorEffect::GetType () const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kChooseColor.

Tag Identificativo: EFFECT-STD-MTH-012

Reimplementa `game::Effect`.

8.6.3.2 Resolve()

`EffectResult` game::ChooseColorEffect::Resolve (
 `GameState` * state,
 `MatchInstance` * match_instance) [override], [virtual]

Sospende l'esecuzione in attesa dell'input dal target. Se l'input è già presente, aggiorna il colore globale.

Parametri

state	Lo stato del gioco.
match_instance	Istanza della partita.

Restituisce

`EffectResult` Esito dell'operazione o kNeedsInput.

Tag Identificativo: EFFECT-STD-MTH-011

Implementa `game::Effect`.

8.6.4 Documentazione dei membri dato

8.6.4.1 target_username_

`std::string` game::ChooseColorEffect::target_username_ [private]

Utente che deve fornire la scelta del colore.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/game/effects/[standard.hpp](#)
- src/game/effects/[standard.cpp](#)

8.7 Riferimenti per la classe Database

Pattern Singleton che incapsula la connessione univoca e condivisa al DB SQLite.

```
#include <database.hpp>
```

Membri pubblici

- [VoidResult Open](#) (std::string_view path)
Apre o crea il file del database SQLite nel percorso fornito.
- [VoidResult ApplySchema](#) (const char *sql)
Applica comandi SQL diretti al database (utilizzato prevalentemente per i file di Schema).
- void [Close](#) ()
Chiude esplicitamente la connessione SQLite attiva.
- bool [IsOpen](#) () const
Verifica se il DB è attualmente pronto all'uso.
- [VoidResult Exec](#) (const char *sql, std::vector< [DbValue](#) > params={})
Esegue una query che non restituisce righe (es. INSERT, UPDATE, DELETE).
- [Result](#)< std::vector< [DbRow](#) > > [Query](#) (const char *sql, std::vector< [DbValue](#) > params={})
Esegue una query che restituisce una lista di risultati (es. SELECT generiche).
- [Result](#)< std::optional< [DbRow](#) > > [QueryOne](#) (const char *sql, std::vector< [DbValue](#) > params={})
Esegue una query mirata alla restituzione di al massimo 1 record (es. SELECT con LIMIT 1).

Membri pubblici statici

- static [Database & Get](#) ()
Recupera l'unica istanza attiva del [Database](#).

Membri privati

- [Database](#) ()=default
- [~Database](#) ()
- [Database](#) (const [Database](#) &)=delete
- [Database & operator=](#) (const [Database](#) &)=delete
- sqlite3_stmt * [Prepare](#) (const char *sql, const std::vector< [DbValue](#) > ¶ms)
- [DbRow ReadRow](#) (sqlite3_stmt *stmt)

Attributi privati

- sqlite3 * [db_](#) = nullptr

8.7.1 Descrizione dettagliata

Pattern Singleton che incapsula la connessione univoca e condivisa al DB SQLite.

- Tutti i metodi di query sono "No-Throw" e ritornano risultati monadici ([Result](#)) per una corretta gestione degli errori senza eccezioni C++.

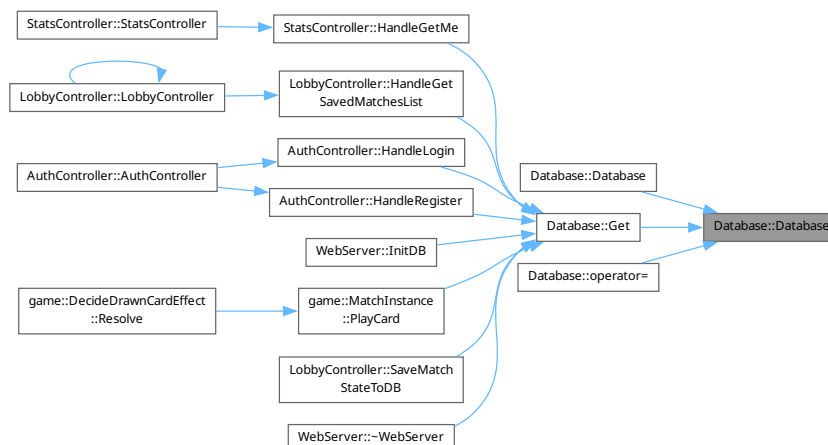
Tag Identificativo: DB-CLS-001

8.7.2 Documentazione dei costruttori e dei distruttori

8.7.2.1 Database() ^[1/2]

Database::Database () [private], [default]

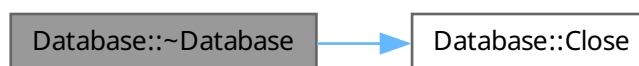
Questo è il grafo dei chiamanti di questa funzione:



8.7.2.2 ~Database()

Database::~Database () [inline], [private]

Questo è il grafo delle chiamate per questa funzione:



8.7.2.3 Database() ^[2/2]

Database::Database (
 const Database &) [private], [delete]

Questo è il grafo delle chiamate per questa funzione:



8.7.3 Documentazione delle funzioni membro

8.7.3.1 ApplySchema()

```
VoidResult Database::ApplySchema (  
    const char * sql)
```

Applica comandi SQL diretti al database (utilizzato prevalentemente per i file di Schema).

Parametri

sql	Stringa query SQL raw.
-----	------------------------

Restituisce

VoidResult.

Tag Identificativo: DB-MTH-003

Questo è il grafo delle chiamate per questa funzione:



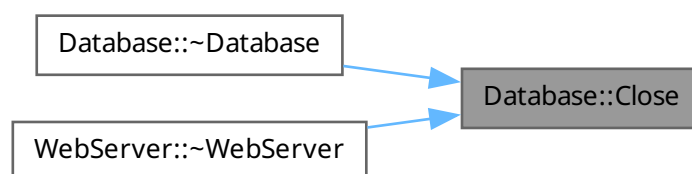
8.7.3.2 Close()

```
void Database::Close ()
```

Chiude esplicitamente la connessione SQLite attiva.

Tag Identificativo: DB-MTH-004

Questo è il grafo dei chiamanti di questa funzione:



8.7.3.3 Exec()

```
VoidResult Database::Exec (  
    const char * sql,  
    std::vector< DbValue > params = {})
```

Esegue una query che non restituisce righe (es. INSERT, UPDATE, DELETE).

Parametri

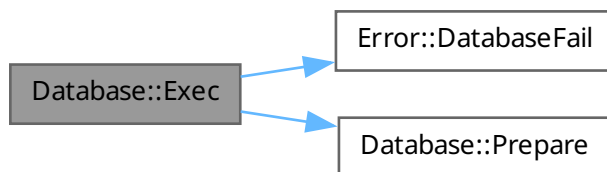
sql	La stringa SQL (che può contenere segnaposto ?).
params	I parametri tipizzati opzionali da bindare sulla query.

Restituisce

VoidResult Errore in caso di fallimento della query o violazione constraint.

Tag Identificativo: DB-MTH-006

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.7.3.4 Get()

[Database](#) & Database::Get () [static]

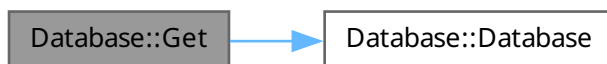
Recupera l'unica istanza attiva del [Database](#).

Restituisce

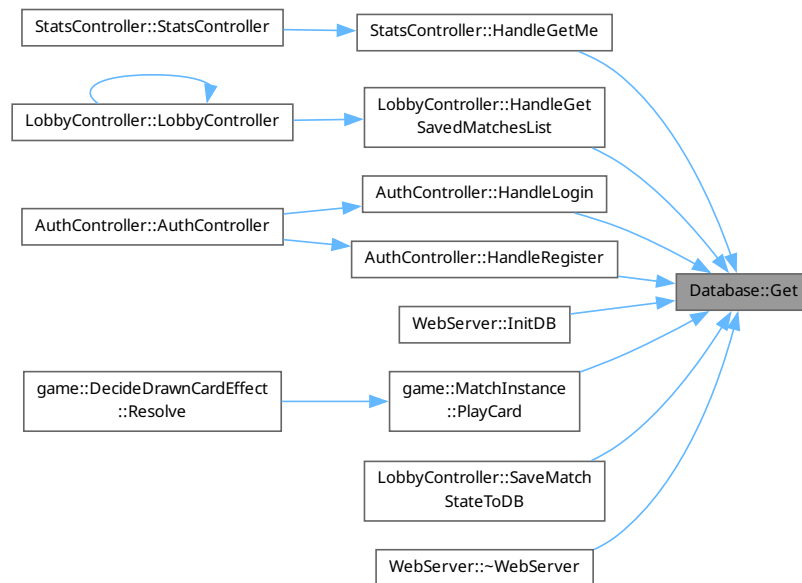
[Database&](#) Riferimento al Singleton.

Tag Identificativo: DB-MTH-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.7.3.5 IsOpen()

```
bool Database::IsOpen () const
```

Verifica se il DB è attualmente pronto all'uso.

Restituisce

true se aperto.

Tag Identificativo: DB-MTH-005

8.7.3.6 Open()

```
VoidResult Database::Open (
    std::string_view path)
```

Apre o crea il file del database SQLite nel percorso fornito.

Parametri

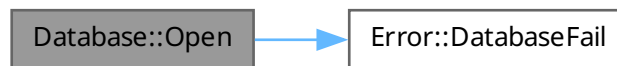
path	Percorso su disco.
------	--------------------

Restituisce

[VoidResult](#) Risultato del tentativo di apertura.

Tag Identificativo: DB-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.7.3.7 operator=()

```
Database & Database::operator= (  
    const Database & )    [private], [delete]
```

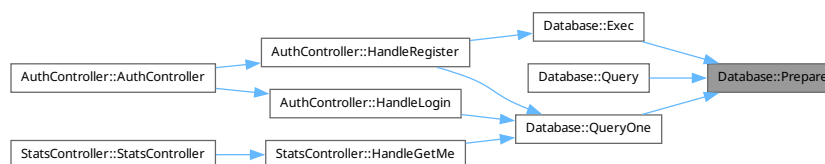
Questo è il grafo delle chiamate per questa funzione:



8.7.3.8 Prepare()

```
sqlite3_stmt * Database::Prepare (
    const char * sql,
    const std::vector< DbValue > & params) [private]
```

Questo è il grafo dei chiamanti di questa funzione:



8.7.3.9 Query()

```
Result< std::vector< DbRow > > Database::Query (
    const char * sql,
    std::vector< DbValue > params = {})
```

Esegue una query che restituisce una lista di risultati (es. SELECT generiche).

Parametri

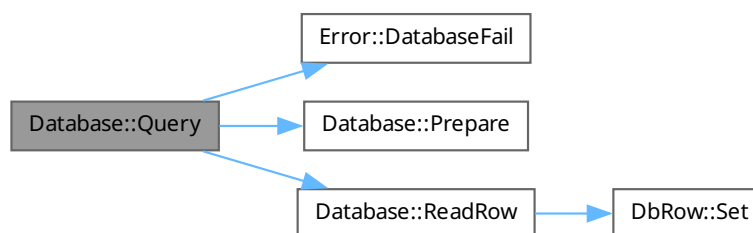
sql	Stringa query parametrizzata.
params	Lista di parametri associati ai segnaposto.

Restituisce

`Result<std::vector<DbRow>>` Il vettore di righe risultanti.

Tag Identificativo: DB-MTH-007

Questo è il grafo delle chiamate per questa funzione:



8.7.3.10 QueryOne()

```
Result< std::optional< DbRow > > Database::QueryOne (
    const char * sql,
    std::vector< DbValue > params = {})
```

Esegue una query mirata alla restituzione di al massimo 1 record (es. SELECT con LIMIT 1).

Parametri

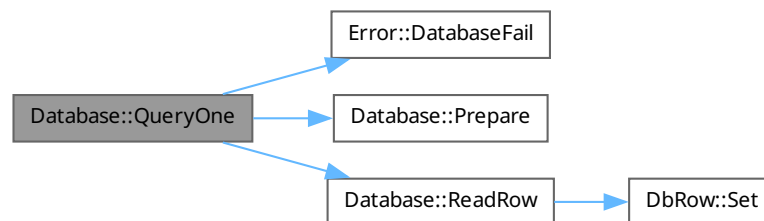
sql	Stringa query parametrizzata.
params	Lista di parametri.

Restituisce

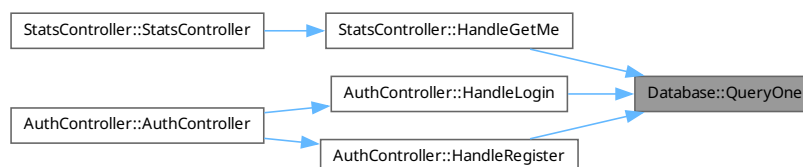
`Result<std::optional<DbRow>>` Risultato contenente il record o vuoto (`std::nullopt`).

Tag Identificativo: DB-MTH-008

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



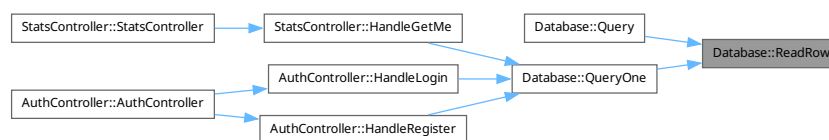
8.7.3.11 ReadRow()

```
DbRow Database::ReadRow (
    sqlite3_stmt * stmt) [private]
```

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.7.4 Documentazione dei membri dato

8.7.4.1 db_

```
sqlite3* Database::db_ = nullptr [private]
```

La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/database.hpp](#)
- [src/database.cpp](#)

8.8 Riferimenti per la classe DbRow

Rappresenta una singola riga restituita da una query SQL. Memorizza i dati usando le chiavi di colonna come indici della mappa interna.

```
#include <database.hpp>
```

Membri pubblici

- void **Set** (const std::string &col, DbValue val)
Imposta un valore nella riga per una specifica colonna.
- template<typename T>
T **Get** (const std::string &col) const
Estrae un valore forzandone il tipo previsto (Lancia eccezione se la colonna manca).
- template<typename T>
T **GetOr** (const std::string &col, T fallback) const
Estrae un valore in modo sicuro fornendo un fallback nel caso sia assente o NULL.
- bool **Has** (const std::string &col) const
Verifica se la riga possiede la colonna specificata.

Attributi privati

- std::unordered_map< std::string, DbValue > data__

8.8.1 Descrizione dettagliata

Rappresenta una singola riga restituita da una query SQL. Memorizza i dati usando le chiavi di colonna come indici della mappa interna.

Tag Identificativo: DB-ROW-001

8.8.2 Documentazione delle funzioni membro

8.8.2.1 Get()

```
template<typename T>
T DbRow::Get (
    const std::string & col) const [inline]
```

Estrae un valore forzandone il tipo previsto (Lancia eccezione se la colonna manca).

Parametri dei template

T	Tipo di dato atteso (es. int, std::string).
---	---

Parametri

col	Nome della colonna.
-----	---------------------

Restituisce

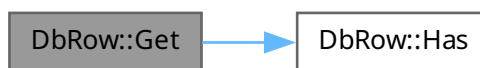
T Valore estratto.

Eccezioni

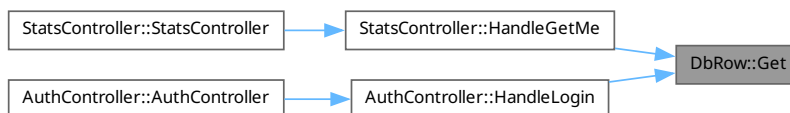
<code>std::runtime_error</code>	se la colonna non esiste.
---------------------------------	---------------------------

Tag Identificativo: DB-ROW-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.8.2.2 GetOr()

```
template<typename T>  
T DbRow::GetOr (  
    const std::string & col,  
    T fallback) const [inline]
```

Estrae un valore in modo sicuro fornendo un fallback nel caso sia assente o NULL.

Parametri dei template

T	Tipo di dato atteso.
---	----------------------

Parametri

col	Nome della colonna.
fallback	Valore di default in caso di colonna NULL/mancante.

Restituisce

T Il valore estratto o il fallback.

Tag Identificativo: DB-ROW-MTH-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.8.2.3 Has()

```
bool DbRow::Has (  
    const std::string & col) const
```

Verifica se la riga possiede la colonna specificata.

Parametri

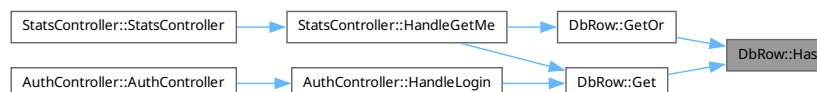
col	Nome della colonna.
-----	---------------------

Restituisce

true se presente, false altrimenti.

Tag Identificativo: DB-ROW-MTH-004

Questo è il grafo dei chiamanti di questa funzione:



8.8.2.4 Set()

```
void DbRow::Set (
    const std::string & col,
    DbValue val)
```

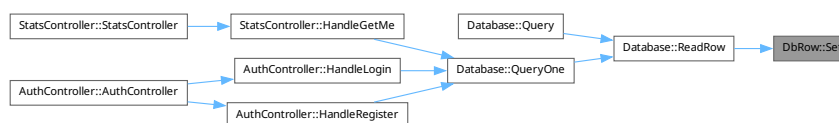
Imposta un valore nella riga per una specifica colonna.

Parametri

col	Il nome della colonna.
val	Il valore tipizzato.

Tag Identificativo: DB-ROW-MTH-001

Questo è il grafo dei chiamanti di questa funzione:



8.8.3 Documentazione dei membri data

8.8.3.1 data_

```
std::unordered_map<std::string, DbValue> DbRow::data_ [private]
```

La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/database.hpp](#)
- [src/database.cpp](#)

8.9 Riferimenti per la classe `game::DecideDrawnCardEffect`

Effetto interattivo che chiede al giocatore se vuole giocare la carta appena pescata.

```
#include <standard.hpp>
```

Diagramma delle classi per `game::DecideDrawnCardEffect`

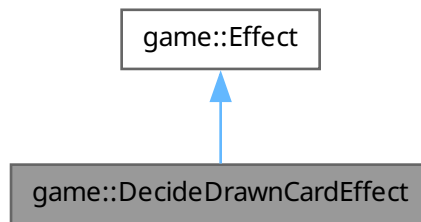
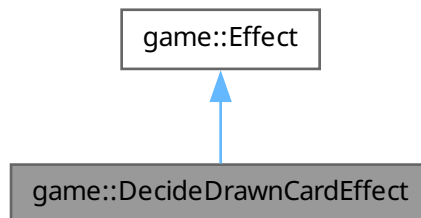


Diagramma di collaborazione per `game::DecideDrawnCardEffect`:



Membri pubblici

- [`DecideDrawnCardEffect`](#) (`const std::string &username, uint16_t card_id`)
Costruttore dell'effetto.
- [`EffectResult Resolve`](#) ([`GameState` *state, `MatchInstance` *match](#)) override
Sospende il motore di gioco in attesa dell'input dell'utente.
- [`EffectType GetType`](#) () const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da `game::Effect`

- virtual [`~Effect`](#) ()=default

Attributi privati

- std::string [username__](#)
- uint16_t [card_id__](#)

8.9.1 Descrizione dettagliata

Effetto interattivo che chiede al giocatore se vuole giocare la carta appena pescata.

- Si innesca quando un giocatore pesca una carta dal mazzo (non per penalità) e questa risulta essere giocabile secondo le regole correnti.

Tag Identificativo: EFFECT-STD-002

8.9.2 Documentazione dei costruttori e dei distruttori

8.9.2.1 DecideDrawnCardEffect()

```
game::DecideDrawnCardEffect::DecideDrawnCardEffect (
    const std::string & username,
    uint16_t card_id)  [inline], [explicit]
```

Costruttore dell'effetto.

Parametri

username	Nome del giocatore che ha appena pescato.
card_id	L'ID a 16-bit della carta appena pescata.

8.9.3 Documentazione delle funzioni membro

8.9.3.1 GetType()

```
EffectType game::DecideDrawnCardEffect::GetType () const  [inline], [override], [virtual]
```

Restituisce il tipo enumerativo dell'effetto.

Restituisce

[EffectType](#) Identificatore kDecideDrawnCard.

Tag Identificativo: EFFECT-STD-MTH-004

Reimplementa [game::Effect](#).

8.9.3.2 Resolve()

```
EffectResult game::DecideDrawnCardEffect::Resolve (
    GameState * state,
    MatchInstance * match)  [override], [virtual]
```

Sospende il motore di gioco in attesa dell'input dell'utente.

Parametri

state	Lo stato del gioco.
match	Istanza della partita.

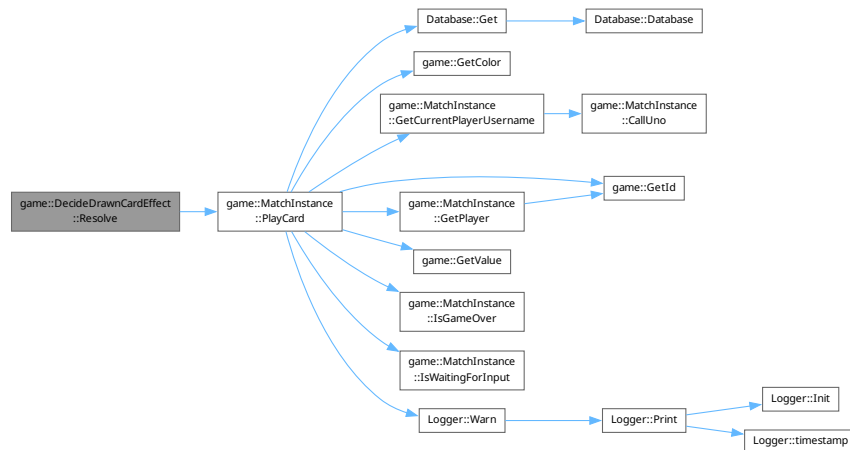
Restituisce

EffectResult Restituisce uno stato kNeedsInput per chiedere al client cosa fare.

Tag Identificativo: EFFECT-STD-MTH-003

Implementa **game::Effect**.

Questo è il grafo delle chiamate per questa funzione:



8.9.4 Documentazione dei membri dato

8.9.4.1 card_id_

```
uint16_t game::DecideDrawnCardEffect::card_id_  [private]
```

L'ID della carta su cui decidere.

8.9.4.2 username_

```
std::string game::DecideDrawnCardEffect::username__ [private]
```

L'utente che deve prendere la decisione.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- `include/game/effects/standard.hpp`
- `src/game/effects/standard.cpp`

8.10 Riferimenti per la classe `game::DecideSwapTargetEffect`

Sospende il gioco per chiedere a un utente con chi desidera scambiare le carte.

```
#include <decide_swap_target.hpp>
```

Diagramma delle classi per `game::DecideSwapTargetEffect`

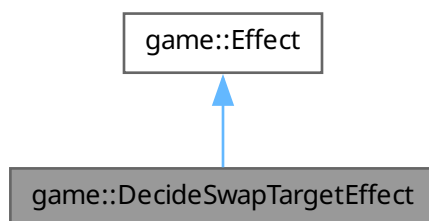
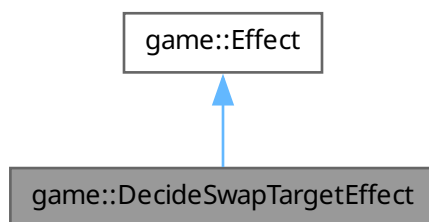


Diagramma di collaborazione per `game::DecideSwapTargetEffect`:



Membri pubblici

- [DecideSwapTargetEffect](#) (std::string username)
Costruttore dell'effetto interattivo.
- [EffectResult Resolve](#) ([GameState](#) *state, [MatchInstance](#) *match) override
Sospende l'esecuzione in attesa dell'input dal giocatore. Se l'input (username avversario) è già stato fornito e memorizzato nello stato, convalida la mossa e procede.
- [EffectType GetType](#) () const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da [game::Effect](#)

- virtual [~Effect](#) ()=default

Attributi privati

- std::string [username__](#)

8.10.1 Descrizione dettagliata

Sospende il gioco per chiedere a un utente con chi desidera scambiare le carte.

- Questo effetto non scambia fisicamente le carte, ma raccoglie l'input dell'utente. Una volta ricevuto l'input (il nome del target), tipicamente accoda un [PassHandsEffect](#) o un effetto simile mirato ai due giocatori coinvolti.

Tag Identificativo: EFFECT-SWAP-001

8.10.2 Documentazione dei costruttori e dei distruttori

8.10.2.1 [DecideSwapTargetEffect\(\)](#)

```
game::DecideSwapTargetEffect::DecideSwapTargetEffect (
    std::string username)  [inline], [explicit]
```

Costruttore dell'effetto interattivo.

Parametri

username	Nome del giocatore a cui spetta la decisione.
----------	---

8.10.3 Documentazione delle funzioni membro

8.10.3.1 GetType()

`EffectType` game::DecideSwapTargetEffect::GetType () const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kDecideSwapTarget.

Tag Identificativo: EFFECT-SWAP-MTH-002

Reimplementa `game::Effect`.

8.10.3.2 Resolve()

`EffectResult` game::DecideSwapTargetEffect::Resolve (
`GameState * state`,
`MatchInstance * match`) [override], [virtual]

Sospende l'esecuzione in attesa dell'input dal giocatore. Se l'input (username avversario) è già stato fornito e memorizzato nello stato, convalida la mossa e procede.

Parametri

state	Lo stato del gioco.
match	Istanza della partita.

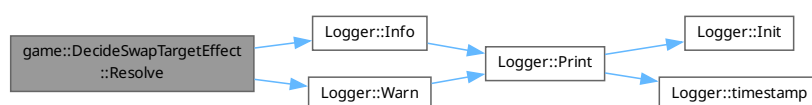
Restituisce

`EffectResult` Restituisce kNeedsInput per interrogare il client.

Tag Identificativo: EFFECT-SWAP-MTH-001

Implementa `game::Effect`.

Questo è il grafo delle chiamate per questa funzione:



8.10.4 Documentazione dei membri dato

8.10.4.1 username_

```
std::string game::DecideSwapTargetEffect::username_ [private]
```

L'utente che deve selezionare il target.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/game/effects/decide_swap_target.hpp](#)
- [src/game/effects/decide_swap_target.cpp](#)

8.11 Riferimenti per la classe `game::DrawEffect`

Effetto penalità/obbligo che fa pescare un numero di carte a un giocatore. Usato per l'inizio del turno, le carte +2, +4, o per penalità da mod.

```
#include <standard.hpp>
```

Diagramma delle classi per `game::DrawEffect`

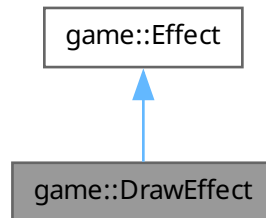
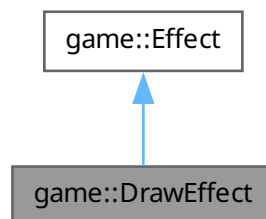


Diagramma di collaborazione per `game::DrawEffect`:



Membri pubblici

- [DrawEffect](#) (int count, const std::string &target_username)
Costruttore dell'effetto di pescaggio.
- [EffectResult Resolve](#) ([GameState](#) *state, [MatchInstance](#) *match_instance) override
Risolve l'effetto prelevando le carte dal mazzo e aggiungendole alla mano del bersaglio. Gestisce automaticamente il rimescolamento del mazzo degli scarti se necessario.
- [EffectType GetType](#) () const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da [game::Effect](#)

- virtual [~Effect](#) ()=default

Attributi privati

- int [count_](#)
- std::string [target_username_](#)

8.11.1 Descrizione dettagliata

Effetto penalità/obbligo che fa pescare un numero di carte a un giocatore. Usato per l'inizio del turno, le carte +2, +4, o per penalità da mod.

Tag Identificativo: EFFECT-STD-003

8.11.2 Documentazione dei costruttori e dei distruttori

8.11.2.1 DrawEffect()

```
game::DrawEffect::DrawEffect (
    int count,
    const std::string & target_username) [inline], [explicit]
```

Costruttore dell'effetto di pescaggio.

Parametri

count	Il numero di carte da far pescare.
target_username	L'utente che subirà l'effetto.

8.11.3 Documentazione delle funzioni membro

8.11.3.1 GetType()

`EffectType` game::DrawEffect::GetType () const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kDraw.

Tag Identificativo: EFFECT-STD-MTH-006

Reimplementa `game::Effect`.

8.11.3.2 Resolve()

`EffectResult` game::DrawEffect::Resolve (
`GameState * state`,
`MatchInstance * match_instance`) [override], [virtual]

Risolve l'effetto prelevando le carte dal mazzo e aggiungendole alla mano del bersaglio. Gestisce automaticamente il rimescolamento del mazzo degli scarti se necessario.

Parametri

state	Lo stato del gioco.
match_instance	Istanza della partita.

Restituisce

`EffectResult` Esito dell'operazione.

Tag Identificativo: EFFECT-STD-MTH-005

Implementa `game::Effect`.

Questo è il grafo delle chiamate per questa funzione:



8.11.4 Documentazione dei membri dato

8.11.4.1 count_

```
int game::DrawEffect::count_ [private]
```

Quantità di carte.

8.11.4.2 target_username_

```
std::string game::DrawEffect::target_username_ [private]
```

Bersaglio dell'effetto.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- `include/game/effects/standard.hpp`
- `src/game/effects/standard.cpp`

8.12 Riferimenti per la classe game::DrawStackingRule

Diagramma delle classi per game::DrawStackingRule

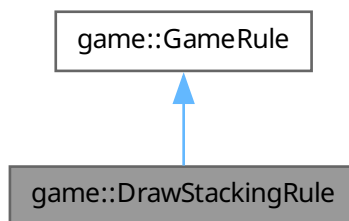
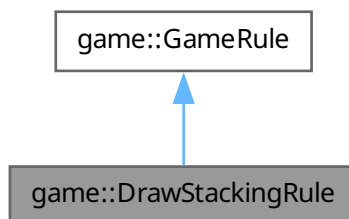


Diagramma di collaborazione per game::DrawStackingRule:



Membri pubblici

- void [ValidatePlay](#) ([GameState](#) *state, [CardPlayedEvent](#) &event) override
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- void [OnCardPlayed](#) ([GameState](#) *state, [CardPlayedEvent](#) &event) override
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se [ValidatePlay](#) ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di [Effect](#) nella coda degli effetti di [GameState](#).

Membri pubblici ereditati da [game::GameRule](#)

- virtual [~GameRule](#) ()=default
- virtual void [OnCardDrawn](#) ([GameState](#) *state, [CardDrawnEvent](#) &event)

8.12.1 Documentazione delle funzioni membro

8.12.1.1 OnCardPlayed()

```
void game::DrawStackingRule::OnCardPlayed (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

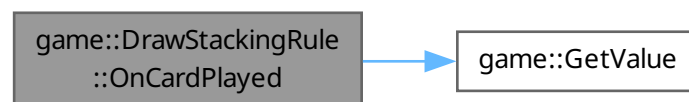
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se [ValidatePlay](#) ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di [Effect](#) nella coda degli effetti di [GameState](#).

Parametri

state	Lo stato attuale del gioco.
event	L'evento contenente la giocata risolta.

Reimplementa [game::GameRule](#).

Questo è il grafo delle chiamate per questa funzione:



8.12.1.2 ValidatePlay()

```
void game::DrawStackingRule::ValidatePlay (  
    GameState * state,  
    CardPlayedEvent & event) [inline], [override], [virtual]
```

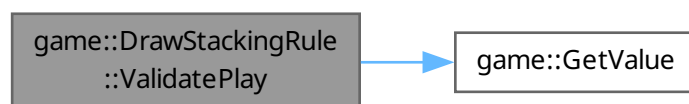
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

Parametri

state	Lo stato attuale del gioco.
event	L'evento che porta i dati della giocata. Il metodo dovrebbe impostare event.is_valid_play a false se la mossa è illegale.

Reimplementa [game::GameRule](#).

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa classe è stata generata a partire dal seguente file:

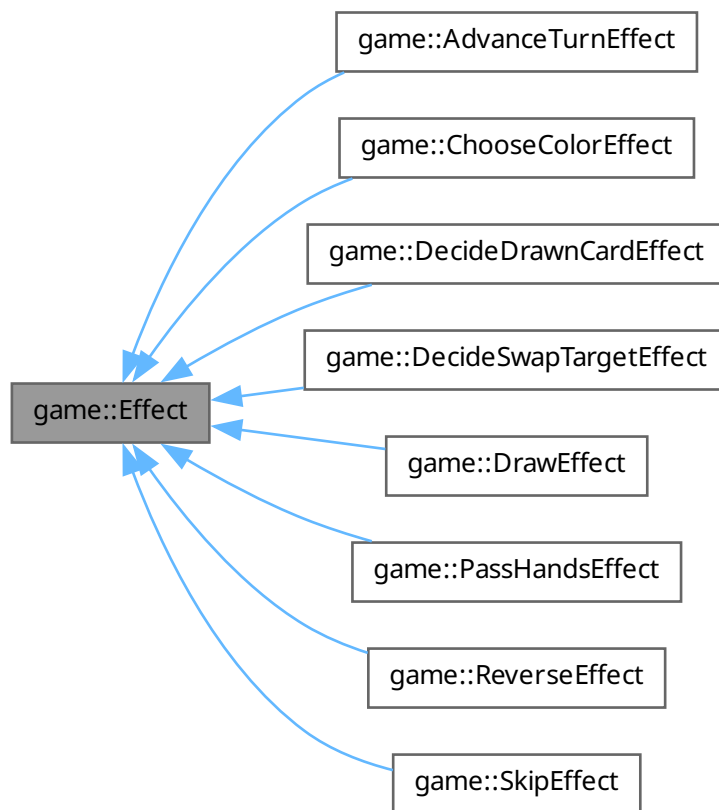
- [src/game/rules/draw_stacking.cpp](#)

8.13 Riferimenti per la classe game::Effect

Classe base astratta per tutti gli effetti applicabili nel gioco.

```
#include <effect.hpp>
```

Diagramma delle classi per game::Effect



Membri pubblici

- virtual `~Effect()`=default
- virtual `EffectType GetType()` const
Recupera il tipo dell'effetto implementato.
- virtual `EffectResult Resolve(GameState *state, MatchInstance *match)=0`
Esegue e applica l'effetto allo stato del gioco attuale.

8.13.1 Descrizione dettagliata

Classe base astratta per tutti gli effetti applicabili nel gioco.

8.13.2 Documentazione dei costruttori e dei distruttori

8.13.2.1 ~Effect()

virtual `game::Effect::~~Effect()` [virtual], [default]

8.13.3 Documentazione delle funzioni membro

8.13.3.1 `GetType()`

```
virtual EffectType game::Effect::GetType () const [inline], [virtual]
```

Recupera il tipo dell'effetto implementato.

Restituisce

[EffectType](#) Identificatore dell'effetto (default a `kCustom`).

Reimplementata in [game::AdvanceTurnEffect](#), [game::ChooseColorEffect](#), [game::DecideDrawnCardEffect](#), [game::DecideSwapTargetEffect](#), [game::DrawEffect](#), [game::ReverseEffect](#), e [game::SkipEffect](#).

8.13.3.2 `Resolve()`

```
virtual EffectResult game::Effect::Resolve (
    GameState * state,
    MatchInstance * match) [pure virtual]
```

Esegue e applica l'effetto allo stato del gioco attuale.

Parametri

<code>state</code>	Puntatore allo stato globale della partita (GameState).
<code>match</code>	Puntatore all'istanza della partita (MatchInstance) per coordinare input/bot.

Restituisce

[EffectResult](#) Il risultato dell'applicazione (risolto, errore o necessita input).

Implementato in [game::AdvanceTurnEffect](#), [game::ChooseColorEffect](#), [game::DecideDrawnCardEffect](#), [game::DecideSwapTargetEffect](#), [game::DrawEffect](#), [game::PassHandsEffect](#), [game::ReverseEffect](#), e [game::SkipEffect](#).

La documentazione per questa classe è stata generata a partire dal seguente file:

- `include/common/game/effect.hpp`

8.14 Riferimenti per la struct `game::EffectResult`

Struttura restituita dopo il tentativo di risoluzione di un effetto.

```
#include <effect.hpp>
```

Attributi pubblici

- [EffectStatus](#) status
- std::string [input_type](#)
- std::string [target_player](#)
- std::string [input_context](#) = ""

8.14.1 Descrizione dettagliata

Struttura restituita dopo il tentativo di risoluzione di un effetto.

8.14.2 Documentazione dei membri dato

8.14.2.1 input_context

```
std::string game::EffectResult::input_context = ""
```

Un payload JSON generico con informazioni contestuali per il frontend.

8.14.2.2 input_type

```
std::string game::EffectResult::input_type
```

Il tipo di input richiesto (se status == kNeedsInput).

8.14.2.3 status

```
EffectStatus game::EffectResult::status
```

Lo stato di risoluzione dell'effetto.

8.14.2.4 target_player

```
std::string game::EffectResult::target_player
```

Il giocatore da cui si attende l'input.

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/common/game/effect.hpp`

8.15 Riferimenti per la struct Error

Incapsula le informazioni riguardanti un errore verificatosi nel sistema.

```
#include <error.hpp>
```

Tipi pubblici

- enum class [Code](#) {
[kBadRequest](#) , [kInvalidInput](#) , [kNotFound](#) , [kUnauthorised](#) ,
[kConflict](#) , [kDatabaseFailure](#) , [kInternalError](#) }

Codici generici che classificano la tipologia di errore.

Memberi pubblici

- std::string [HttpStatus](#) () const

Mappa il codice di errore interno al corrispondente Status [Code](#) HTTP. Utile per rispondere automaticamente al client in modo standard (es. nelle REST API).

Memberi pubblici statici

- static [Error BadRequest](#) (const string &msg)
Crea un errore di tipo [kBadRequest](#).
- static [Error InvalidInput](#) (const string &msg)
Crea un errore di tipo [kInvalidInput](#).
- static [Error NotFound](#) (const string &msg)
Crea un errore di tipo [kNotFound](#).
- static [Error Unauthorised](#) (const string &msg)
Crea un errore di tipo [kUnauthorised](#).
- static [Error Conflict](#) (const string &msg)
Crea un errore di tipo [kConflict](#).
- static [Error DatabaseFail](#) (const string &msg)
Crea un errore di DB [kDatabaseFailure](#).
- static [Error Internal](#) (const string &msg)
Crea un errore generico [kInternalError](#).

Attributi pubblici

- [Code](#) [code](#)
- string [message](#)

8.15.1 Descrizione dettagliata

Incapsula le informazioni riguardanti un errore verificatosi nel sistema.

- Contiene un codice standard interno ed un messaggio descrittivo per un facile instradamento e risposta verso il client.

Tag Identificativo: ERR-STR-001

8.15.2 Documentazione dei tipi enumerati (enum)

8.15.2.1 Code

enum class `Error::Code` [strong]

Codici generici che classificano la tipologia di errore.

Tag Identificativo: ERR-ENUM-001

Valori del tipo enumerato

kBadRequest	Richiesta malformata.
kInvalidInput	Dati in ingresso non validi (es. Type mismatch in JSON).
kNotFound	Risorsa non trovata (es. file o lobby inesistente).
kUnauthorised	Mancanza di permessi o token JWT non valido.
kConflict	Conflitto nello stato delle risorse (es. utente già presente).
kDatabaseFailure	Errore interno al driver SQLite.
kInternalError	Eccezione non gestita o errore di logica del server.

8.15.3 Documentazione delle funzioni membro

8.15.3.1 BadRequest()

`Error` `Error::BadRequest` (
 const string & msg) [inline], [static]

Crea un errore di tipo `kBadRequest`.

Tag Identificativo: ERR-MTH-001

8.15.3.2 Conflict()

`Error` `Error::Conflict` (
 const string & msg) [inline], [static]

Crea un errore di tipo `kConflict`.

Tag Identificativo: ERR-MTH-005

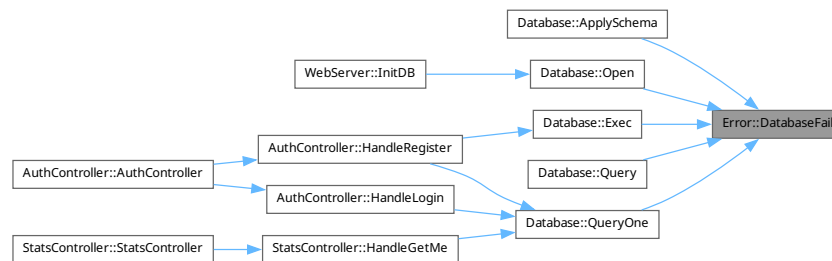
8.15.3.3 DatabaseFail()

Error Error::DatabaseFail (
 const string & msg) [inline], [static]

Crea un errore di DB kDatabaseFailure.

Tag Identificativo: ERR-MTH-006

Questo è il grafo dei chiamanti di questa funzione:



8.15.3.4 HttpStatus()

std::string Error::HttpStatus () const [inline]

Mappa il codice di errore interno al corrispondente Status Code HTTP. Utile per rispondere automaticamente al client in modo standard (es. nelle REST API).

Restituisce

std::string Status code formattato (es. "404 Not Found").

Tag Identificativo: ERR-MTH-008

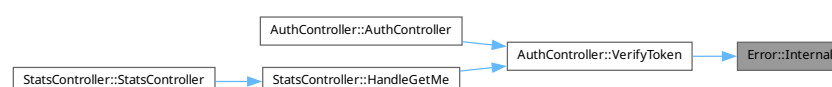
8.15.3.5 Internal()

Error Error::Internal (
 const string & msg) [inline], [static]

Crea un errore generico kInternalError.

Tag Identificativo: ERR-MTH-007

Questo è il grafo dei chiamanti di questa funzione:



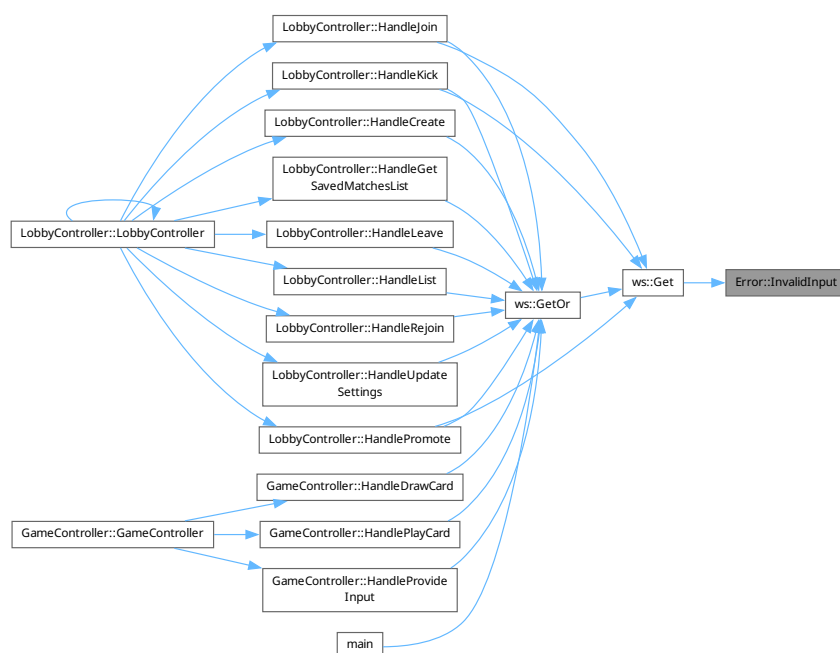
8.15.3.6 InvalidInput()

Error Error::InvalidInput (
 const string & msg) [inline], [static]

Crea un errore di tipo kInvalidInput.

Tag Identificativo: ERR-MTH-002

Questo è il grafo dei chiamanti di questa funzione:



8.15.3.7 NotFound()

Error Error::NotFound (
 const string & msg) [inline], [static]

Crea un errore di tipo kNotFound.

Tag Identificativo: ERR-MTH-003

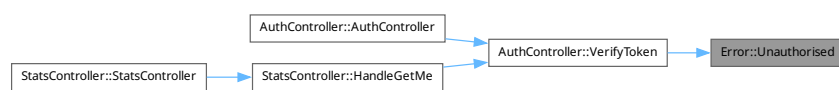
8.15.3.8 Unauthorized()

Error Error::Unauthorised (
const string & msg) [inline], [static]

Crea un errore di tipo kUnauthorised.

Tag Identificativo: ERR-MTH-004

Questo è il grafo dei chiamanti di questa funzione:



8.15.4 Documentazione dei membri dato

8.15.4.1 code

Code Error::code

Tipologia dell'errore.

8.15.4.2 message

string Error::message

Dettaglio testuale umano/leggibile.

La documentazione per questa struct è stata generata a partire dal seguente file:

- include/error.hpp

8.16 Riferimenti per la classe `game::ForcePlayRule`

Diagramma delle classi per `game::ForcePlayRule`

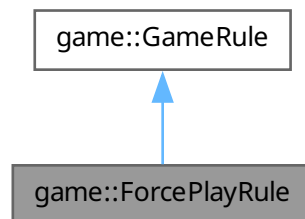
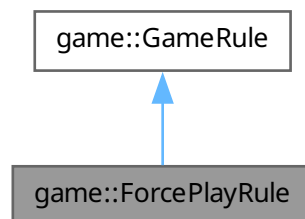


Diagramma di collaborazione per `game::ForcePlayRule`:



Membrì pubblici

- void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event) override

Membrì pubblici ereditati da `game::GameRule`

- virtual `~GameRule` ()=default
- virtual void `ValidatePlay` (`GameState` *state, `CardPlayedEvent` &event)
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- virtual void `OnCardPlayed` (`GameState` *state, `CardPlayedEvent` &event)
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.

8.16.1 Documentazione delle funzioni membro

8.16.1.1 OnCardDrawn()

```
void game::ForcePlayRule::OnCardDrawn (  
    GameState * state,  
    CardDrawnEvent & event) [inline], [override], [virtual]
```

Reimplementa `game::GameRule`.

La documentazione per questa classe è stata generata a partire dal seguente file:

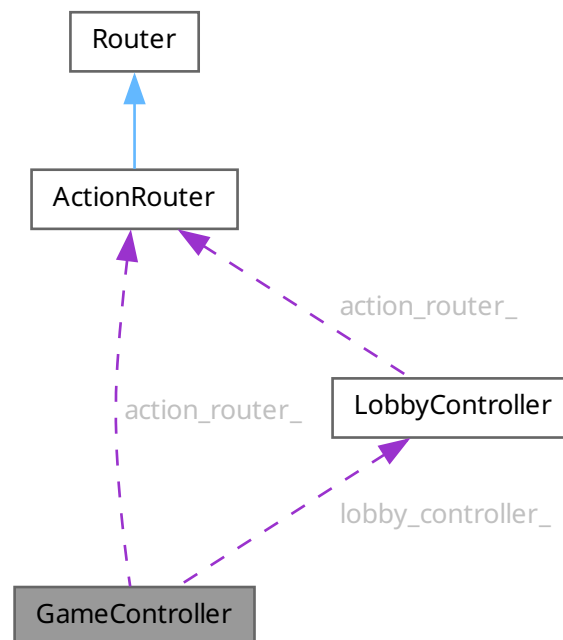
- `src/game/rules/force_play.cpp`

8.17 Riferimenti per la classe GameController

Riceve e processa l'input dei giocatori durante una partita attiva.

```
#include <game_controller.hpp>
```

Diagramma di collaborazione per GameController:



Membri pubblici

- [GameController](#) ([WebServer](#) &server, [LobbyController](#) &lobby_controller)
Costruttore del controller di gioco. Registra gli handler delle azioni WebSocket game_* sull'[ActionRouter](#).

Membri privati

- void [HandlePlayCard](#) ([WsContext](#) context, const nlohmann::json &message)
Gestisce la richiesta di giocare una carta.
- void [HandleDrawCard](#) ([WsContext](#) context, const nlohmann::json &message)
Gestisce la richiesta di pescare una carta dal mazzo.
- void [HandleProvideInput](#) ([WsContext](#) context, const nlohmann::json &message)
Gestisce l'invio di un input da parte del client per un effetto in sospeso (es. scelta colore).
- void [HandleCallUno](#) ([WsContext](#) context, const nlohmann::json &message)
Gestisce la dichiarazione di "UNO" da parte del giocatore.
- void [OnTurnStarted](#) ([Lobby](#) *active_lobby)
Callback/Hook scatenato all'inizio di ogni nuovo turno di un giocatore. Avvia/reimposta il timer per l'AFK in base alle impostazioni della lobby.
- void [BroadcastGameState](#) ([Lobby](#) *current_lobby)
Invia lo stato del gioco censurato a tutti i membri connessi. Iterando su ogni utente, usa Match←Instance::SerializePlayerState per nascondere le mani avversarie e inoltra il messaggio WS.
- void [SetTurnTimer](#) (uint32_t lobby_id, int timeout_ms, std::function< void()> callback)
Crea o aggiorna il timer del turno per la protezione AFK.
- void [ClearTurnTimer](#) (uint32_t lobby_id)
Ferma e distrugge il timer attivo per una determinata lobby. Usato quando un giocatore esegue un'azione valida in tempo o la partita finisce.

Attributi privati

- [ActionRouter](#) & action_router_
- [LobbyController](#) & lobby_controller_
- std::unordered_map< uint32_t, struct us_timer_t * > [active_turn_timers_](#)
Mappa che associa l'ID di una [Lobby](#) al puntatore del timer libuv corrente. Utilizzato per attivare la protezione AFK (es. il bot subentra o la mossa viene saltata).

8.17.1 Descrizione dettagliata

Riceve e processa l'input dei giocatori durante una partita attiva.

- Lavora a stretto contatto con il [LobbyController](#) per identificare la partita associata all'utente. Questa classe è anche responsabile del ciclo di vita dei timer dei turni (libuv) per gestire i giocatori AFK o il takeover dei bot.

Tag Identificativo: CTRL-GAME-001

Invia lo stato del gioco censurato a tutti i membri connessi. Iterando su ogni utente, usa `MatchInstance::SerializePlayerState` per nascondere le mani avversarie e inoltra il messaggio WS.

Serializes complete match configurations, sending unique filtered game boards down to each user.

Parametri

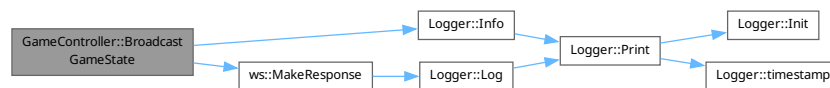
<code>current_lobby</code>	Puntatore alla lobby da aggiornare.
----------------------------	-------------------------------------

Tag Identificativo: CTRL-GAME-FLW-002

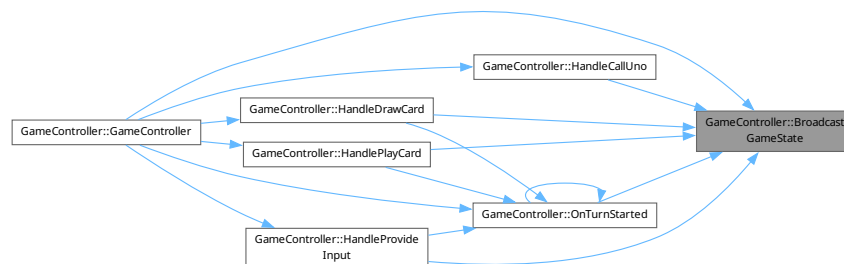
Parametri

<code>current_lobby</code>	Target room pointer whose context needs to be synchronized.
----------------------------	---

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.2 ClearTurnTimer()

```
void GameController::ClearTurnTimer (
    uint32_t lobby_id) [private]
```

Ferma e distrugge il timer attivo per una determinata lobby. Usato quando un giocatore esegue un'azione valida in tempo o la partita finisce.

Disarms and frees loop memory allocations associated with an outstanding turn timer.

Parametri

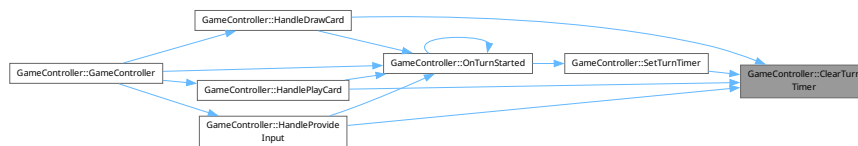
lobby_id	ID della lobby di cui cancellare il timer.
----------	--

Tag Identificativo: CTRL-GAME-TMR-002

Parametri

lobby_id	Target internal tracker key identifying the active timer slot.
----------	--

Questo è il grafo dei chiamanti di questa funzione:



8.17.3.3 HandleCallUno()

```
void GameController::HandleCallUno (
    WsContext context,
    const nlohmann::json & message) [private]
```

Gestisce la dichiarazione di "UNO" da parte del giocatore.

Flags a player's safety verification state within the game rule evaluation layers.

Parametri

context	Contesto del socket chiamante.
message	Payload JSON della richiesta.

Tag Identificativo: CTRL-GAME-ACT-004

Parametri

context	Signaling packet metadata tracking incoming user sockets.
message	Received payload data map document.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.4 HandleDrawCard()

```
void GameController::HandleDrawCard (
    WsContext context,
    const nlohmann::json & message) [private]
```

Gestisce la richiesta di pescare una carta dal mazzo.

Signals the running match instance engine to allocate a fresh card to the requesting user.

Parametri

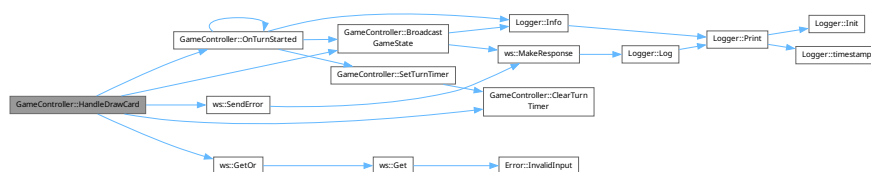
context	Contesto del socket chiamante.
message	Payload JSON della richiesta.

Tag Identificativo: CTRL-GAME-ACT-002

Parametri

context	Signaling packet metadata tracking incoming user sockets.
message	JSON input structure containing callback identifiers.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.5 HandlePlayCard()

```
void GameController::HandlePlayCard (
    WsContext context,
    const nlohmann::json & message) [private]
```

Gestisce la richiesta di giocare una carta.

Intercepts explicit match transaction inputs, routing candidate card identifiers to the engine.

Parametri

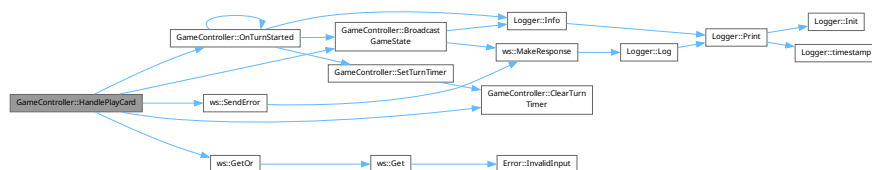
context	Contesto del socket chiamante.
message	Payload JSON contenente il campo card_id.

Tag Identificativo: CTRL-GAME-ACT-001

Parametri

context	Signaling packet metadata tracking incoming user sockets.
message	JSON input structure carrying data properties.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.6 HandleProvideInput()

```
void GameController::HandleProvideInput (
    WsContext context,
    const nlohmann::json & message) [private]
```

Gestisce l'invio di un input da parte del client per un effetto in sospeso (es. scelta colore).

Forwards localized resolution responses (like color selections) down into pending effect queues.

Parametri

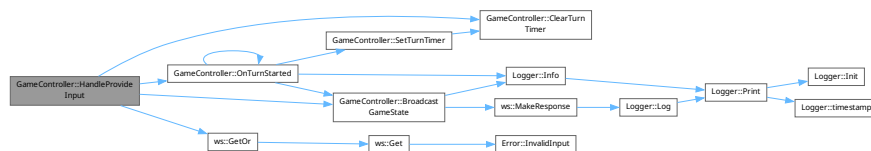
context	Contesto del socket chiamante.
message	Payload JSON contenente il dato scelto.

Tag Identificativo: CTRL-GAME-ACT-003

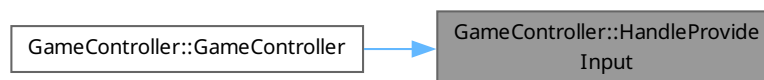
Parametri

context	Signaling packet metadata tracking incoming user sockets.
message	Input document carrying state selections.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.7 OnTurnStarted()

```
void GameController::OnTurnStarted (
    Lobby * active_lobby) [private]
```

Callback/Hook scatenato all'inizio di ogni nuovo turno di un giocatore. Avvia/reimposta il timer per l'AFK in base alle impostazioni della lobby.

Sets turn constraints, routing automatic simulation threads on bot turns or player timeouts.

Parametri

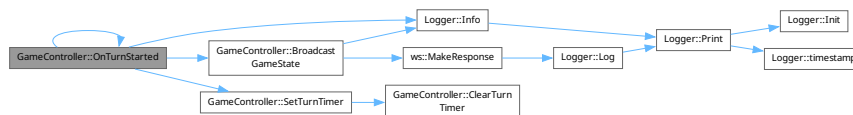
active_lobby	Puntatore alla lobby di cui è iniziato il turno.
--------------	--

Tag Identificativo: CTRL-GAME-FLW-001

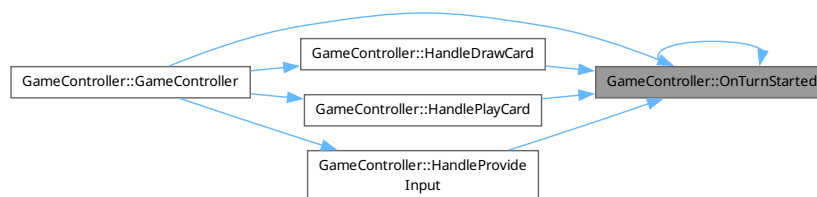
Parametri

active_lobby	Target active game room layout evaluated.
--------------	---

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.3.8 SetTurnTimer()

```
void GameController::SetTurnTimer (
    uint32_t lobby_id,
    int timeout_ms,
    std::function< void()> callback) [private]
```

Crea o aggiorna il timer del turno per la protezione AFK.

Schedules a single-shot internal event loop timer to execute a deferred task.

Parametri

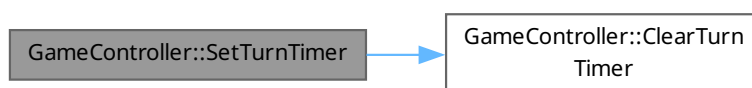
lobby_id	ID della lobby in corso.
timeout_ms	Millisecondi prima dello scatto del timeout (es. 15000ms).
callback	Funzione da eseguire alla scadenza del timer (es. auto-play o bot).

Tag Identificativo: CTRL-GAME-TMR-001

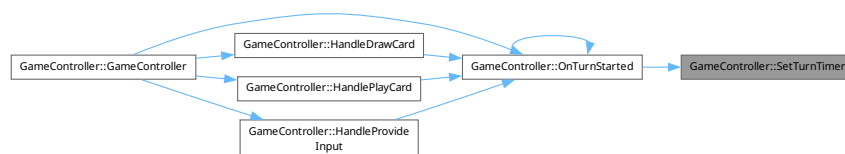
Parametri

lobby_id	Numerical identifier targeting a specific active room map.
timeout_ms	Length of duration in milliseconds before firing execution.
callback	Target functional callback block fired upon timeout completion.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.17.4 Documentazione dei membri dato

8.17.4.1 action_router_

[ActionRouter](#)& GameController::action_router_ [private]

Riferimento al router WebSocket.

8.17.4.2 active_turn_timers_

std::unordered_map<uint32_t, struct us_timer_t*> GameController::active_turn_timers_ [private]

Mappa che associa l'ID di una [Lobby](#) al puntatore del timer libuv corrente. Utilizzato per attivare la protezione AFK (es. il bot subentra o la mossa viene saltata).

-

8.17.4.3 lobby_controller_

[LobbyController](#)& GameController::lobby_controller_ [private]

Riferimento per accedere alle lobby in memoria.

La documentazione per questa classe è stata generata a partire dai seguenti file:

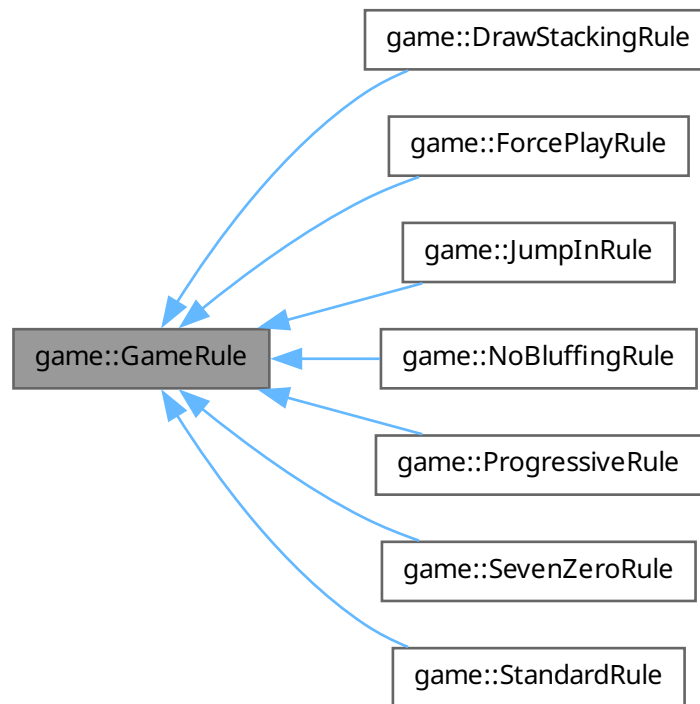
- include/controllers/[game_controller.hpp](#)
- src/controllers/[game_controller.cpp](#)

8.18 Riferimenti per la classe game::GameRule

Interfaccia base che definisce il ciclo di vita per l'interpretazione di una giocata.

```
#include <gamerule.hpp>
```

Diagramma delle classi per game::GameRule



Membri pubblici

- virtual `~GameRule()` = default
- virtual void `ValidatePlay(GameState *state, CardPlayedEvent &event)`
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- virtual void `OnCardPlayed(GameState *state, CardPlayedEvent &event)`
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.
- virtual void `OnCardDrawn(GameState *state, CardDrawnEvent &event)`

8.18.1 Descrizione dettagliata

Interfaccia base che definisce il ciclo di vita per l'interpretazione di una giocata.

8.18.2 Documentazione dei costruttori e dei distruttori

8.18.2.1 ~GameRule()

virtual game::GameRule::~~GameRule() [virtual], [default]

8.18.3 Documentazione delle funzioni membro

8.18.3.1 OnCardDrawn()

```
virtual void game::GameRule::OnCardDrawn (
    GameState * state,
    CardDrawnEvent & event) [inline], [virtual]
```

Reimplementata in [game::ForcePlayRule](#), e [game::ProgressiveRule](#).

8.18.3.2 OnCardPlayed()

```
virtual void game::GameRule::OnCardPlayed (
    GameState * state,
    CardPlayedEvent & event) [inline], [virtual]
```

Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di [Effect](#) nella coda degli effetti di [GameState](#).

Parametri

state	Lo stato attuale del gioco.
event	L'evento contenente la giocata risolta.

Reimplementata in [game::DrawStackingRule](#), [game::JumpInRule](#), [game::SevenZeroRule](#), e [game::StandardRule](#).

8.18.3.3 ValidatePlay()

```
virtual void game::GameRule::ValidatePlay (
    GameState * state,
    CardPlayedEvent & event) [inline], [virtual]
```

Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

Parametri

state	Lo stato attuale del gioco.
event	L'evento che porta i dati della giocata. Il metodo dovrebbe impostare <code>event.is_valid_play</code> a false se la mossa è illegale.

Reimplementata in [game::DrawStackingRule](#), [game::JumpInRule](#), [game::NoBluffingRule](#), [game::SevenZeroRule](#), e [game::StandardRule](#).

La documentazione per questa classe è stata generata a partire dal seguente file:

- `include/common/game/gamerule.hpp`

8.19 Riferimenti per la struct `game::GameState`

Struttura dati monolitica che contiene l'intera "fotografia" della partita ad un dato istante. Viene manipolata dalle classi [Effect](#) e [MatchInstance](#) e serializzata in JSON.

```
#include <game_state.hpp>
```

Diagramma di collaborazione per `game::GameState`:



Attributi pubblici

- [MatchStatus](#) `status` = [MatchStatus::kWaitingToStart](#)
- `std::string` `winner`
- `std::vector< Player >` `players`
- `int` `current_player_index` = 0
- `int` `play_direction` = 1
- `int` `pending_draws` = 0
- [LastPlay](#) `last_play`
- `std::vector< CompactCard >` `draw_pile`
- `std::vector< CompactCard >` `discard_pile`
- [Color](#) `active_color` = [Color::kRed](#)
- `std::deque< std::unique_ptr< Effect > >` `effect_queue`
- `std::string` `pending_player`
- `std::string` `pending_input_type`
- `std::string` `pending_input_context`
- `std::string` `provided_input`
- `std::chrono::steady_clock::time_point` `turn_end_time`

8.19.1 Descrizione dettagliata

Struttura dati monolitica che contiene l'intera "fotografia" della partita ad un dato istante. Viene manipolata dalle classi [Effect](#) e [MatchInstance](#) e serializzata in JSON.

Tag Identificativo: GAME-STRUCT-003

8.19.2 Documentazione dei membri dato

8.19.2.1 `active_color`

`Color` `game::GameState::active_color` = `Color::kRed`

Colore attualmente obbligatorio per rispondere (cambia con carte Jolly).

8.19.2.2 `current_player_index`

`int` `game::GameState::current_player_index` = 0

Indice del giocatore con il turno attivo.

8.19.2.3 `discard_pile`

`std::vector<CompactCard>` `game::GameState::discard_pile`

Mazzo degli scarti al centro del tavolo.

8.19.2.4 `draw_pile`

`std::vector<CompactCard>` `game::GameState::draw_pile`

Mazzo delle carte da cui pescare.

8.19.2.5 `effect_queue`

`std::deque<std::unique_ptr<Effect> >` `game::GameState::effect_queue`

Coda di effetti da risolvere (Architecture asincrona per mosse a catena).

8.19.2.6 `last_play`

`LastPlay` `game::GameState::last_play`

Provenienza dell'ultima carta giocata, usata per le animazioni client.

8.19.2.7 `pending_draws`

`int` `game::GameState::pending_draws` = 0

Carte accumulate che il prossimo giocatore dovrà pescare (es. catena +2/+4).

8.19.2.8 pending_input_context

`std::string game::GameState::pending_input_context`

Payload JSON aggiuntivo per aiutare il client nel rendering dell'input box.

8.19.2.9 pending_input_type

`std::string game::GameState::pending_input_type`

Tipo di input richiesto (es. "choose_color").

8.19.2.10 pending_player

`std::string game::GameState::pending_player`

L'utente che DEVE fornire un input prima che il gioco continui.

8.19.2.11 play_direction

`int game::GameState::play_direction = 1`

Direzione di gioco (1 = orario, -1 = antiorario).

8.19.2.12 players

`std::vector<Player> game::GameState::players`

Lista ordinata (per turno) dei giocatori.

8.19.2.13 provided_input

`std::string game::GameState::provided_input`

Risposta asincrona memorizzata appena inviata dal giocatore per l'effetto.

8.19.2.14 status

`MatchStatus game::GameState::status = MatchStatus::kWaitingToStart`

Lo stato di progressione.

8.19.2.15 turn_end_time

`std::chrono::steady_clock::time_point game::GameState::turn_end_time`

Timestamp in cui scatterà l'AFK/Timeout del turno corrente.

8.19.2.16 winner

```
std::string game::GameState::winner
```

Username del vincitore (se conclusa).

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/game/game_state.hpp`

8.20 Riferimenti per la classe HttpRouter

Gestisce la registrazione e il routing delle chiamate HTTP (GET, POST e middleware).

```
#include <http_router.hpp>
```

Diagramma delle classi per HttpRouter

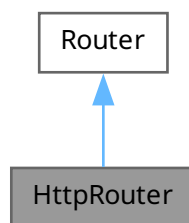
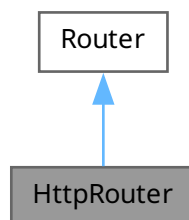


Diagramma di collaborazione per HttpRouter:



Composti

- struct `Route`

Struttura interna che memorizza temporaneamente le rotte prima del caricamento.

Membri pubblici

- [HttpRouter](#) ()=default
- [HttpRouter](#) & [Get](#) (const std::string &path, [HttpHandler](#) handler)
Registra un gestore per una rotta in GET. Supporta i percorsi con wildcard nativi di uWS (es. `"/\/*"`).
- [HttpRouter](#) & [Post](#) (const std::string &path, [HttpHandler](#) handler)
Registra un gestore per una rotta in POST.
- [HttpRouter](#) & [OnAny](#) (std::function< bool([ApiResponse](#) *, [AppRequest](#) *)> handler)
Registra un middleware che viene eseguito prima di qualsiasi richiesta HTTP. Molto utile per configurare logging globali o Header CORS.
- void [Attach](#) (uWS::SSLApp &app)
Applica tutte le rotte accumulate (ed i relativi middleware) all'istanza nativa di uWebSockets. Deve essere chiamato esattamente una volta dal [WebServer](#) prima di `Run()`.

Membri pubblici ereditati da [Router](#)

- virtual [~Router](#) ()=default
- [Router](#) (const [Router](#) &)=delete
Elimina il costruttore di copia.
- [Router](#) & [operator=](#) (const [Router](#) &)=delete
Elimina l'operatore di assegnamento.

Membri privati

- [HttpHandler](#) [WrapWithWildcards](#) ([HttpHandler](#) handler)
Avvolge un handler specifico affinché esegua prima le wildcard registrate.

Attributi privati

- std::vector< [Route](#) > [routes__](#)
- std::vector< std::function< bool([ApiResponse](#) *, [AppRequest](#) *)> > [wildcards__](#)

Altri membri ereditati

Membri protetti ereditati da [Router](#)

- [Router](#) ()=default
Costruttore protetto, istanziabile solo dalle classi derivate.

8.20.1 Descrizione dettagliata

Gestisce la registrazione e il routing delle chiamate HTTP (GET, POST e middleware).

- L'approccio differisce leggermente dall'[ActionRouter](#) perché uWebSockets richiede di registrare esplicitamente le rotte HTTP direttamente sulla sua istanza nativa.

Tag Identificativo: HTTP-RTR-CLS-001

8.20.2 Documentazione dei costruttori e dei distruttori

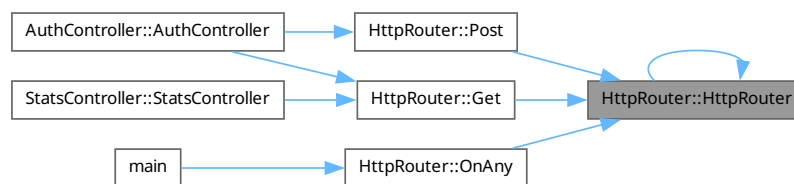
8.20.2.1 `HttpRouter()`

`HttpRouter::HttpRouter ()` [default]

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.20.3 Documentazione delle funzioni membro

8.20.3.1 `Attach()`

```
void HttpRouter::Attach (
    uWS::SSLApp & app)
```

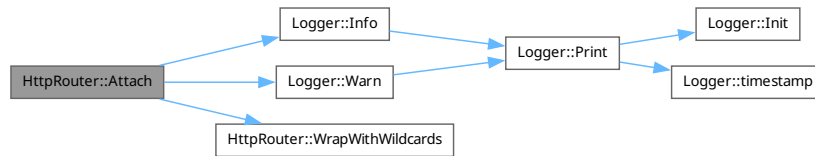
Applica tutte le rotte accumulate (ed i relativi middleware) all'istanza nativa di `uWebSockets`. Deve essere chiamato esattamente una volta dal [WebServer](#) prima di `Run()`.

Parametri

app	Riferimento all'istanza <code>uWebSockets</code> corrente.
-----	--

Tag Identificativo: HTTP-RTR-MTH-004

Questo è il grafo delle chiamate per questa funzione:



8.20.3.2 Get()

`HttpRouter` & `HttpRouter::Get` (
 const std::string & path,
`HttpHandler` handler)

Registra un gestore per una rotta in GET. Supporta i percorsi con wildcard nativi di uWS (es. `"/\/*"`).

Parametri

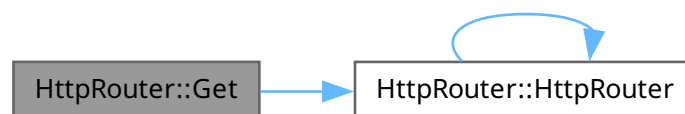
path	Il percorso URI, deve iniziare con <code>"/"</code> .
handler	La funzione da eseguire.

Restituisce

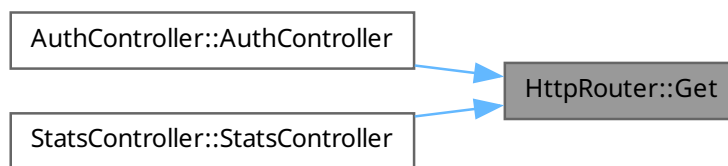
`HttpRouter`& Riferimento per il chaining.

Tag Identificativo: HTTP-RTR-MTH-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.20.3.3 `OnAny()`

```

HttpRouter & HttpRouter::OnAny (
    std::function< bool(AppResponse *, AppRequest *)> handler)
  
```

Registra un middleware che viene eseguito prima di qualsiasi richiesta HTTP. Molto utile per configurare logging globali o Header CORS.

- **ATTENZIONE:** Se restituisce false per interrompere la catena, il middleware deve occuparsi direttamente di inviare la risposta HTTP al client.

Parametri

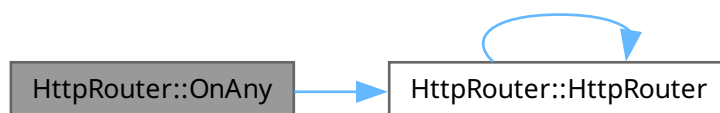
handler	La funzione middleware.
---------	-------------------------

Restituisce

[HttpRouter](#)& Riferimento per il chaining.

Tag Identificativo: HTTP-RTR-MTH-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.20.3.4 Post()

[HttpRouter](#) & [HttpRouter::Post](#) (
const std::string & path,
[HttpHandler](#) handler)

Registra un gestore per una rotta in POST.

Parametri

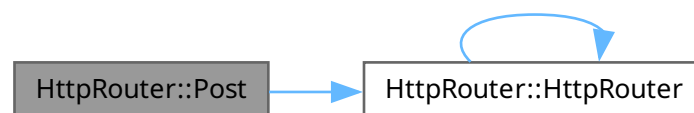
path	Il percorso URI, deve iniziare con ”/”.
handler	La funzione da eseguire.

Restituisce

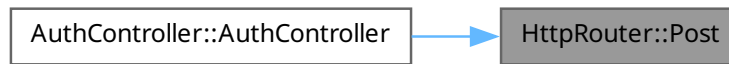
[HttpRouter](#)& Riferimento per il chaining.

Tag Identificativo: HTTP-RTR-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.20.3.5 `WrapWithWildcards()`

```
HttpHandler HttpRouter::WrapWithWildcards (  
    HttpHandler handler) [private]
```

Avvolge un handler specifico affinché esegua prima le wildcard registrate.

Parametri

handler	L'handler originale da decorare.
---------	----------------------------------

Restituisce

[HttpHandler](#) L'handler impacchettato con la logica dei middleware.

Tag Identificativo: HTTP-RTR-PRIV-001

Questo è il grafo dei chiamanti di questa funzione:



8.20.4 Documentazione dei membri dato

8.20.4.1 `routes_`

```
std::vector<Route> HttpRouter::routes_ [private]
```

8.20.4.2 wildcards_

```
std::vector<std::function<bool(AppResponse\*, AppRequest\*)> > HttpRouter::wildcards_ [private]
```

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/[http_router.hpp](#)
- src/[http_router.cpp](#)

8.21 Riferimenti per la classe `game::JumpInRule`

Diagramma delle classi per `game::JumpInRule`

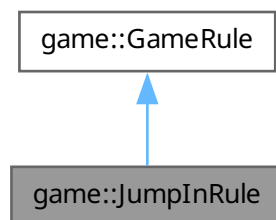
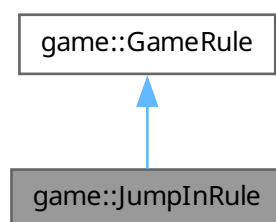


Diagramma di collaborazione per `game::JumpInRule`:



Membri pubblici

- void [ValidatePlay](#) ([GameState](#) *state, [CardPlayedEvent](#) &event) override
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- void [OnCardPlayed](#) ([GameState](#) *state, [CardPlayedEvent](#) &event) override
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se [ValidatePlay](#) ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di [Effect](#) nella coda degli effetti di [GameState](#).

Membri pubblici ereditati da `game::GameRule`

- virtual `~GameRule()` = default
- virtual void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event)

8.21.1 Documentazione delle funzioni membro

8.21.1.1 `OnCardPlayed()`

```
void game::JumpInRule::OnCardPlayed (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

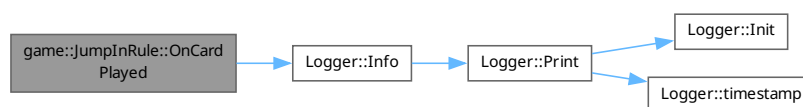
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.

Parametri

state	Lo stato attuale del gioco.
event	L'evento contenente la giocata risolta.

Reimplementa `game::GameRule`.

Questo è il grafo delle chiamate per questa funzione:



8.21.1.2 `ValidatePlay()`

```
void game::JumpInRule::ValidatePlay (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

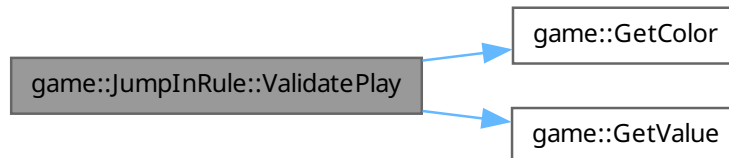
Parametri

state	Lo stato attuale del gioco.
-------	-----------------------------

event	L'evento che porta i dati della giocata. Il metodo dovrebbe impostare event.is_valid_play a false se la mossa è illegale.
-------	---

Reimplementa `game::GameRule`.

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa classe è stata generata a partire dal seguente file:

- `src/game/rules/jump_in.cpp`

8.22 Riferimenti per la struct JwtPayload

Rappresenta i dati (payload) estratti da un token JWT verificato con successo.

```
#include <auth_controller.hpp>
```

Attributi pubblici

- `std::string username`

8.22.1 Descrizione dettagliata

Rappresenta i dati (payload) estratti da un token JWT verificato con successo.

Tag Identificativo: CTRL-AUTH-STR-001

8.22.2 Documentazione dei membri dato

8.22.2.1 username

`std::string JwtPayload::username`

Il nome identificativo dell'utente autenticato (campo 'sub' del token).

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/controllers/auth_controller.hpp`

8.23 Riferimenti per la struct `game::LastPlay`

Descrive l'ultima carta giocata e la sua provenienza, così che il client possa animare la carta dalla specifica posizione (slot) della mano di chi l'ha giocata.

```
#include <game_state.hpp>
```

Attributi pubblici

- `bool valid = false`
- `std::string player`
- `int hand_index = -1`
- `CompactCard card {}`

8.23.1 Descrizione dettagliata

Descrive l'ultima carta giocata e la sua provenienza, così che il client possa animare la carta dalla specifica posizione (slot) della mano di chi l'ha giocata.

Tag Identificativo: GAME-STRUCT-004

8.23.2 Documentazione dei membri dato

8.23.2.1 card

`CompactCard game::LastPlay::card {}`

La carta giocata (id/colore/valore compatti).

8.23.2.2 hand_index

```
int game::LastPlay::hand_index = -1
```

Indice nello slot della mano da cui è partita la carta.

8.23.2.3 player

```
std::string game::LastPlay::player
```

Username di chi ha giocato la carta.

8.23.2.4 valid

```
bool game::LastPlay::valid = false
```

True se contiene una giocata valida da animare.

La documentazione per questa struct è stata generata a partire dal seguente file:

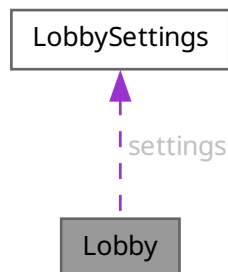
- [include/game/game_state.hpp](#)

8.24 Riferimenti per la struct Lobby

Aggrega l'intero stato strutturale di una stanza di gioco.

```
#include <lobby.hpp>
```

Diagramma di collaborazione per Lobby:



Attributi pubblici

- `uint32_t id`
- `std::string invite_code`
- `std::string host`
- `std::vector< LobbyMember > members`
- `std::string name`
- `LobbySettings settings`
- `std::unique_ptr< game::MatchInstance > match`

Istanza del motore di gioco. Se `nullptr`, la lobby è in fase di attesa. Se popolato, una partita è correntemente in corso.

8.24.1 Descrizione dettagliata

Aggrega l'intero stato strutturale di una stanza di gioco.

Tag Identificativo: CMN-LOBBY-STR-003

8.24.2 Documentazione dei membri dato

8.24.2.1 host

`std::string Lobby::host`

Username del creatore/host attuale della lobby.

8.24.2.2 id

`uint32_t Lobby::id`

Identificativo numerico interno univoco della lobby.

8.24.2.3 invite_code

`std::string Lobby::invite_code`

Codice di invito a 6 caratteri alfanumerici.

8.24.2.4 match

`std::unique_ptr<game::MatchInstance> Lobby::match`

Istanza del motore di gioco. Se `nullptr`, la lobby è in fase di attesa. Se popolato, una partita è correntemente in corso.

•

8.24.2.5 members

```
std::vector<LobbyMember> Lobby::members
```

Lista dei partecipanti (Umani e Bot).

8.24.2.6 name

```
std::string Lobby::name
```

Nome testuale della lobby (per visualizzazione in lista).

8.24.2.7 settings

```
LobbySettings Lobby::settings
```

Impostazioni attuali della partita in questa lobby.

La documentazione per questa struct è stata generata a partire dal seguente file:

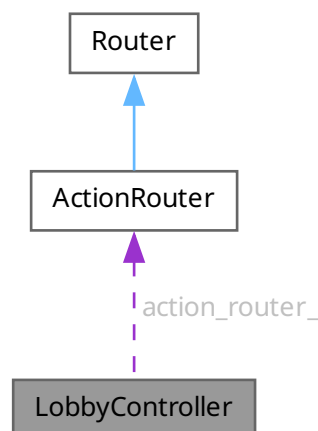
- `include/common/lobby.hpp`

8.25 Riferimenti per la classe LobbyController

Gestisce lo stato in memoria di tutte le lobby attive.

```
#include <lobby_controller.hpp>
```

Diagramma di collaborazione per LobbyController:



Membri pubblici

- [LobbyController](#) ([WebServer](#) &webserver)
Costruttore del [LobbyController](#). Registra gli handler delle azioni WebSocket sul router fornito. Mantiene un riferimento a app per i broadcast pub/sub e il timer di sfratto.
- [~LobbyController](#) ()
Distruttore. Si occupa di ripulire i timer libuv associati.
- void [OnOpen](#) ([AppWebSocket](#) *ws, [PerSocketData](#) *sd)
Handler chiamato quando un client stabilisce una nuova connessione WebSocket. Se l'utente era già in una lobby (scenario di riconnessione), aggiorna il puntatore del socket e ripristina la sua presenza.
- void [OnClose](#) ([AppWebSocket](#) *ws, [PerSocketData](#) *sd)
Handler chiamato quando un client chiude la connessione. Segna il membro come disconnesso e registra il timestamp. Non rimuove immediatamente l'utente, ma avvia un periodo di tolleranza (grace period).
- [Lobby](#) * [GetLobbyByCode](#) (const std::string &code)
Recupera un puntatore a una lobby tramite il suo codice di invito.
- [Lobby](#) * [GetLobbyById](#) (const uint32_t id)
Recupera un puntatore a una lobby tramite il suo ID numerico interno.
- void [OnGameStarted](#) ([GameStartedCallback](#) callback)
Registra una callback da eseguire all'inizio di una partita.
- void [OnPlayerReplaced](#) ([PlayerReplacedCallback](#) callback)
Registra una callback da eseguire quando un giocatore viene sostituito.
- void [SaveMatchStateToDB](#) ([Lobby](#) &lobby)
Salva lo stato corrente della partita associata alla lobby nel database.

Attributi pubblici statici

- static constexpr int [kReconnectGraceMs](#) = 1'000 * 30
Tempo di tolleranza (grace period) prima che un utente disconnesso venga espulso.
- static constexpr int [kMaxMembers](#) = 4
Numero massimo di giocatori consentiti in una singola lobby.

Membri privati

- void [SyncBots](#) ([Lobby](#) &lobby)
Sincronizza lo stato dei bot all'interno della lobby.
- void [CheckMatchIntegrity](#) ([Lobby](#) &lobby)
Verifica l'integrità dei dati della partita corrente (es. riallineamento host).
- void [HandleCreate](#) ([WsContext](#) ctx, const [json](#) &msg)
Gestisce la richiesta di creazione di una nuova lobby.
- void [HandleJoin](#) ([WsContext](#) ctx, const [json](#) &msg)
Gestisce la richiesta di unione a una lobby tramite codice.
- void [HandleRejoin](#) ([WsContext](#) ctx, const [json](#) &msg)
Gestisce la richiesta di riconnessione a una lobby.
- void [HandleLeave](#) ([WsContext](#) ctx, const [json](#) &msg)
Gestisce l'uscita volontaria di un utente dalla lobby.
- void [HandleList](#) ([WsContext](#) ctx, const [json](#) &msg)
Gestisce la richiesta della lista di lobby pubbliche attive.
- void [HandlePromote](#) ([WsContext](#) ctx, const [json](#) &msg)
Promuove un giocatore specifico al ruolo di Host della lobby.
- void [HandleKick](#) ([WsContext](#) ctx, const [json](#) &msg)

- Espelle un giocatore dalla lobby (eseguibile solo dall'Host).
- void [HandleUpdateSettings](#) ([WsContext](#) ctx, const [json](#) &msg)
Aggiorna le impostazioni della lobby (eseguibile solo dall'Host).
- void [HandleGetSavedMatchesList](#) ([WsContext](#) ctx, const [json](#) &msg)
Richiede la lista delle partite salvate precedentemente.
- void [HandleResumeSavedMatch](#) ([WsContext](#) ctx, const [json](#) &msg)
Riprende una partita salvata a partire dal suo ID.
- void [HandleDeleteSavedMatch](#) ([WsContext](#) ctx, const [json](#) &msg)
Elimina in modo permanente una partita salvata dal DB.
- void [HandleStartGame](#) ([WsContext](#) context, const nlohmann::json &message)
Avvia la partita per la lobby corrente. Inizializza l'istanza di gioco.
- void [BroadcastUpdate](#) (const [Lobby](#) &lobby) const
Invia in broadcast l'evento lobby_updated a tutti i membri connessi.
- bool [RemoveMember](#) (uint32_t lobby_id, const string &username, bool explicit_leave=true, const string &request_id="")
Rimuove un membro dalla lobby. Distrugge la lobby se risulta vuota.
- [Lobby](#) * [FindLobbyForUser](#) (const std::string &username)
Trova a quale lobby appartiene attualmente un dato utente.

Membri privati statici

- static std::string [GenerateInviteCode](#) ()
Genera un codice alfanumerico univoco di 6 caratteri (Invite Code).
- static [json](#) [MemberListJson](#) (const [Lobby](#) &lobby)
Serializza la lista dei membri in formato JSON per l'invio via WebSocket.

Attributi privati

- [GameStartedCallback](#) game_started_callback__
- [PlayerReplacedCallback](#) player_replaced_callback__
- [ActionRouter](#) & action_router__
- uWS::SSLApp & app__
- std::unordered_map< uint32_t, [Lobby](#) > lobbies__
- std::unordered_map< std::string, uint32_t > code_to_id__
- std::atomic< uint32_t > next_id__ {1}
- us_timer_t * eviction_timer__ = nullptr

8.25.1 Descrizione dettagliata

Gestisce lo stato in memoria di tutte le lobby attive.

- Questa classe non utilizza un database: le lobby sono effimere e vivono solo per la durata della sessione del server. Gestisce la creazione, l'unione, le disconnessioni temporanee e la riconnessione dei client tramite WebSocket.

Tag Identificativo: LOBBY-CTRL-000

-

8.25.2 Documentazione dei costruttori e dei distruttori

8.25.2.1 LobbyController()

LobbyController::LobbyController (
[WebServer](#) & server)

Costruttore del [LobbyController](#). Registra gli handler delle azioni WebSocket sul router fornito. Mantiene un riferimento a app per i broadcast pub/sub e il timer di sfratto.

Constructs the [LobbyController](#) instance and establishes central inbound routing maps.

- Parametri

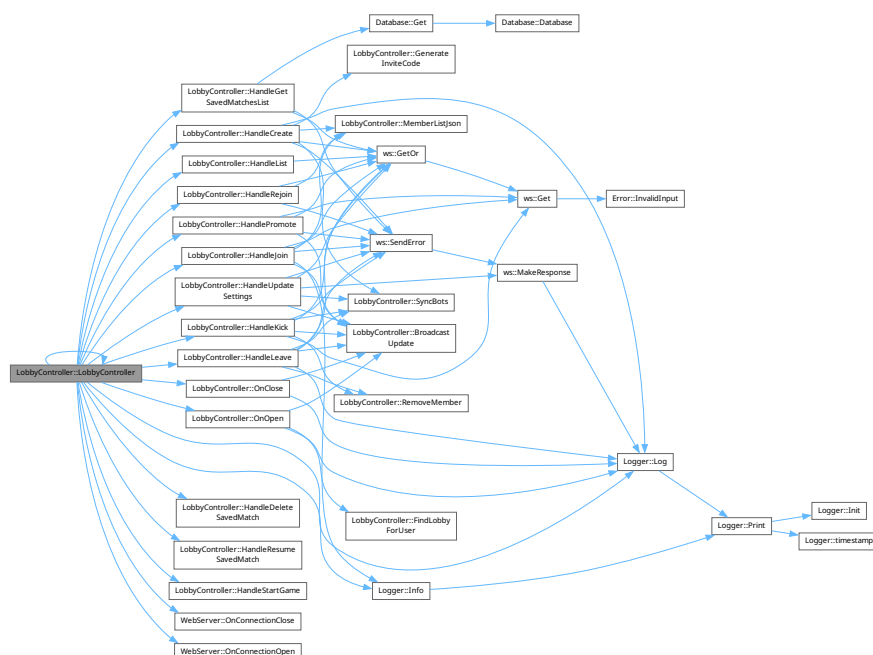
webserver	Riferimento al WebServer principale.
-----------	--

Tag Identificativo: LOBBY-CTRL-001

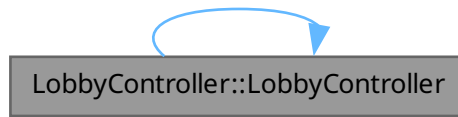
Parametri

server	Reference to the hosting asynchronous web server infrastructure.
--------	--

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.2.2 ~LobbyController()

LobbyController::~~LobbyController ()

Distruttore. Si occupa di ripulire i timer libuv associati.

Destructor that handles explicit system loop timer releases.

Tag Identificativo: LOBBY-CTRL-002

8.25.3 Documentazione delle funzioni membro

8.25.3.1 BroadcastUpdate()

```
void LobbyController::BroadcastUpdate (  
    const Lobby & lobby) const [private]
```

Invia in broadcast l'evento lobby__updated a tutti i membri connessi.

Blasts generalized configuration status changes down to all users currently subscribed to the room thread.

Parametri

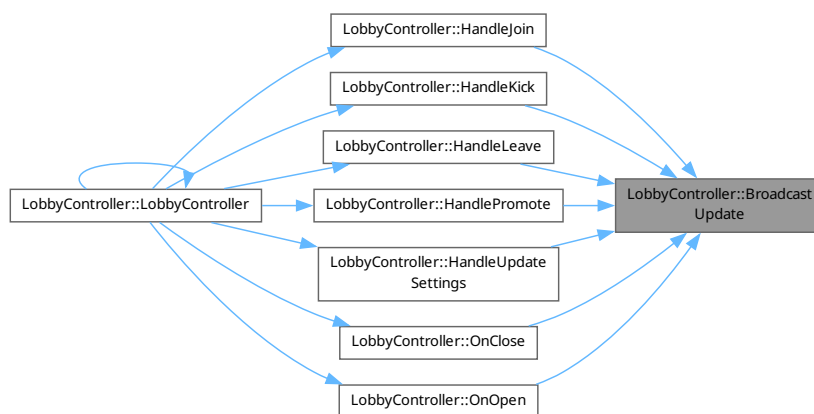
lobby	La lobby da cui inviare il broadcast.
-------	---------------------------------------

Tag Identificativo: LOBBY-UTIL-003

Parametri

lobby	Targeted state structure tracking information.
-------	--

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.2 CheckMatchIntegrity()

```
void LobbyController::CheckMatchIntegrity (
    Lobby & lobby) [private]
```

Verifica l'integrità dei dati della partita corrente (es. riallineamento host).

Validates remaining room configurations and cleans up abandoned empty environments.

Parametri

lobby	La lobby da verificare.
-------	-------------------------

Tag Identificativo: LOBBY-PRIV-002

Parametri

lobby	Reference to the checked targeted room instance.
-------	--

8.25.3.3 FindLobbyForUser()

```
Lobby* LobbyController::FindLobbyForUser (
    const std::string & username) [private]
```

Trova a quale lobby appartiene attualmente un dato utente.

Scans active registries matching connected users back into their hosted environment pointer.

Parametri

username	L'username da cercare.
----------	------------------------

Restituisce

Lobby* Puntatore alla lobby, nullptr se non si trova in nessuna lobby.

Tag Identificativo: LOBBY-PRIV-004

Parametri

username	Unique string tracking key mapping current entity identity.
----------	---

Restituisce

Lobby* Pointer to target lobby framework, or nullptr on map miss.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.4 GenerateInviteCode()

```
string LobbyController::GenerateInviteCode () [static], [private]
```

Genera un codice alfanumerico univoco di 6 caratteri (Invite Code).

Helper parsing cryptographic entropy arrays to output a random unique string identifier.

Restituisce

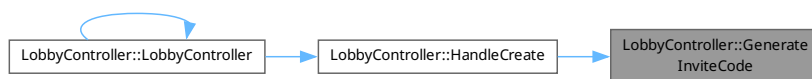
`std::string` Il codice di invito generato.

Tag Identificativo: LOBBY-UTIL-001

Restituisce

`string` Cryptographically randomized alphanumeric joining token.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.5 GetLobbyByCode()

```
Lobby* LobbyController::GetLobbyByCode (
    const std::string & code) [inline]
```

Recupera un puntatore a una lobby tramite il suo codice di invito.

- Parametri

code	Codice alfanumerico di 6 caratteri (es. "XK4F9Z").
------	--

Restituisce

`Lobby*` Puntatore alla lobby, o `nullptr` se non trovata.

Tag Identificativo: LOBBY-CTRL-005

8.25.3.6 GetLobbyById()

`Lobby * LobbyController::GetLobbyById (`
 `const uint32_t id) [inline]`

Recupera un puntatore a una lobby tramite il suo ID numerico interno.

- Parametri

id	ID univoco generato dal server.
----	---------------------------------

Restituisce

Lobby* Puntatore alla lobby, o nullptr se non trovata.

Tag Identificativo: LOBBY-CTRL-006

8.25.3.7 HandleCreate()

`void LobbyController::HandleCreate (`
 `WsContext ctx,`
 `const json & message) [private]`

Gestisce la richiesta di creazione di una nuova lobby.

Allocates structural memory maps for initializing a novel game instance channel.

Parametri

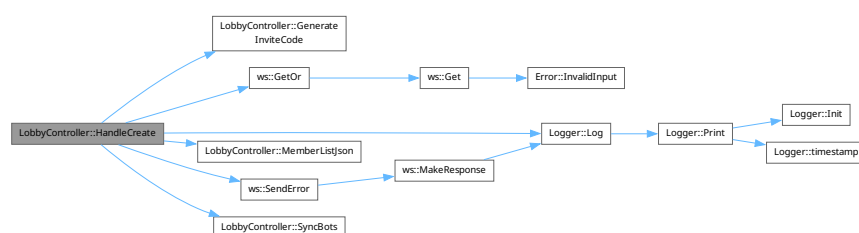
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-001

Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Received raw JSON document mapping initialization preferences.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.8 HandleDeleteSavedMatch()

```
void LobbyController::HandleDeleteSavedMatch (  
    WsContext context,  
    const json & message) [private]
```

Elimina in modo permanente una partita salvata dal DB.

Removes a saved historical game session completely from persistence tracking.

Parametri

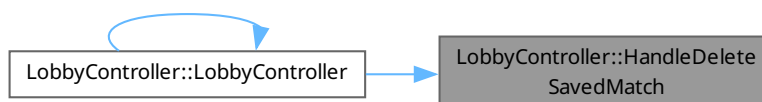
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-011

Parametri

context	Payload context wrapping request sockets and raw buffers.
message	Structure holding the specific match_id string to purge.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.9 HandleGetSavedMatchesList()

```
void LobbyController::HandleGetSavedMatchesList (
    WsContext ctx,
    const json & message) [private]
```

Richiede la lista delle partite salvate precedentemente.

Queries database layers compiling historical session records for connected users.

Parametri

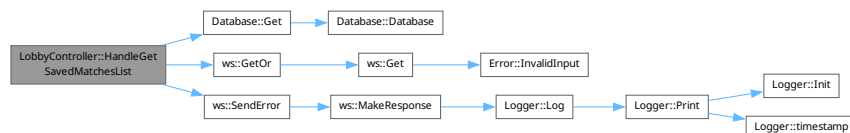
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-009

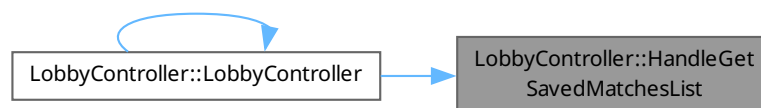
Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Inbound payload message.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.10 HandleJoin()

```
void LobbyController::HandleJoin (
    WsContext ctx,
    const json & message) [private]
```

Gestisce la richiesta di unione a una lobby tramite codice.

Attaches incoming network contexts to matching pre-configured room environments.

Parametri

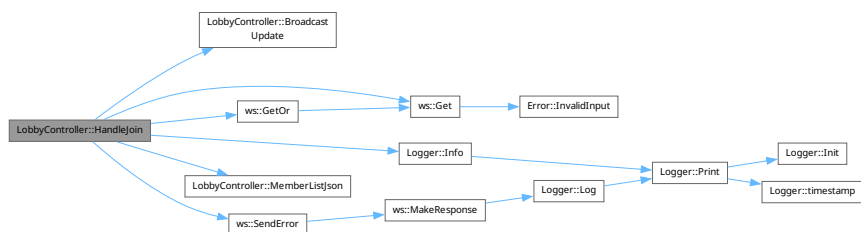
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-002

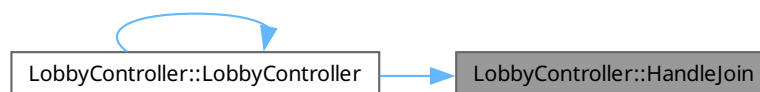
Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Received payload string mapping specific target identification tokens.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.11 HandleKick()

```
void LobbyController::HandleKick (
    WsContext ctx,
    const json & message) [private]
```

Espelle un giocatore dalla lobby (eseguibile solo dall'Host).

Evicts a target entity out of the active channel scope maps.

Parametri

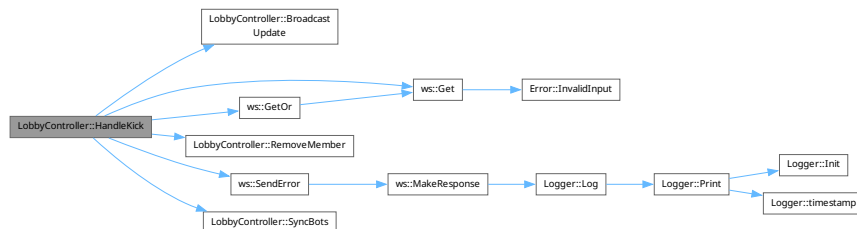
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-007

Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Input mapping parameters declaring execution constraints.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.12 HandleLeave()

```
void LobbyController::HandleLeave (  
    WsContext ctx,  
    const json & message) [private]
```

Gestisce l'uscita volontaria di un utente dalla lobby.

Cleanly unsubscribes channels when an explicit leave event is sent by a player.

Parametri

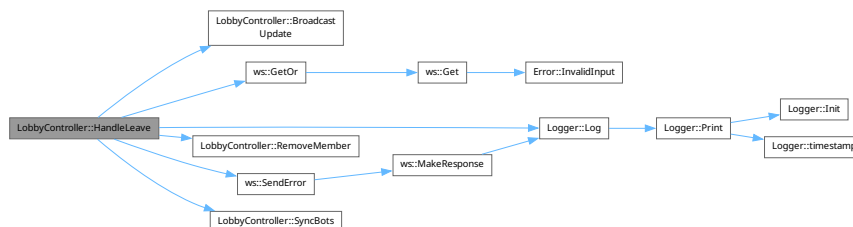
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-004

Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	JSON representation of the leave request payload.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.13 HandleList()

```
void LobbyController::HandleList (  
    WsContext ctx,  
    const json & message) [private]
```

Gestisce la richiesta della lista di lobby pubbliche attive.

Returns a public directory of all active joinable lobbies.

Parametri

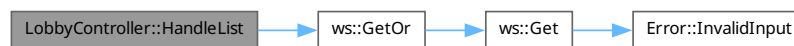
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-005

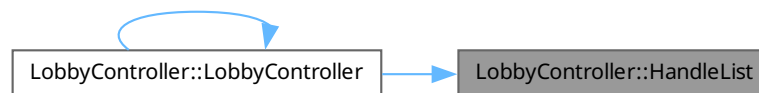
Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	JSON message block from the client requesting room states.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.14 HandlePromote()

```
void LobbyController::HandlePromote (
    WsContext ctx,
    const json & message) [private]
```

Promuove un giocatore specifico al ruolo di Host della lobby.

Transfers room ownership permissions to another human user in the lobby.

Parametri

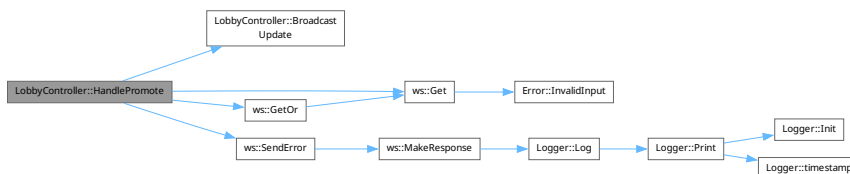
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-006

Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Payload specifying targeted user label parameters.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.15 HandleRejoin()

```
void LobbyController::HandleRejoin (
    WsContext ctx,
    const json & message) [private]
```

Gestisce la richiesta di riconnessione a una lobby.

Resolves tracking references for re-hooking dropped sessions immediately without data loss.

Parametri

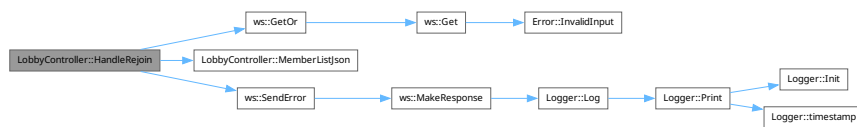
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-003

Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Raw input structure indicating desired identity verification keys.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.16 HandleResumeSavedMatch()

```
void LobbyController::HandleResumeSavedMatch (
    WsContext context,
    const json & message) [private]
```

Riprende una partita salvata a partire dal suo ID.

Restores a previously saved match state from the DB and broadcasts it to participants.

Parametri

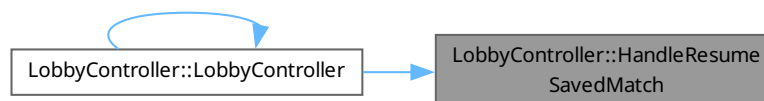
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-010

Parametri

context	Payload context wrapping request sockets and raw buffers.
message	Structure containing the match_id targeting the saved state.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.17 HandleStartGame()

```
void LobbyController::HandleStartGame (
    WsContext context,
    const nlohmann::json & message) [private]
```

Avvia la partita per la lobby corrente. Inizializza l'istanza di gioco.

Commits initialization arrays instantiating a fresh active MatchInstance environment.

Parametri

context	Contesto WebSocket della richiesta.
---------	-------------------------------------

message	Payload JSON associato.
---------	-------------------------

Tag Identificativo: LOBBY-ACT-012

Parametri

context	Payload context wrapping request sockets and raw buffers.
message	Structured message parameter data from client frames.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.18 HandleUpdateSettings()

```
void LobbyController::HandleUpdateSettings (
    WsContext ctx,
    const json & message) [private]
```

Aggiorna le impostazioni della lobby (eseguibile solo dall'Host).

Parses room modification payloads and adjusts internal structures on mutations.

Parametri

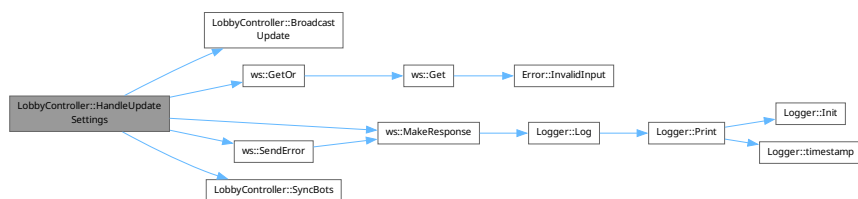
ctx	Contesto WebSocket della richiesta.
msg	Payload JSON associato.

Tag Identificativo: LOBBY-ACT-008

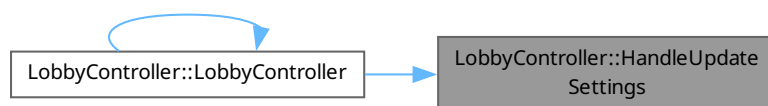
Parametri

ctx	Payload context wrapping request sockets and raw buffers.
message	Document declaration holding updated setting keys.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.19 MemberListJson()

```
json LobbyController::MemberListJson (
    const Lobby & lobby) [static], [private]
```

Serializza la lista dei membri in formato JSON per l'invio via WebSocket.

Encodes the current room roster status flags into a compact JSON array template.

Parametri

lobby	La lobby di cui serializzare i membri.
-------	--

Restituisce

json Oggetto JSON rappresentante i membri.

Tag Identificativo: LOBBY-UTIL-002

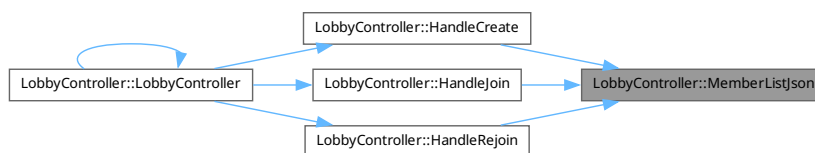
Parametri

lobby	The targeted room reference evaluated.
-------	--

Restituisce

json The serialized list array.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.20 OnClose()

```
void LobbyController::OnClose (
    AppWebSocket * ws,
    PerSocketData * sd)
```

Handler chiamato quando un client chiude la connessione. Segna il membro come disconnesso e registra il timestamp. Non rimuove immediatamente l'utente, ma avvia un periodo di tolleranza (grace period).

Intercepts socket drop frames, setting up transient disconnection grace boundaries.

- Parametri

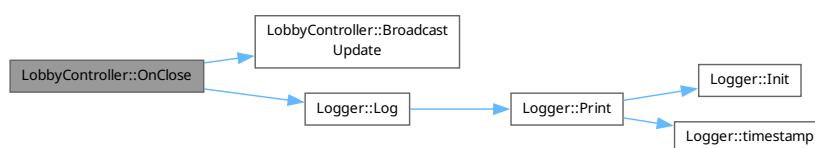
ws	Puntatore al socket WebSocket disconnesso.
sd	Dati associati al socket.

Tag Identificativo: LOBBY-CTRL-004

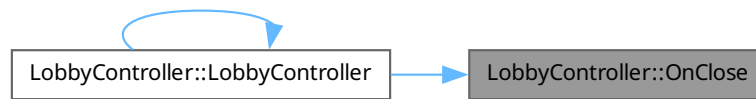
Parametri

ws	Connection reference which dropped out of visibility frames.
sd	Socket data structure tracking current connection information.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.21 OnGameStarted()

```
void LobbyController::OnGameStarted (  
    GameStartedCallback callback) [inline]
```

Registra una callback da eseguire all'inizio di una partita.

Parametri

callback	La funzione da invocare.
----------	--------------------------

Tag Identificativo: LOBBY-CTRL-007

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.22 OnOpen()

```
void LobbyController::OnOpen (  
    AppWebSocket * ws,  
    PerSocketData * sd)
```

Handler chiamato quando un client stabilisce una nuova connessione WebSocket. Se l'utente era già in una lobby (scenario di riconnessione), aggiorna il puntatore del socket e ripristina la sua presenza.

Triggers immediately on connection open, restoring user binding back into their active room session context.

- Parametri

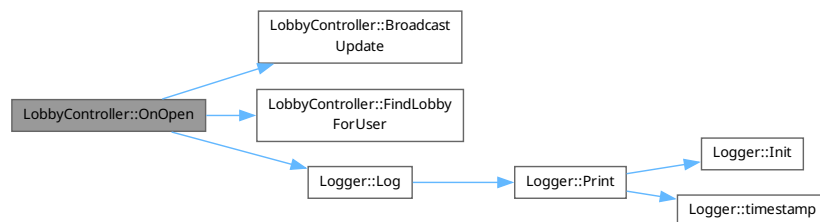
ws	Puntatore al socket WebSocket.
sd	Dati associati al socket (contiene username, ecc.).

Tag Identificativo: LOBBY-CTRL-003

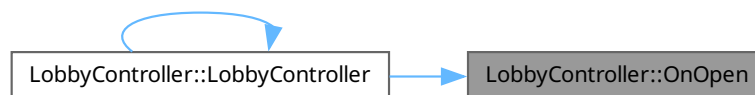
Parametri

ws	Incoming pointer to the active raw client socket instance.
sd	Extracted metadata context state owned by the underlying connection.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.25.3.23 OnPlayerReplaced()

```
void LobbyController::OnPlayerReplaced (
    PlayerReplacedCallback callback) [inline]
```

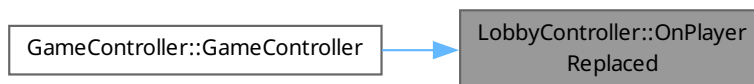
Registra una callback da eseguire quando un giocatore viene sostituito.

Parametri

callback	La funzione da invocare.
--------------------------	--------------------------

Tag Identificativo: LOBBY-CTRL-008

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.24 RemoveMember()

```

bool LobbyController::RemoveMember (
    uint32_t lobby_id,
    const string & username,
    bool explicit_leave = true,
    const string & request_id = "") [private]
  
```

Rimuove un membro dalla lobby. Distrugge la lobby se risulta vuota.

Removes an active member and handles hot-swapping logic or graceful database saving.

Parametri

lobby_id	ID della lobby.
username	Username del giocatore da rimuovere.
explicit_leave	True se l'utente ha abbandonato volontariamente.
request_id	ID della richiesta (opzionale, per tracciamento).

Restituisce

true se la lobby esiste ancora, false se è stata distrutta.

Tag Identificativo: LOBBY-PRIV-003

Parametri

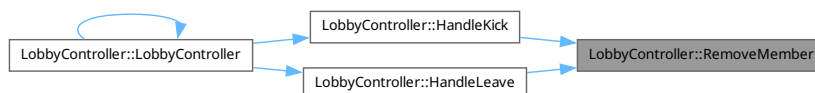
lobby_id	Unique internal numerical tracker identifying the instance folder.
username	String literal targeting unique tracking tag of the client entity.
explicit_leave	True if the action represents an intentional exit command.

request_id	Contextual callback tracker for matching active frontend promises.
------------	--

Restituisce

true If the lobby survived the removal mutation.

Questo è il grafo dei chiamanti di questa funzione:



8.25.3.25 SaveMatchStateToDB()

```
void LobbyController::SaveMatchStateToDB (
    Lobby & lobby)
```

Salva lo stato corrente della partita associata alla lobby nel database.

Commits the structured snapshot of an ongoing match to the SQLite storage layer.

Parametri

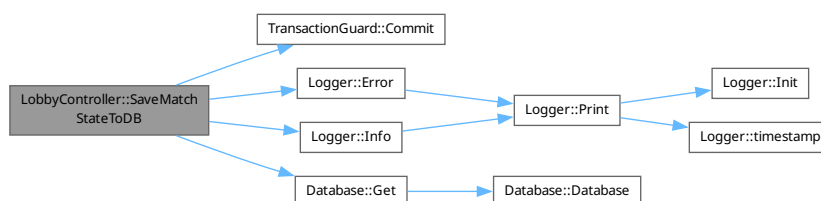
lobby	Riferimento alla lobby di cui salvare lo stato.
-------	---

Tag Identificativo: LOBBY-CTRL-009

Parametri

lobby	Reference to the target active lobby containing the current game match.
-------	---

Questo è il grafo delle chiamate per questa funzione:



8.25.3.26 SyncBots()

```
void LobbyController::SyncBots (
    Lobby & lobby) [private]
```

Sincronizza lo stato dei bot all'interno della lobby.

Re-evaluates configuration limits, adding or purging bot instances until matching thresholds are synchronized.

Parametri

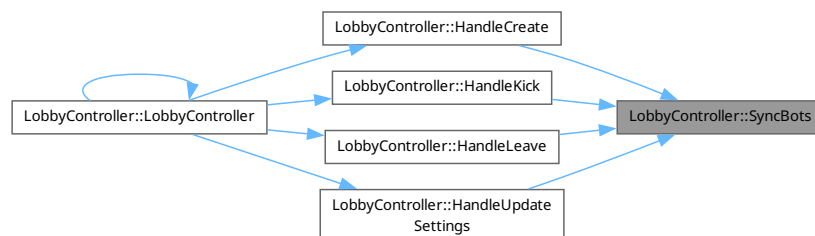
lobby	La lobby da aggiornare.
-------	-------------------------

Tag Identificativo: LOBBY-PRIV-001

Parametri

lobby	Active room reference requiring bot balancing.
-------	--

Questo è il grafo dei chiamanti di questa funzione:



8.25.4 Documentazione dei membri dato

8.25.4.1 action_router_

```
ActionRouter& LobbyController::action_router_ [private]
```

Riferimento al router di azioni WS.

8.25.4.2 app_

```
uWS::SSLApp& LobbyController::app_ [private]
```

Riferimento all'istanza uWebSockets.

8.25.4.3 code_to_id_

```
std::unordered_map<std::string, uint32_t> LobbyController::code_to_id_ [private]
```

Indice secondario per ricerca rapida.

8.25.4.4 eviction_timer_

```
us_timer_t* LobbyController::eviction_timer_ = nullptr [private]
```

Timer libuv per l'eviction dei disconnessi.

8.25.4.5 game_started_callback_

```
GameStartedCallback LobbyController::game_started_callback_ [private]
```

Callback per l'avvio partita.

8.25.4.6 kMaxMembers

```
int LobbyController::kMaxMembers = 4 [static], [constexpr]
```

Numero massimo di giocatori consentiti in una singola lobby.

Tag Identificativo: LOBBY-CFG-002

8.25.4.7 kReconnectGraceMs

```
int LobbyController::kReconnectGraceMs = 1'000 * 30 [static], [constexpr]
```

Tempo di tolleranza (grace period) prima che un utente disconnesso venga espulso.

Tag Identificativo: LOBBY-CFG-001

8.25.4.8 lobbies_

```
std::unordered_map<uint32_t, Lobby> LobbyController::lobbies_ [private]
```

Memorizzazione primaria delle lobby.

8.25.4.9 next_id_

```
std::atomic<uint32_t> LobbyController::next_id_ {1} [private]
```

Contatore thread-safe per gli ID delle lobby.

8.25.4.10 player_replaced_callback_

```
PlayerReplacedCallback LobbyController::player_replaced_callback_ [private]
```

Callback per sostituzione giocatori.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/controllers/lobby_controller.hpp](#)
- [src/controllers/lobby_controller.cpp](#)

8.26 Riferimenti per la struct LobbyMember

Rappresenta un singolo partecipante all'interno di una lobby.

```
#include <lobby.hpp>
```

8.26.1 Descrizione dettagliata

Rappresenta un singolo partecipante all'interno di una lobby.

Tag Identificativo: CMN-LOBBY-STR-002

La documentazione per questa struct è stata generata a partire dal seguente file:

- [include/common/lobby.hpp](#)

8.27 Riferimenti per la struct LobbySettings

Contiene la configurazione dettagliata e le regole di una specifica lobby. Queste impostazioni sono serializzabili in JSON per essere trasmesse ai client.

```
#include <lobby.hpp>
```

8.27.1 Descrizione dettagliata

Contiene la configurazione dettagliata e le regole di una specifica lobby. Queste impostazioni sono serializzabili in JSON per essere trasmesse ai client.

Tag Identificativo: CMN-LOBBY-STR-001

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/common/lobby.hpp`

8.28 Riferimenti per la struct Logger

Fornisce metodi statici per stampare messaggi di log formattati sullo standard output.

```
#include <logger.hpp>
```

Membri pubblici statici

- `template<typename... Args>`
`static void Info (Args &&... args)`
Registra un messaggio di informazione generico (Verde).
- `template<typename... Args>`
`static void Warn (Args &&... args)`
Registra un avviso non bloccante (Giallo).
- `template<typename... Args>`
`static void Error (Args &&... args)`
Registra un errore critico (Rosso).
- `template<typename... Args>`
`static void WS (Args &&... args)`
Registra un evento specifico del layer WebSocket (Ciano).
- `template<typename... Args>`
`static void Lobby (Args &&... args)`
Registra un evento specifico del `LobbyController` (Giallo scuro).
- `template<typename... Args>`
`static void HTTP (Args &&... args)`
Registra un evento specifico delle richieste HTTP (Magenta).
- `template<typename... Args>`
`static void Log (Args &&... args)`
Registra un log di debug base (Bianco).
- `static constexpr std::string_view ColorWhite ()`
- `static constexpr std::string_view ColorReset ()`

Membri privati statici

- static void `Init` ()
Abilita il supporto ANSI sui terminali Windows.
- static std::string `timestamp` ()
Genera un timestamp formattato con millisecondi (es. "[14:32:01.045]").
- template<typename... Args>
static void `Print` (std::string_view color, std::string_view tag, Args &&... args)
Funzione core per la formattazione e la stampa. Utilizza le fold expressions del C++17 per spaccettare in modo efficiente gli argomenti variadici.

Attributi privati statici

- static constexpr std::string_view `Reset` = "\033[0m"
- static constexpr std::string_view `Gray` = "\033[90m"
- static constexpr std::string_view `Red` = "\033[31m"
- static constexpr std::string_view `Green` = "\033[32m"
- static constexpr std::string_view `Yellow` = "\033[33m"
- static constexpr std::string_view `Blue` = "\033[34m"
- static constexpr std::string_view `Magenta` = "\033[35m"
- static constexpr std::string_view `Cyan` = "\033[36m"
- static constexpr std::string_view `White` = "\033[37m"
- static constexpr std::string_view `Bold` = "\033[1m"
- static bool `is_initialized` = false

8.28.1 Descrizione dettagliata

Fornisce metodi statici per stampare messaggi di log formattati sullo standard output.

- Implementa il supporto per i colori ANSI cross-platform (abilitando il VT processing su Windows) e formatta automaticamente i messaggi con timestamp al millisecondo e tag allineati.

Tag Identificativo: LOG-CLS-001

8.28.2 Documentazione delle funzioni membro

8.28.2.1 ColorReset()

```
constexpr std::string_view Logger::ColorReset () [inline], [static], [constexpr]
```

8.28.2.2 ColorWhite()

```
constexpr std::string_view Logger::ColorWhite () [inline], [static], [constexpr]
```

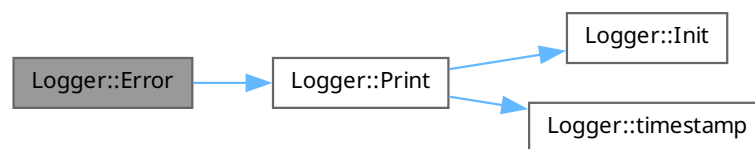
8.28.2.3 Error()

```
template<typename... Args>
void Logger::Error (
    Args &&... args) [inline], [static]
```

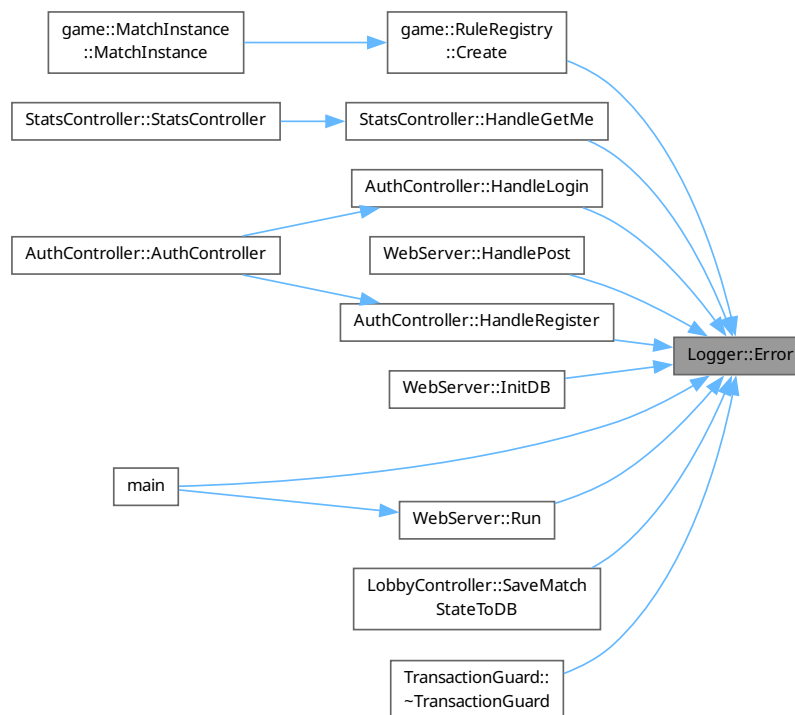
Registra un errore critico (Rosso).

Tag Identificativo: LOG-MTH-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.28.2.4 HTTP()

```
template<typename... Args>  
void Logger::HTTP (  
    Args &&... args)  [inline], [static]
```

Registra un evento specifico delle richieste HTTP (Magenta).

Tag Identificativo: LOG-MTH-006

Questo è il grafo delle chiamate per questa funzione:



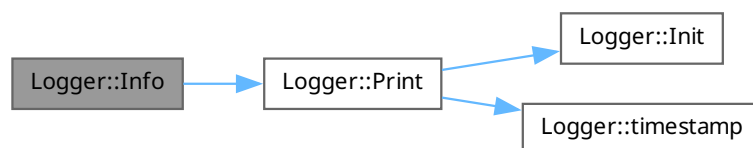
8.28.2.5 Info()

```
template<typename... Args>  
void Logger::Info (  
    Args &&... args)  [inline], [static]
```

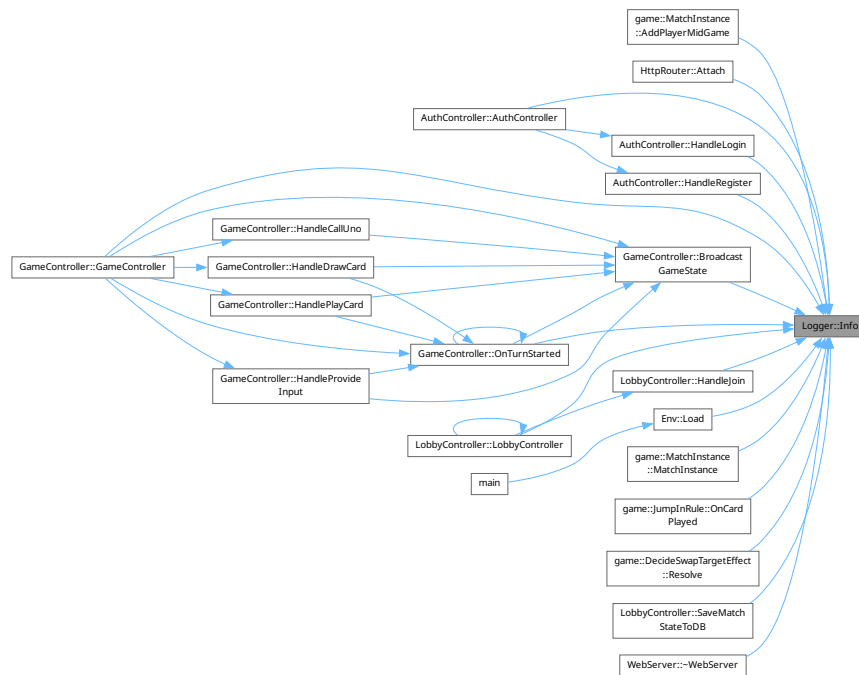
Registra un messaggio di informazione generico (Verde).

Tag Identificativo: LOG-MTH-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:

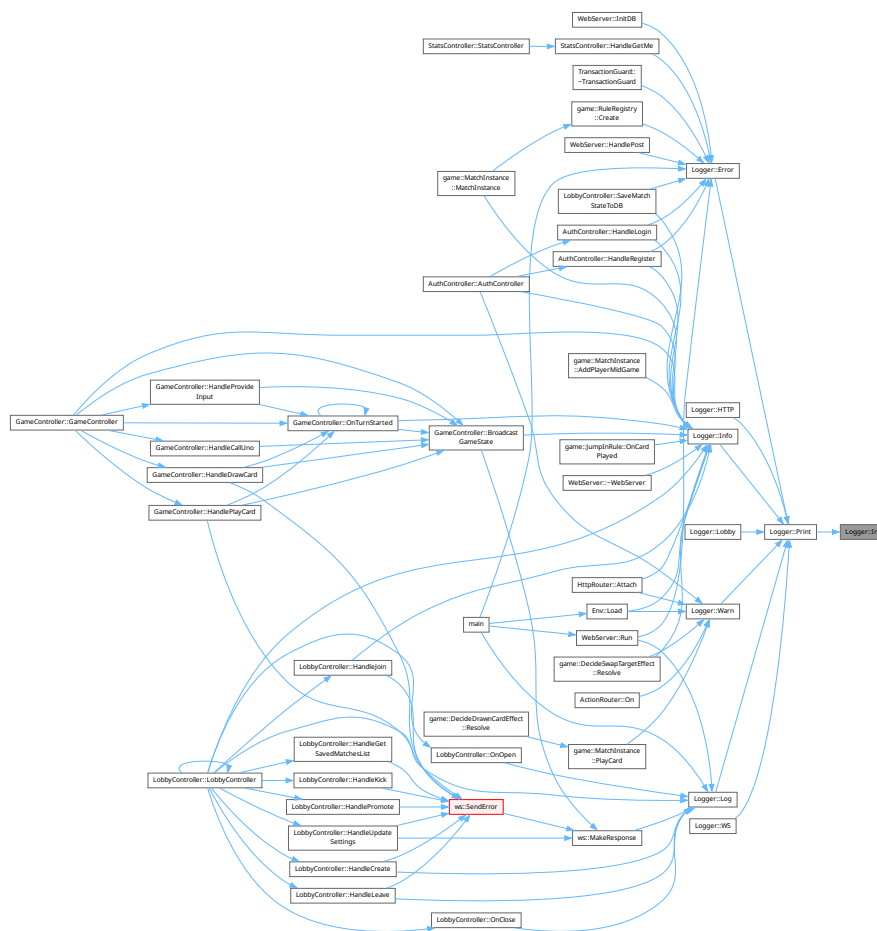


8.28.2.6 Init()

```
void Logger::Init () [inline], [static], [private]
```

Abilita il supporto ANSI sui terminali Windows.

Questo è il grafo dei chiamanti di questa funzione:



```
template<typename... Args>
void Logger::Lobby (
    Args &&... args) [inline], [static]
```

Generato da Doxygen

Tag Identificativo: LOG-MTH-005

Questo è il grafo delle chiamate per questa funzione:



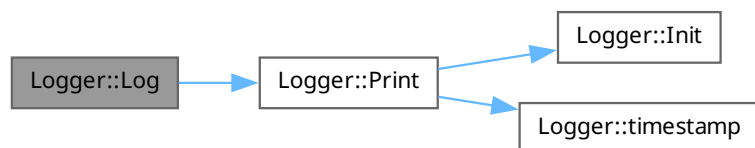
8.28.2.8 Log()

```
template<typename... Args>
void Logger::Log (
    Args &&... args) [inline], [static]
```

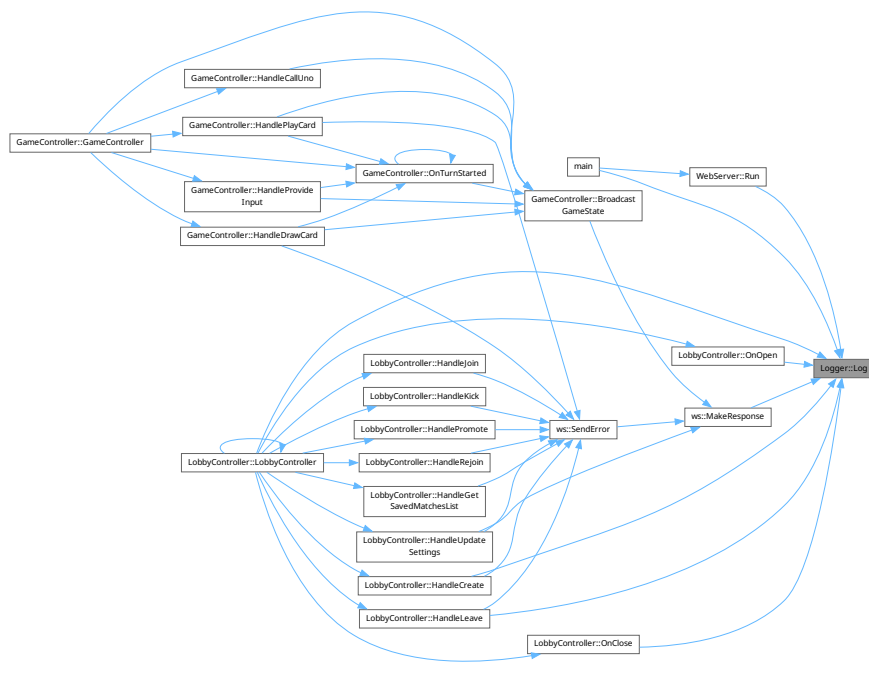
Registra un log di debug base (Bianco).

Tag Identificativo: LOG-MTH-007

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



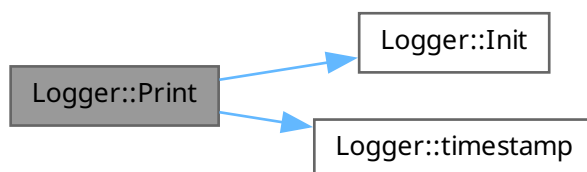
8.28.2.9 Print()

```
template<typename... Args>
void Logger::Print (
    std::string_view color,
    std::string_view tag,
    Args &&... args) [inline], [static], [private]
```

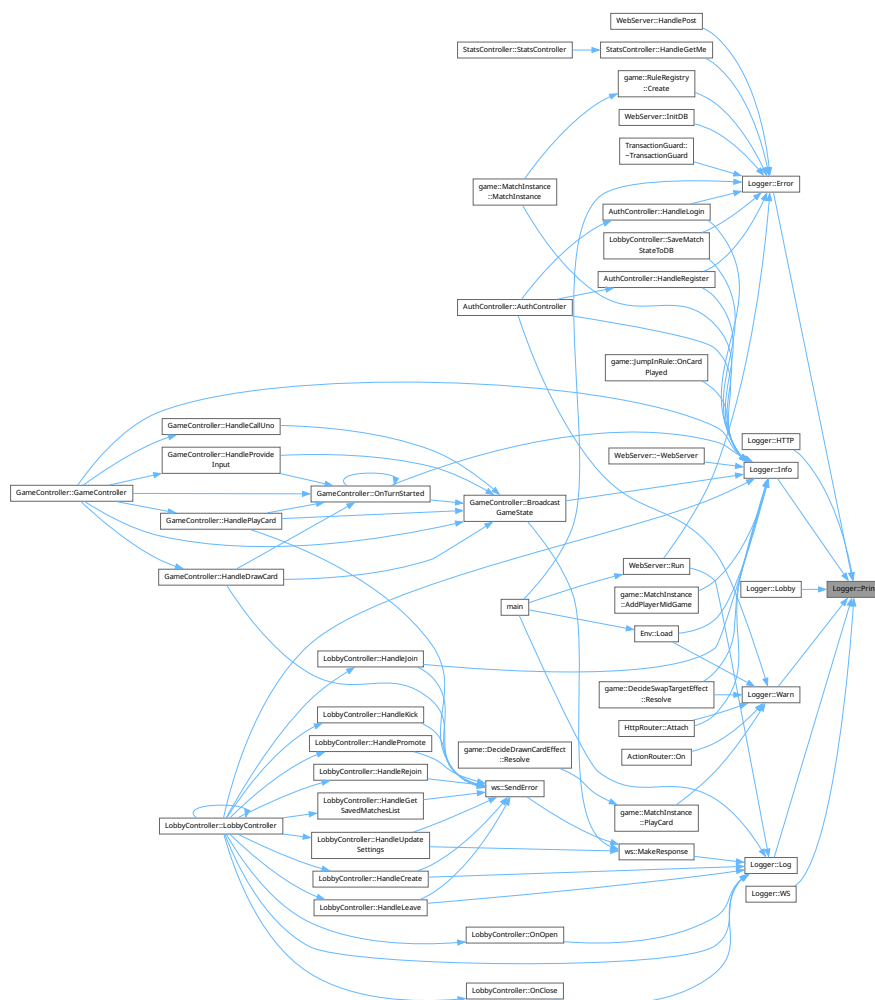
Funzione core per la formattazione e la stampa. Utilizza le fold expressions del C++17 per spaccettare in modo efficiente gli argomenti variadici.

Tag Identificativo: LOG-PRIV-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.28.2.10 timestamp()

`std::string Logger::timestamp ()` [inline], [static], [private]

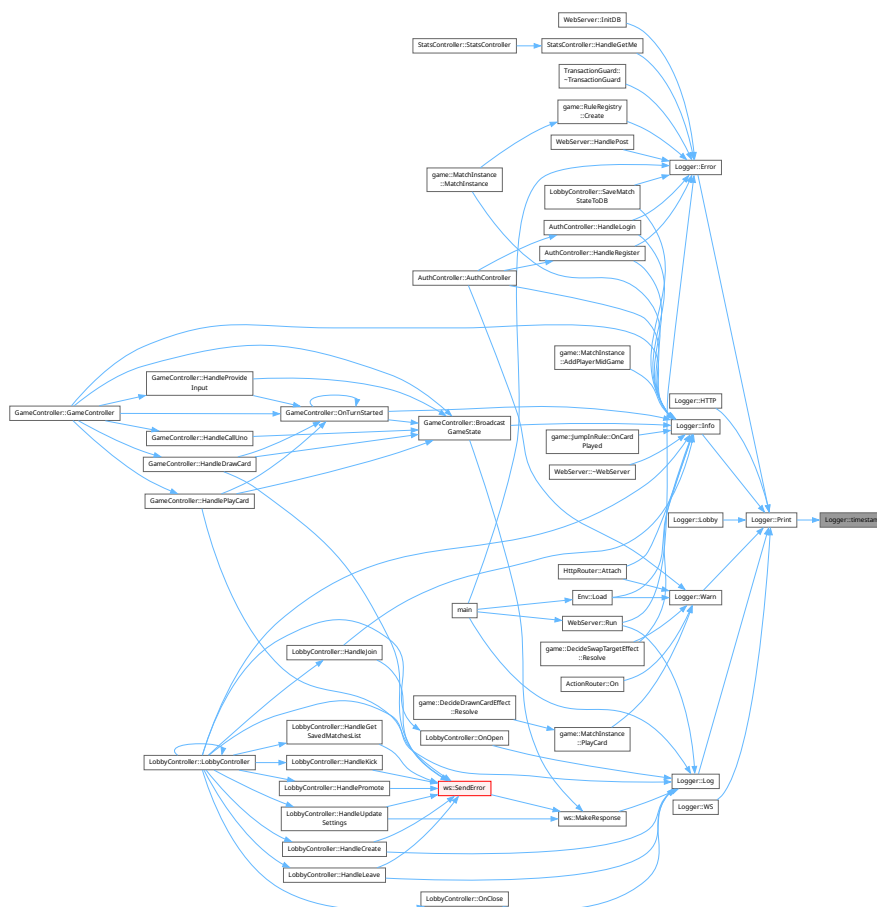
Genera un timestamp formattato con millisecondi (es. "[14:32:01.045]").

Restituisce

`std::string` Timestamp colorato in grigio.

Tag Identificativo: LOG-PRIV-002

Questo è il grafo dei chiamanti di questa funzione:



8.28.2.11 Warn()

```
template<typename... Args>
void Logger::Warn (
    Args &&... args) [inline], [static]
```

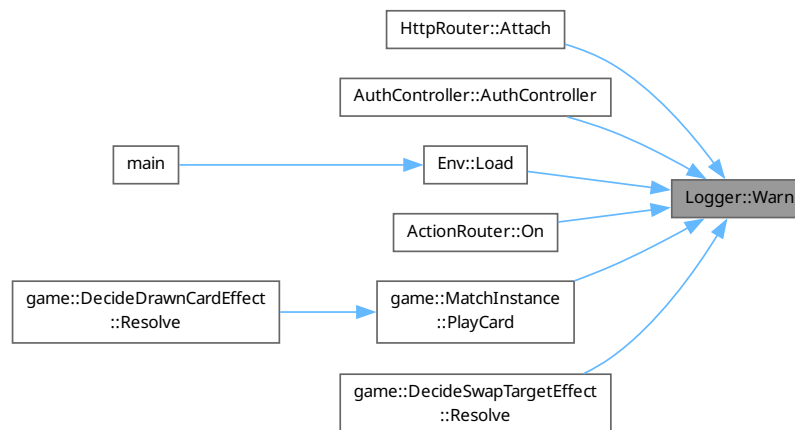
Registra un avviso non bloccante (Giallo).

Tag Identificativo: LOG-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



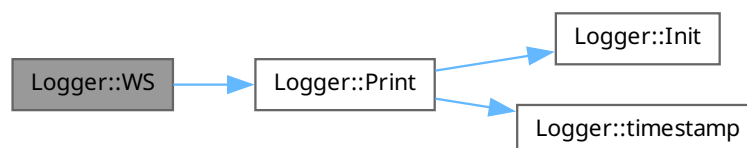
8.28.2.12 WS()

```
template<typename... Args>
void Logger::WS (
    Args &&... args) [inline], [static]
```

Registra un evento specifico del layer WebSocket (Ciano).

Tag Identificativo: LOG-MTH-004

Questo è il grafo delle chiamate per questa funzione:



8.28.3 Documentazione dei membri dato

8.28.3.1 Blue

```
std::string_view Logger::Blue = "\033[34m" [static], [constexpr], [private]
```


8.28.3.2 Bold

```
std::string_view Logger::Bold = "\033[1m" [static], [constexpr], [private]
```

8.28.3.3 Cyan

```
std::string_view Logger::Cyan = "\033[36m" [static], [constexpr], [private]
```

8.28.3.4 Gray

```
std::string_view Logger::Gray = "\033[90m" [static], [constexpr], [private]
```

8.28.3.5 Green

```
std::string_view Logger::Green = "\033[32m" [static], [constexpr], [private]
```

8.28.3.6 is_initialized

```
bool Logger::is_initialized = false [inline], [static], [private]
```

8.28.3.7 Magenta

```
std::string_view Logger::Magenta = "\033[35m" [static], [constexpr], [private]
```

8.28.3.8 Red

```
std::string_view Logger::Red = "\033[31m" [static], [constexpr], [private]
```

8.28.3.9 Reset

```
std::string_view Logger::Reset = "\033[0m" [static], [constexpr], [private]
```

8.28.3.10 White

```
std::string_view Logger::White = "\033[37m" [static], [constexpr], [private]
```

8.28.3.11 Yellow

```
std::string_view Logger::Yellow = "\033[33m" [static], [constexpr], [private]
```

La documentazione per questa struct è stata generata a partire dal seguente file:

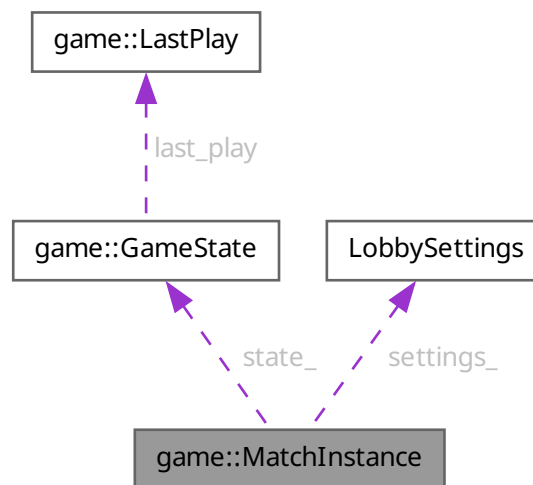
- [include/logger.hpp](#)

8.29 Riferimenti per la classe `game::MatchInstance`

Rappresenta un'istanza attiva di una partita (Match).

```
#include <match_instance.hpp>
```

Diagramma di collaborazione per `game::MatchInstance`:



Memberi pubblici

- [MatchInstance](#) (const std::vector< std::pair< std::string, bool > > &players_info, const [LobbySettings](#) &settings)
Costruttore per una nuova partita partendo dalle informazioni dei giocatori e impostazioni.
- [MatchInstance](#) (const json &saved_state, const [LobbySettings](#) &settings)
Costruttore per ricaricare una partita partendo da uno stato salvato nel database.
- void [Start](#) ()
Inizializza il match, crea il mazzo iniziale e stabilisce i turni.
- void [Tick](#) ()
Aggiornamento periodico della partita (utilizzato per elaborare gli effetti in coda o timeout).
- bool [PlayCard](#) (const std::string &username, uint16_t card_id)
Richiesta da parte di un utente di giocare una determinata carta.
- bool [DrawCard](#) (const std::string &username)
Richiesta da parte di un utente di pescare una carta dal mazzo.
- void [ProvideInput](#) (const std::string &username, const std::string &input)
Fornisce un input ad un effetto in sospeso (es. Scelta di un colore).
- void [CallUno](#) (const std::string &username)
Dichiara "UNO" per lo specifico giocatore.
- void [TakeBotTurn](#) ()

- Richiama l'intelligenza artificiale per far eseguire il turno al Bot.
- `std::string GetCurrentPlayerUsername () const`
Restituisce il nome utente del giocatore il cui turno è attualmente attivo.
- `void AddPlayerMidGame (const std::string &username, bool is_bot)`
Aggiunge o sostituisce un giocatore a partita in corso (es. Takeover bot).
- `void RemovePlayerMidGame (const std::string &username)`
Rimuove un giocatore a partita in corso, trasformandolo tipicamente in Bot.
- `json ExportState () const`
Esporta l'intero stato del `GameState` in un formato JSON salvabile.
- `bool IsWaitingForInput () const`
Verifica se il motore di gioco è in attesa di un input esplicito da un giocatore.
- `void SetTurnEndTime (std::chrono::steady_clock::time_point end_time)`
Imposta il limite di tempo per la fine del turno corrente.
- `Player * GetPlayer (const std::string &username)`
Recupera i dati interni di un giocatore specifico tramite username.
- `std::string GetPendingPlayer () const`
Restituisce l'utente dal quale si sta attendendo un input obbligatorio.
- `std::string GetPendingInputType () const`
Restituisce il tipo di input in sospeso (es. "choose_color").
- `std::string GetPendingInputContext () const`
Restituisce dati testuali JSON contestuali alla richiesta di input.
- `bool IsGameOver () const`
Determina se la partita ha raggiunto uno stato terminale.
- `std::string GetWinner () const`
Recupera l'username del vincitore (se la partita è conclusa).
- `nlohmann::json SerializePlayerState (const std::string &username) const`
Crea un JSON che rappresenta lo stato di gioco mascherato, specifico per il punto di vista del giocatore fornito (nascondendo mani avversarie).
- `std::string GetMatchId () const`
Recupera l'UUID univoco della partita (per salvataggio su DB).
- `void SetMatchId (const std::string &id)`
Assegna un UUID persistente alla partita.

Membru privati

- `bool HasMod (const std::string &name) const`
Checks whether a given optional rule mod is currently active for this match.
- `void GenerateDeck ()`
Inizializza il mazzo di carte basandosi sulle `LobbySettings` fornite.

Attributi privati

- `GameState state__`
- `LobbySettings settings__`
- `std::string match_id__`
- `std::unordered_map< std::string, PlayerSessionStats > session_stats__`
- `std::vector< std::unique_ptr< GameRule > > active_rules__`

8.29.1 Descrizione dettagliata

Rappresenta un'istanza attiva di una partita (Match).

- Mantiene lo stato di gioco interno ([GameState](#)), gestisce il flusso dei turni, l'esecuzione delle mosse (giocare carte, pescare), e processa gli effetti e le regole attive applicate a questa partita specifica.

Tag Identificativo: MATCH-INST-000

•

8.29.2 Documentazione dei costruttori e dei distruttori

8.29.2.1 MatchInstance() ^[1/2]

```
game::MatchInstance::MatchInstance (
    const std::vector< std::pair< std::string, bool > > & players_info,
    const LobbySettings & settings) [explicit]
```

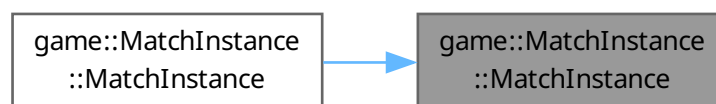
Costruttore per una nuova partita partendo dalle informazioni dei giocatori e impostazioni.

Parametri

players_info	Vettore di coppie {username, is_bot} indicanti i partecipanti.
settings	Impostazioni correnti della lobby.

Tag Identificativo: MATCH-INST-001

Questo è il grafo dei chiamanti di questa funzione:



8.29.2.2 MatchInstance()

```
game::MatchInstance::MatchInstance (
    const json & saved_state,
    const LobbySettings & settings) [explicit]
```

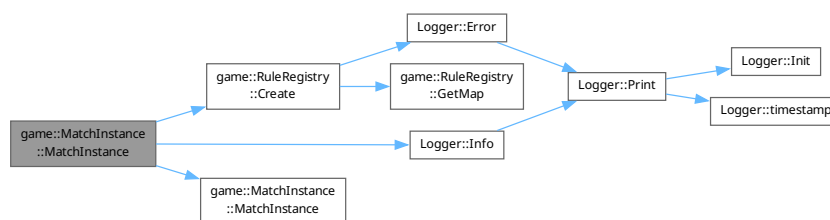
Costruttore per ricaricare una partita partendo da uno stato salvato nel database.

Parametri

saved_state	Il JSON contenente lo stato serializzato.
settings	Impostazioni della lobby associate al salvataggio.

Tag Identificativo: MATCH-INST-002

Questo è il grafo delle chiamate per questa funzione:



8.29.3 Documentazione delle funzioni membro

8.29.3.1 AddPlayerMidGame()

```
void game::MatchInstance::AddPlayerMidGame (
    const std::string & username,
    bool is_bot)
```

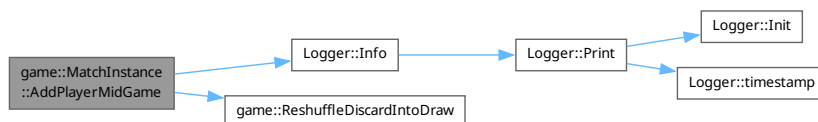
Aggiunge o sostituisce un giocatore a partita in corso (es. Takeover bot).

Parametri

username	Il nuovo giocatore.
is_bot	True se si tratta di un bot, False per un umano.

Tag Identificativo: MATCH-INST-011

Questo è il grafo delle chiamate per questa funzione:



8.29.3.2 CallUno()

```
void game::MatchInstance::CallUno (
    const std::string & username)
```

Dichiara "UNO" per lo specifico giocatore.

Parametri

username	Il giocatore che sta chiamando UNO.
----------	-------------------------------------

Tag Identificativo: MATCH-INST-008

Questo è il grafo dei chiamanti di questa funzione:



8.29.3.3 DrawCard()

```
bool game::MatchInstance::DrawCard (
    const std::string & username)
```

Richiesta da parte di un utente di pescare una carta dal mazzo.

Parametri

username	L'utente che intende pescare.
----------	-------------------------------

Restituisce

true se l'azione è permessa ed eseguita, false altrimenti.

Tag Identificativo: MATCH-INST-006

8.29.3.4 ExportState()

```
json game::MatchInstance::ExportState () const
```

Esporta l'intero stato del [GameState](#) in un formato JSON salvabile.

Restituisce

json Stato serializzato.

Tag Identificativo: MATCH-INST-013

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



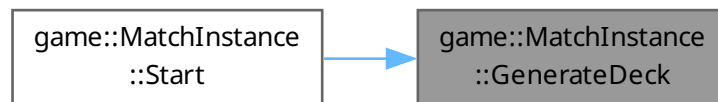
8.29.3.5 GenerateDeck()

```
void game::MatchInstance::GenerateDeck () [private]
```

Inizializza il mazzo di carte basandosi sulle [LobbySettings](#) fornite.

Tag Identificativo: MATCH-PRIV-002

Questo è il grafo dei chiamanti di questa funzione:



8.29.3.6 GetCurrentPlayerUsername()

```
std::string game::MatchInstance::GetCurrentPlayerUsername () const
```

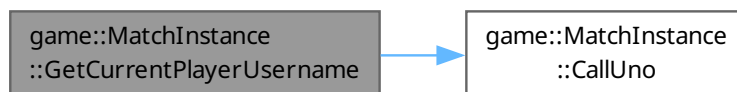
Restituisce il nome utente del giocatore il cui turno è attualmente attivo.

Restituisce

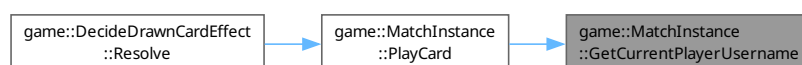
std::string Nome del giocatore.

Tag Identificativo: MATCH-INST-010

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.29.3.7 GetMatchId()

```
std::string game::MatchInstance::GetMatchId () const [inline]
```

Recupera l'UUID univoco della partita (per salvataggio su DB).

Restituisce

std::string ID della partita.

Tag Identificativo: MATCH-INST-023

8.29.3.8 GetPendingInputContext()

```
std::string game::MatchInstance::GetPendingInputContext () const [inline]
```

Restituisce dati testuali JSON contestuali alla richiesta di input.

Restituisce

std::string Il contesto dell'input.

Tag Identificativo: MATCH-INST-019

8.29.3.9 GetPendingInputType()

```
std::string game::MatchInstance::GetPendingInputType () const [inline]
```

Restituisce il tipo di input in sospeso (es. "choose_color").

Restituisce

std::string Codice dell'input.

Tag Identificativo: MATCH-INST-018

8.29.3.10 GetPendingPlayer()

`std::string game::MatchInstance::GetPendingPlayer () const [inline]`

Restituisce l'utente dal quale si sta attendendo un input obbligatorio.

Restituisce

`std::string Username` o stringa vuota se non c'è attesa.

Tag Identificativo: MATCH-INST-017

8.29.3.11 GetPlayer()

`Player * game::MatchInstance::GetPlayer (`
`const std::string & username)`

Recupera i dati interni di un giocatore specifico tramite username.

Parametri

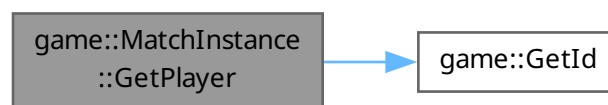
username	Identificativo dell'utente.
----------	-----------------------------

Restituisce

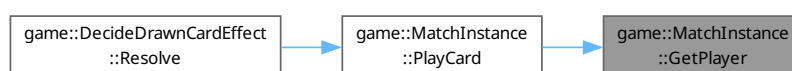
`Player*` Puntatore ai dati del giocatore o `nullptr` se non esiste.

Tag Identificativo: MATCH-INST-016

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.29.3.12 GetWinner()

std::string game::MatchInstance::GetWinner () const [inline]

Recupera l'username del vincitore (se la partita è conclusa).

Restituisce

std::string Username del vincitore.

Tag Identificativo: MATCH-INST-021

8.29.3.13 HasMod()

bool game::MatchInstance::HasMod (
 const std::string & name) const [inline], [private]

Checks whether a given optional rule mod is currently active for this match.

Parametri

name	The mod identifier (e.g. "seven_zero", "draw_stacking", "no_bluffing").
------	---

Restituisce

true if the mod is enabled in this lobby's settings.

Tag Identificativo: MATCH-INST-PRIV-001

8.29.3.14 IsGameOver()

bool game::MatchInstance::IsGameOver () const [inline]

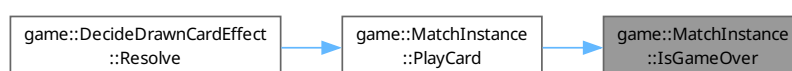
Determina se la partita ha raggiunto uno stato terminale.

Restituisce

true se terminata.

Tag Identificativo: MATCH-INST-020

Questo è il grafo dei chiamanti di questa funzione:



8.29.3.15 IsWaitingForInput()

```
bool game::MatchInstance::IsWaitingForInput () const    [inline]
```

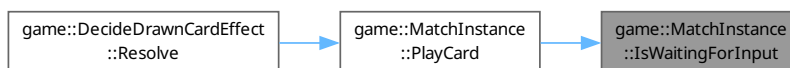
Verifica se il motore di gioco è in attesa di un input esplicito da un giocatore.

Restituisce

true se l'effetto corrente ha bisogno di input.

Tag Identificativo: MATCH-INST-014

Questo è il grafo dei chiamanti di questa funzione:



8.29.3.16 PlayCard()

```
bool game::MatchInstance::PlayCard (  
    const std::string & username,  
    uint16_t card_id)
```

Richiesta da parte di un utente di giocare una determinata carta.

Parametri

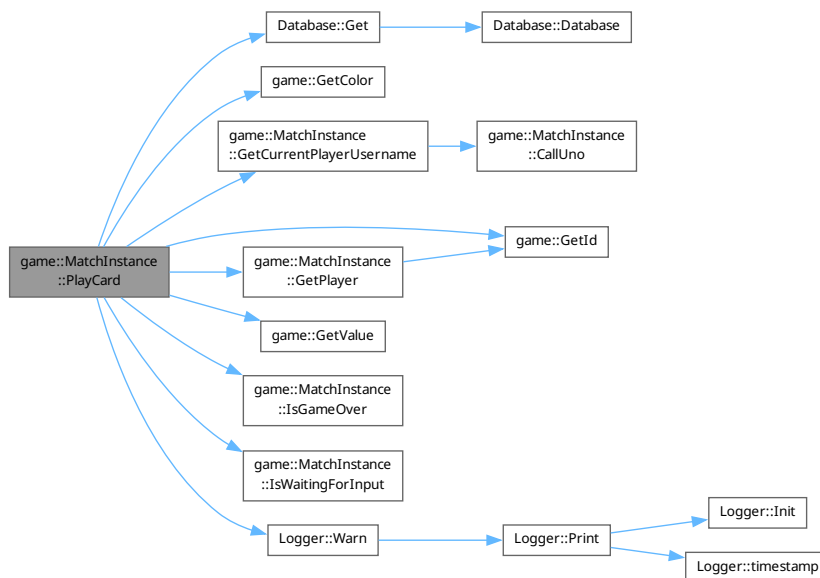
username	L'utente che sta giocando la carta.
card_id	L'identificativo a 16-bit della carta compatta.

Restituisce

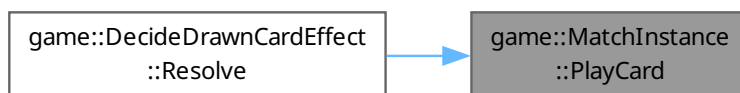
true se la giocata è stata validata ed elaborata, false altrimenti.

Tag Identificativo: MATCH-INST-005

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.29.3.17 ProvideInput()

```
void game::MatchInstance::ProvideInput (
    const std::string & username,
    const std::string & input)
```

Fornisce un input ad un effetto in sospeso (es. Scelta di un colore).

Parametri

username	Il giocatore che fornisce l'input.
----------	------------------------------------

input	Stringa rappresentante la scelta effettuata.
-------	--

Tag Identificativo: MATCH-INST-007

8.29.3.18 RemovePlayerMidGame()

```
void game::MatchInstance::RemovePlayerMidGame (  
    const std::string & username)
```

Rimuove un giocatore a partita in corso, trasformandolo tipicamente in Bot.

Parametri

username	Il giocatore disconnesso/uscito.
----------	----------------------------------

Tag Identificativo: MATCH-INST-012

8.29.3.19 SerializePlayerState()

```
nlohmann::json game::MatchInstance::SerializePlayerState (  
    const std::string & username) const
```

Crea un JSON che rappresenta lo stato di gioco mascherato, specifico per il punto di vista del giocatore fornito (nascondendo mani avversarie).

- Parametri

username	Il giocatore per il quale si genera la vista.
----------	---

Restituisce

nlohmann::json Lo stato censurato e pronto all'invio al frontend.

Tag Identificativo: MATCH-INST-022

8.29.3.20 SetMatchId()

```
void game::MatchInstance::SetMatchId (  
    const std::string & id) [inline]
```

Assegna un UUID persistente alla partita.

Parametri

id	L'identificativo da assegnare.
----	--------------------------------

Tag Identificativo: MATCH-INST-024

8.29.3.21 SetTurnEndTime()

```
void game::MatchInstance::SetTurnEndTime (  
    std::chrono::steady_clock::time_point end_time) [inline]
```

Imposta il limite di tempo per la fine del turno corrente.

Parametri

end_time	Il timestamp in cui il turno scadrà (triggerando l'AFK o bot).
----------	--

Tag Identificativo: MATCH-INST-015

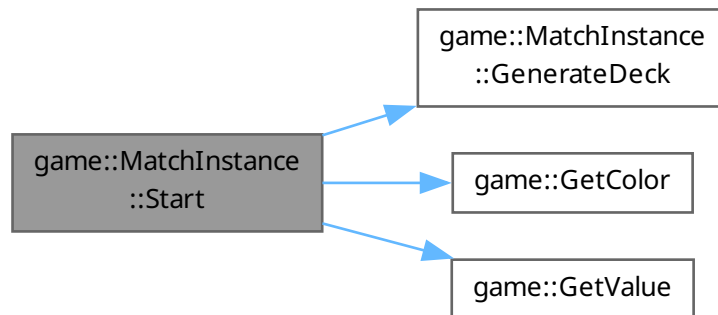
8.29.3.22 Start()

```
void game::MatchInstance::Start ()
```

Inizializza il match, crea il mazzo iniziale e stabilisce i turni.

Tag Identificativo: MATCH-INST-003

Questo è il grafo delle chiamate per questa funzione:



8.29.3.23 TakeBotTurn()

```
void game::MatchInstance::TakeBotTurn ()
```

Richiama l'intelligenza artificiale per far eseguire il turno al Bot.

Tag Identificativo: MATCH-INST-009

8.29.3.24 Tick()

```
void game::MatchInstance::Tick ()
```

Aggiornamento periodico della partita (utilizzato per elaborare gli effetti in coda o timeout).

Tag Identificativo: MATCH-INST-004

8.29.4 Documentazione dei membri dato

8.29.4.1 active_rules_

```
std::vector<std::unique_ptr<GameRule> > game::MatchInstance::active_rules_ [private]
```

Set di regole attive.

8.29.4.2 match_id_

std::string game::MatchInstance::match_id_ [private]

Identificatore univoco del match sul database.

8.29.4.3 session_stats_

std::unordered_map<std::string, [PlayerSessionStats](#)> game::MatchInstance::session_stats_ [private]

Statistiche raccolte durante il match.

8.29.4.4 settings_

[LobbySettings](#) game::MatchInstance::settings_ [private]

Le regole e preferenze della partita.

8.29.4.5 state_

[GameState](#) game::MatchInstance::state_ [private]

Lo stato di gioco centrale.

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/game/[match_instance.hpp](#)
- src/game/[match_instance.cpp](#)

8.30 Riferimenti per la classe game::NoBluffingRule

Diagramma delle classi per game::NoBluffingRule

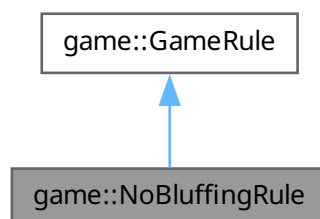
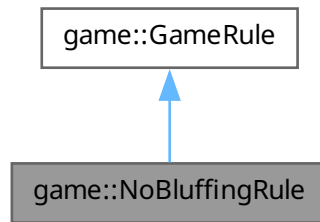


Diagramma di collaborazione per `game::NoBluffingRule`:



Membri pubblici

- void `ValidatePlay` (`GameState` *state, `CardPlayedEvent` &event) override
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

Membri pubblici ereditati da `game::GameRule`

- virtual `~GameRule` ()=default
- virtual void `OnCardPlayed` (`GameState` *state, `CardPlayedEvent` &event)
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.
- virtual void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event)

8.30.1 Documentazione delle funzioni membro

8.30.1.1 `ValidatePlay()`

```
void game::NoBluffingRule::ValidatePlay (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

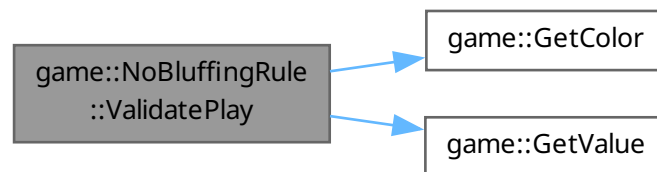
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

Parametri

state	Lo stato attuale del gioco.
event	L'evento che porta i dati della giocata. Il metodo dovrebbe impostare <code>event.is_valid_play</code> a false se la mossa è illegale.

Reimplementa `game::GameRule`.

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa classe è stata generata a partire dal seguente file:

- `src/game/rules/no_bluffing.cpp`

8.31 Riferimenti per la classe `game::PassHandsEffect`

Effetto che esegue lo scambio delle carte in mano ai giocatori.

```
#include <pass_hands.hpp>
```

Diagramma delle classi per `game::PassHandsEffect`

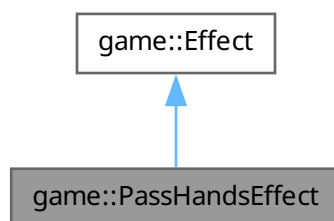
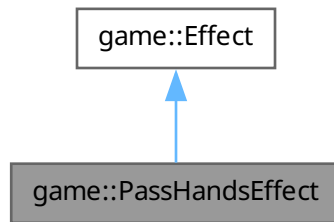


Diagramma di collaborazione per `game::PassHandsEffect`:



Membri pubblici

- `EffectResult Resolve (GameState *state, MatchInstance *match)` override

Risolve l'effetto scambiando i vettori hand all'interno di `GameState::players`. Se innescato da una rotazione globale (es. regola dello "0"), sposta le mani di una posizione seguendo la direzione di gioco corrente.

Membri pubblici ereditati da `game::Effect`

- virtual `~Effect ()`=default
- virtual `EffectType GetType ()` const
Recupera il tipo dell'effetto implementato.

8.31.1 Descrizione dettagliata

Effetto che esegue lo scambio delle carte in mano ai giocatori.

Tag Identificativo: EFFECT-PASS-001

8.31.2 Documentazione delle funzioni membro

8.31.2.1 Resolve()

```

EffectResult game::PassHandsEffect::Resolve (
    GameState * state,
    MatchInstance * match) [override], [virtual]
  
```

Risolve l'effetto scambiando i vettori hand all'interno di `GameState::players`. Se innescato da una rotazione globale (es. regola dello "0"), sposta le mani di una posizione seguendo la direzione di gioco corrente.

Parametri

state	Lo stato attuale del gioco.
match	Istanza della partita.

Restituisce

[EffectResult](#) L'esito dell'operazione.

Tag Identificativo: EFFECT-PASS-MTH-001

Implementa [game::Effect](#).

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/game/effects/[pass_hands.hpp](#)
- src/game/effects/[pass_hands.cpp](#)

8.32 Riferimenti per la struct PerSocketData

Dati di stato mantenuti all'interno di ciascuna connessione uWS.

```
#include <websocket_context.hpp>
```

Attributi pubblici

- std::string [username](#)
- std::string [room](#)
- std::string [lobby_code](#)

8.32.1 Descrizione dettagliata

Dati di stato mantenuti all'interno di ciascuna connessione uWS.

- La libreria uWebSockets detiene la proprietà della memoria di questa struttura per l'intera durata della vita del socket. Viene usata per tracciare l'identità dell'utente connesso senza dover usare mappe di lookup esterne.

Tag Identificativo: WS-CTX-STR-001

8.32.2 Documentazione dei membri dato

8.32.2.1 lobby_code

std::string PerSocketData::lobby_code

Il codice invito della lobby in cui si trova l'utente.

8.32.2.2 room

std::string PerSocketData::room

Stanza di broadcast uWS corrente (es. ID della lobby).

8.32.2.3 username

std::string PerSocketData::username

Username impostato post-upgrade dopo la verifica del JWT.

La documentazione per questa struct è stata generata a partire dal seguente file:

- include/[websocket_context.hpp](#)

8.33 Riferimenti per la struct game::Player

Rappresenta un giocatore reale o Bot all'interno della partita in corso.

```
#include <game_state.hpp>
```

Membri pubblici

- [Player](#) (std::string u, std::vector< [CompactCard](#) > h, bool b, bool uno)
Costruttore completo del [Player](#).
- [Player](#) (std::string u, bool b, bool uno)
Costruttore senza carte in mano (fase di preparazione).

Attributi pubblici

- std::string [username](#)
- std::vector< [CompactCard](#) > [hand](#)
- bool [is_bot](#)
- bool [has_called_uno](#) = false

8.33.1 Descrizione dettagliata

Rappresenta un giocatore reale o Bot all'interno della partita in corso.

Tag Identificativo: GAME-STRUCT-002

8.33.2 Documentazione dei costruttori e dei distruttori

8.33.2.1 Player() [1/2]

```
game::Player::Player (  
    std::string u,  
    std::vector< CompactCard > h,  
    bool b,  
    bool uno)    [inline]
```

Costruttore completo del [Player](#).

Parametri

u	Username.
h	Mano iniziale di carte.
b	True se Bot.
uno	True se ha già chiamato UNO.

8.33.2.2 Player() [2/2]

```
game::Player::Player (  
    std::string u,  
    bool b,  
    bool uno)    [inline]
```

Costruttore senza carte in mano (fase di preparazione).

8.33.3 Documentazione dei membri dato

8.33.3.1 hand

```
std::vector<CompactCard> game::Player::hand
```

Le carte attualmente in mano al giocatore.

8.33.3.2 `has_called_uno`

```
bool game::Player::has_called_uno = false
```

Flag che indica se ha dichiarato "UNO" correttamente.

8.33.3.3 `is_bot`

```
bool game::Player::is_bot
```

Flag che indica se il giocatore è guidato dalla CPU.

8.33.3.4 `username`

```
std::string game::Player::username
```

Nome utente identificativo.

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/game/game_state.hpp`

8.34 Riferimenti per la struct `game::PlayerSessionStats`

Traccia le statistiche delle carte giocate/pescate in una singola sessione.

```
#include <game_state.hpp>
```

Attributi pubblici

- `int color_counts [5] = {0}`
- `int value_counts [15] = {0}`

8.34.1 Descrizione dettagliata

Traccia le statistiche delle carte giocate/pescate in una singola sessione.

Tag Identificativo: GAME-STRUCT-001

8.34.2 Documentazione dei membri dato

8.34.2.1 color_counts

```
int game::PlayerSessionStats::color_counts[5] = {0}
```

Conteggio frequenze per i vari Colori (incluso Wild).

8.34.2.2 value_counts

```
int game::PlayerSessionStats::value_counts[15] = {0}
```

Conteggio frequenze per i Valori (0-9, e carte speciali).

La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/game/game_state.hpp`

8.35 Riferimenti per la classe game::ProgressiveRule

Diagramma delle classi per game::ProgressiveRule

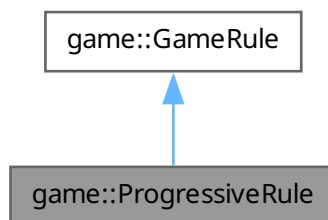
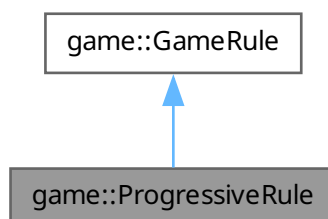


Diagramma di collaborazione per game::ProgressiveRule:



Membri pubblici

- void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event) override

Membri pubblici ereditati da `game::GameRule`

- virtual `~GameRule` ()=default
- virtual void `ValidatePlay` (`GameState` *state, `CardPlayedEvent` &event)
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- virtual void `OnCardPlayed` (`GameState` *state, `CardPlayedEvent` &event)
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.

8.35.1 Documentazione delle funzioni membro

8.35.1.1 `OnCardDrawn()`

```
void game::ProgressiveRule::OnCardDrawn (
    GameState * state,
    CardDrawnEvent & event) [inline], [override], [virtual]
```

Reimplementa `game::GameRule`.

La documentazione per questa classe è stata generata a partire dal seguente file:

- `src/game/rules/progressive.cpp`

8.36 Riferimenti per la classe `game::ReverseEffect`

Effetto che inverte il senso di rotazione dei turni (Carta Inverti).

```
#include <standard.hpp>
```

Diagramma delle classi per `game::ReverseEffect`

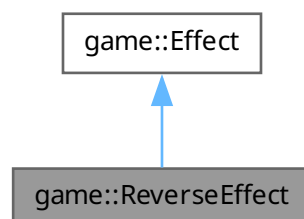
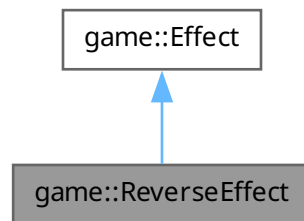


Diagramma di collaborazione per game::ReverseEffect:



Membri pubblici

- `EffectResult Resolve (GameState *state, MatchInstance *match_instance)` override
Inverte la direzione di gioco (es. da 1 a -1 o viceversa). In 2 giocatori funge da Skip.
- `EffectType GetType ()` const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da `game::Effect`

- virtual `~Effect ()`=default

8.36.1 Descrizione dettagliata

Effetto che inverte il senso di rotazione dei turni (Carta Inverti).

Tag Identificativo: EFFECT-STD-005

8.36.2 Documentazione delle funzioni membro

8.36.2.1 GetType()

`EffectType game::ReverseEffect::GetType ()` const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kReverse.

Tag Identificativo: EFFECT-STD-MTH-010

Reimplementa `game::Effect`.

8.36.2.2 Resolve()

```
EffectResult game::ReverseEffect::Resolve (  
    GameState * state,  
    MatchInstance * match_instance)  [override], [virtual]
```

Inverte la direzione di gioco (es. da 1 a -1 o viceversa). In 2 giocatori funge da Skip.

Parametri

state	Lo stato del gioco.
match_instance	Istanza della partita.

Restituisce

[EffectResult](#) Esito dell'operazione.

Tag Identificativo: EFFECT-STD-MTH-009

Implementa [game::Effect](#).

La documentazione per questa classe è stata generata a partire dai seguenti file:

- include/game/effects/[standard.hpp](#)
- src/game/effects/[standard.cpp](#)

8.37 Riferimenti per la struct `HttpRouter::Route`

Struttura interna che memorizza temporaneamente le rotte prima del caricamento.

Membri pubblici

- [Route](#) (std::string m, std::string p, [HttpHandler](#) h)

Attributi pubblici

- std::string [method](#)
- std::string [path](#)
- [HttpHandler](#) [handler](#)

8.37.1 Descrizione dettagliata

Struttura interna che memorizza temporaneamente le rotte prima del caricamento.

Tag Identificativo: HTTP-RTR-STR-001

8.37.2 Documentazione dei costruttori e dei distruttori

8.37.2.1 `Route()`

```
HttpRouter::Route::Route (  
    std::string m,  
    std::string p,  
    HttpHandler h) [inline]
```

8.37.3 Documentazione dei membri dato

8.37.3.1 `handler`

[HttpHandler](#) `HttpRouter::Route::handler`

Funzione di callback associata.

8.37.3.2 `method`

`std::string` `HttpRouter::Route::method`

Metodo HTTP ("GET" o "POST").

8.37.3.3 `path`

`std::string` `HttpRouter::Route::path`

Percorso URI registrato.

La documentazione per questa struct è stata generata a partire dal seguente file:

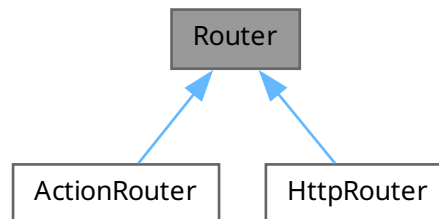
- `include/http_router.hpp`

8.38 Riferimenti per la classe Router

Interfaccia base non istanziabile per tutti i router dell'applicazione.

```
#include <router.hpp>
```

Diagramma delle classi per Router



Membri pubblici

- virtual `~Router()`=default
- `Router(const Router &)=delete`
Elimina il costruttore di copia.
- `Router & operator=(const Router &)=delete`
Elimina l'operatore di assegnamento.

Membri protetti

- `Router()`=default
Costruttore protetto, istanziabile solo dalle classi derivate.

8.38.1 Descrizione dettagliata

Interfaccia base non istanziabile per tutti i router dell'applicazione.

- Questa classe impone la regola di non-copiabilità. Dato che sia `ActionRouter` che `HttpRouter` possiedono lo stato interno degli handler, duplicarli porterebbe a comportamenti indefiniti e corruzione della memoria.

Tag Identificativo: RTR-CLS-001

8.38.2 Documentazione dei costruttori e dei distruttori

8.38.2.1 ~Router()

virtual Router::~Router () [virtual], [default]

8.38.2.2 Router() [1/2]

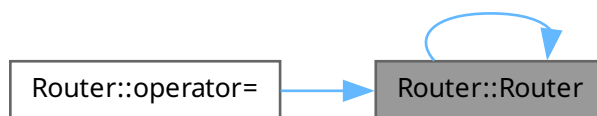
Router::Router (
const Router &) [delete]

Elimina il costruttore di copia.

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.38.2.3 Router() [2/2]

Router::Router () [protected], [default]

Costruttore protetto, istanziabile solo dalle classi derivate.

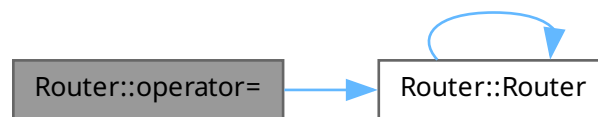
8.38.3 Documentazione delle funzioni membro

8.38.3.1 operator=()

```
Router & Router::operator= (
    const Router & ) [delete]
```

Elimina l'operatore di assegnamento.

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa classe è stata generata a partire dal seguente file:

- include/[router.hpp](#)

8.39 Riferimenti per la struct `game::RuleRegistrar`

Struttura di supporto per l'auto-registrazione delle regole.

```
#include <rule_registry.hpp>
```

Membri pubblici

- [RuleRegistrar](#) (const std::string &name, [RuleFactory](#) factory)
Costruttore che esegue l'effettiva registrazione della Factory.

8.39.1 Descrizione dettagliata

Struttura di supporto per l'auto-registrazione delle regole.

- Questa struttura è concepita per essere istanziata globalmente/staticamente nei file sorgente (.cpp) delle regole stesse. Al momento dell'inizializzazione del programma, il costruttore viene eseguito e inietta la regola nel [RuleRegistry](#).

Tag Identificativo: RULE-REG-STR-001

8.39.2 Documentazione dei costruttori e dei distruttori

8.39.2.1 RuleRegistrar()

```
game::RuleRegistrar::RuleRegistrar (  
    const std::string & name,  
    RuleFactory factory) [inline]
```

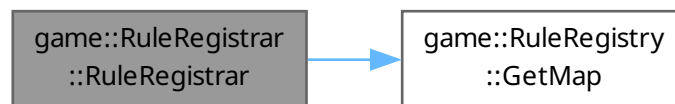
Costruttore che esegue l'effettiva registrazione della Factory.

Parametri

name	Il nome con cui la regola sarà conosciuta nel sistema.
factory	La funzione in grado di istanziare la regola.

Tag Identificativo: RULE-REG-MTH-003

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa struct è stata generata a partire dal seguente file:

- `include/game/rule_registry.hpp`

8.40 Riferimenti per la classe game::RuleRegistry

Registro globale (Singleton) delle Factory delle regole di gioco.

```
#include <rule_registry.hpp>
```

Membri pubblici statici

- static `std::unordered_map< std::string, RuleFactory > & GetMap ()`
Fornisce l'accesso alla mappa globale delle regole registrate. Utilizza il pattern "Meyers' Singleton" per garantire che la mappa venga inizializzata in modo thread-safe al primo utilizzo.
- static `std::unique_ptr< GameRule > Create (const std::string &name)`
Istanza una nuova regola di gioco a partire dal suo nome.

8.40.1 Descrizione dettagliata

Registro globale (Singleton) delle Factory delle regole di gioco.

- Mantiene una mappa che associa una stringa identificativa (il nome della mod/regola) alla funzione [RuleFactory](#) in grado di allocare la classe specifica.

Tag Identificativo: RULE-REG-CLS-001

8.40.2 Documentazione delle funzioni membro

8.40.2.1 Create()

```
std::unique_ptr< GameRule > game::RuleRegistry::Create (  
    const std::string & name)    [inline], [static]
```

Istanza una nuova regola di gioco a partire dal suo nome.

- Cerca il nome della regola nella mappa e, se trovata, invoca la sua Factory per restituirne un'istanza allocata in memoria dinamica.

Parametri

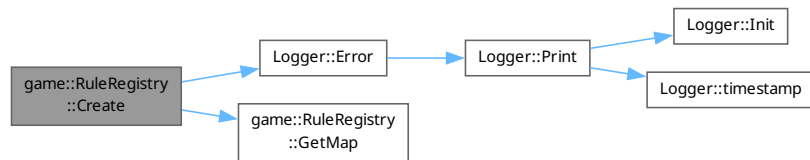
name	L'identificativo testuale della regola (es. "seven_zero").
------	--

Restituisce

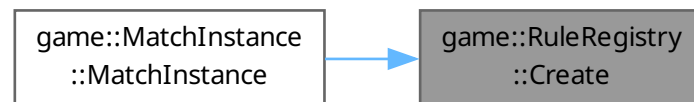
`std::unique_ptr<GameRule>` Puntatore univoco alla nuova istanza, oppure `nullptr` se la regola non esiste.

Tag Identificativo: RULE-REG-MTH-002

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.40.2.2 GetMap()

```
std::unordered_map< std::string, RuleFactory > & game::RuleRegistry::GetMap () [inline], [static]
```

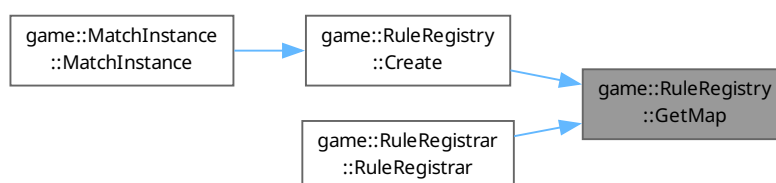
Fornisce l'accesso alla mappa globale delle regole registrate. Utilizza il pattern "Meyers' Singleton" per garantire che la mappa venga inizializzata in modo thread-safe al primo utilizzo.

Restituisce

`std::unordered_map<std::string, RuleFactory>&` Riferimento alla mappa delle regole.

Tag Identificativo: RULE-REG-MTH-001

Questo è il grafo dei chiamanti di questa funzione:



La documentazione per questa classe è stata generata a partire dal seguente file:

- [include/game/rule_registry.hpp](#)

8.41 Riferimenti per la classe `game::SevenZeroRule`

Diagramma delle classi per `game::SevenZeroRule`

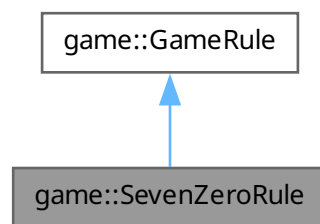
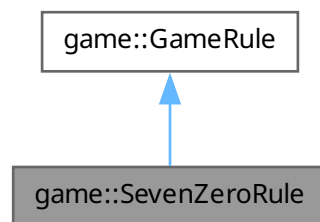


Diagramma di collaborazione per `game::SevenZeroRule`:



Membri pubblici

- `~SevenZeroRule()` `override=default`
- `void ValidatePlay (GameState *state, CardPlayedEvent &event)` `override`
Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.
- `void OnCardPlayed (GameState *state, CardPlayedEvent &event)` `override`
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.

Membri pubblici ereditati da `game::GameRule`

- virtual `~GameRule()` = default
- virtual void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event)

8.41.1 Documentazione dei costruttori e dei distruttori

8.41.1.1 `~SevenZeroRule()`

`game::SevenZeroRule::~SevenZeroRule()` [override], [default]

8.41.2 Documentazione delle funzioni membro

8.41.2.1 `OnCardPlayed()`

```
void game::SevenZeroRule::OnCardPlayed (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

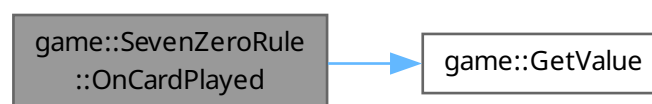
Scatena le conseguenze derivanti da una giocata convalidata. Viene chiamato solo se `ValidatePlay` ha permesso la giocata. Tipicamente usato per istanziare ed accodare le istanze figlie di `Effect` nella coda degli effetti di `GameState`.

Parametri

state	Lo stato attuale del gioco.
event	L'evento contenente la giocata risolta.

Reimplementa `game::GameRule`.

Questo è il grafo delle chiamate per questa funzione:



8.41.2.2 ValidatePlay()

```
void game::SevenZeroRule::ValidatePlay (
    GameState * state,
    CardPlayedEvent & event) [inline], [override], [virtual]
```

Valuta se la carta giocata rispetta le regole imposte. Viene chiamato prima che la carta venga effettivamente accettata dal motore.

Parametri

state	Lo stato attuale del gioco.
event	L'evento che porta i dati della giocata. Il metodo dovrebbe impostare event.is_valid_play a false se la mossa è illegale.

Reimplementa `game::GameRule`.

La documentazione per questa classe è stata generata a partire dal seguente file:

- `src/game/rules/70.cpp`

8.42 Riferimenti per la classe `game::SkipEffect`

Effetto che salta il turno del giocatore successivo (Carta Divieto).

```
#include <standard.hpp>
```

Diagramma delle classi per `game::SkipEffect`

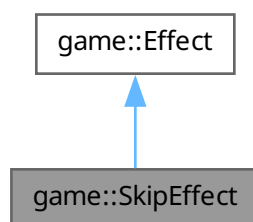
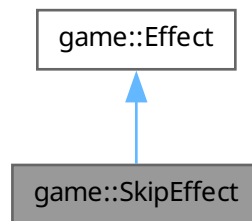


Diagramma di collaborazione per game::SkipEffect:



Membri pubblici

- `EffectResult Resolve (GameState *state, MatchInstance *match_instance)` override
Avanza l'indice del giocatore di un passo addizionale, di fatto saltandolo.
- `EffectType GetType ()` const override
Restituisce il tipo enumerativo dell'effetto.

Membri pubblici ereditati da `game::Effect`

- virtual `~Effect ()`=default

8.42.1 Descrizione dettagliata

Effetto che salta il turno del giocatore successivo (Carta Divieto).

Tag Identificativo: EFFECT-STD-004

8.42.2 Documentazione delle funzioni membro

8.42.2.1 GetType()

`EffectType game::SkipEffect::GetType ()` const [inline], [override], [virtual]

Restituisce il tipo enumerativo dell'effetto.

Restituisce

`EffectType` Identificatore kSkip.

Tag Identificativo: EFFECT-STD-MTH-008

Reimplementa `game::Effect`.

8.42.2.2 Resolve()

```
EffectResult game::SkipEffect::Resolve (  
    GameState * state,  
    MatchInstance * match_instance)  [override], [virtual]
```

Avanza l'indice del giocatore di un passo addizionale, di fatto saltandolo.

Parametri

state	Lo stato del gioco.
match_instance	Istanza della partita.

Restituisce

EffectResult Esito dell'operazione.

Tag Identificativo: EFFECT-STD-MTH-007

Implementa [game::Effect](#).

La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/game/effects/standard.hpp](#)
- [src/game/effects/standard.cpp](#)

8.43 Riferimenti per la classe game::StandardRule

Validatore e processore principale delle meccaniche standard del gioco.

```
#include <standard.hpp>
```

Diagramma delle classi per game::StandardRule

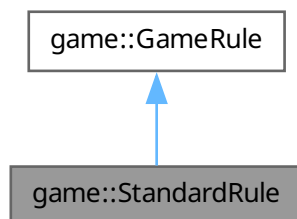
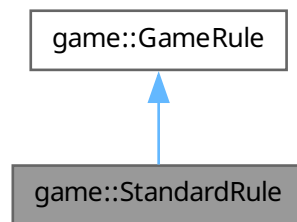


Diagramma di collaborazione per game::StandardRule:



Membri pubblici

- `~StandardRule()` override=default
- void `ValidatePlay` (`GameState` *state, `CardPlayedEvent` &event) override
 Intercetta il tentativo di giocata valutandone la legalità. Verifica se la carta giocata corrisponde in colore o in valore/simbolo alla carta in cima agli scarti (o al colore attivo per i Jolly). Modifica event.`is_valid_play` in caso negativo.
- void `OnCardPlayed` (`GameState` *state, `CardPlayedEvent` &event) override
 Esegue le conseguenze di una giocata confermata valida. Mette in coda in `GameState::effect_queue` le classi `Effect` pertinenti basate sull'enumeratore `Value` estratto dalla carta compatta giocata.

Membri pubblici ereditati da game::GameRule

- virtual `~GameRule()`=default
- virtual void `OnCardDrawn` (`GameState` *state, `CardDrawnEvent` &event)

Membri privati

- std::string `GetNextPlayer` (`GameState` *state)
 Calcola chi sarà il giocatore a subire le penalità o il prossimo a giocare. Metodo di supporto utilizzato internamente per determinare il target (es. per `SkipEffect` o `DrawEffect`).

8.43.1 Descrizione dettagliata

Validatore e processore principale delle meccaniche standard del gioco.

- Assicura che la mossa sia legale rispetto allo stato attuale del tavolo e, se convalidata, decodifica il valore della carta istanziando gli Effetti corrispondenti (es. se la carta è un "Pesca Due", accoda un `DrawEffect` e un `AdvanceTurnEffect`).

Tag Identificativo: RULE-STD-001

8.43.2 Documentazione dei costruttori e dei distruttori

8.43.2.1 ~StandardRule()

game::StandardRule::~StandardRule () [override], [default]

8.43.3 Documentazione delle funzioni membro

8.43.3.1 GetNextPlayer()

std::string game::StandardRule::GetNextPlayer (
[GameState](#) * state) [private]

Calcola chi sarà il giocatore a subire le penalità o il prossimo a giocare. Metodo di supporto utilizzato internamente per determinare il target (es. per [SkipEffect](#) o [DrawEffect](#)).

- Parametri

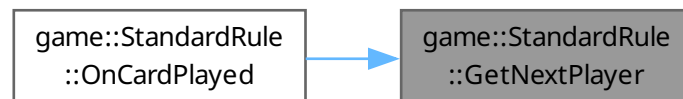
state	Puntatore allo stato del gioco.
-------	---------------------------------

Restituisce

std::string Username del giocatore bersaglio calcolato seguendo la direzione di gioco.

Tag Identificativo: RULE-STD-PRIV-001

Questo è il grafo dei chiamanti di questa funzione:



8.43.3.2 OnCardPlayed()

```
void game::StandardRule::OnCardPlayed (
    GameState * state,
    CardPlayedEvent & event) [override], [virtual]
```

Esegue le conseguenze di una giocata confermata valida. Mette in coda in `GameState::effect_queue` le classi `Effect` pertinenti basate sull'enumeratore `Value` estratto dalla carta compatta giocata.

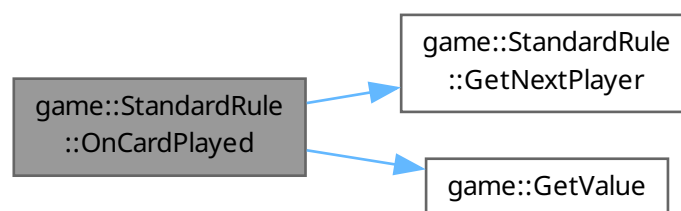
Parametri

state	Puntatore allo stato di gioco.
event	L'evento della carta giocata.

Tag Identificativo: RULE-STD-MTH-002

Reimplementa `game::GameRule`.

Questo è il grafo delle chiamate per questa funzione:



8.43.3.3 ValidatePlay()

```
void game::StandardRule::ValidatePlay (
    GameState * state,
    CardPlayedEvent & event) [override], [virtual]
```

Intercetta il tentativo di giocata valutandone la legalità. Verifica se la carta giocata corrisponde in colore o in valore/simbolo alla carta in cima agli scarti (o al colore attivo per i Jolly). Modifica `event.is_valid_play` in caso negativo.

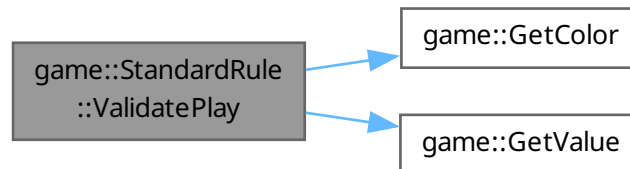
Parametri

state	Puntatore allo stato di gioco attuale.
event	Struttura evento contenente i dettagli della giocata.

Tag Identificativo: RULE-STD-MTH-001

Reimplementa [game::GameRule](#).

Questo è il grafo delle chiamate per questa funzione:



La documentazione per questa classe è stata generata a partire dai seguenti file:

- [include/game/rules/standard.hpp](#)
- [src/game/rules/standard.cpp](#)

8.44 Riferimenti per la classe StatsController

Esponde gli endpoint REST necessari al frontend per il pannello Profilo e Leaderboard.

```
#include <stats_controller.hpp>
```

Membri pubblici

- [StatsController](#) ([HttpRouter](#) &router)
Costruttore del [StatsController](#). Registra le rotte HTTP (metodi GET) sull'[HttpRouter](#) fornito.

Membri privati

- void [HandleGetMe](#) ([AppResponse](#) *res, [AppRequest](#) *req)
Handler per la rotta GET /stats/me.
- void [HandleGetLeaderboard](#) ([AppResponse](#) *res, [AppRequest](#) *req)
Handler per la rotta GET /stats/leaderboard.

8.44.1 Descrizione dettagliata

Esponde gli endpoint REST necessari al frontend per il pannello Profilo e Leaderboard.

Tag Identificativo: CTRL-STATS-001

8.44.2 Documentazione dei costruttori e dei distruttori

8.44.2.1 StatsController()

StatsController::StatsController (
[HttpRouter](#) & router) [explicit]

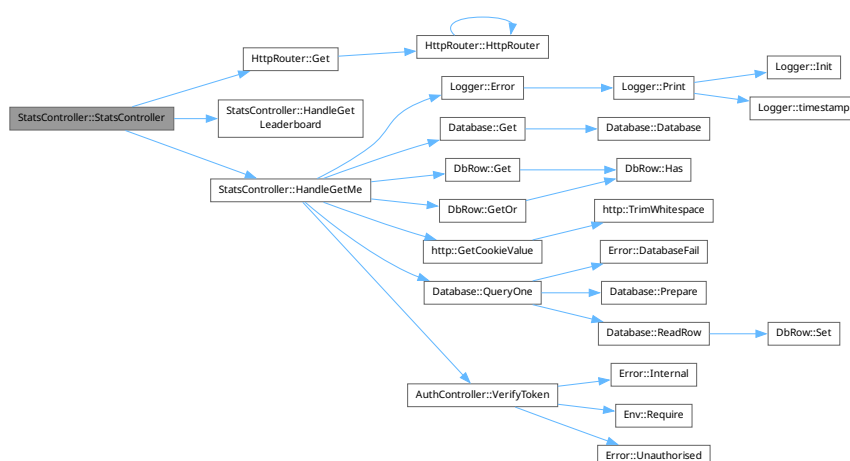
Costruttore del [StatsController](#). Registra le rotte HTTP (metodi GET) sull'[HttpRouter](#) fornito.

Parametri

router	Il router HTTP centrale dell'applicazione.
--------	--

Tag Identificativo: CTRL-STATS-MTH-001

Questo è il grafo delle chiamate per questa funzione:



8.44.3 Documentazione delle funzioni membro

8.44.3.1 HandleGetLeaderboard()

```
void StatsController::HandleGetLeaderboard (
    AppResponse * res,
    AppRequest * req) [private]
```

Handler per la rotta GET /stats/leaderboard.

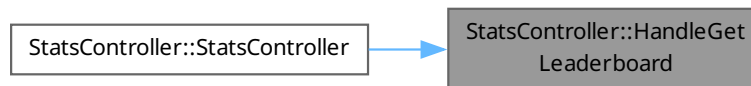
- Esegue una query sul database per restituire la top 50 dei giocatori ordinata per numero di vittorie decrescenti.

Parametri

res	Puntatore alla risposta HTTP.
req	Puntatore alla richiesta HTTP.

Tag Identificativo: CTRL-STATS-ACT-002

Questo è il grafo dei chiamanti di questa funzione:



8.44.3.2 HandleGetMe()

```
void StatsController::HandleGetMe (
    AppResponse * res,
    AppRequest * req) [private]
```

Handler per la rotta GET /stats/me.

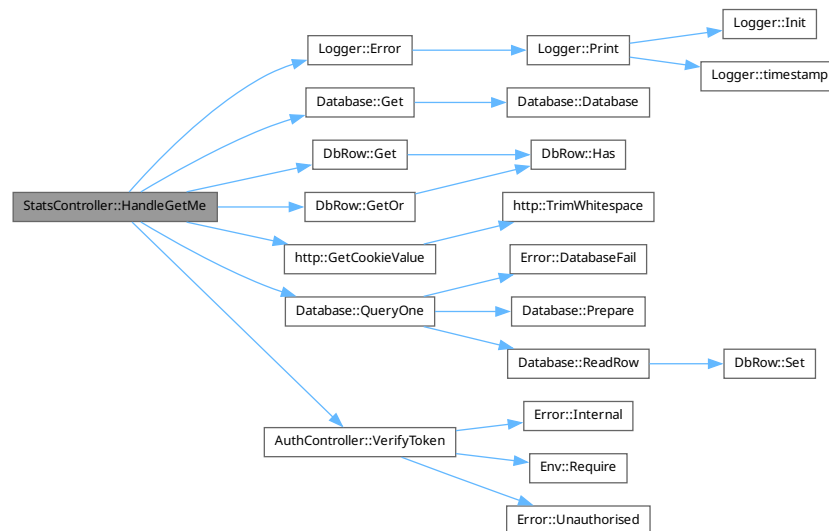
- Estrae le statistiche personali e la posizione in classifica (rank) del giocatore attualmente autenticato (valutando il JWT nell'header).

Parametri

res	Puntatore alla risposta HTTP (uWS).
req	Puntatore alla richiesta HTTP (uWS).

Tag Identificativo: CTRL-STATS-ACT-001

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



La documentazione per questa classe è stata generata a partire dai seguenti file:

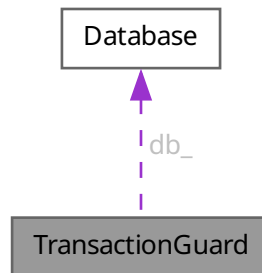
- include/controllers/[stats_controller.hpp](#)
- src/controllers/[stats_controller.cpp](#)

8.45 Riferimenti per la classe TransactionGuard

Implementa l'esecuzione sicura di transazioni database tramite il paradigma RAIL.

```
#include <database.hpp>
```

Diagramma di collaborazione per TransactionGuard:



Memberi pubblici

- [TransactionGuard](#) ([Database](#) &db)
Inizia la transazione eseguendo "BEGIN TRANSACTION".
- [~TransactionGuard](#) ()
Distruttore: Esegue "ROLLBACK" se la transazione non è stata completata regolarmente.
- void [Commit](#) ()
Marca la transazione per un completamento vincente (esegue "COMMIT"). Se chiamato, annulla l'effetto di rollback alla distruzione del Guard.
- [TransactionGuard](#) (const [TransactionGuard](#) &)=delete
- [TransactionGuard](#) & operator= (const [TransactionGuard](#) &)=delete

Attributi privati

- [Database](#) & db_
- bool [committed_](#)

8.45.1 Descrizione dettagliata

Implementa l'esecuzione sicura di transazioni database tramite il paradigma RAII.

- Garantisce che un eventuale fallimento bloccante all'interno del suo scope C++ scateni automaticamente un ROLLBACK (se non è stato chiamato esplicitamente [Commit\(\)](#)), evitando inconsistenze nel database.

Tag Identificativo: DB-TX-001

8.45.2 Documentazione dei costruttori e dei distruttori

8.45.2.1 TransactionGuard() ^[1/2]

TransactionGuard::TransactionGuard (
 Database & db) [inline], [explicit]

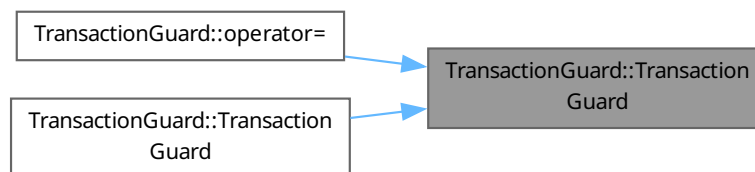
Inizia la transazione eseguendo "BEGIN TRANSACTION".

Parametri

db	Il database su cui operare.
----	-----------------------------

Tag Identificativo: DB-TX-MTH-001

Questo è il grafo dei chiamanti di questa funzione:



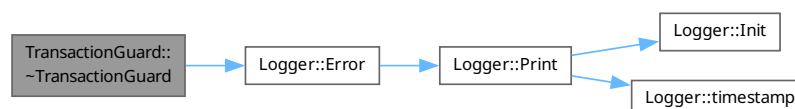
8.45.2.2 ~TransactionGuard()

TransactionGuard::~~TransactionGuard () [inline]

Distruttore: Esegue "ROLLBACK" se la transazione non è stata completata regolarmente.

Tag Identificativo: DB-TX-MTH-002

Questo è il grafo delle chiamate per questa funzione:

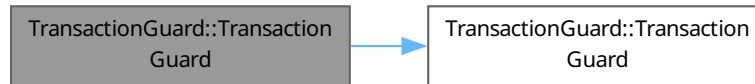


8.45.2.3 TransactionGuard()

[2/2]

```
TransactionGuard::TransactionGuard (  
    const TransactionGuard & ) [delete]
```

Questo è il grafo delle chiamate per questa funzione:



8.45.3 Documentazione delle funzioni membro

8.45.3.1 Commit()

```
void TransactionGuard::Commit () [inline]
```

Marca la transazione per un completamento vincente (esegue "COMMIT"). Se chiamato, annulla l'effetto di rollback alla distruzione del Guard.

Tag Identificativo: DB-TX-MTH-003

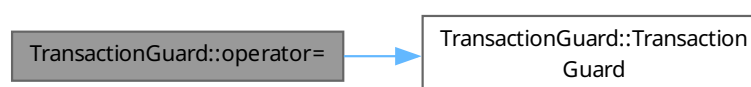
Questo è il grafo dei chiamanti di questa funzione:



8.45.3.2 operator=()

```
TransactionGuard & TransactionGuard::operator= (  
    const TransactionGuard & ) [delete]
```

Questo è il grafo delle chiamate per questa funzione:



8.45.4 Documentazione dei membri dato

8.45.4.1 committed_

bool TransactionGuard::committed_ [private]

8.45.4.2 db_

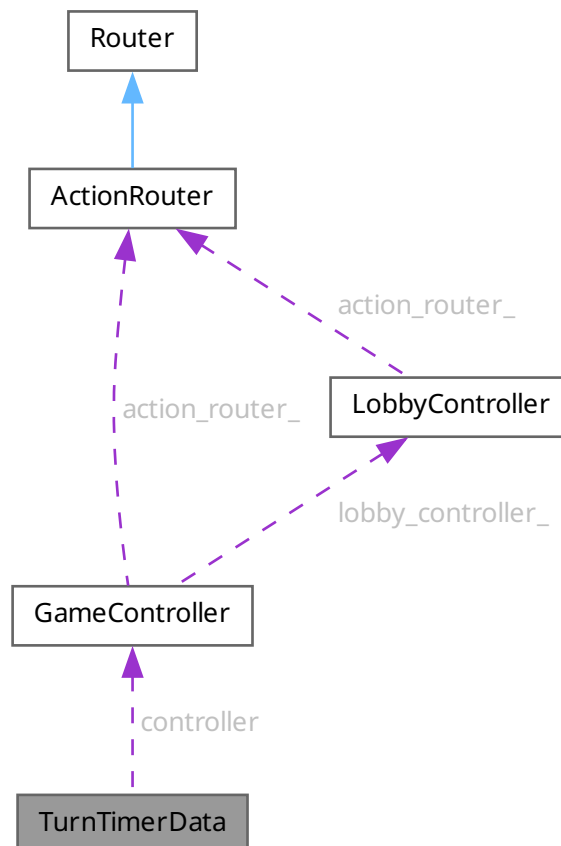
[Database&](#) TransactionGuard::db_ [private]

La documentazione per questa classe è stata generata a partire dal seguente file:

- [include/database.hpp](#)

8.46 Riferimenti per la struct TurnTimerData

Diagramma di collaborazione per TurnTimerData:



Attributi pubblici

- `std::function< void()>` [callback](#)
- `uint32_t` [lobby_id](#)
- [GameController](#) * [controller](#)

8.46.1 Documentazione dei membri dato

8.46.1.1 callback

`std::function<void()>` TurnTimerData::callback

8.46.1.2 controller

[GameController](#)* TurnTimerData::controller

8.46.1.3 lobby_id

`uint32_t` TurnTimerData::lobby_id

La documentazione per questa struct è stata generata a partire dal seguente file:

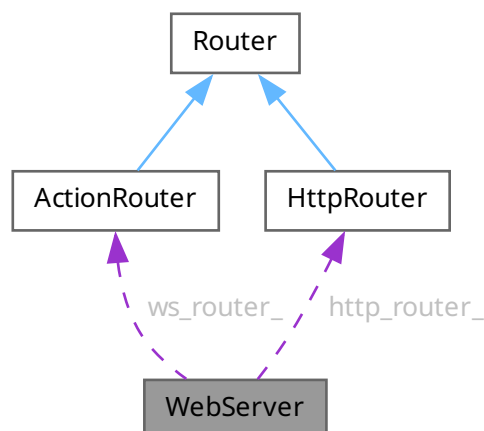
- `src/controllers/`[game_controller.cpp](#)

8.47 Riferimenti per la classe WebServer

Motore principale dell'applicazione backend.

```
#include <webserver.hpp>
```

Diagramma di collaborazione per WebServer:



Tipi pubblici

- using [ConnectionHandler](#) = std::function<void([AppWebSocket*](#), [PerSocketData*](#))>
Callback invocata quando un WebSocket viene aperto o chiuso.

Memberi pubblici

- [WebServer](#) (int port, std::string_view keyFile="key.pem", std::string_view certFile="cert.pem", std::string_view dbFile="game.db")
Costruttore del [WebServer](#).
- [~WebServer](#) ()
Distruttore. Si occupa di rilasciare eventuali risorse pendenti.
- [WebServer](#) (const [WebServer](#) &)=delete
- [WebServer](#) & operator= (const [WebServer](#) &)=delete
- void [Run](#) ()
Avvia il loop di ascolto del server (metodo bloccante). Mette in ascolto uWebSockets sulla porta specificata.
- [ActionRouter](#) & [GetActionRouter](#) ()
Recupera un riferimento al router delle azioni WebSocket.
- [HttpRouter](#) & [GetHTTPRouter](#) ()
Recupera un riferimento al router delle richieste HTTP.
- uWS::SSLApp & [GetApp](#) ()
Recupera l'istanza sottostante dell'applicazione uWebSockets.
- void [OnConnectionOpen](#) ([ConnectionHandler](#) handler)
Aggiunge una callback da eseguire all'apertura di una nuova connessione WS.
- void [OnConnectionClose](#) ([ConnectionHandler](#) handler)
Aggiunge una callback da eseguire alla chiusura di una connessione WS.

Memberi privati

- bool [InitDB](#) ()
Inizializza la connessione al database e ne applica lo schema se necessario.
- void [RegisterRoutes](#) ()
Registra le rotte interne, applica i wildcard (middleware) all'app uWS.
- void [HandlePost](#) ([AppResponse](#) *, [AppRequest](#) *)
Handler generico per richieste HTTP POST (delegato all'[HttpRouter](#)).
- void [HandleGet](#) ([AppResponse](#) *, [AppRequest](#) *)
Handler generico per richieste HTTP GET (delegato all'[HttpRouter](#)).
- void [OnSocketOpen](#) ([AppWebSocket](#) *)
Gestisce l'evento di upgrade WebSocket andato a buon fine.
- void [OnSocketMessage](#) ([AppWebSocket](#) *, std::string_view, uWS::OpCode)
Riceve i frame WebSocket in ingresso e li inoltra all'[ActionRouter](#).
- void [OnSocketClosed](#) ([AppWebSocket](#) *)
Gestisce l'evento di disconnessione di un socket.

Memberi privati statici

- static std::string [ReadFile](#) (std::string_view path)
Utilità interna per leggere il contenuto completo di un file (es. per risorse statiche).
- static std::string [GetMimeType](#) (const string &path)
Deduce il MimeType di un file basandosi sull'estensione.

Attributi privati

- int [port_](#)
- string [db_file_](#)
- uWS::SSLApp [app_](#)
- std::map< string, [AppWebSocket](#) * > [connections_](#)
- [ActionRouter](#) [ws_router_](#)
- [HttpRouter](#) [http_router_](#)
- std::vector< [ConnectionHandler](#) > [on_open_hooks_](#)
- std::vector< [ConnectionHandler](#) > [on_close_hooks_](#)

8.47.1 Descrizione dettagliata

Motore principale dell'applicazione backend.

- Inizializza l'applicazione SSL (se configurata), configura i router HTTP e WebSocket, mantiene un registro delle connessioni attive e fornisce hook per eventi di apertura/chiusura connessione. Non supporta copia o assegnamento (classe non copiabile).

Tag Identificativo: SRV-CORE-000

•

8.47.2 Documentazione delle ridefinizioni dei tipi (typedef)

8.47.2.1 ConnectionHandler

```
using WebServer::ConnectionHandler = std::function<void(AppWebSocket*, PerSocketData*)>
```

Callback invocata quando un WebSocket viene aperto o chiuso.

Tag Identificativo: SRV-TYP-001

8.47.3 Documentazione dei costruttori e dei distruttori

8.47.3.1 WebServer() ^[1/2]

```
WebServer::WebServer (
    int port,
    std::string_view keyFile = "key.pem",
    std::string_view certFile = "cert.pem",
    std::string_view dbFile = "game.db") [explicit]
```

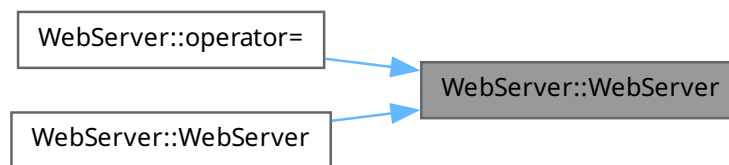
Costruttore del [WebServer](#).

Parametri

port	La porta su cui il server starà in ascolto (es. 443 o 8080).
keyFile	Percorso al file della chiave privata SSL.
certFile	Percorso al file del certificato SSL.
dbFile	Percorso al file del database SQLite.

Tag Identificativo: SRV-CORE-001

Questo è il grafo dei chiamanti di questa funzione:



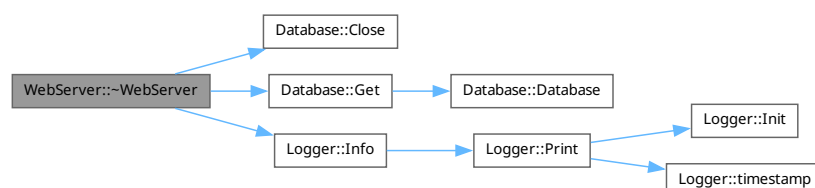
8.47.3.2 ~WebServer()

WebServer::~~WebServer ()

Distruttore. Si occupa di rilasciare eventuali risorse pendenti.

Tag Identificativo: SRV-CORE-002

Questo è il grafo delle chiamate per questa funzione:



8.47.3.3 WebServer() ^[2/2]

WebServer::WebServer (
 const [WebServer](#) &) [delete]

Questo è il grafo delle chiamate per questa funzione:



8.47.4 Documentazione delle funzioni membro

8.47.4.1 GetActionRouter()

[ActionRouter](#) & WebServer::GetActionRouter () [inline]

Recupera un riferimento al router delle azioni WebSocket.

Restituisce

[ActionRouter](#)& [Router](#) per registrare gli handler dei messaggi WS.

Tag Identificativo: SRV-CORE-004

Questo è il grafo dei chiamanti di questa funzione:



8.47.4.2 GetApp()

`uWS::SSLApp & WebServer::GetApp ()` [inline]

Recupera l'istanza sottostante dell'applicazione uWebSockets.

Restituisce

`uWS::SSLApp&` L'applicazione uWS.

Tag Identificativo: SRV-CORE-006

8.47.4.3 GetHTTPRouter()

`HttpRouter & WebServer::GetHTTPRouter ()` [inline]

Recupera un riferimento al router delle richieste HTTP.

Restituisce

`HttpRouter&` `Router` per registrare le rotte REST/HTTP.

Tag Identificativo: SRV-CORE-005

Questo è il grafo dei chiamanti di questa funzione:



8.47.4.4 GetMimeType()

```
string WebServer::GetMimeType (  
    const string & path) [static], [private]
```

Deduce il MimeType di un file basandosi sull'estensione.

Parametri

path	Percorso del file.
------	--------------------

Restituisce

std::string Il MIME Type (es. "text/html").

Tag Identificativo: SRV-UTIL-002

8.47.4.5 HandleGet()

```
void WebServer::HandleGet (  
    AppResponse * res,  
    AppRequest * req) [private]
```

Handler generico per richieste HTTP GET (delegato all'[HttpRouter](#)).

Tag Identificativo: SRV-PRIV-004

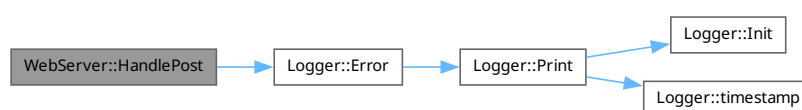
8.47.4.6 HandlePost()

```
void WebServer::HandlePost (  
    AppResponse * response,  
    AppRequest * request) [private]
```

Handler generico per richieste HTTP POST (delegeto all'[HttpRouter](#)).

Tag Identificativo: SRV-PRIV-003

Questo è il grafo delle chiamate per questa funzione:



8.47.4.7 InitDB()

```
bool WebServer::InitDB () [private]
```

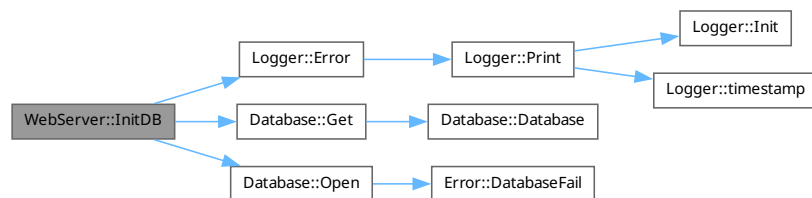
Inizializza la connessione al database e ne applica lo schema se necessario.

Restituisce

true se l'inizializzazione ha avuto successo, false altrimenti.

Tag Identificativo: SRV-PRIV-001

Questo è il grafo delle chiamate per questa funzione:



8.47.4.8 OnConnectionClose()

```
void WebServer::OnConnectionClose (
    ConnectionHandler handler)
```

Aggiunge una callback da eseguire alla chiusura di una connessione WS.

Parametri

handler	La funzione da registrare.
---------	----------------------------

Tag Identificativo: SRV-CORE-008

Questo è il grafo dei chiamanti di questa funzione:



8.47.4.9 OnConnectionOpen()

```
void WebServer::OnConnectionOpen (  
    ConnectionHandler handler)
```

Aggiunge una callback da eseguire all'apertura di una nuova connessione WS.

Parametri

handler	La funzione da registrare.
---------	----------------------------

Tag Identificativo: SRV-CORE-007

Questo è il grafo dei chiamanti di questa funzione:



8.47.4.10 OnSocketClosed()

```
void WebServer::OnSocketClosed (  
    AppWebSocket * socket) [private]
```

Gestisce l'evento di disconnessione di un socket.

Tag Identificativo: SRV-PRIV-007

8.47.4.11 OnSocketMessage()

```
void WebServer::OnSocketMessage (  
    AppWebSocket * ,  
    std::string_view ,  
    uWS::OpCode ) [private]
```

Riceve i frame WebSocket in ingresso e li inoltra all'[ActionRouter](#).

Tag Identificativo: SRV-PRIV-006

8.47.4.12 OnSocketOpen()

```
void WebServer::OnSocketOpen (  
    AppWebSocket * socket) [private]
```

Gestisce l'evento di upgrade WebSocket andato a buon fine.

Tag Identificativo: SRV-PRIV-005

8.47.4.13 operator=()

```
WebServer & WebServer::operator= (  
    const WebServer & ) [delete]
```

Questo è il grafo delle chiamate per questa funzione:



8.47.4.14 ReadFile()

```
string WebServer::ReadFile (  
    std::string_view path) [static], [private]
```

Utilità interna per leggere il contenuto completo di un file (es. per risorse statiche).

Parametri

path	Percorso del file.
------	--------------------

Restituisce

std::string Il contenuto del file.

Tag Identificativo: SRV-UTIL-001

8.47.4.15 RegisterRoutes()

```
void WebServer::RegisterRoutes () [private]
```

Registra le rotte interne, applica i wildcard (middleware) all'app uWS.

Tag Identificativo: SRV-PRIV-002

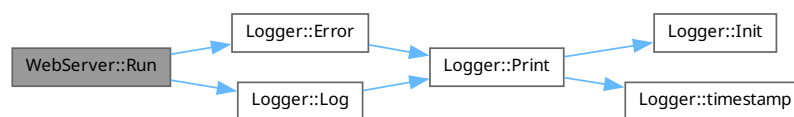
8.47.4.16 Run()

```
void WebServer::Run ()
```

Avvia il loop di ascolto del server (metodo bloccante). Mette in ascolto uWebSockets sulla porta specificata.

Tag Identificativo: SRV-CORE-003

Questo è il grafo delle chiamate per questa funzione:



Questo è il grafo dei chiamanti di questa funzione:



8.47.5 Documentazione dei membri data

8.47.5.1 app_

```
uWS::SSLApp WebServer::app_ [private]
```

Istanza principale di uWebSockets (SSL/TLS).

8.47.5.2 connections_

`std::map<string, AppWebSocket\*> WebServer::connections_` [private]

Mappa degli utenti connessi (Username -> Socket).

8.47.5.3 db_file_

`string WebServer::db_file_` [private]

Percorso del file DB sqlite.

8.47.5.4 http_router_

[HttpRouter](#) `WebServer::http_router_` [private]

Gestore del dispatching delle richieste HTTP.

8.47.5.5 on_close_hooks_

`std::vector<ConnectionHandler> WebServer::on_close_hooks_` [private]

Lista di hook di chiusura connessione.

8.47.5.6 on_open_hooks_

`std::vector<ConnectionHandler> WebServer::on_open_hooks_` [private]

Lista di hook di apertura connessione.

8.47.5.7 port_

`int WebServer::port_` [private]

Porta di ascolto del server.

8.47.5.8 ws_router_

[ActionRouter](#) `WebServer::ws_router_` [private]

Gestore del dispatching dei messaggi WebSocket.

La documentazione per questa classe è stata generata a partire dai seguenti file:

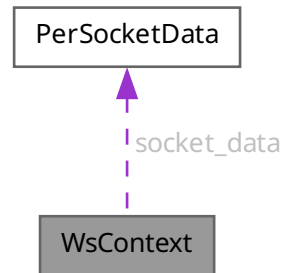
- `include/webserver.hpp`
- `src/webserver.cpp`

8.48 Riferimenti per la struct WsContext

Contesto di esecuzione passato ad ogni gestore del [ActionRouter](#).

```
#include <websocket_context.hpp>
```

Diagramma di collaborazione per WsContext:



Attributi pubblici

- [AppWebSocket](#) * [socket](#)
- [PerSocketData](#) * [socket_data](#)
- [uWS::OpCode](#) [op_code](#)

8.48.1 Descrizione dettagliata

Contesto di esecuzione passato ad ogni gestore del [ActionRouter](#).

- Incapsula tutto ciò di cui un handler ha bisogno per processare un messaggio e rispondere, eliminando la necessità di accedere a stati globali.

Tag Identificativo: WS-CTX-STR-002

8.48.2 Documentazione dei membri dato

8.48.2.1 op_code

[uWS::OpCode](#) [WsContext::op_code](#)

Codice operativo del frame WebSocket (Es. Testo o Binario).

8.48.2.2 `socket`

[AppWebSocket*](#) WsContext::socket

Puntatore al socket che ha inviato il messaggio.

8.48.2.3 `socket_data`

[PerSocketData*](#) WsContext::socket_data

Dati di sessione associati al socket chiamante.

La documentazione per questa struct è stata generata a partire dal seguente file:

- [include/websocket_context.hpp](#)

Capitolo 9

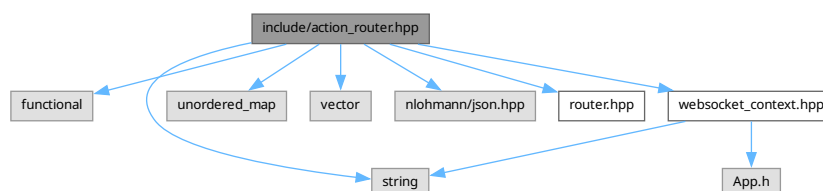
Documentazione dei file

9.1 Riferimenti per il file include/action_router.hpp

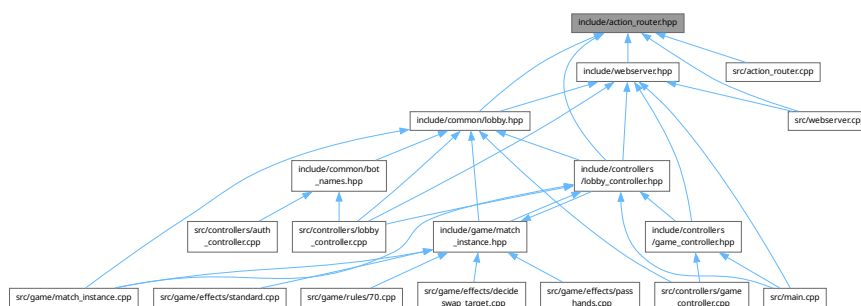
Implementazione del router per lo smistamento dei messaggi WebSocket.

```
#include <functional>
#include <string>
#include <unordered_map>
#include <vector>
#include <nlohmann/json.hpp>
#include <router.hpp>
#include <websocket_context.hpp>
```

Grafo delle dipendenze di inclusione per action_router.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class `ActionRouter`
Smista i messaggi WebSocket ai vari controller in base al tipo di azione.

Ridefinizioni di tipo (typedef)

- using `json` = `nlohmann::json`
- using `ActionHandler` = `std::function<bool(WsContext, const json&)>`
Firma per i gestori delle azioni WebSocket.

9.1.1 Descrizione dettagliata

Implementazione del router per lo smistamento dei messaggi WebSocket.

- Indirizza i messaggi JSON in arrivo ai gestori appropriati ispezionando il campo "action" all'interno del payload.

9.1.2 Documentazione delle ridefinizioni di tipo (typedef)

9.1.2.1 ActionHandler

```
using ActionHandler = std::function<bool(WsContext, const json&)>
```

Firma per i gestori delle azioni WebSocket.

- Restituisce un booleano:
- true: Continua il dispatch (passa al gestore specifico successivo).
- false: Interrompe il dispatch (usato dai middleware/wildcard per bloccare l'azione, es. guardia di autenticazione).

Tag Identificativo: ACT-RTR-TYP-001

9.1.2.2 json

```
using json = nlohmann::json
```

9.2 action_router.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <functional>
00003 #include <string>
00004 #include <unordered_map>
00005 #include <vector>
00006 #include <nlohmann/json.hpp>
00007 #include <router.hpp>
00008 #include <websocket_context.hpp>
00009
00016
00017 using json = nlohmann::json;
00018
00027 using ActionHandler = std::function<bool(WsContext, const json&)>;
00028
00037 class ActionRouter : public Router {
00038 public:
00039     ActionRouter() = default;
00040
00048     ActionRouter& On(const std::string& action, ActionHandler handler);
00049
00057     ActionRouter& OnAny(ActionHandler handler);
00058
00068     bool Dispatch(WsContext ctx, const json& msg) const;
00069
00070 private:
00071     std::unordered_map<std::string, ActionHandler> handlers_;
00072
00073     std::vector<ActionHandler> wildcards_;
00074 };

```

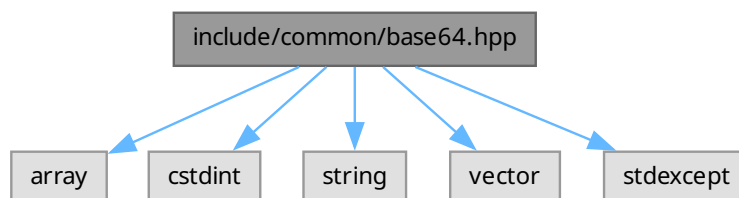
9.3 Riferimenti per il file include/common/base64.hpp

```

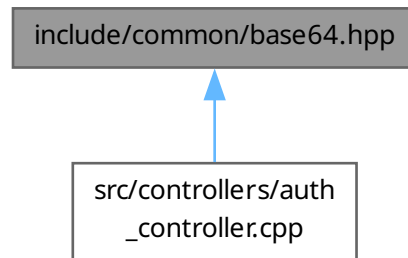
#include <array>
#include <cstdint>
#include <string>
#include <vector>
#include <stdexcept>

```

Grafo delle dipendenze di inclusione per base64.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Namespace

- namespace [Base64](#)

Funzioni

- `std::string` [Base64::Encode](#) (`const std::vector< uint8_t > &data`)
- `std::vector< uint8_t >` [Base64::Decode](#) (`const std::string &encoded`)

Variabili

- `constexpr char` [Base64::kAlphabet](#) [] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
- `constexpr uint8_t` [Base64::kInvalid](#) = 0xFF
- `constexpr std::array< uint8_t, 256 >` [Base64::kDecodeTable](#)

9.4 base64.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <array>
00003 #include <cstdint>
00004 #include <string>
00005 #include <vector>
00006 #include <stdexcept>
00007
00008 namespace Base64 {
00009
00010 inline constexpr char kAlphabet[] =
00011     "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
00012
00013 inline constexpr uint8_t kInvalid = 0xFF;
00014
00015 inline constexpr std::array<uint8_t, 256> kDecodeTable = []() {
00016     std::array<uint8_t, 256> table{};
00017     table.fill(kInvalid);
00018     for (uint8_t i = 0; i < 64; ++i) {
00019         table[static_cast<uint8_t>(kAlphabet[i])] = i;
00020     }
00021 }
00022

```

```

00021     return table;
00022 }();
00023
00024 // Takes arbitrary binary data and returns its base64 representation.
00025 // Every 3 input bytes → 4 output characters; '=' padding fills the tail.
00026 //
00027 // Bit arithmetic:
00028 //   Given bytes [A, B, C] packed into a 24-bit integer:
00029 //     bits 23-18 → index 0
00030 //     bits 17-12 → index 1
00031 //     bits 11- 6 → index 2
00032 //     bits  5- 0 → index 3
00033 //
00034 // @param data   Raw bytes to encode.
00035 // @returns      Base64-encoded ASCII string.
00036 inline std::string Encode(const std::vector<uint8_t>& data) {
00037     std::string out;
00038     out.reserve(((data.size() + 2) / 3) * 4);
00039
00040     for (size_t i = 0; i < data.size(); i += 3) {
00041         uint32_t b = static_cast<uint32_t>(data[i]) « 16;
00042         if (i + 1 < data.size()) b |= static_cast<uint32_t>(data[i + 1]) « 8;
00043         if (i + 2 < data.size()) b |= static_cast<uint32_t>(data[i + 2]);
00044
00045         out += kAlphabet[(b » 18) & 0x3F];
00046         out += kAlphabet[(b » 12) & 0x3F];
00047
00048         out += (i + 1 < data.size()) ? kAlphabet[(b » 6) & 0x3F] : '=';
00049         out += (i + 2 < data.size()) ? kAlphabet[ b      & 0x3F] : '=';
00050     }
00051     return out;
00052 }
00053
00054 // Inverse of Encode. Validates every character against kDecodeTable and
00055 // throws std::invalid_argument on malformed input.
00056 //
00057 // @param encoded Base64 string (may include '=' padding).
00058 // @returns        Decoded raw bytes.
00059 // @throws         std::invalid_argument if 'encoded' contains illegal chars
00060 //                 or has a length that is not a multiple of 4.
00061 inline std::vector<uint8_t> Decode(const std::string& encoded) {
00062     if (encoded.size() % 4 != 0) {
00063         throw std::invalid_argument("base64::Decode: length must be a multiple of 4");
00064     }
00065
00066     std::vector<uint8_t> out;
00067     out.reserve((encoded.size() / 4) * 3);
00068
00069     for (size_t i = 0; i < encoded.size(); i += 4) {
00070         auto idx = [&](size_t pos) -> uint32_t {
00071             uint8_t c = static_cast<uint8_t>(encoded[pos]);
00072             uint8_t v = kDecodeTable[c];
00073             if (v == kInvalid && encoded[pos] != '=') {
00074                 throw std::invalid_argument(
00075                     std::string("base64::Decode: invalid character '" + encoded[pos] + "'");
00076             }
00077             return (encoded[pos] == '=') ? 0u : static_cast<uint32_t>(v);
00078         };
00079
00080         // Pack four 6-bit values back into a 24-bit integer.
00081         uint32_t b = (idx(i) « 18) | (idx(i+1) « 12) | (idx(i+2) « 6) | idx(i+3);
00082
00083         out.push_back(static_cast<uint8_t>((b » 16) & 0xFF));
00084         if (encoded[i + 2] != '=') out.push_back(static_cast<uint8_t>((b » 8) & 0xFF));
00085         if (encoded[i + 3] != '=') out.push_back(static_cast<uint8_t>( b      & 0xFF));
00086     }
00087     return out;
00088 }
00089
00090 }

```

9.5 Riferimenti per il file include/common/bot_names.hpp

Raccoglitore di costanti per l'inizializzazione del motore AI (Bot).

```

#include <vector>
#include <string>

```


9.6 bot_names.hpp

Vai alla documentazione di questo file.

```
00001 #pragma once
00002 #include <vector>
00003 #include <string>
00004 #include <common/lobby.hpp>
00005
00010
00011 namespace game {
00012
00019     inline const std::vector<std::string> kReservedBotNames = {
00020         "GLaDOS", "HAL9000", "Bender", "Wall-E", "R2-D2", "C-3PO",
00021         "Data", "Lore", "T-800", "Johnny5", "Marvin", "BB-8", "Ultron",
00022         "Megatron", "Optimus", "Cortana", "Siri", "Alexa", "ChatGPT",
00023         "Aatrox", "Adventurer", "Alchemist", "Alda", "Alixer", "Amelia",
00024         "Andrew", "Angler Angus", "Anita", "Apprentice", "Arthur",
00025         "Auction Agents", "Auction Master", "Baker", "Banker", "Bartender",
00026         "Bazaar Agent", "Bazaar", "Bea", "Biblio", "Billy Joe", "Bingo",
00027         "Blacksmith", "Bobby Joe", "Builder", "Carnival Cowboy",
00028         "Carnival Fisherman", "Carnival Leader", "Carnival Pirateman",
00029         "Carpenter", "Chantelle", "Christopher", "Clerk Seraphine",
00030         "Coach Jackrabbit", "Cole", "Curator", "Derpy", "Diana", "Diaz",
00031         "Duke", "Doug", "Dusk", "Elizabeth", "Erihann", "Fann",
00032         "Farm Merchant", "Farmer", "Fear Mongerer", "Felix", "Finnegan",
00033         "Fishing Merchant", "Fisherman Gerald", "Fisherwoman Enid", "Foxy",
00034         "Gavin", "George", "Gladiator", "Guy", "Gwynnie", "Hootie",
00035         "Hoppity", "Hub Selector", "Jack", "Jacob", "Jacobus", "Jamie",
00036         "Jax", "Jerry", "Jim Bob", "Kat", "Karis", "Leo", "Liam",
00037         "Librarian", "Liz", "Lonely Philosopher", "Lucius", "Lukas the Aquarist",
00038         "Lumber Jack", "Lumber Merchant", "Lynn", "Mad Redstone Engineer",
00039         "Maddox The Slayer", "Malik", "Marco", "Marina", "Maths Enjoyer",
00040         "Maxwell", "Mine Merchant", "Mort", "Ophelia", "Oringo", "Ozanne",
00041         "Pat", "Paul", "Pig Shop", "Plumber Joe", "Rabbit Bro", "Rabbit Cousin",
00042         "Rabbit Daddy", "Rabbit Dog", "Rabbit Granny", "Rabbit Sis",
00043         "Rabbit Uncle", "Richard", "Rosetta", "Ryu", "Salesman", "Scoop",
00044         "Scorpius", "Security Sloth", "Seymour", "Shania", "Shifty",
00045         "Sirius", "SkyBlock Animation", "Smithmonger", "Stella", "Taylor",
00046         "Tia The Fairy", "Tom", "Udium", "Vex", "Vincent", "Weaponsmith",
00047         "Wizard", "Wool Weaver", "Zog", "Zarina", "Farmhand", "Windmill Operator",
00048         "Beth", "Farmer Jon", "Friendly Hiker", "Hungry Hiker", "Hunter Ava",
00049         "Jake", "Mason", "Moby", "Shepherd", "Talbot", "Tammy", "Tony",
00050         "Treasure Hunter", "Trevor The Trapper", "Gold Forger", "Iron Forger",
00051         "Lazy Miner", "Rusty", "Lapis Miner", "Lift Operator", "Redstone Miner",
00052         "Rhys", "Walter", "Banker Broadjaw", "Bednom", "Brammor", "Brarnas",
00053         "Brynmor", "Bubu", "Bulvar", "Bylma", "Castle Guard", "Cold Enjoyer",
00054         "Dalbrek", "Dalir", "Dirt Guy", "Don Espresso", "Dr. Stone", "Dulin",
00055         "Emissary Braum", "Emissary Carlton", "Emissary Ceanna", "Emissary Eliza",
00056         "Emissary Fraiser", "Emissary Lilith", "Emissary Lissandra", "Emissary Wilson",
00057         "Emkam", "Emmor", "Erren", "Fetchur", "Forger", "Fragilis", "Fred",
00058         "Geo", "Gimley", "Grandan", "Gwendolyn", "Hornum", "Jotraeline Greatforge",
00059         "Lumina", "Marigold", "Old Man Garry", "Puzzler", "Queen Mismyla",
00060         "Redros", "Royal Guard", "Royal Resident", "Royal Residents", "Sargwyn",
00061         "Scout Plinius", "Scout Scardius", "Sor'Hen", "Silnar", "Station Master",
00062         "Tal Ker", "Tarwen", "Thondin", "Thormyr", "Ticket Master", "Tornora",
00063         "Bears", "Chunk", "Emissary Sisko", "Gemma", "Geonathan Greatforge",
00064         "Kalhuiki Door Guardian", "Keepers", "King Yolkar", "Odawa",
00065         "Professor Robot", "Queen's Guards", "Sludge", "Xalx", "Archaeologist",
00066         "Bramass Beastslayer", "Grandma Wolf", "Haymitch", "Ike", "Michael",
00067         "Rick", "Shaggy", "Spider Tamer", "Alchemage", "Alwin", "An", "Aranya",
00068         "Arba", "Arbadak", "Aviar", "Aura", "Arch", "Avorius", "Azax", "Baar",
00069         "Barbarian Emissary", "Barbarian Guard", "Barter", "Belanor", "Bromm",
00070         "Bruto", "Bruuh", "Captain Ahone", "Carrolyn", "Chief Scorn", "Chihai",
00071         "Chak", "Chuk", "Colore", "Corm", "Counsellor", "Cyndarin", "Dean",
00072         "Deng", "Desperate Engineer", "Drakuu", "Edelis", "Elle", "Elmar",
00073         "Eludore", "Etc", "Ezekiel", "Forast", "Gnyl", "Grelus", "Gris",
00074         "Grog", "Hilda", "Igor", "Igrupan", "Jine", "Kaus", "Keran", "Kheharad",
00075         "Krevius", "Kutral", "Kuudra Archaeologist", "Kuudra Believer",
00076         "Kuudra Gatekeeper", "Kuudra Loremaster", "Lasea", "Lys", "Mage Emissary",
00077         "Mage Guard", "Maggie", "Marthos", "Master Tao", "Mazakala", "Mollim",
00078         "Nall", "Nuvian", "Odexar", "Odger", "Offea", "Pablo", "Pam",
00079         "Piglin Guard", "Plenk", "Pomtair", "Porc", "Prie", "Queen Nyx",
00080         "Rescue Recruiter", "Rhanora", "Rollim", "Rulenor", "Scholar Alluin",
00081         "Seffea", "Selenar", "Sirih", "Strux", "Suus", "Truu", "Turd", "Udel",
00082         "Ugo", "Uly", "Undercover Agent", "Uvia", "Velyna", "Vulcan", "Yoink",
00083         "Ziri", "Weird Sailor", "Gregory", "Guber", "Lone Adventurer",
00084         "Pearl Dealer", "Tyzzo", "Campfire Adept", "Campfire Initiate", "Charlie",
00085         "Gustave", "Juliette", "Kelly", "Master Tactician Funk", "Melancholic Viking",
00086         "Melody", "Molbert", "Old Shaman Nyko", "Romero", "Ryan", "Vanessa",
00087         "Worker Xavier", "Agatha", "Alan", "Albert", "Amaury", "Auryon",
00088         "Banker Bonsai", "Boris", "Coral", "David Hunterborough", "Dax", "Hina",
00089         "Jaeger", "Kiara", "Kysha", "Loras", "Martin", "Nemo", "Oden", "Pounce",
00090         "Rosemary", "Salvage Old Attributes", "Sawyer", "Soul Campfire Adept",
00091         "Soul Campfire Initiate", "Swoop", "Talia", "Teddy", "Tilus", "Vaeri",
    }
```

```

00092    "Captain Baha", "Hattie", "Junker Joel", "Roddy", "Banker Barry", "Daan",
00093    "Dirk", "Einary", "Frosty", "Frozen Alex", "Gary", "Generow", "Gulliver",
00094    "Helena", "Hendrik", "Maaik", "Mees", "Sherry", "St. Jerry", "Terry",
00095    "Adventurer Saul", "Croesus", "Guildford", "Dungeon Hub Selector",
00096    "Duncan", "Three Weirdos", "Tomioka", "Trinity", "Winona", "Zodd",
00097    "Sam", "Pesthunter Pamela", "Pesthunter Phillip", "Pete", "Maeve",
00098    "Spaceman", "Ravenous Rhino", "Pest Wrangler", "Pest Wrangler?",
00099    "Vinyl Collector", "Bob", "Alabaster", "Alatar", "Argofay Bughunter",
00100    "Argofay Bugshopper", "Argofay Branchstrutter", "Argofay Serialbather",
00101    "Argofay Sonfather", "Argofay Threebrother", "Argofay Trafficker",
00102    "Argofay Trailblazer", "Argofay Tencounter", "Argofay Treemerger",
00103    "Arora", "Ashera", "Barry", "Chef", "Chester", "Chicken", "Cosmo",
00104    "Cow", "Cowboy Nick", "Creed", "Cryptosis", "Dackinoru", "Deer",
00105    "Detective Amog", "Disinfestor", "Dr. Emmett", "Dr. Hibble", "Dr. Phear",
00106    "Dust", "Elise", "Enigma", "Fafnir", "Fairylosopher", "Fimplex",
00107    "Frankie", "Garlaciuss", "Grandma", "Gunther", "Harriette", "Hound",
00108    "Inverted Sirius", "Jacqueline", "Joey McPizze", "Joliet", "Julian",
00109    "Kay", "Kiermet", "Lazarus", "Lonely Ávaeikx", "Marcia", "Mole",
00110    "Motes Grubber", "Mushroom Guy", "Nene", "Phaser", "Porhtal", "Ramero",
00111    "Reed", "Ribbit", "Rock", "Roger", "Roy", "Seraphine", "Seskel",
00112    "Sheep", "Shifted", "Sick Farmer", "Skylark", "Soma", "Sorcerer Okron",
00113    "Stain", "Tel Kar", "Tybalt", "Unbound Explorer", "Unhinged Kloon",
00114    "Violetta", "Vreike", "Wizardman", "YoshuayrreJ", "Agent Kapustin",
00115    "Agent Komarov", "Agent Kruglov", "Agent Kvasov", "Armorsmith", "Cashier",
00116    "Dante", "Dante Goons", "Elle of the Nether", "Game Agent", "Malmar",
00117    "Minikloon", "Nitroholi", "Potato King", "Raffle Box", "Relenter",
00118    "Resistance Generals", "Rezzus", "Technoblade", "Technoshop",
00119    "The Lost Adventurer", "Warren", "Worker Emma", "Dean", "Garrett", "Hannah", "Allie",
00120    "Tucker", "Jhon", "Arrodes", "Klein Moretti", "Sauron Einhorn Medici", "Eldyn THE GOAT",
00121    "<script>console.log('no XSS!')</script>", "Birdie"
00122  };
00123 }

```

9.7 Riferimenti per il file include/common/env.hpp

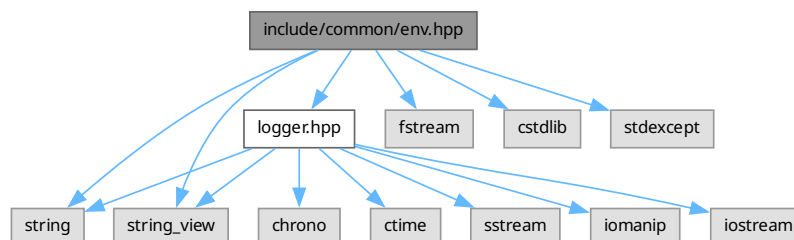
Gestione delle variabili d'ambiente e caricamento dal file di configurazione locale .env.

```

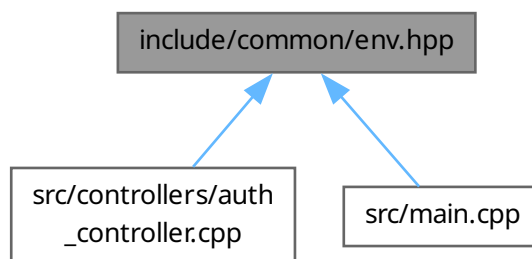
#include <string>
#include <string_view>
#include <fstream>
#include <cstdlib>
#include <stdexcept>
#include <logger.hpp>

```

Grafo delle dipendenze di inclusione per env.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Namespace

- namespace [Env](#)

Funzioni

- `std::string Env::Require (const std::string &key)`
Recupera il valore di una variabile d'ambiente obbligatoria.
- `std::string Env::Get (const std::string &key, const std::string &fallback="")`
Recupera il valore di una variabile d'ambiente, fornendo un fallback opzionale.
- `void Env::SetEnv (const std::string &key, const std::string &value)`
Funzione interna helper per impostare una variabile d'ambiente in modo cross-platform.
- `std::string Env::Trim (const std::string &s)`
Funzione interna helper per rimuovere gli spazi bianchi ASCII all'inizio e alla fine di una stringa. Rimuove i caratteri ' ', '\t', '\r', '\n'. Previene comportamenti indefiniti (UB) restituendo una stringa vuota qualora la stringa d'ingresso sia composta interamente da spazi.
- `int Env::Load (std::string_view path=".env")`
Legge il file riga per riga e inietta ogni coppia CHIAVE=VALORE nell'ambiente di processo.

9.7.1 Descrizione dettagliata

Gestione delle variabili d'ambiente e caricamento dal file di configurazione locale .env.

- Formato del file supportato:
- `KEY=VALUE` (Assegnamento base)
- `KEY="VALUE"` (I doppi apici circostanti vengono rimossi automaticamente)
- `# commento` (Le righe che iniziano con '#' vengono ignorate)
- Righe vuote saltate silenziosamente
- Tutti i valori vengono iniettati nell'ambiente di processo nativo del sistema operativo (tramite `setenv` o `_putenv_s`), rendendoli visibili anche a processi figli o librerie di terze parti che chiamano direttamente `std::getenv()`.

9.8 env.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include <string_view>
00004 #include <fstream>
00005 #include <cstdlib>
00006 #include <stdexcept>
00007 #include <logger.hpp>
00008
00021
00022 namespace Env {
00023
00034 inline std::string Require(const std::string& key) {
00035     const char* val = std::getenv(key.c_str());
00036     if (!val) {
00037         throw std::runtime_error("[Env] Required variable '" + key + "' is not set");
00038     }
00039     return val;
00040 }
00041
00051 inline std::string Get(const std::string& key,
00052                       const std::string& fallback = "") {
00053     const char* val = std::getenv(key.c_str());
00054     return val ? std::string(val) : fallback;
00055 }
00056
00065 inline void SetEnv(const std::string& key, const std::string& value) {
00066     #ifdef _WIN32
00067         _putenv_s(key.c_str(), value.c_str());
00068     #else
00069         ::setenv(key.c_str(), value.c_str(), 1);
00070     #endif
00071 }
00072
00081 inline std::string Trim(const std::string& s) {
00082     constexpr std::string_view kWs = " \t\r\n";
00083     auto start = s.find_first_not_of(kWs);
00084     if (start == std::string::npos) return "";
00085     auto end = s.find_last_not_of(kWs);
00086     return s.substr(start, end - start + 1);
00087 }
00088
00098 inline int Load(std::string_view path = ".env") {
00099     std::ifstream file(path.data());
00100     if (!file.is_open()) {
00101         Logger::Warn("[Env] .env not found at '" + std::string(path) +
00102                     "' — relying on existing environment variables");
00103         return 0;
00104     }
00105
00106     int count = 0;
00107     std::string line;
00108
00109     // std::getline estrae i caratteri fino al '\n', scartando il delimitatore stesso.
00110     while (std::getline(file, line)) {
00111         // Ignora le righe vuote e i commenti (primo carattere non vuoto è '#').
00112         if (line.empty() || line[0] == '#') continue;
00113
00114         auto eq = line.find('=');
00115         if (eq == std::string::npos) continue; // nessuna '=' → malformata, salta
00116
00117         std::string key = Trim(line.substr(0, eq));
00118         std::string value = Trim(line.substr(eq + 1));
00119
00120         // Rimuove eventuali doppi apici circostanti opzionali: "value" → value
00121         if (value.size() >= 2 && value.front() == '"' && value.back() == '"') {
00122             value = value.substr(1, value.size() - 2);
00123         }
00124
00125         if (key.empty()) continue;
00126
00127         SetEnv(key, value);
00128         ++count;
00129     }
00130
00131     Logger::Info("[Env] Loaded " + std::to_string(count) +
00132                " variable(s) from '" + std::string(path) + "'");
00133     return count;
00134 }
00135
00136 }

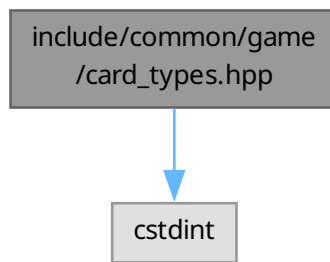
```

9.9 Riferimenti per il file include/common/ ↵ game/card_types.hpp

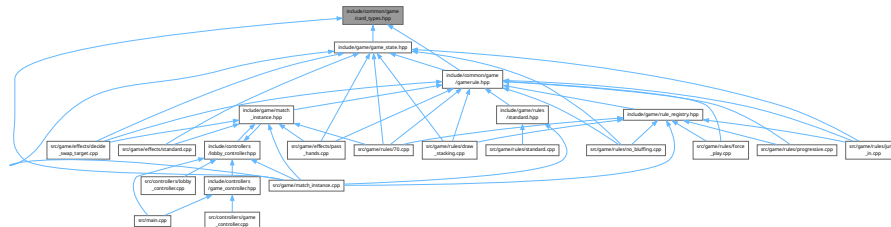
Definisce le strutture dati fondamentali e i tipi per la rappresentazione delle carte nel gioco.

```
#include <cstdint>
```

Grafo delle dipendenze di inclusione per `card_types.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Namespace

- namespace `game`

Ridefinizioni di tipo (typedef)

- using `game::CompactCard = uint32_t`

Definizione del tipo **CompactCard**. Impacchetta le informazioni di una carta all'interno di un intero a 32 bit. Formato binario: cccccccv vvvvvv uuuuuuuuuuuuuuuuuuuuu.

Tipi enumerati (enum)

- enum class `game::Color` : `uint8_t` {
`game::kRed` = 0 , `game::kBlue` = 1 , `game::kGreen` = 2 , `game::kYellow` = 3 ,
`game::kWild` = 4 }
 Enumera i possibili colori che una carta può assumere.
- enum class `game::Value` : `uint8_t` {
`game::k0` = 0 , `game::k1` , `game::k2` , `game::k3` ,
`game::k4` , `game::k5` , `game::k6` , `game::k7` ,
`game::k8` , `game::k9` , `game::kSkip` = 10 , `game::kReverse` = 11 ,
`game::kDraw2` = 12 , `game::kWild` = 13 , `game::kWildDraw4` = 14 }
 Enumera i possibili valori o azioni associati a una carta.

Funzioni

- `Color game::GetColor (CompactCard c)`
 Estrae il colore da una carta compatta (Primi 8 bit).
- `Value game::GetValue (CompactCard c)`
 Estrae il valore o l'azione da una carta compatta (Secondi 8 bit).
- `uint16_t game::GetId (CompactCard c)`
 Estrae l'identificativo univoco a 16-bit della carta (Ultimi 16 bit).
- `CompactCard game::MakeCard (Color c, Value v, uint16_t id)`
 Crea una nuova `CompactCard` assemblando colore, valore e ID.

9.9.1 Descrizione dettagliata

Definisce le strutture dati fondamentali e i tipi per la rappresentazione delle carte nel gioco.

Tag Identificativo: FEAT-CG-001 Rappresentazione Compatta delle Carte: Utilizzo di un formato intero a 32-bit (Com

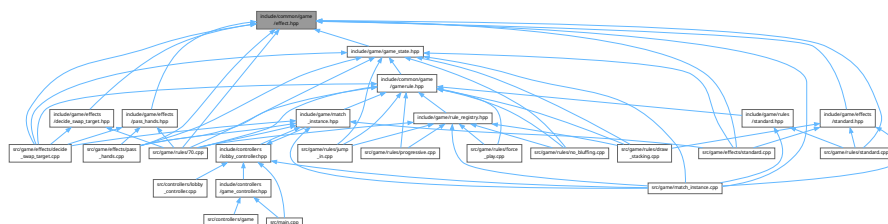
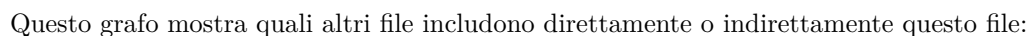
9.10 card_types.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include <cstdint>
00004
00011 namespace game {
00012     using CompactCard = uint32_t;
00023
00028     enum class Color : uint8_t {
00029         kRed = 0,
00030         kBlue = 1,
00031         kGreen = 2,
00032         kYellow = 3,
00033         kWild = 4
00034     };
00035
00040     enum class Value : uint8_t {
00041         k0 = 0, k1, k2, k3, k4, k5, k6, k7, k8, k9,
00042         kSkip = 10,
00043         kReverse = 11,
00044         kDraw2 = 12,
00045         kWild = 13,
```

9.11 Riferimenti per il file `include/common/game/effect.hpp`

```
#include <string>
Grafo delle dipendenze di inclusione per effect.hpp:
```



Composti

- struct `game::EffectResult`
Struttura restituita dopo il tentativo di risoluzione di un effetto.
- class `game::Effect`
Classe base astratta per tutti gli effetti applicabili nel gioco.

Namespace

- namespace `game`

Tipi enumerati (enum)

- enum class `game::EffectStatus` { `game::kResolved` , `game::kNeedsInput` , `game::kError` }
Enumera i possibili esiti della risoluzione di un effetto.
- enum class `game::EffectType` {
`game::kAdvanceTurn` , `game::kDraw` , `game::kSkip` , `game::kPlayCard` ,
`game::kDecideDrawnCard` , `game::kChooseColor` , `game::kReverse` , `game::kDecideSwapTarget` ,
`game::kPassHands` , `game::kCustom` }
Codici per l'identificazione a runtime del tipo di effetto in corso.

9.11.1 Descrizione dettagliata

Sistema di base per la definizione e la risoluzione degli effetti di gioco.

Tag Identificativo: FEAT-CG-002 Gestione Asincrona degli Effetti (Effect Engine): Architettura polimorfica basata su e

9.12 effect.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include <string>
00004
00011
00012 namespace game {
00013
00014     struct GameState;
00015     class MatchInstance;
00016
00021     enum class EffectStatus {
00022         kResolved,
00023         kNeedsInput,
00024         kError
00025     };
00026
00031     struct EffectResult {
00032         EffectStatus status;
00033         std::string input_type;
00034         std::string target_player;
00035         std::string input_context = "";
00036     };
00037
00042     enum class EffectType {
00043         kAdvanceTurn,
00044         kDraw,
00045         kSkip,
```



```
00046     kPlayCard,  
00047     kDecideDrawnCard,  
00048     kChooseColor,  
00049     kReverse,  
00050     kDecideSwapTarget,  
00051     kPassHands,  
00052     kCustom  
00053 };  
00054  
00055 class Effect {  
00056 public:  
00061     virtual ~Effect() = default;  
00062  
00067     virtual EffectType GetType() const { return EffectType::kCustom; }  
00068  
00075     virtual EffectResult Resolve(GameState* state, MatchInstance* match) = 0;  
00076 };  
00077  
00078 }
```

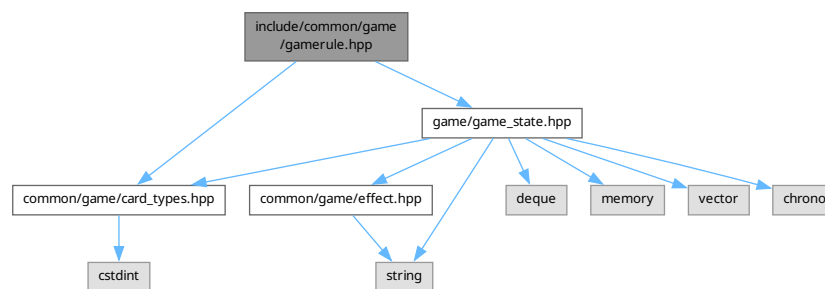
9.13 Riferimenti per il file `include/common/game/gamerule.hpp`

Definisce il motore base per l'applicazione delle regole del gioco.

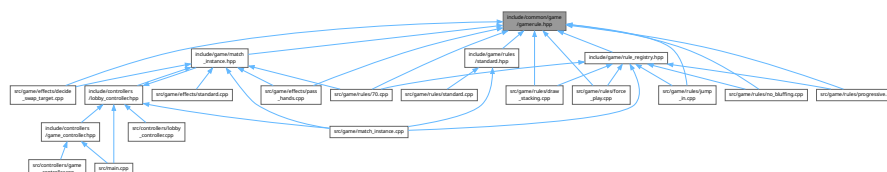
```
#include <common/game/card_types.hpp>
```

```
#include <game/game_state.hpp>
```

Grafo delle dipendenze di inclusione per `gamerule.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- struct `game::CardPlayedEvent`

Struttura che incapsula i dati riguardanti l'evento di una carta messa in gioco.

- struct `game::CardDrawnEvent`
- class `game::GameRule`

Interfaccia base che definisce il ciclo di vita per l'interpretazione di una giocata.

Namespace

- namespace [game](#)

9.13.1 Descrizione dettagliata

Definisce il motore base per l'applicazione delle regole del gioco.

Tag Identificativo: FEAT-CG-003 Validazione Flessibile delle Regole (Rule Engine): Intercettazione delle giocate tramite

9.14 gamerule.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include <common/game/card_types.hpp>
00004 #include <game/game_state.hpp>
00005
00012
00013 namespace game {
00014
00019     struct CardPlayedEvent {
00020         std::string player_username;
00021         CompactCard played_card;
00022
00023         bool is_valid_play = true;
00024         bool is_handled = false;
00025         bool is_out_of_turn = false;
00026     };
00027
00028 // Add below CardPlayedEvent
00029     struct CardDrawnEvent {
00030         std::string player_username;
00031         CompactCard drawn_card;
00032         bool is_handled = false;
00033
00034         bool keep_drawing = false; // Hook for "Progressive" mod
00035         bool force_play = false; // Hook for "Force Play" mod
00036     };
00037
00038
00043     class GameRule {
00044     public:
00045         virtual ~GameRule() = default;
00046
00053         virtual void ValidatePlay(GameState* state, CardPlayedEvent& event) {}
00054
00062         virtual void OnCardPlayed(GameState* state, CardPlayedEvent& event) {}
00063
00064         virtual void OnCardDrawn(GameState* state, CardDrawnEvent& event) {}
00065     };
00066 }
```

9.15 Riferimenti per il file include/common/http.hpp

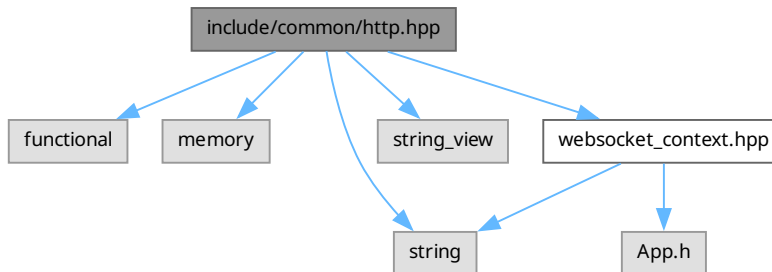
Funzioni di utilità per la gestione del protocollo HTTP.

```
#include <functional>
#include <memory>
#include <string>
```

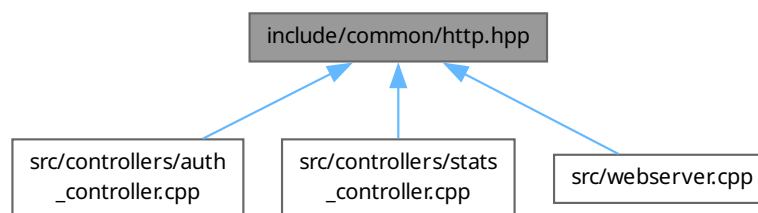
```
#include <string_view>
```

```
#include <websocket_context.hpp>
```

Grafo delle dipendenze di inclusione per http.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Namespace

- namespace [http](#)

Funzioni

- void [http::ReadBody](#) ([AppResponse](#) *res, size_t max_bytes, std::function< void(const std::string &)> callback)
Accumula asincronamente un intero body HTTP frammentato (chunked) da uWebSockets.
- constexpr std::string_view [http::TrimWhitespace](#) (std::string_view str)
Rimuove gli spazi bianchi ASCII (spazi, tabulazioni, a capo) all'inizio e alla fine di una stringa. Essendo constexpr, se valutata a tempo di compilazione non genera overhead.
- std::optional< std::string > [http::GetCookieValue](#) (std::string_view cookie_header, std::string_view target_key)
Estrae in modo sicuro il valore di un singolo cookie dalla stringa raw dell'header.

9.15.1 Descrizione dettagliata

Funzioni di utilità per la gestione del protocollo HTTP.

- Poiché uWebSockets utilizza un'architettura non-bloccante event-driven, questo file contiene helper cruciali per semplificare l'estrazione in sicurezza di body chunked e il parsing degli header (es. Cookies).

9.16 http.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <functional>
00003 #include <memory>
00004 #include <string>
00005 #include <string_view>
00006 #include <websocket_context.hpp>
00007
00015
00016 namespace http {
00017
00031 inline void ReadBody(AppResponse* res, size_t max_bytes, std::function<void(const std::string&> callback) {
00032     auto is_alive = std::make_shared<bool>(true);
00033     auto buffer = std::make_shared<std::string>();
00034
00035     res->onAborted([is_alive] { *is_alive = false; });
00036
00037     res->onData([is_alive, buffer, max_bytes, callback = std::move(callback), res](std::string_view chunk, bool isLast)
00038         mutable {
00039         if (!*is_alive) return;
00040
00041         buffer->append(chunk.data(), chunk.size());
00042
00043         if (buffer->size() > max_bytes) {
00044             res->writeStatus("413 Payload Too Large")->end();
00045             return;
00046         }
00047         if (isLast) callback(*buffer);
00048     });
00049 }
00050
00058 constexpr std::string_view TrimWhitespace(std::string_view str) {
00059     size_t first = str.find_first_not_of(" \t\r\n");
00060     if (first == std::string_view::npos) {
00061         return {};
00062     }
00063     size_t last = str.find_last_not_of(" \t\r\n");
00064     return str.substr(first, (last - first + 1));
00065 }
00066
00077 inline std::optional<std::string> GetCookieValue(std::string_view cookie_header, std::string_view target_key) {
00078     size_t start = 0;
00079
00080     while (start < cookie_header.size()) {
00081         size_t end = cookie_header.find(';', start);
00082
00083         std::string_view pair = (end == std::string_view::npos)
00084             ? cookie_header.substr(start)
00085             : cookie_header.substr(start, end - start);
00086
00087         // Separa la coppia in Chiave e Valore (Key/Value)
00088         size_t eq_pos = pair.find('=');
00089         if (eq_pos != std::string_view::npos) {
00090             std::string_view key = TrimWhitespace(pair.substr(0, eq_pos));
00091
00092             if (key == target_key) {
00093                 return std::string(TrimWhitespace(pair.substr(eq_pos + 1)));
00094             }
00095         }
00096
00097         if (end == std::string_view::npos) {
00098             break;
00099         }
00100     }

```

```

00101     start = end + 1;
00102 }
00103
00104     return "";
00105 }
00106
00107 }

```

9.17 Riferimenti per il file include/common/lobby.hpp

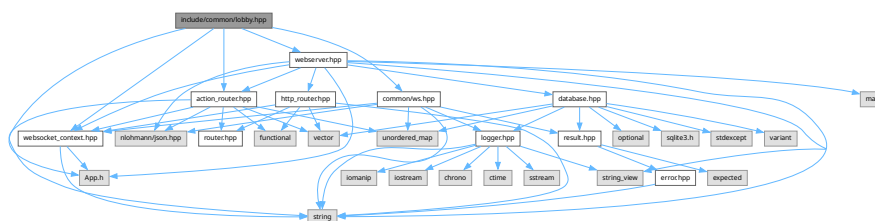
Definisce le strutture dati base per la rappresentazione delle lobby.

```

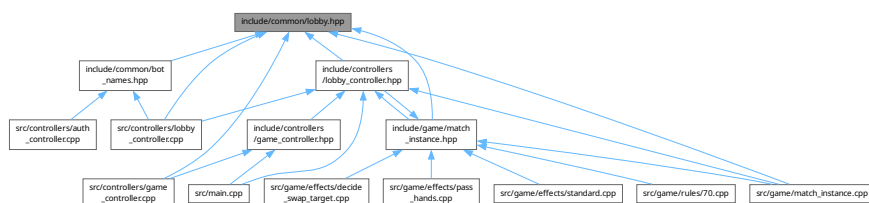
#include <common/ws.hpp>
#include <App.h>
#include <action_router.hpp>
#include <websocket_context.hpp>
#include <webserver.hpp>

```

Grafo delle dipendenze di inclusione per lobby.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- struct [Lobby](#)

Aggrega l'intero stato strutturale di una stanza di gioco.

Namespace

- namespace [game](#)

Tipi enumerati (enum)

- enum class `BotTakeoverMode` { `kPlayInstantly` , `kWaitUntilTurnEnd` }
Modalità di comportamento quando un bot subentra o viene sostituito.

9.17.1 Descrizione dettagliata

Definisce le strutture dati base per la rappresentazione delle lobby.

- Disaccoppia lo stato dei dati puri (settings, membri, identificatori) dalla logica di gestione che risiede nel `LobbyController`.

9.17.2 Documentazione dei tipi enumerati

9.17.2.1 BotTakeoverMode

enum class `BotTakeoverMode` [strong]

Modalità di comportamento quando un bot subentra o viene sostituito.

Tag Identificativo: CMN-LOBBY-ENUM-001

Valori del tipo enumerato

<code>kPlayInstantly</code>	Il bot gioca istantaneamente appena è il suo turno.
<code>kWaitUntilTurnEnd</code>	Il bot attende lo scadere del timer del turno prima di giocare.

9.18 lobby.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <common/ws.hpp>
00003 #include <App.h>
00004 #include <action_router.hpp>
00005 #include <websocket_context.hpp>
00006 #include <webserver.hpp>
00007
00014
00015 namespace game {
00016     class MatchInstance;
00017 }
00018
00024 enum class BotTakeoverMode {
00025     kPlayInstantly,
00026     kWaitUntilTurnEnd
00027 };
00028
00029 NLOHMANN_JSON_SERIALIZE_ENUM(BotTakeoverMode, {
00030     {BotTakeoverMode::kPlayInstantly, 0},
00031     {BotTakeoverMode::kWaitUntilTurnEnd, 1}
00032 })
00033
```

```

00040 struct LobbySettings {
00041     bool is_public = false;
00042     int turn_time_limit_ms = 15'000;
00043     std::vector<std::string> active_mods;
00044
00045     bool save_state = false;
00046     bool quit_deletes_match = false;
00047
00048     int bot_count = 0;
00049     BotTakeoverMode bot_mode = BotTakeoverMode::kWaitUntilTurnEnd;
00050
00051     bool allow_bot_takeover = true;
00052     bool allow_bot_replacement = true;
00053
00054     int starting_cards    = 7;
00055     int count_zeros      = 1;
00056     int count_numbered   = 2;
00057     int count_skips       = 2;
00058     int count_reverses   = 2;
00059     int count_draw_two   = 2;
00060     int count_wild       = 4;
00061     int count_wild_draw_four = 4;
00062 };
00063
00064 NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE_WITH_DEFAULT(LobbySettings,
00065     turn_time_limit_ms, active_mods, bot_count, bot_mode, starting_cards,
00066     allow_bot_takeover, allow_bot_replacement, quit_deletes_match, save_state, is_public
00067 )
00068
00069
00074 struct LobbyMember {
00075     std::string username;
00076     AppWebSocket* socket;
00077     bool is_connected;
00078     bool is_bot;
00079
00080     std::chrono::steady_clock::time_point disconnected_at{};
00081
00082     LobbyMember(std::string u, AppWebSocket* s, bool c, bool b) : username(std::move(u)), socket(s), is_connected(c),
00083     is_bot(b) {}
00084
00085
00090 struct Lobby {
00091     uint32_t id;
00092     std::string invite_code;
00093     std::string host;
00094     std::vector<LobbyMember> members;
00095     std::string name;
00096     LobbySettings settings;
00097
00101     std::unique_ptr<game::MatchInstance> match;
00102 };

```

9.19 Riferimenti per il file include/common/ws.hpp

Definizione del protocollo di messaggistica WebSocket bidirezionale.

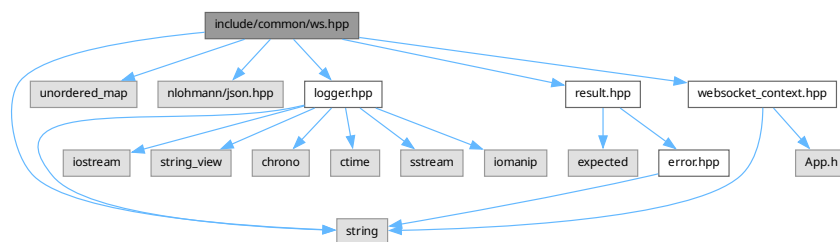
```

#include <string>
#include <unordered_map>
#include <nlohmann/json.hpp>
#include <logger.hpp>
#include <result.hpp>

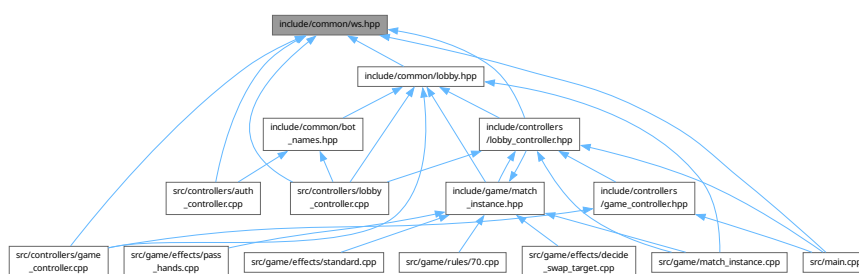
```

```
#include <websocket_context.hpp>
```

Grafo delle dipendenze di inclusione per ws.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Namespace

- namespace [ws](#)

Tipi enumerati (enum)

- enum class [ws::ServerAction](#) {
[ws::kSuccess](#) , [ws::kError](#) , [ws::kLobbyList](#) , [ws::kLobbyUpdated](#) ,
[ws::kLobbyJoined](#) , [ws::kLobbyLeft](#) , [ws::kLobbyEvicted](#) , [ws::kGameStateUpdated](#) ,
[ws::kGameOver](#) , [ws::kChatMessage](#) }
 Azioni inviate dal Server ai Client connessi.
- enum class [ws::ClientAction](#) {
[ws::kLobbyStartMatch](#) , [ws::kLobbyList](#) , [ws::kLobbyCreate](#) , [ws::kLobbyRejoin](#) ,
[ws::kLobbyJoin](#) , [ws::kLobbyLeave](#) , [ws::kLobbyUpdateSettings](#) , [ws::kLobbyDeleteSavedMatch](#) ,
[ws::kLobbyListSavedMatches](#) , [ws::kLobbyResumeSavedMatch](#) , [ws::kLobbyPromote](#) , [ws::kLobbyKick](#) ,
[ws::kGameSubmitInput](#) , [ws::kGamePlayCard](#) , [ws::kGameDrawCard](#) , [ws::kGameCallUno](#) ,
[ws::kGameExit](#) , [ws::kChatSend](#) }
 Azioni (richieste) inviate dai Client verso il Server.

Funzioni

- `template<typename T>`
`Result< T > ws::Get` (`const nlohmann::json &json`, `std::string_view key`)
 Estrae un campo obbligatorio da un payload JSON, ritornando un `Result` monadico.
- `template<typename T>`
`T ws::GetOr` (`const nlohmann::json &json`, `std::string_view key`, `const T &default_value`)
 Estrae un campo opzionale da un JSON, applicando un fallback in caso di assenza/errore.
- `nlohmann::json ws::MakeResponse` (`ws::ServerAction action`, `const std::string &request_id=""`)
 Costruisce l'intelaiatura di base per un messaggio JSON di risposta del server.
- `void ws::SendError` (`AppWebSocket *ws`, `uWS::OpCode op`, `const std::string &reason`, `const std::string &request_id`)
 Invia un payload JSON formattato come errore sul socket specifico.

Variabili

- `const std::unordered_map< ServerAction, std::string > ws::kServerActionStr`
- `const std::unordered_map< ClientAction, std::string > ws::kClientActionStr`
- `const std::unordered_map< std::string, ClientAction > ws::kStrClientAction`

9.19.1 Descrizione dettagliata

Definizione del protocollo di messaggistica WebSocket bidirezionale.

- Contiene le enumerazioni delle azioni (Client -> Server e Server -> Client), costanti di mappatura stringa/enum e funzioni di utilità per l'estrazione sicura dei payload dai JSON in ingresso.

9.20 ws.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <unordered_map>
00005 #include <nlohmann/json.hpp>
00006 #include <logger.hpp>
00007 #include <result.hpp>
00008 #include <websocket_context.hpp>
00009
00017
00018 namespace ws {
00024     enum class ServerAction {
00025         kSuccess,
00026         kError,
00027
00028         kLobbyList,
00029         kLobbyUpdated,
00030         kLobbyJoined,
00031         kLobbyLeft,
00032         kLobbyEvicted,
00033
00034         kGameStateUpdated,
00035         kGameOver,
00036
00037         kChatMessage,
00038     };
00039
00045     enum class ClientAction {
00046         kLobbyStartMatch,
00047         kLobbyList,
```

```

00048     kLobbyCreate,
00049     kLobbyRejoin,
00050     kLobbyJoin,
00051     kLobbyLeave,
00052     kLobbyUpdateSettings,
00053     kLobbyDeleteSavedMatch,
00054     kLobbyListSavedMatches,
00055     kLobbyResumeSavedMatch,
00056     kLobbyPromote,
00057     kLobbyKick,
00058
00059     kGameSubmitInput,
00060     kGamePlayCard,
00061     kGameDrawCard,
00062     kGameCallUno,
00063     kGameExit,
00064
00065     kChatSend,
00066 };
00067
00068 inline const std::unordered_map<ServerAction, std::string> kServerActionStr {
00069     { ServerAction::kSuccess, "success" },
00070     { ServerAction::kError, "error" },
00071     { ServerAction::kLobbyList, "lobby_list" },
00072     { ServerAction::kLobbyEvicted, "lobby_evicted" },
00073     { ServerAction::kLobbyUpdated, "lobby_updated" },
00074     { ServerAction::kLobbyJoined, "lobby_joined" },
00075     { ServerAction::kLobbyLeft, "lobby_left" },
00076
00077     { ServerAction::kGameStateUpdated, "game_state_updated" },
00078     { ServerAction::kGameOver, "game_over" },
00079
00080     { ServerAction::kChatMessage, "chat_message" },
00081 };
00082
00083 inline const std::unordered_map<ClientAction, std::string> kClientActionStr {
00084     { ClientAction::kGamePlayCard, "game_play_card" },
00085     { ClientAction::kGameDrawCard, "game_draw_card" },
00086     { ClientAction::kGameCallUno, "game_call_uno" },
00087
00088     { ClientAction::kChatSend, "chat_send" },
00089
00090     { ClientAction::kLobbyList, "lobby_list" },
00091     { ClientAction::kLobbyStartMatch, "lobby_start_match" },
00092     { ClientAction::kLobbyCreate, "lobby_create" },
00093     { ClientAction::kLobbyRejoin, "lobby_rejoin" },
00094     { ClientAction::kLobbyJoin, "lobby_join" },
00095     { ClientAction::kLobbyLeave, "lobby_leave" },
00096     { ClientAction::kLobbyUpdateSettings, "lobby_update_settings" },
00097     { ClientAction::kLobbyDeleteSavedMatch, "lobby_delete_saved_match" },
00098     { ClientAction::kLobbyListSavedMatches, "lobby_list_saved_matches" },
00099     { ClientAction::kLobbyResumeSavedMatch, "lobby_resume_saved_match" },
00100     { ClientAction::kLobbyPromote, "lobby_promote" },
00101     { ClientAction::kLobbyKick, "lobby_kick" },
00102
00103     { ClientAction::kGameSubmitInput, "game_submit_input" },
00104     { ClientAction::kGamePlayCard, "game_play_card" },
00105     { ClientAction::kGameDrawCard, "game_draw_card" },
00106     { ClientAction::kGameCallUno, "game_call_uno" },
00107     { ClientAction::kGameExit, "game_exit" },
00108
00109     { ClientAction::kChatSend, "chat_send" },
00110 };
00111
00112
00113 inline const std::unordered_map<std::string, ClientAction> kStrClientAction {
00114     { "lobby_list", ClientAction::kLobbyList },
00115     { "lobby_start_match", ClientAction::kLobbyStartMatch },
00116     { "lobby_create", ClientAction::kLobbyCreate },
00117     { "lobby_join", ClientAction::kLobbyJoin },
00118     { "lobby_leave", ClientAction::kLobbyLeave },
00119
00120     { "play_card", ClientAction::kGamePlayCard },
00121     { "draw_card", ClientAction::kGameDrawCard },
00122     { "call_uno", ClientAction::kGameCallUno },
00123
00124     { "chat_send", ClientAction::kChatSend },
00125
00126     { "lobby_list", ClientAction::kLobbyList },
00127     { "lobby_create", ClientAction::kLobbyCreate },
00128     { "lobby_rejoin", ClientAction::kLobbyRejoin },
00129     { "lobby_join", ClientAction::kLobbyJoin },
00130     { "lobby_leave", ClientAction::kLobbyLeave },
00131     { "lobby_update_settings", ClientAction::kLobbyUpdateSettings },
00132     { "lobby_delete_saved_match", ClientAction::kLobbyDeleteSavedMatch },
00133     { "lobby_list_saved_matches", ClientAction::kLobbyListSavedMatches },
00134     { "lobby_resume_saved_match", ClientAction::kLobbyResumeSavedMatch },

```

```

00135     { "lobby_promote", ClientAction::kLobbyPromote           },
00136     { "lobby_kick", ClientAction::kLobbyKick                 },
00137
00138     { "game_submit_input", ClientAction::kGameSubmitInput    },
00139     { "game_play_card", ClientAction::kGamePlayCard          },
00140     { "game_draw_card", ClientAction::kGameDrawCard          },
00141     { "game_call_uno", ClientAction::kGameCallUno            },
00142     { "game_exit", ClientAction::kGameExit                   },
00143
00144     { "chat_send", ClientAction::kChatSend                    },
00145 };
00146
00157 template <typename T>
00158 inline Result<T> Get(const nlohmann::json& json, std::string_view key) {
00159     auto it = json.find(key);
00160     if (it == json.end()) {
00161         return std::unexpected(Error::InvalidInput("Missing required field: '" + std::string(key) + "'"));
00162     }
00163     if (it->is_null()) {
00164         return std::unexpected(Error::InvalidInput("Required field '" + std::string(key) + "' cannot be null"));
00165     }
00166
00167     if constexpr (std::is_same_v<T, std::string>) {
00168         if (it->is_string()) return it->get<std::string>();
00169         if (it->is_number_integer()) return std::to_string(it->get<int64_t>());
00170     }
00171     else if constexpr (std::is_integral_v<T> && !std::is_same_v<T, bool>) {
00172         if (it->is_number_integer()) return it->get<T>();
00173         if (it->is_string()) {
00174             try {
00175                 if constexpr (std::is_signed_v<T>) return static_cast<T>(std::stoll(it->get<std::string>()));
00176                 else return static_cast<T>(std::stoull(it->get<std::string>()));
00177             } catch (...) {}
00178         }
00179     }
00180     else if constexpr (std::is_same_v<T, bool>) {
00181         if (it->is_boolean()) return it->get<bool>();
00182     }
00183     else if constexpr (std::is_floating_point_v<T>) {
00184         if (it->is_number()) return it->get<T>();
00185     }
00186
00187     return std::unexpected(Error::InvalidInput("Type mismatch for field: '" + std::string(key) + "'"));
00188 }
00189
00199 template <typename T>
00200 inline T GetOr(const nlohmann::json& json, std::string_view key, const T& default_value) {
00201     auto res = Get<T>(json, key);
00202     return res ? *res : default_value;
00203 }
00204
00212 inline nlohmann::json MakeResponse(ws::ServerAction action, const std::string& request_id = "") {
00213     // Force nlohmann::json to treat this strictly as an object map
00214     nlohmann::json json = nlohmann::json::object({
00215         {"action", kServerActionStr.at(action)}
00216     });
00217
00218     if (!request_id.empty()) {
00219         json["request_id"] = request_id;
00220     }
00221
00222     Logger::Log("[WS] response sent: ", kServerActionStr.at(action), "(", request_id, ")");
00223     return json;
00224 }
00225
00234 inline void SendError(AppWebSocket* ws, uWS::OpCode op, const std::string& reason, const std::string& request_id) {
00235     auto msg = MakeResponse(ServerAction::kError, request_id);
00236     msg["reason"] = reason;
00237     ws->send(msg.dump(), op);
00238 }
00239 }

```

9.21 Riferimenti per il file include/controllers/auth_controller.hpp ↵

Controller per l'autenticazione, registrazione e gestione dei token JWT.

```

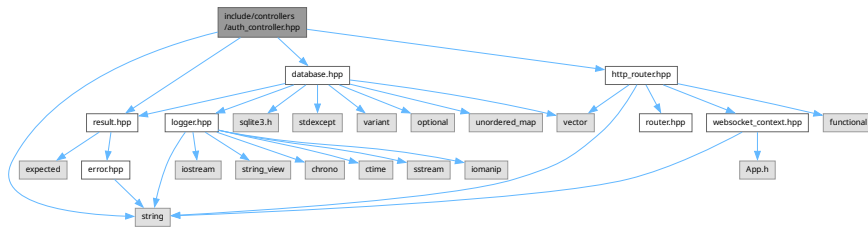
#include <string>
#include <result.hpp>

```

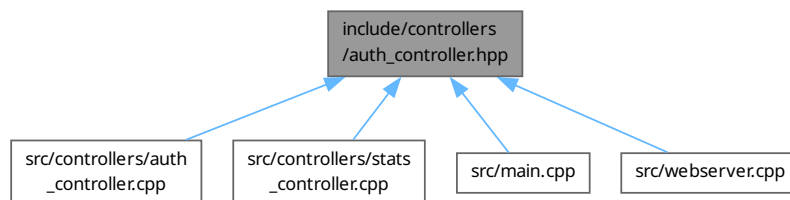
```
#include <database.hpp>
```

```
#include <http_router.hpp>
```

Grafo delle dipendenze di inclusione per `auth_controller.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- struct [JwtPayload](#)
Rappresenta i dati (payload) estratti da un token JWT verificato con successo.
- class [AuthController](#)
Gestisce le rotte HTTP per il login/registrazione e la sicurezza crittografica.

9.21.1 Descrizione dettagliata

Controller per l'autenticazione, registrazione e gestione dei token JWT.

- Gestisce l'identità degli utenti nel sistema implementando endpoint HTTP sicuri. Si occupa dell'hashing crittografico delle password e della generazione/validazione dei token Web JSON (JWT) utilizzati per l'autorizzazione WebSocket.

9.22 auth_controller.hpp

Vai alla documentazione di questo file.

```

00001 #pragma once
00002 #include <string>
00003 #include <result.hpp>
00004 #include <database.hpp>
00005 #include <http_router.hpp>
00006
00014
00020 struct JwtPayload {
00021     std::string username;
00022 };
00023
00039 class AuthController {
00040 public:
00047     explicit AuthController(HttpRouter& router);
00048
00057     static Result<JwtPayload> VerifyToken(const std::string& token);
00058
00059 private:
00060     // --- HTTP Route Handlers ---
00061
00070     void HandleRegister(AppResponse* res, AppRequest* req);
00071
00080     void HandleLogin (AppResponse* res, AppRequest* req);
00081
00089     void HandleLogout (AppResponse* res, AppRequest* req);
00090
00098     void HandleMe (AppResponse* res, AppRequest* req);
00099
00100     // --- Helpers Crittografici ---
00101
00108     static std::string HashPassword (const std::string& password);
00109
00117     static bool VerifyPassword(const std::string& password, const std::string& stored);
00118
00125     static std::string IssueToken(const std::string& username);
00126
00127     // --- Parametri di validazione e Sicurezza ---
00128
00129     static constexpr int kMinUsernameLen = 3;
00130     static constexpr int kMaxUsernameLen = 32;
00131     static constexpr int kMinPasswordLen = 8;
00132     static constexpr int kMaxBodyBytes = 4096;
00133
00134     // Parametri PBKDF2 (Aumentare kIterations alza il costo per i tentativi di brute-force)
00135
00136     static constexpr int kSaltBytes = 16;
00137     static constexpr int kHashBytes = 32;
00138     static constexpr int kIterations = 200'000;
00139 };

```

9.23 Riferimenti per il file include/ ↵ controllers/game_controller.hpp

Controller per la gestione degli eventi in-game e del flusso dei turni.

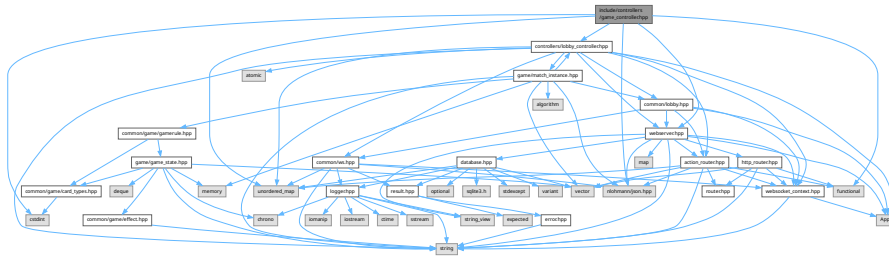
```

#include <webserver.hpp>
#include <controllers/lobby_controller.hpp>
#include <nlohmann/json.hpp>
#include <unordered_map>
#include <functional>

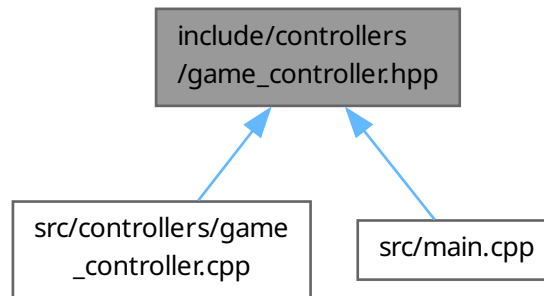
```

```
#include <stdint>
```

Grafo delle dipendenze di inclusione per `game_controller.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class [GameController](#)

Riceve e processa l'input dei giocatori durante una partita attiva.

9.23.1 Descrizione dettagliata

Controller per la gestione degli eventi in-game e del flusso dei turni.

- Smista i messaggi WebSocket riguardanti le azioni di gioco (giocare carte, pescare, dichiarare UNO) alle rispettive istanze di MatchInstance.

9.24 game_controller.hpp

Vai alla documentazione di questo file.

```

00001 #pragma once
00002
00003 #include <webserver.hpp>
00004 #include <controllers/lobby_controller.hpp>
00005 #include <nlohmann/json.hpp>
00006 #include <unordered_map>
00007 #include <functional>
00008 #include <cstdint>
00009
00016
00025 class GameController {
00026 public:
00034     GameController(WebServer& server, LobbyController& lobby_controller);
00035
00036 private:
00037     ActionRouter& action_router_;
00038     LobbyController& lobby_controller_;
00039
00043     std::unordered_map<uint32_t, struct us_timer_t*> active_turn_timers_;
00044
00045     // --- WebSocket Event Handlers ---
00046
00053     void HandlePlayCard(WsContext context, const nlohmann::json& message);
00054
00061     void HandleDrawCard(WsContext context, const nlohmann::json& message);
00062
00069     void HandleProvideInput(WsContext context, const nlohmann::json& message);
00070
00077     void HandleCallUno(WsContext context, const nlohmann::json& message);
00078
00079     // --- Core Game Flow ---
00080
00087     void OnTurnStarted(Lobby* active_lobby);
00088
00096     void BroadcastGameState(Lobby* current_lobby);
00097
00098     // --- Timer Management Helpers ---
00099
00107     void SetTurnTimer(uint32_t lobby_id, int timeout_ms, std::function<void()> callback);
00108
00115     void ClearTurnTimer(uint32_t lobby_id);
00116 };

```

9.25 Riferimenti per il file include/ ↵ controllers/lobby_controller.hpp

Definizione del controller per la gestione delle **Lobby**. Gestisce l'intero I/O tra Client e Server per quanto riguarda l'inizializzazione di **Lobby**, e l'inizio di Partite di Gioco, a cui successivamente passa il controllo.

```

#include <game/match_instance.hpp>
#include <common/lobby.hpp>
#include <common/ws.hpp>
#include <atomic>
#include <string>
#include <unordered_map>
#include <App.h>
#include <action_router.hpp>
#include <websocket_context.hpp>

```


9.25.2.2 PlayerReplacedCallback

using `PlayerReplacedCallback` = std::function<void(Lobby*)>

Callback invocata quando un giocatore viene sostituito (es. da un bot).

Tag Identificativo: LOBBY-TYP-002

9.26 lobby_controller.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <game/match_instance.hpp>
00003 #include <common/lobby.hpp>
00004 #include <common/ws.hpp>
00005 #include <atomic>
00006 #include <string>
00007 #include <unordered_map>
00008 #include <App.h>
00009 #include <action_router.hpp>
00010 #include <websocket_context.hpp>
00011 #include <webserver.hpp>
00012
00019
00025 using GameStartedCallback = std::function<void(Lobby*)>;
00026
00032 using PlayerReplacedCallback = std::function<void(Lobby*)>;
00033
00042 class LobbyController {
00043 public:
00051     LobbyController(WebServer& webserver);
00052
00057     ~LobbyController();
00058
00067     void OnOpen(AppWebSocket* ws, PerSocketData* sd);
00068
00077     void OnClose(AppWebSocket* ws, PerSocketData* sd);
00078
00085     Lobby* GetLobbyByCode(const std::string& code) {
00086         auto it = code_to_id_.find(code);
00087         if (it != code_to_id_.end()) {
00088             return &lobbies_.at(it->second);
00089         }
00090         return nullptr;
00091     }
00092
00099     Lobby* GetLobbyById(const uint32_t id) {
00100         auto it = lobbies_.find(id);
00101         if (it != lobbies_.end()) {
00102             return &it->second;
00103         }
00104         return nullptr;
00105     }
00106
00112     void OnGameStarted(GameStartedCallback callback) {
00113         game_started_callback_ = std::move(callback);
00114     }
00115
00121     void OnPlayerReplaced(PlayerReplacedCallback callback) {
00122         player_replaced_callback_ = std::move(callback);
00123     }
00124
00130     void SaveMatchStateToDB(Lobby& lobby);
00131
00136     static constexpr int kReconnectGraceMs = 1'000 * 30;
00137
00142     static constexpr int kMaxMembers = 4;
00143
00144 private:
00150     void SyncBots(Lobby& lobby);
00151
00157     void CheckMatchIntegrity(Lobby& lobby);
00158
00165     void HandleCreate(WsContext ctx, const json& msg);

```

```

00166
00173 void HandleJoin(WsContext ctx, const json& msg);
00174
00181 void HandleRejoin(WsContext ctx, const json& msg);
00182
00189 void HandleLeave(WsContext ctx, const json& msg);
00190
00197 void HandleList(WsContext ctx, const json& msg);
00198
00205 void HandlePromote(WsContext ctx, const json& msg);
00206
00213 void HandleKick(WsContext ctx, const json& msg);
00214
00221 void HandleUpdateSettings(WsContext ctx, const json& msg);
00222
00229 void HandleGetSavedMatchesList(WsContext ctx, const json& msg);
00230
00237 void HandleResumeSavedMatch(WsContext ctx, const json& msg);
00238
00245 void HandleDeleteSavedMatch(WsContext ctx, const json& msg);
00246
00253 void HandleStartGame(WsContext context, const nlohmann::json& message);
00254
00260 static std::string GenerateInviteCode();
00261
00268 static json MemberListJson(const Lobby& lobby);
00269
00275 void BroadcastUpdate(const Lobby& lobby) const;
00276
00286 bool RemoveMember(uint32_t lobby_id, const string& username, bool explicit_leave = true, const string& request_id
= "");
00287
00294 Lobby* FindLobbyForUser(const std::string& username);
00295
00296 GameStartedCallback game_started_callback_;
00297 PlayerReplacedCallback player_replaced_callback_;
00298
00299 ActionRouter& action_router_;
00300 uWS::SSLApp& app_;
00301
00302 std::unordered_map<uint32_t, Lobby> lobbies_;
00303 std::unordered_map<std::string, uint32_t> code_to_id_;
00304
00305 std::atomic<uint32_t> next_id_{1};
00306 us_timer_t* eviction_timer_ = nullptr;
00307 };

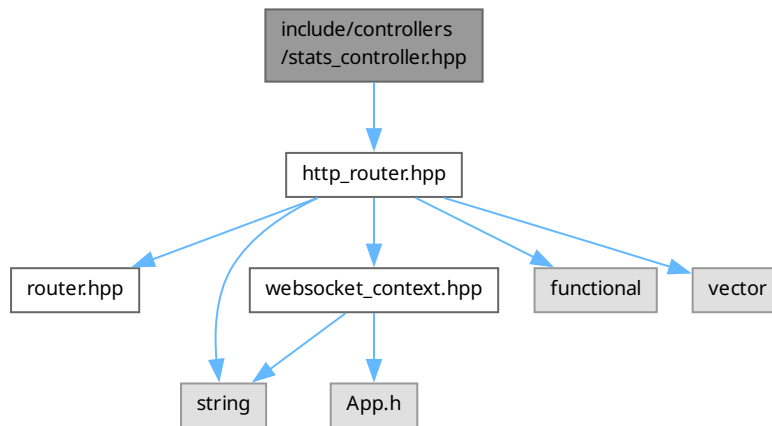
```

9.27 Riferimenti per il file include/ ↩ controllers/stats_controller.hpp

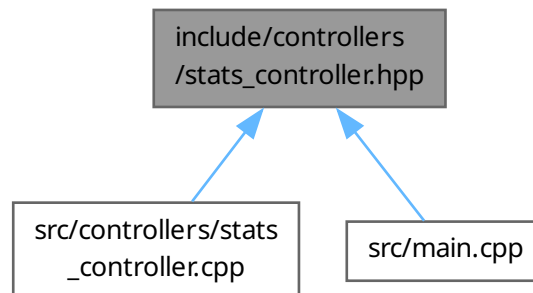
Controller HTTP per la gestione delle statistiche utenti e classifiche.

```
#include <http_router.hpp>
```

Grafo delle dipendenze di inclusione per stats_controller.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class [StatsController](#)

Esponde gli endpoint REST necessari al frontend per il pannello Profilo e Leaderboard.

9.27.1 Descrizione dettagliata

Controller HTTP per la gestione delle statistiche utenti e classifiche.

- Gestisce richieste RESTful (GET) interagendo col [Database](#) per calcolare il ranking globale (win rate, totali partite, ecc).

Composti

- class [DbRow](#)
Rappresenta una singola riga restituita da una query SQL. Memorizza i dati usando le chiavi di colonna come indici della mappa interna.
- class [Database](#)
Pattern Singleton che incapsula la connessione univoca e condivisa al DB SQLite.
- class [TransactionGuard](#)
Implementa l'esecuzione sicura di transazioni database tramite il paradigma RAII.

Ridefinizioni di tipo (typedef)

- using [DbValue](#) = std::variant<int, double, std::string, std::nullptr_t>
Variante che incapsula i tipi di dati base ammessi nel database SQLite.

9.29.1 Descrizione dettagliata

Astrazione leggera e sicura (Type-Safe) attorno alla libreria SQLite3.

- Fornisce accesso Singleton al database, con un sistema di query basato su std::variant per prevenire SQL Injection, ed un costrutto RAII per le transazioni.

9.29.2 Documentazione delle ridefinizioni di tipo (typedef)

9.29.2.1 DbValue

```
using DbValue = std::variant<int, double, std::string, std::nullptr_t>
```

Variante che incapsula i tipi di dati base ammessi nel database SQLite.

Tag Identificativo: DB-TYP-001

9.30 database.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include "logger.hpp"
00003 #include <sqlite3.h>
00004 #include <stdexcept>
00005 #include <variant>
00006 #include <vector>
00007 #include <optional>
00008 #include <unordered_map>
00009 #include <result.hpp>
00010
00017
00023 using DbValue = std::variant<int, double, std::string, std::nullptr_t>;
00024
00031 class DbRow {
00032 public:
00039     void Set(const std::string& col, DbValue val);
00040
00049     template<typename T>
00050     T Get(const std::string& col) const {
00051         if (!Has(col)) {
00052             throw std::runtime_error("Column not found: " + col);
00053         }
00054         return std::get<T>(data_.at(col));
00055     };
00056
00065     template<typename T>
00066     T GetOr(const std::string& col, T fallback) const {
00067         if (!Has(col) || std::holds_alternative<std::nullptr_t>(data_.at(col))) {
00068             return fallback;
00069         }
00070         return std::get<T>(data_.at(col));
00071     };
00072
00079     bool Has(const std::string& col) const;
00080
00081 private:
00082     std::unordered_map<std::string, DbValue> data_;
00083 };
00084
00092 class Database {
00093 public:
00099     static Database& Get();
00100
00107     VoidResult Open(std::string_view path);
00108
00115     VoidResult ApplySchema(const char* sql);
00116
00121     void Close();
00122
00128     bool IsOpen() const;
00129
00137     VoidResult Exec (const char* sql, std::vector<DbValue> params = {});
00138
00146     Result<std::vector<DbRow>> Query (const char* sql, std::vector<DbValue> params = {});
00147
00155     Result<std::optional<DbRow>> QueryOne (const char* sql, std::vector<DbValue> params = {});
00156
00157 private:
00158     Database() = default;
00159     ~Database() { Close(); }
00160     Database(const Database&) = delete;
00161     Database& operator=(const Database&) = delete;
00162
00163     sqlite3* db_ = nullptr;
00164
00165     sqlite3_stmt* Prepare(const char* sql, const std::vector<DbValue>& params);
00166     DbRow ReadRow(sqlite3_stmt* stmt);
00167 };
00168
00177 class TransactionGuard {
00178 public:
00184     explicit TransactionGuard(Database& db) : db_(db), committed_(false) {
00185         auto res = db_.Exec("BEGIN TRANSACTION;");
00186         if (!res) {
00187             throw std::runtime_error("Failed to begin transaction: " + res.error().message);
00188         }
00189     }
00190
00195     ~TransactionGuard() {
00196         if (!committed_) {
00197             auto _ = db_.Exec("ROLLBACK;");

```

```

00198         Logger::Error("[DB] Transaction failed, rolling back...");
00199     }
00200 }
00201
00202 void Commit() {
00203     if (!committed_) {
00204         auto res = db_.Exec("COMMIT;");
00205         if (!res) {
00206             throw std::runtime_error("Failed to COMMIT transaction: " + res.error().message);
00207         }
00208         committed_ = true;
00209     }
00210 }
00211
00212 TransactionGuard(const TransactionGuard&) = delete;
00213 TransactionGuard& operator=(const TransactionGuard&) = delete;
00214
00215 private:
00216     Database& db_;
00217     bool committed_;
00218 };

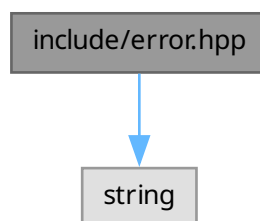
```

9.31 Riferimenti per il file `include/error.hpp`

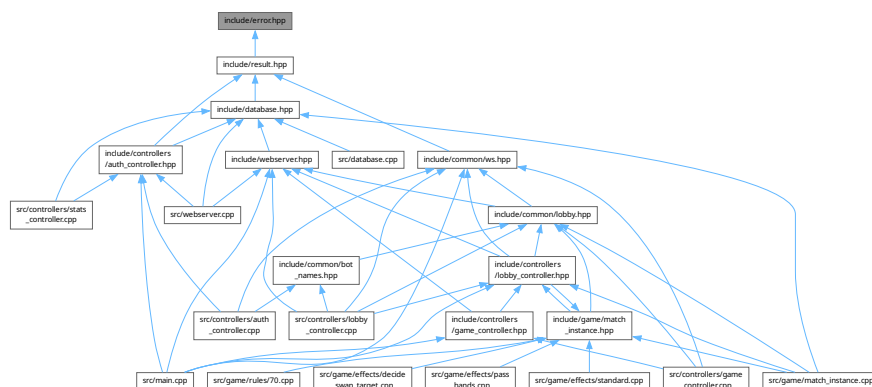
Definizione degli errori standard di sistema e protocollo.

```
#include <string>
```

Grafo delle dipendenze di inclusione per `error.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- struct `Error`

Incapsula le informazioni riguardanti un errore verificatosi nel sistema.

9.31.1 Descrizione dettagliata

Definizione degli errori standard di sistema e protocollo.

9.32 error.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <string>
00003
00004 using std::string;
00005
00010
00018 struct Error {
00019
00025     enum class Code {
00026         kBadRequest,
00027         kInvalidInput,
00028         kNotFound,
00029         kUnauthorised,
00030         kConflict,
00031         kDatabaseFailure,
00032         kInternalError,
00033     };
00034
00035     Code code;
00036     string message;
00037
00038     // --- Factory Methods ---
00039
00041     static Error BadRequest (const string& msg) { return {Code::kBadRequest, msg}; }
00043     static Error InvalidInput (const string& msg) { return {Code::kInvalidInput, msg}; }
00045     static Error NotFound (const string& msg) { return {Code::kNotFound, msg}; }
00047     static Error Unauthorised (const string& msg) { return {Code::kUnauthorised, msg}; }
00049     static Error Conflict (const string& msg) { return {Code::kConflict, msg}; }
00051     static Error DatabaseFail (const string& msg) { return {Code::kDatabaseFailure, msg}; }
00053     static Error Internal (const string& msg) { return {Code::kInternalError, msg}; }
00054
00061     std::string HttpStatus() const {
00062         switch (code) {
00063             case Code::kBadRequest: return "400 Bad Request";
00064             case Code::kInvalidInput: return "422 Unprocessable Entity";
00065             case Code::kNotFound: return "404 Not Found";
00066             case Code::kUnauthorised: return "401 Unauthorized";
00067             case Code::kConflict: return "409 Conflict";
00068             case Code::kDatabaseFailure: return "500 Internal Server Error";
00069             default: return "500 Internal Server Error";
00070         }
00071     }
00072 };
```

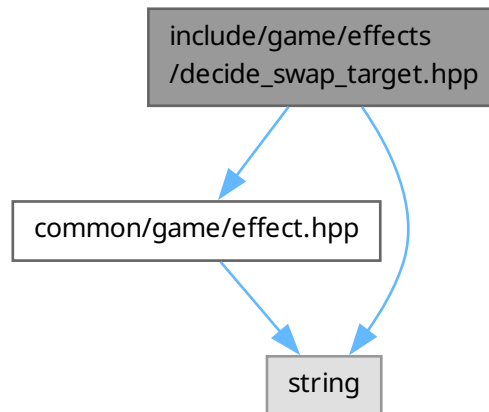
9.33 Riferimenti per il file `include/game/effects/decide_swap_target.hpp`

Effetto interattivo per la scelta del bersaglio per lo scambio delle carte.

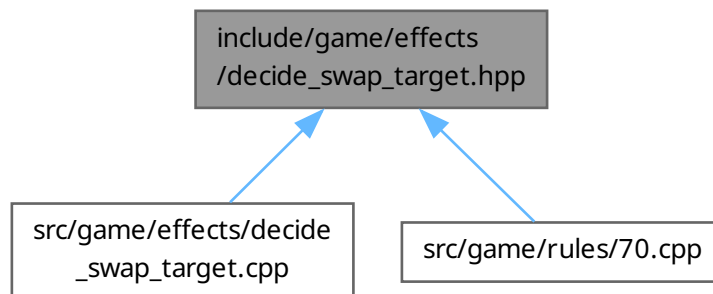

```
#include <common/game/effect.hpp>
```

```
#include <string>
```

Grafo delle dipendenze di inclusione per decide_swap_target.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class [game::DecideSwapTargetEffect](#)

Sospende il gioco per chiedere a un utente con chi desidera scambiare le carte.

Namespace

- namespace [game](#)

9.33.1 Descrizione dettagliata

Effetto interattivo per la scelta del bersaglio per lo scambio delle carte.

- Richiede un'azione esplicita da parte di un giocatore per decidere con quale avversario scambiare la propria intera mano di carte (tipico della carta "7").

9.34 `decide_swap_target.hpp`

[Vai alla documentazione di questo file.](#)

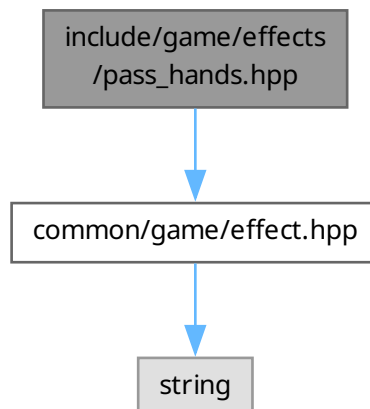
```
00001 #pragma once
00002
00003 #include <common/game/effect.hpp>
00004 #include <string>
00005
00012
00013 namespace game {
00014
00023     class DecideSwapTargetEffect : public Effect {
00024     public:
00029         explicit DecideSwapTargetEffect(std::string username) : username_(std::move(username)) {}
00030
00040         EffectResult Resolve(GameState* state, MatchInstance* match) override;
00041
00047         EffectType GetType() const override { return EffectType::kDecideSwapTarget; }
00048
00049     private:
00050         std::string username_;
00051     };
00052 }
```

9.35 Riferimenti per il file `include/game/effects/pass_hands.hpp` ↩

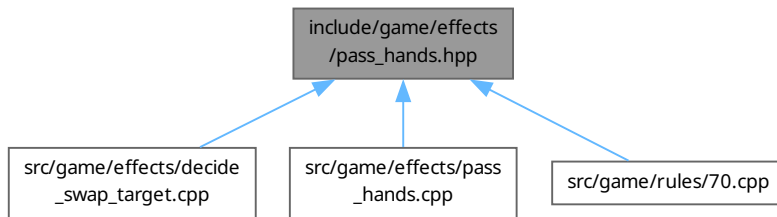
Implementazione dell'effetto di rotazione o scambio delle mani di carte.

```
#include <common/game/effect.hpp>
```

Grafo delle dipendenze di inclusione per `pass_hands.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class `game::PassHandsEffect`
Effetto che esegue lo scambio delle carte in mano ai giocatori.

Namespace

- namespace `game`

9.35.1 Descrizione dettagliata

Implementazione dell'effetto di rotazione o scambio delle mani di carte.

- Questo effetto gestisce lo spostamento fisico delle carte tra i giocatori. Viene tipicamente utilizzato nelle mod "7-0" (dove lo 0 fa ruotare le mani di tutti nella direzione di gioco) oppure dopo che un giocatore ha scelto un target con cui scambiare.

9.36 pass_hands.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <common/game/effect.hpp>
00003
00011
00012 namespace game {
00013
00019     class PassHandsEffect : public Effect {
00020     public:
00030         EffectResult Resolve(GameState* state, MatchInstance* match) override;
00031     };
00032 }
  
```


Tipi enumerati (enum)

- enum class `game::MatchStatus` { `game::kWaitingToStart` , `game::kPlaying` , `game::kFinished` }
Fasi del ciclo di vita globale di una partita.

Funzioni

- void `game::ReshuffleDiscardIntoDraw` (`GameState` *game_state)
Utilità di sistema che sposta e rimescola le carte dagli scarti al mazzo principale. Viene invocata automaticamente dal motore di gioco quando draw_pile si esaurisce.

9.37.1 Descrizione dettagliata

Strutture dati fondazionali per la rappresentazione dello stato del gioco "UNO".

- Contiene i dati puri disaccoppiati dalla logica di instradamento. Lo stato del gioco è il contesto su cui agiscono gli Effetti (Effect) e i Validatori delle regole (GameRule).

9.38 game_state.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include <common/game/card_types.hpp>
00004 #include <common/game/effect.hpp>
00005 #include <deque>
00006 #include <memory>
00007 #include <string>
00008 #include <vector>
00009 #include <chrono>
00010
00017
00018 namespace game {
00019
00025     struct PlayerSessionStats {
00026         int color_counts[5] = {0};
00027         int value_counts[15] = {0};
00028     };
00029
00035     struct Player {
00036         std::string username;
00037         std::vector<CompactCard> hand;
00038         bool is_bot;
00039         bool has_called_uno = false;
00040
00048         Player(std::string u, std::vector<CompactCard> h, bool b, bool uno) : username(u), hand(h), is_bot(b),
has_called_uno(uno) {};
00049
00053         Player(std::string u, bool b, bool uno) : username(u), is_bot(b), has_called_uno(uno) {};
00054     };
00055
00061     enum class MatchStatus {
00062         kWaitingToStart,
00063         kPlaying,
00064         kFinished
00065     };
00066
00073     struct LastPlay {
00074         bool valid = false;
00075         std::string player;
00076         int hand_index = -1;
00077         CompactCard card{};
00078     };
00079
00086     struct GameState {
00087         MatchStatus status = MatchStatus::kWaitingToStart;
```


Composti

- class `game::MatchInstance`
Rappresenta un'istanza attiva di una partita (Match).

Namespace

- namespace `game`

9.39.1 Descrizione dettagliata

Definizione della classe principale che gestisce l'intero ciclo di vita di una partita di gioco: una state machine che applica gli effetti di ogni carta giocata in base a delle regole che ne definiscono le varie proprietà.

9.40 match_instance.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002
00003 #include "common/lobby.hpp"
00004 #include <common/game/gamerule.hpp>
00005 #include <controllers/lobby_controller.hpp>
00006 #include <nlohmann/json.hpp>
00007 #include <string>
00008 #include <vector>
00009 #include <memory>
00010 #include <algorithm>
00011
00018
00019 struct LobbySettings;
00020
00021 namespace game {
00022
00031     class MatchInstance {
00032     public:
00039         explicit MatchInstance(const std::vector<std::pair<std::string, bool>& players_info, const LobbySettings& settings);
00040
00047         explicit MatchInstance(const json& saved_state, const LobbySettings& settings);
00048
00053         void Start();
00054
00059         void Tick();
00060
00068         bool PlayCard(const std::string& username, uint16_t card_id);
00069
00076         bool DrawCard(const std::string& username);
00077
00084         void ProvideInput(const std::string& username, const std::string& input);
00085
00091         void CallUno(const std::string& username);
00092
00097         void TakeBotTurn();
00098
00104         std::string GetCurrentPlayerUsername() const;
00105
00112         void AddPlayerMidGame(const std::string& username, bool is_bot);
00113
00119         void RemovePlayerMidGame(const std::string& username);
00120
00126         json ExportState() const;
00127
00133         bool IsWaitingForInput() const { return !state_.pending_player.empty(); }
00134
00140         void SetTurnEndTime(std::chrono::steady_clock::time_point end_time) { state_.turn_end_time = end_time; }
00141
00148         Player* GetPlayer(const std::string& username);
00149
00155         std::string GetPendingPlayer() const { return state_.pending_player; }
```

```

00156     std::string GetPendingInputType() const { return state_.pending_input_type; }
00162
00163     std::string GetPendingInputContext() const { return state_.pending_input_context; }
00169
00170     bool IsGameOver() const { return state_.status == MatchStatus::kFinished; }
00176
00177     std::string GetWinner() const { return state_.winner; }
00183
00184     nlohmann::json SerializePlayerState(const std::string& username) const;
00192
00193     std::string GetMatchId() const { return match_id_; }
00199
00200     void SetMatchId(const std::string& id) { match_id_ = id; }
00206
00207 private:
00208     GameState state_;
00209     LobbySettings settings_;
00210     std::string match_id_;
00211
00212     std::unordered_map<std::string, PlayerSessionStats> session_stats_;
00213     std::vector<std::unique_ptr<GameRule>> active_rules_;
00214
00215     bool HasMod(const std::string& name) const {
00222         return std::find(settings_.active_mods.begin(), settings_.active_mods.end(), name)
00223             != settings_.active_mods.end();
00224     }
00225
00226     void GenerateDeck();
00231 };
00232
00233 }
00234

```

9.41 Riferimenti per il file `include/game/rule_registry.hpp`

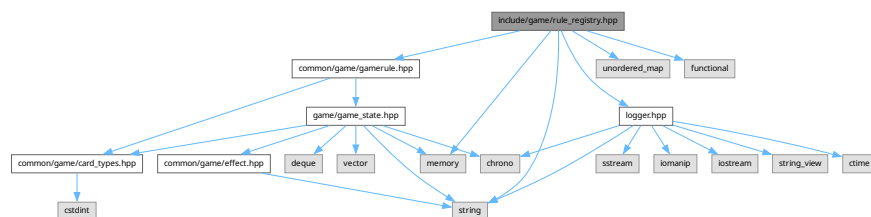
Gestione della registrazione e istanziazione dinamica delle regole di gioco.

```

#include "common/game/gamerule.hpp"
#include <unordered_map>
#include <functional>
#include <string>
#include <memory>
#include "logger.hpp"

```

Grafo delle dipendenze di inclusione per `rule_registry.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class `game::RuleRegistry`
Registro globale (Singleton) delle Factory delle regole di gioco.
- struct `game::RuleRegistrar`
Struttura di supporto per l'auto-registrazione delle regole.

Namespace

- namespace `game`

Ridefinizioni di tipo (typedef)

- using `game::RuleFactory` = `std::function<std::unique_ptr<GameRule>()>`
Definisce il tipo di funzione capace di generare una nuova istanza di una regola.

9.42.1 Descrizione dettagliata

Gestione della registrazione e istanziazione dinamica delle regole di gioco.

- Questo file definisce un sistema basato su Factory Pattern per permettere la creazione di regole (GameRule) a runtime partendo dal loro nome testuale (es. dal JSON). Utilizza il meccanismo di auto-registrazione statica per disaccoppiare l'aggiunta di nuove regole dal codice core del motore.

9.42 rule_registry.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include "common/game/gamerule.hpp"
00003 #include <unordered_map>
00004 #include <functional>
00005 #include <string>
00006 #include <memory>
00007 #include "logger.hpp"
00008
00017 namespace game {
00018
00025     using RuleFactory = std::function<std::unique_ptr<GameRule>()>;
00026
00034     class RuleRegistry {
00035     public:
00043         std::unordered_map<std::string, RuleFactory>& GetMap() {
00044             static std::unordered_map<std::string, RuleFactory> registry;
00045             return registry;
00046         }
00047
00056         static std::unique_ptr<GameRule> Create(const std::string& name) {
00057             auto& map = GetMap();
00058             if (map.find(name) != map.end()) {
00059                 return map[name]();
00060             }
00061             Logger::Error("[RuleRegistry] Failed to find rule: ", name);
00062             return nullptr;
00063         }
00064     };
00065
00074     struct RuleRegistrar {
00081         RuleRegistrar(const std::string& name, RuleFactory factory) {
00082             RuleRegistry::GetMap()[name] = std::move(factory);
00083         }
00084     };
00085
00086 }
```

9.43 Riferimenti per il file `include/game/effects/standard.hpp`

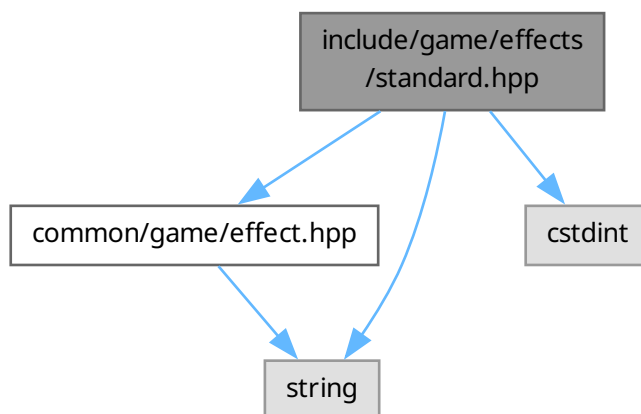
Implementazione degli effetti standard associati alle carte di base.

```
#include <common/game/effect.hpp>
```

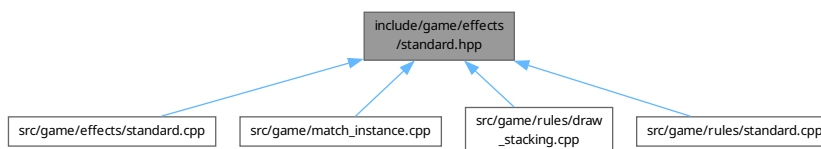
```
#include <string>
```

```
#include <cstdint>
```

Grafo delle dipendenze di inclusione per `standard.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class `game::AdvanceTurnEffect`
Effetto base che avanza il turno al giocatore successivo. Viene solitamente accodato alla fine di una giocata standard o dopo la risoluzione degli altri effetti.
- class `game::DecideDrawnCardEffect`
Effetto interattivo che chiede al giocatore se vuole giocare la carta appena pescata.
- class `game::DrawEffect`
Effetto penalità/obbligo che fa pescare un numero di carte a un giocatore. Usato per l'inizio del turno, le carte +2, +4, o per penalità da mod.

- class `game::SkipEffect`
Effetto che salta il turno del giocatore successivo (Carta Divieto).
- class `game::ReverseEffect`
Effetto che inverte il senso di rotazione dei turni (Carta Inverti).
- class `game::ChooseColorEffect`
Effetto interattivo che impone la selezione del nuovo colore attivo (Carte Jolly).

Namespace

- namespace `game`

9.43.1 Descrizione dettagliata

Implementazione degli effetti standard associati alle carte di base.

- Questo file definisce l'insieme degli effetti classici (es. Salto del turno, Inversione, Pesca carte, Scelta colore) che vengono accodati e risolti dal motore di gioco durante una partita.

9.44 standard.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <common/game/effect.hpp>
00003 #include <string>
00004 #include <stdint>
00005
00012
00013 namespace game {
00014
00021     class AdvanceTurnEffect : public Effect {
00022     public:
00030         EffectResult Resolve(GameState* game_state, MatchInstance* match_instance) override;
00031
00037         EffectType GetType() const override { return EffectType::kAdvanceTurn; }
00038     };
00039
00047     class DecideDrawnCardEffect : public Effect {
00048     public:
00054         explicit DecideDrawnCardEffect(const std::string& username, uint16_t card_id) : username_(username),
card_id_(card_id) {}
00055
00063         EffectResult Resolve(GameState* state, MatchInstance* match) override;
00064
00070         EffectType GetType() const override { return EffectType::kDecideDrawnCard; }
00071     private:
00072         std::string username_;
00073         uint16_t card_id_;
00074     };
00075
00082     class DrawEffect : public Effect {
00083     public:
00089         explicit DrawEffect(int count, const std::string& target_username) : count_(count),
target_username_(target_username) {}
00090
00099         EffectResult Resolve(GameState* state, MatchInstance* match_instance) override;
00100
00106         EffectType GetType() const override { return EffectType::kDraw; }
00107     private:
00108         int count_;
00109         std::string target_username_;
00110     };
00111
00117     class SkipEffect : public Effect {
00118     public:
00126         EffectResult Resolve(GameState* state, MatchInstance* match_instance) override;
```

```

00127     EffectType GetType() const override { return EffectType::kSkip; }
00133 };
00134
00135 class ReverseEffect : public Effect {
00141 public:
00142     EffectResult Resolve(GameState* state, MatchInstance* match_instance) override;
00150
00151     EffectType GetType() const override { return EffectType::kReverse; }
00157 };
00158
00159 class ChooseColorEffect : public Effect {
00165 public:
00166     explicit ChooseColorEffect(const std::string& target_username) : target_username_(target_username) {}
00171
00172     EffectResult Resolve(GameState* state, MatchInstance* match_instance) override;
00180
00181     EffectType GetType() const override { return EffectType::kChooseColor; }
00187 private:
00188     std::string target_username_;
00189 };
00190
00191 }

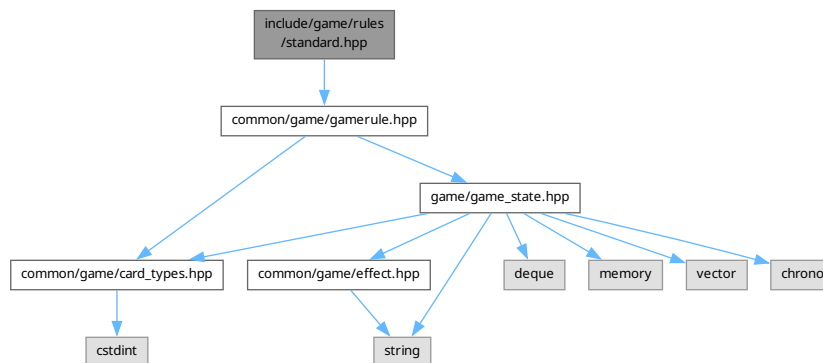
```

9.45 Riferimenti per il file `include/game/rules/standard.hpp`

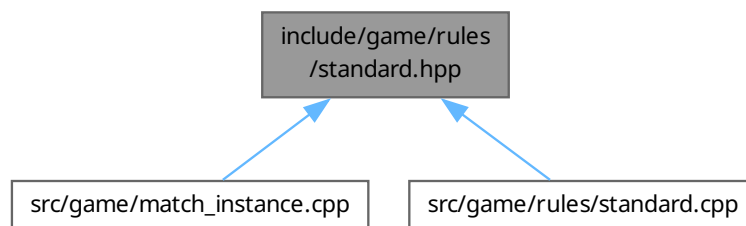
Implementazione del set di regole classiche di base del gioco UNO.

```
#include <common/game/gamerule.hpp>
```

Grafo delle dipendenze di inclusione per `standard.hpp`:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class [game::StandardRule](#)
Validatore e processore principale delle meccaniche standard del gioco.

Namespace

- namespace [game](#)

9.45.1 Descrizione dettagliata

Implementazione del set di regole classiche di base del gioco UNO.

- Questa regola è responsabile della validazione primaria (colore su colore, valore su valore) e dell'inserimento nella coda di risoluzione degli effetti associati ad ogni tipo di carta speciale giocata.

9.46 standard.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <common/game/gamerule.hpp>
00003
00011
00012 namespace game {
00013
00023     class StandardRule : public GameRule {
00024     public:
00025         ~StandardRule() override = default;
00026
00035         void ValidatePlay(GameState* state, CardPlayedEvent& event) override;
00036
00045         void OnCardPlayed(GameState* state, CardPlayedEvent& event) override;
00046
00047     private:
00055         std::string GetNextPlayer(GameState* state);
00056     };
00057 }
```

9.47 Riferimenti per il file include/http_router.hpp

Implementazione del router per le richieste HTTP/REST.

```
#include <router.hpp>
#include <websocket_context.hpp>
#include <functional>
#include <string>
```


9.47.2 Documentazione delle ridefinizioni di tipo (typedef)

9.47.2.1 HttpHandler

using `HttpHandler` = std::function<void(`AppResponse*`, `AppRequest*`)>

Firma per i gestori delle richieste HTTP. Riceve i puntatori agli oggetti `AppResponse` e `AppRequest` di uWS.

Tag Identificativo: HTTP-RTR-TYP-001

9.48 http_router.hpp

[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <router.hpp>
00003 #include <websocket_context.hpp>
00004 #include <functional>
00005 #include <string>
00006 #include <vector>
00007
00014
00021 using HttpHandler = std::function<void(AppResponse*, AppRequest*)>;
00022
00030 class HttpRouter : public Router {
00031 public:
00032     HttpRouter() = default;
00033
00042     HttpRouter& Get(const std::string& path, HttpHandler handler);
00043
00051     HttpRouter& Post(const std::string& path, HttpHandler handler);
00052
00062     HttpRouter& OnAny(std::function<bool(AppResponse*, AppRequest*)> handler);
00063
00070     void Attach(uWS::SSLApp& app);
00071
00072 private:
00078     struct Route {
00079         std::string method;
00080         std::string path;
00081         HttpHandler handler;
00082
00083         Route(std::string m, std::string p, HttpHandler h) : method(m), path(p), handler(h) {}
00084     };
00085
00086     std::vector<Route> routes_;
00087     std::vector<std::function<bool(AppResponse*, AppRequest*)>» wildcards_;
00088
00095     HttpHandler WrapWithWildcards(HttpHandler handler);
00096 };
```

9.49 Riferimenti per il file include/logger.hpp

Sistema di logging thread-safe e formattato a colori per la console.

```
#include <iostream>
#include <string>
#include <string_view>
```



```

00029 static constexpr std::string_view Reset = "\033[0m";
00030 static constexpr std::string_view Gray = "\033[90m";
00031 static constexpr std::string_view Red = "\033[31m";
00032 static constexpr std::string_view Green = "\033[32m";
00033 static constexpr std::string_view Yellow = "\033[33m";
00034 static constexpr std::string_view Blue = "\033[34m";
00035 static constexpr std::string_view Magenta = "\033[35m";
00036 static constexpr std::string_view Cyan = "\033[36m";
00037 static constexpr std::string_view White = "\033[37m";
00038 static constexpr std::string_view Bold = "\033[1m";
00039
00040 inline static bool is_initialized = false;
00041
00042 static void Init() {
00043     if (is_initialized) return;
00044 #ifdef _WIN32
00045     HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
00046     if (hOut != INVALID_HANDLE_VALUE) {
00047         DWORD dwMode = 0;
00048         if (GetConsoleMode(hOut, &dwMode)) {
00049             dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
00050             SetConsoleMode(hOut, dwMode);
00051         }
00052     }
00053 #endif
00054     is_initialized = true;
00055 }
00056
00057 static std::string timestamp() {
00058     auto now = std::chrono::system_clock::now();
00059     auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
00060         now.time_since_epoch()) % 1000;
00061
00062     time_t t = std::chrono::system_clock::to_time_t(now);
00063     tm tm_buf;
00064 #ifdef _WIN32
00065     localtime_s(&tm_buf, &t); // In windows gli argomenti sono swappati
00066 #else
00067     localtime_r(&t, &tm_buf); // POSIX
00068 #endif
00069
00070     char buf[20];
00071     strftime(buf, sizeof(buf), "%H:%M:%S", &tm_buf);
00072
00073     std::ostringstream oss;
00074     oss << Gray << "[" << buf << " " << std::setfill('0') << std::setw(3) << ms.count() << "]" << Reset;
00075     return oss.str();
00076 }
00077
00078 template<typename... Args>
00079 static void Print(std::string_view color, std::string_view tag, Args&&... args) {
00080     if (!is_initialized) Init();
00081
00082     std::ostringstream oss;
00083     oss << timestamp() << " ";
00084
00085     oss << Gray << "[" << Reset << color << Bold << std::left << std::setw(5) << tag << Reset << Gray << "]" << Reset;
00086
00087     // C++17 Fold Expression to unpack arguments efficiently
00088     (oss << ... << std::forward<Args>(args));
00089
00090     std::cout << oss.str() << "\n";
00091 }
00092
00093 public:
00094     template<typename... Args>
00095     static void Info(Args&&... args) { Print(Green, "INFO", std::forward<Args>(args)...); }
00096
00097     template<typename... Args>
00098     static void Warn(Args&&... args) { Print(Yellow, "WARN", std::forward<Args>(args)...); }
00099
00100     template<typename... Args>
00101     static void Error(Args&&... args) { Print(Red, "ERROR", std::forward<Args>(args)...); }
00102
00103     template<typename... Args>
00104     static void WS(Args&&... args) { Print(Cyan, " WS ", std::forward<Args>(args)...); }
00105
00106     template<typename... Args>
00107     static void Lobby(Args&&... args) { Print(Yellow, "LOBBY", std::forward<Args>(args)...); }
00108
00109     template<typename... Args>
00110     static void HTTP(Args&&... args) { Print(Magenta, "HTTP", std::forward<Args>(args)...); }
00111
00112     template<typename... Args>
00113     static void Log(Args&&... args) { Print(White, " LOG", std::forward<Args>(args)...); }
00114
00115     static constexpr std::string_view ColorWhite() { return White; }

```

```
00158 static constexpr std::string_view ColorReset() { return Reset; }
00159 };
```

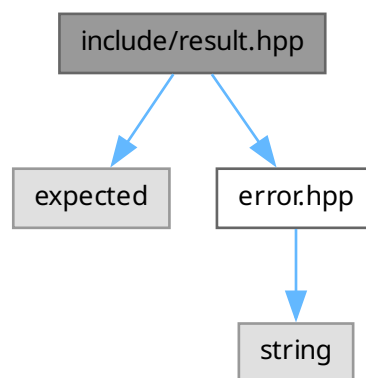
9.51 Riferimenti per il file include/result.hpp

Definisce alias per la gestione funzionale degli errori tramite std::expected.

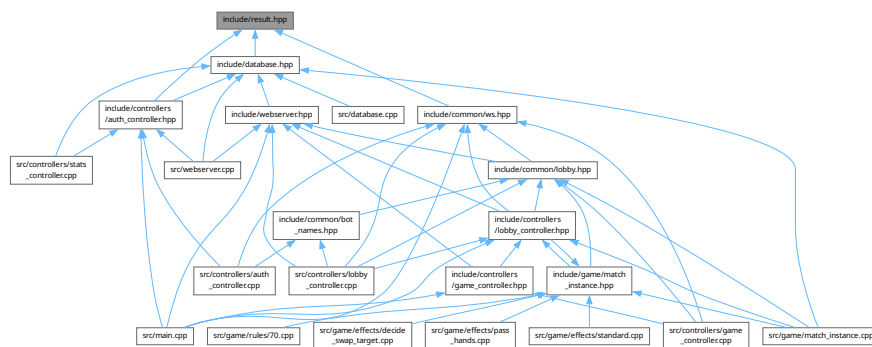
```
#include <expected>
```

```
#include <error.hpp>
```

Grafo delle dipendenze di inclusione per result.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Ridefinizioni di tipo (typedef)

- `template<typename T>`
`using Result = std::expected<T, Error>`

Rappresenta un'operazione che può restituire un valore di tipo T o un Error.

- `using VoidResult = std::expected<void, Error>`

Rappresenta un'operazione senza tipo di ritorno che può fallire restituendo un Error. Tipicamente usato per query al DB (UPDATE/DELETE) che non ritornano set di dati.

9.51.1 Descrizione dettagliata

Definisce alias per la gestione funzionale degli errori tramite `std::expected`.

- Sfrutta la funzionalità C++23 (o backport) `std::expected` per ritornare risultati in maniera monadica, evitando l'overhead delle eccezioni (`try/catch`).

9.51.2 Documentazione delle ridefinizioni di tipo (typedef)

9.51.2.1 Result

```
template<typename T>
using Result = std::expected<T, Error>
```

Rappresenta un'operazione che può restituire un valore di tipo `T` o un `Error`.

Parametri dei template

T	Il tipo del dato ritornato in caso di successo.
---	---

Tag Identificativo: RES-TYP-001

9.51.2.2 VoidResult

```
using VoidResult = std::expected<void, Error>
```

Rappresenta un'operazione senza tipo di ritorno che può fallire restituendo un `Error`. Tipicamente usato per query al DB (`UPDATE/DELETE`) che non ritornano set di dati.

Tag Identificativo: RES-TYP-002

9.52 result.hpp

[Vai alla documentazione di questo file.](#)

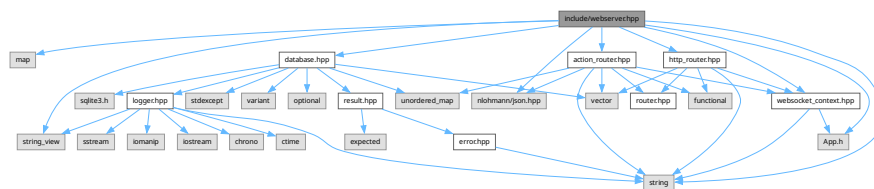
```
00001 #pragma once
00002 #include <expected>
00003 #include <error.hpp>
00004
00011
00018 template<typename T>
00019 using Result = std::expected<T, Error>;
00020
00027 using VoidResult = std::expected<void, Error>;
```


9.55 Riferimenti per il file include/webserver.hpp

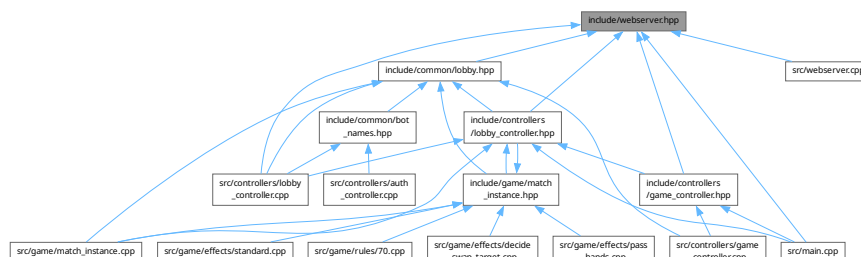
Definizione della classe principale [WebServer](#) che gestisce l'intero ciclo di vita dell'applicazione di rete.

```
#include <map>
#include <string>
#include <string_view>
#include <App.h>
#include <nlohmann/json.hpp>
#include <database.hpp>
#include <action_router.hpp>
#include <http_router.hpp>
#include <websocket_context.hpp>
```

Grafo delle dipendenze di inclusione per webserver.hpp:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- class [WebServer](#)

Motore principale dell'applicazione backend.

9.55.1 Descrizione dettagliata

Definizione della classe principale [WebServer](#) che gestisce l'intero ciclo di vita dell'applicazione di rete.

Questo file contiene l'infrastruttura base per l'avvio del server HTTP e WebSocket basato su uWebSockets. Gestisce le rotte, l'inizializzazione del database e le connessioni attive.

9.56 webserver.hpp

[Vai alla documentazione di questo file.](#)

```

00001 #pragma once
00002 #include <map>
00003 #include <string>
00004 #include <string_view>
00005 #include <App.h>
00006 #include <nlohmann/json.hpp>
00007 #include <database.hpp>
00008 #include <action_router.hpp>
00009 #include <http_router.hpp>
00010 #include <websocket_context.hpp>
00011
00019
00028 class WebServer {
00029 public:
00038     explicit WebServer(int port,
00039                        std::string_view keyFile = "key.pem",
00040                        std::string_view certFile = "cert.pem",
00041                        std::string_view dbFile = "game.db");
00042
00047     ~WebServer();
00048
00049     WebServer(const WebServer&) = delete;
00050     WebServer& operator=(const WebServer&) = delete;
00051
00057     void Run();
00058
00064     ActionRouter& GetActionRouter() { return ws_router_; }
00065
00071     HttpRouter& GetHTTPRouter() { return http_router_; }
00072
00078     uWS::SSLApp& GetApp() { return app_; }
00079
00085     using ConnectionHandler = std::function<void(AppWebSocket*, PerSocketData*)>;
00086
00092     void OnConnectionOpen(ConnectionHandler handler);
00093
00099     void OnConnectionClose(ConnectionHandler handler);
00100
00101 private:
00102     int port_;
00103     string db_file_;
00104
00105     uWS::SSLApp app_;
00106     std::map<string, AppWebSocket*> connections_;
00107
00108     ActionRouter ws_router_;
00109     HttpRouter http_router_;
00110
00116     bool InitDB();
00117
00122     void RegisterRoutes();
00123
00128     void HandlePost(AppResponse*, AppRequest*);
00129
00134     void HandleGet (AppResponse*, AppRequest*);
00135
00140     void OnSocketOpen (AppWebSocket*);
00141
00146     void OnSocketMessage(AppWebSocket*, std::string_view, uWS::OpCode);
00147
00152     void OnSocketClosed (AppWebSocket*);
00153
00160     static std::string ReadFile (std::string_view path);
00161
00168     static std::string GetMimeType (const string& path);
00169
00170     std::vector<ConnectionHandler> on_open_hooks_;
00171     std::vector<ConnectionHandler> on_close_hooks_;
00172 };

```

9.57 Riferimenti per il file include/websocket_context.hpp

Definisce i tipi e le strutture di contesto associati alle connessioni WebSocket.

9.57.2 Documentazione delle ridefinizioni di tipo (typedef)

9.57.2.1 AppRequest

```
using AppRequest = uWS::HttpRequest
```

Alias per la richiesta HTTP in ingresso di uWebSockets.

Tag Identificativo: WS-CTX-TYP-002

9.57.2.2 AppResponse

```
using AppResponse = uWS::HttpResponse<true>
```

Alias per la risposta HTTP in uscita di uWebSockets (con supporto SSL).

Tag Identificativo: WS-CTX-TYP-003

9.57.2.3 AppWebSocket

```
using AppWebSocket = uWS::WebSocket<true, true, PerSocketData>
```

Alias per il tipo complesso di socket uWebSockets configurato per SSL e routing.

Tag Identificativo: WS-CTX-TYP-001

9.58 websocket_context.hpp

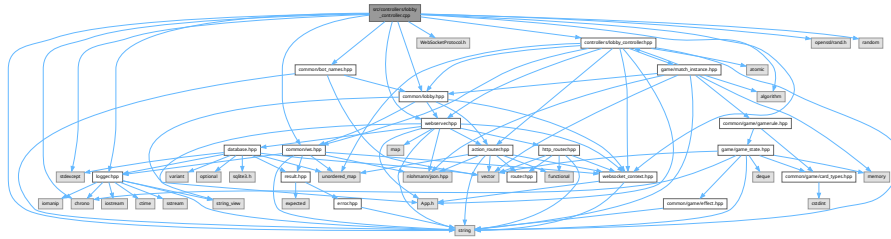
[Vai alla documentazione di questo file.](#)

```
00001 #pragma once
00002 #include <string>
00003 #include <App.h>
00004
00011
00020 struct PerSocketData {
00021     std::string username;
00022     std::string room;
00023     std::string lobby__code;
00024 };
00025
00031 using AppWebSocket = uWS::WebSocket<true, true, PerSocketData>;
00032
00038 using AppRequest  = uWS::HttpRequest;
00039
00045 using AppResponse = uWS::HttpResponse<true>;
00046
00054 struct WsContext {
00055     AppWebSocket* socket;
00056     PerSocketData* socket__data;
00057     uWS::OpCode  op__code;
00058 };
```



```
#include <random>
```

Grafo delle dipendenze di inclusione per lobby_controller.cpp:



Funzioni

- `std::string GetRandomBotName (const Lobby &lobby)`
Selects a random, unique bot name from a predefined thematic list.

Variabili

- static constexpr char `kCodeAlphabet []` = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
- static constexpr int `kCodeLen` = 6
- static constexpr int `kAlphabetLen` = 36

9.62.1 Descrizione dettagliata

Implementation of the `LobbyController` class managing websocket signaling and session lifecycles for game rooms.

9.62.2 Documentazione delle funzioni

9.62.2.1 GetRandomBotName()

```
std::string GetRandomBotName (
    const Lobby & lobby)
```

Selects a random, unique bot name from a predefined thematic list.

- Iterates through the current lobby members to ensure that the selected bot name is not already taken by an existing human or bot. If all predefined names are in use (unlikely but possible), it safely falls back to a sequentially numbered "Bot_N" format.

- Parametri

lobby	The target lobby context used to verify name availability.
-------	--

Restituisce

`std::string` A unique bot name safe for insertion into the lobby.

9.62.3 Documentazione delle variabili

9.62.3.1 kAlphabetLen

```
int kAlphabetLen = 36 [static], [constexpr]
```

9.62.3.2 kCodeAlphabet

```
char kCodeAlphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789" [static], [constexpr]
```

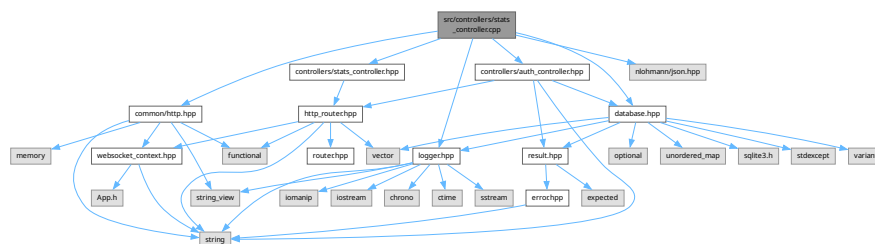
9.62.3.3 kCodeLen

```
int kCodeLen = 6 [static], [constexpr]
```

9.63 Riferimenti per il file `src/controllers/stats_controller.cpp`

```
#include <controllers/stats_controller.hpp>
#include <controllers/auth_controller.hpp>
#include <database.hpp>
#include <common/http.hpp>
#include <nlohmann/json.hpp>
#include <logger.hpp>
```

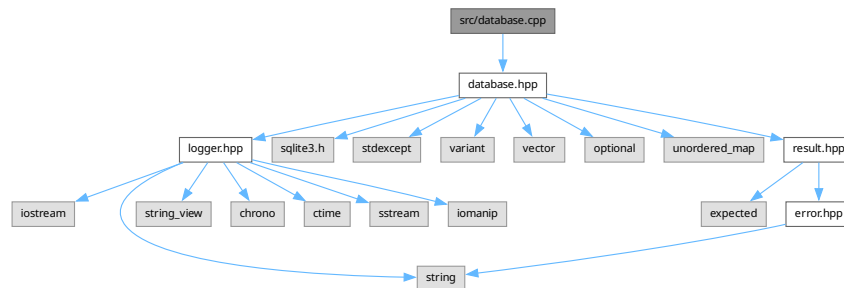
Grafo delle dipendenze di inclusione per `stats_controller.cpp`:



9.64 Riferimenti per il file src/database.cpp

```
#include "database.hpp"
```

Grafo delle dipendenze di inclusione per database.cpp:



9.65 Riferimenti per il file src/game/effects/decide_swap_target.cpp

```
#include <nlohmann/json.hpp>
```

```
#include <common/game/gamerule.hpp>
```

```
#include <common/game/effect.hpp>
```

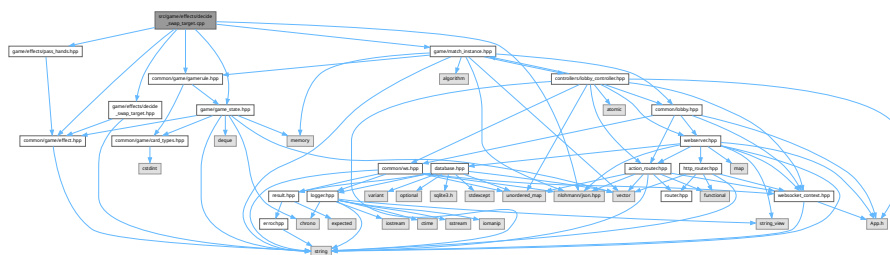
```
#include <game/game_state.hpp>
```

```
#include <game/match_instance.hpp>
```

```
#include <game/effects/decide_swap_target.hpp>
```

```
#include <game/effects/pass_hands.hpp>
```

Grafo delle dipendenze di inclusione per decide_swap_target.cpp:

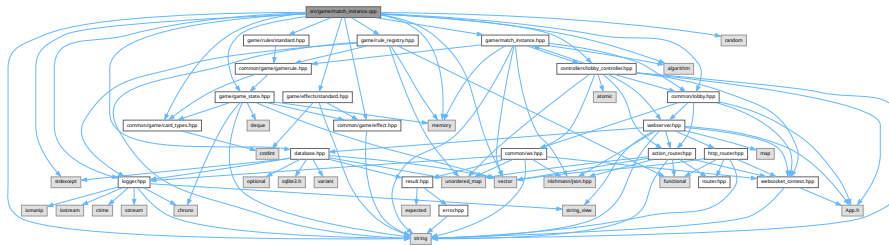


Namespace

- namespace [game](#)


```
#include <vector>
```

Grafo delle dipendenze di inclusione per `match_instance.cpp`:



Namespace

- namespace `game`

Funzioni

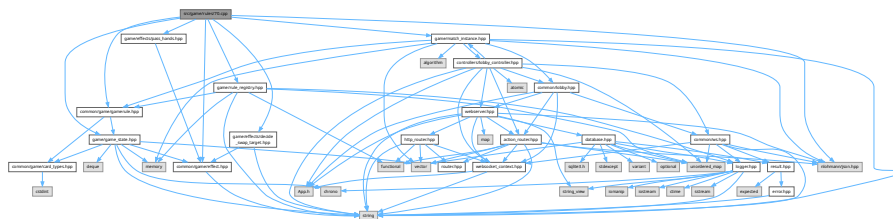
- `void game::ReshuffleDiscardIntoDraw (GameState *game_state)`

Utilità di sistema che sposta e rimescola le carte dagli scarti al mazzo principale. Viene invocata automaticamente dal motore di gioco quando `draw_pile` si esaurisce.

9.68 Riferimenti per il file `src/game/rules/70.cpp`

```
#include <common/game/gamerule.hpp>
#include <common/game/effect.hpp>
#include <game/game_state.hpp>
#include <game/match_instance.hpp>
#include <game/rule_registry.hpp>
#include <game/effects/pass_hands.hpp>
#include <game/effects/decide_swap_target.hpp>
#include <nlohmann/json.hpp>
```

Grafo delle dipendenze di inclusione per 70.cpp:



Composti

- class `game::SevenZeroRule`

Namespace

- namespace [game](#)

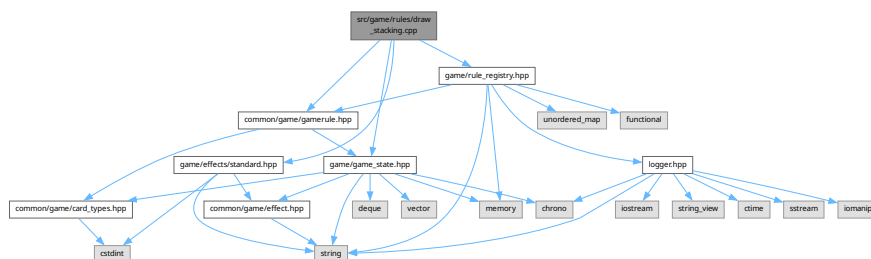
Variabili

- static [RuleRegistrar](#) [game::registrar](#) ("seven_zero", []) { return std::make_unique<[SevenZeroRule](#)>();}

9.69 Riferimenti per il file `src/game/rules/draw_stacking.cpp`

```
#include <common/game/gamerule.hpp>
#include <game/game_state.hpp>
#include <game/rule_registry.hpp>
#include <game/effects/standard.hpp>
```

Grafo delle dipendenze di inclusione per `draw_stacking.cpp`:



Composti

- class [game::DrawStackingRule](#)

Namespace

- namespace [game](#)

Variabili

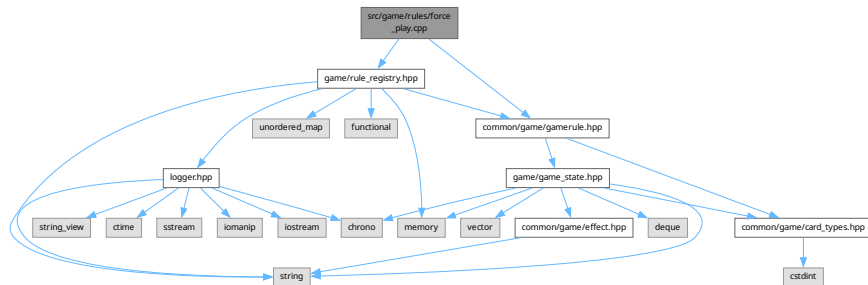
- static [RuleRegistrar](#) [game::registrar](#) ("draw_stacking", []) { return std::make_unique<[DrawStackingRule](#)>();}

9.70 Riferimenti per il file src/game/rules/force_play.cpp

```
#include <common/game/gamerule.hpp>
```

```
#include <game/rule_registry.hpp>
```

Grafo delle dipendenze di inclusione per force_play.cpp:



Composti

- class [game::ForcePlayRule](#)

Namespace

- namespace [game](#)

Variabili

- static [RuleRegistrar](#) [game::registrar](#) ("force_play", []() { return std::make_unique< [ForcePlayRule](#) >();})

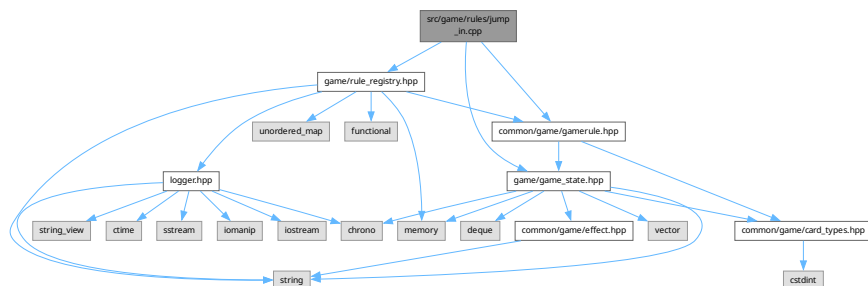
9.71 Riferimenti per il file src/game/rules/jump_in.cpp

```
#include <common/game/gamerule.hpp>
```

```
#include <game/game_state.hpp>
```

```
#include <game/rule_registry.hpp>
```

Grafo delle dipendenze di inclusione per jump_in.cpp:



Composti

- class [game::JumpInRule](#)

Namespace

- namespace [game](#)

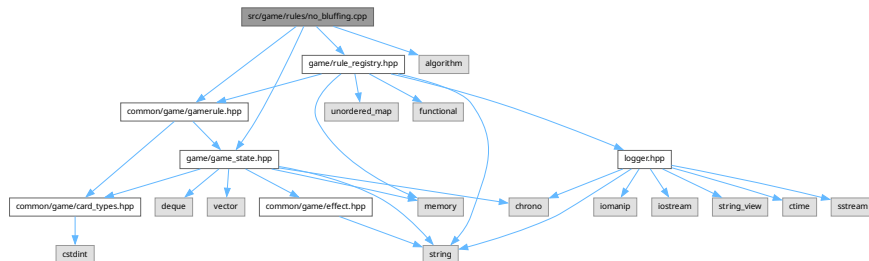
Variabili

- static [RuleRegistrar](#) [game::registrar](#) ("jump_in", []) { return std::make_unique< [JumpInRule](#)>();}

9.72 Riferimenti per il file `src/game/rules/no_bluffing.cpp`

```
#include <common/game/gamerule.hpp>
#include <game/game_state.hpp>
#include <game/rule_registry.hpp>
#include <algorithm>
```

Grafo delle dipendenze di inclusione per `no_bluffing.cpp`:



Composti

- class [game::NoBluffingRule](#)

Namespace

- namespace [game](#)

Variabili

- static [RuleRegistrar](#) [game::registrar](#) ("no_bluffing", []) { return std::make_unique< [NoBluffingRule](#)>();}

Namespace

- namespace [game](#)

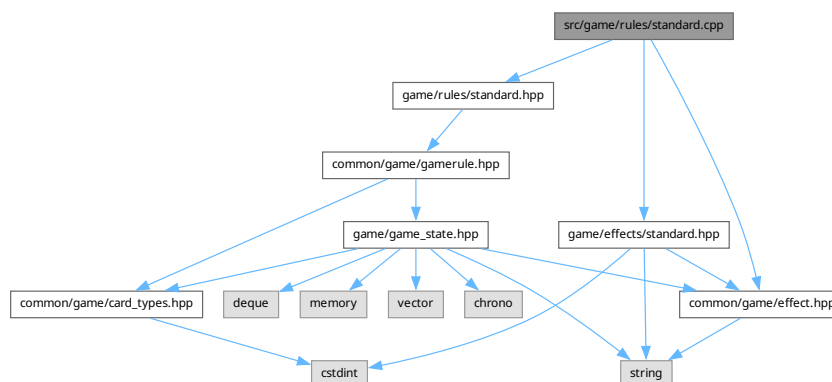
9.75 Riferimenti per il file `src/game/rules/standard.cpp`

```
#include <game/rules/standard.hpp>
```

```
#include "game/effects/standard.hpp"
```

```
#include <common/game/effect.hpp>
```

Grafo delle dipendenze di inclusione per `standard.cpp`:



Namespace

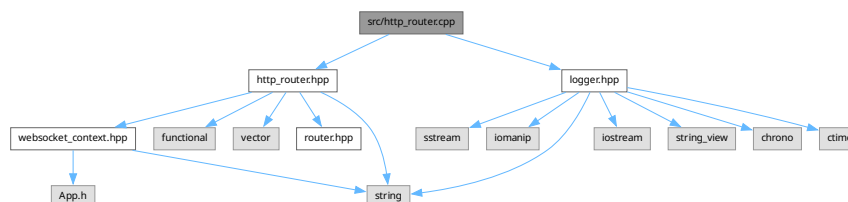
- namespace [game](#)

9.76 Riferimenti per il file `src/http_router.cpp`

```
#include <http_router.hpp>
```

```
#include <logger.hpp>
```

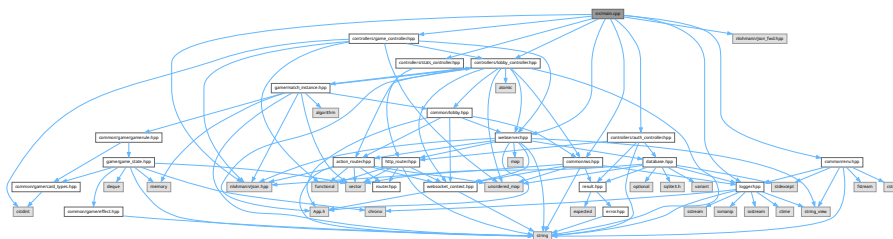
Grafo delle dipendenze di inclusione per `http_router.cpp`:



9.77 Riferimenti per il file src/main.cpp

```
#include <controllers/game_controller.hpp>
#include <controllers/lobby_controller.hpp>
#include <controllers/auth_controller.hpp>
#include <controllers/stats_controller.hpp>
#include <common/env.hpp>
#include <logger.hpp>
#include <common/ws.hpp>
#include <nlohmann/json_fwd.hpp>
#include <webserver.hpp>
#include <nlohmann/json.hpp>
```

Grafo delle dipendenze di inclusione per main.cpp:



Funzioni

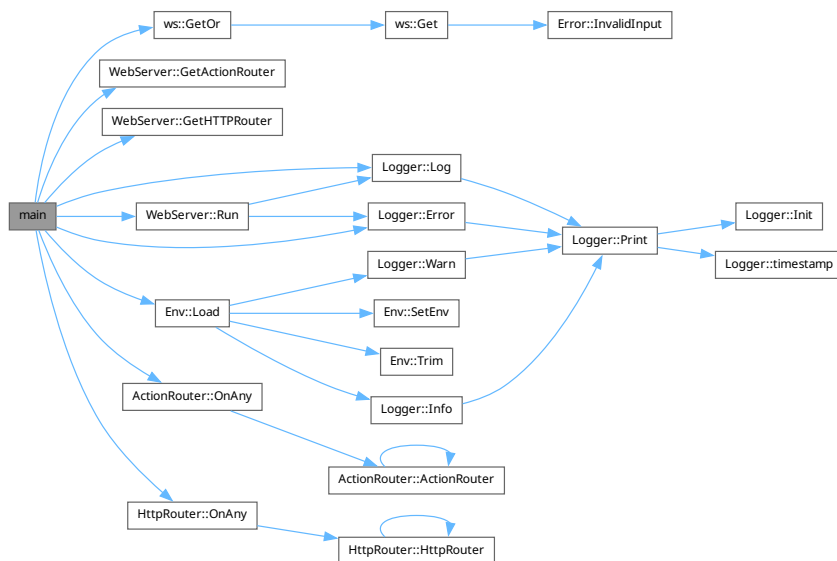
- int `main` ()

9.77.1 Documentazione delle Funzioni

9.77.1.1 main()

int `main` ()

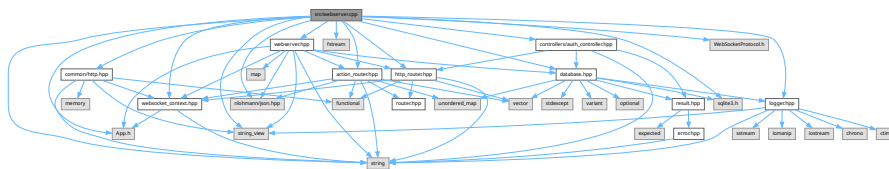
Questo è il grafo delle chiamate per questa funzione:



9.78 Riferimenti per il file `src/webserver.cpp`

```
#include "action_router.hpp"
#include "common/http.hpp"
#include <fstream>
#include "controllers/auth_controller.hpp"
#include "http_router.hpp"
#include "websocket_context.hpp"
#include <WebSocketProtocol.h>
#include <nlohmann/json.hpp>
#include <sqlite3.h>
#include <string>
#include <string_view>
#include <App.h>
#include <webserver.hpp>
#include <database.hpp>
#include <logger.hpp>
```

Grafo delle dipendenze di inclusione per `webserver.cpp`:



Indice analitico

- ~Database
 - Database, [75](#)
- ~Effect
 - game::Effect, [100](#)
- ~GameRule
 - game::GameRule, [120](#)
- ~LobbyController
 - LobbyController, [142](#)
- ~Router
 - Router, [207](#)
- ~SevenZeroRule
 - game::SevenZeroRule, [213](#)
- ~StandardRule
 - game::StandardRule, [218](#)
- ~TransactionGuard
 - TransactionGuard, [225](#)
- ~WebServer
 - WebServer, [231](#)
- action_router.hpp
 - ActionHandler, [244](#)
 - json, [244](#)
- action_router_
 - GameController, [119](#)
 - LobbyController, [163](#)
- ActionHandler
 - action_router.hpp, [244](#)
- ActionRouter, [51](#)
 - ActionRouter, [53](#)
 - Dispatch, [54](#)
 - handlers_, [56](#)
 - On, [54](#)
 - OnAny, [55](#)
 - wildcards_, [56](#)
- active_color
 - game::GameState, [123](#)
- active_rules_
 - game::MatchInstance, [192](#)
- active_turn_timers_
 - GameController, [119](#)
- AddPlayerMidGame
 - game::MatchInstance, [181](#)
- app_
 - LobbyController, [163](#)
 - WebServer, [238](#)
- ApplySchema
 - Database, [76](#)
- AppRequest
 - websocket_context.hpp, [304](#)
- AppResponse
 - websocket_context.hpp, [304](#)
- AppWebSocket
 - websocket_context.hpp, [304](#)
- Attach
 - HttpRouter, [127](#)
- AuthController, [58](#)
 - AuthController, [60](#)
 - HandleLogin, [61](#)
 - HandleLogout, [61](#)
 - HandleMe, [62](#)
 - HandleRegister, [62](#)
 - HashPassword, [63](#)
 - IssueToken, [64](#)
 - kHashBytes, [67](#)
 - kIterations, [67](#)
 - kMaxBodyBytes, [67](#)
 - kMaxUsernameLen, [68](#)
 - kMinPasswordLen, [68](#)
 - kMinUsernameLen, [68](#)
 - kSaltBytes, [68](#)
 - VerifyPassword, [65](#)
 - VerifyToken, [66](#)
- BadRequest
 - Error, [104](#)
- Base64, [25](#)
 - Decode, [25](#)
 - Encode, [25](#)
 - kAlphabet, [26](#)
 - kDecodeTable, [26](#)
 - kInvalid, [26](#)
- Blue
 - Logger, [176](#)
- Bold
 - Logger, [176](#)
- BotTakeoverMode
 - lobby.hpp, [262](#)
- BroadcastGameState
 - GameController, [111](#)
- BroadcastUpdate
 - LobbyController, [142](#)
- callback
 - TurnTimerData, [228](#)
- CallUno
 - game::MatchInstance, [182](#)

- card
 - game::LastPlay, 135
- card_id_
 - game::DecideDrawnCardEffect, 90
- CheckMatchIntegrity
 - LobbyController, 143
- ChooseColorEffect
 - game::ChooseColorEffect, 72
- ClearTurnTimer
 - GameController, 112
- ClientAction
 - ws, 43
- Close
 - Database, 76
- Code
 - Error, 104
- code
 - Error, 107
- code_to_id_
 - LobbyController, 163
- Color
 - game, 33
- color_counts
 - game::PlayerSessionStats, 201
- ColorReset
 - Logger, 167
- ColorWhite
 - Logger, 167
- Commit
 - TransactionGuard, 226
- committed_
 - TransactionGuard, 227
- CompactCard
 - game, 33
- Conflict
 - Error, 104
- ConnectionHandler
 - WebServer, 230
- connections_
 - WebServer, 238
- controller
 - TurnTimerData, 228
- count_
 - game::DrawEffect, 97
- Create
 - game::RuleRegistry, 210
- current_player_index
 - game::GameState, 123
- Cyan
 - Logger, 177
- data_
 - DbRow, 87
- Database, 74
 - ~Database, 75
 - ApplySchema, 76
 - Close, 76
 - Database, 75
 - db_, 83
 - Exec, 77
 - Get, 78
 - IsOpen, 79
 - Open, 79
 - operator=, 80
 - Prepare, 80
 - Query, 81
 - QueryOne, 81
 - ReadRow, 82
- database.hpp
 - DbValue, 277
- DatabaseFail
 - Error, 104
- db_
 - Database, 83
 - TransactionGuard, 227
- db_file_
 - WebServer, 239
- DbRow, 83
 - data_, 87
 - Get, 84
 - GetOr, 85
 - Has, 86
 - Set, 87
- DbValue
 - database.hpp, 277
- DecideDrawnCardEffect
 - game::DecideDrawnCardEffect, 89
- DecideSwapTargetEffect
 - game::DecideSwapTargetEffect, 92
- Decode
 - Base64, 25
- discard_pile
 - game::GameState, 123
- Dispatch
 - ActionRouter, 54
- draw_pile
 - game::GameState, 123
- DrawCard
 - game::MatchInstance, 182
- DrawEffect
 - game::DrawEffect, 95
- drawn_card
 - game::CardDrawnEvent, 69
- effect_queue
 - game::GameState, 123
- EffectStatus
 - game, 33
- EffectType
 - game, 34
- Encode
 - Base64, 25
- Env, 26
 - Get, 27
 - Load, 27
 - Require, 28
 - SetEnv, 29
 - Trim, 29

- Error, 102
 - BadRequest, 104
 - Code, 104
 - code, 107
 - Conflict, 104
 - DatabaseFail, 104
 - HttpStatus, 105
 - Internal, 105
 - InvalidInput, 105
 - kBadRequest, 104
 - kConflict, 104
 - kDatabaseFailure, 104
 - kInternalServerError, 104
 - kInvalidInput, 104
 - kNotFound, 104
 - kUnauthorised, 104
 - Logger, 167
 - message, 107
 - NotFound, 106
 - Unauthorised, 106
- eviction_timer_
 - LobbyController, 164
- Exec
 - Database, 77
- ExportState
 - game::MatchInstance, 183
- FindLobbyForUser
 - LobbyController, 143
- force_play
 - game::CardDrawnEvent, 69
- game, 30
 - Color, 33
 - CompactCard, 33
 - EffectStatus, 33
 - EffectType, 34
 - GetColor, 35
 - GetId, 36
 - GetValue, 36
 - k0, 35
 - k1, 35
 - k2, 35
 - k3, 35
 - k4, 35
 - k5, 35
 - k6, 35
 - k7, 35
 - k8, 35
 - k9, 35
 - kAdvanceTurn, 34
 - kBlue, 33
 - kChooseColor, 34
 - kCustom, 34
 - kDecideDrawnCard, 34
 - kDecideSwapTarget, 34
 - kDraw, 34
 - kDraw2, 35
 - kError, 34
 - kFinished, 35
 - kGreen, 33
 - kNeedsInput, 34
 - kPassHands, 34
 - kPlayCard, 34
 - kPlaying, 35
 - kRed, 33
 - kReservedBotNames, 38
 - kResolved, 34
 - kReverse, 34, 35
 - kSkip, 34, 35
 - kWaitingToStart, 34
 - kWild, 33, 35
 - kWildDraw4, 35
 - kYellow, 33
 - MakeCard, 37
 - MatchStatus, 34
 - registrar, 38, 39
 - ReshuffleDiscardIntoDraw, 38
 - RuleFactory, 33
 - Value, 35
- game::AdvanceTurnEffect, 56
 - GetType, 57
 - Resolve, 57
- game::CardDrawnEvent, 69
 - drawn_card, 69
 - force_play, 69
 - is_handled, 69
 - keep_drawing, 69
 - player_username, 69
- game::CardPlayedEvent, 70
 - is_handled, 70
 - is_out_of_turn, 70
 - is_valid_play, 70
 - played_card, 70
 - player_username, 70
- game::ChooseColorEffect, 71
 - ChooseColorEffect, 72
 - GetType, 73
 - Resolve, 73
 - target_username_, 73
- game::DecideDrawnCardEffect, 88
 - card_id_, 90
 - DecideDrawnCardEffect, 89
 - GetType, 89
 - Resolve, 89
 - username_, 90
- game::DecideSwapTargetEffect, 91
 - DecideSwapTargetEffect, 92
 - GetType, 93
 - Resolve, 93
 - username_, 94
- game::DrawEffect, 94
 - count_, 97
 - DrawEffect, 95
 - GetType, 96
 - Resolve, 96
 - target_username_, 97

- game::DrawStackingRule, 97
 - OnCardPlayed, 98
 - ValidatePlay, 98
- game::Effect, 99
 - ~Effect, 100
 - GetType, 101
 - Resolve, 101
- game::EffectResult, 101
 - input_context, 102
 - input_type, 102
 - status, 102
 - target_player, 102
- game::ForcePlayRule, 108
 - OnCardDrawn, 109
- game::GameRule, 119
 - ~GameRule, 120
 - OnCardDrawn, 121
 - OnCardPlayed, 121
 - ValidatePlay, 121
- game::GameState, 122
 - active_color, 123
 - current_player_index, 123
 - discard_pile, 123
 - draw_pile, 123
 - effect_queue, 123
 - last_play, 123
 - pending_draws, 123
 - pending_input_context, 123
 - pending_input_type, 124
 - pending_player, 124
 - play_direction, 124
 - players, 124
 - provided_input, 124
 - status, 124
 - turn_end_time, 124
 - winner, 124
- game::JumpInRule, 132
 - OnCardPlayed, 133
 - ValidatePlay, 133
- game::LastPlay, 135
 - card, 135
 - hand_index, 135
 - player, 136
 - valid, 136
- game::MatchInstance, 178
 - active_rules_, 192
 - AddPlayerMidGame, 181
 - CallUno, 182
 - DrawCard, 182
 - ExportState, 183
 - GenerateDeck, 183
 - GetCurrentPlayerUsername, 184
 - GetMatchId, 184
 - GetPendingInputContext, 185
 - GetPendingInputType, 185
 - GetPendingPlayer, 185
 - GetPlayer, 186
 - GetWinner, 186
 - HasMod, 187
 - IsGameOver, 187
 - IsWaitingForInput, 187
 - match_id_, 192
 - MatchInstance, 180
 - PlayCard, 188
 - ProvideInput, 189
 - RemovePlayerMidGame, 190
 - SerializePlayerState, 190
 - session_stats_, 193
 - SetMatchId, 190
 - settings_, 193
 - SetTurnEndTime, 191
 - Start, 191
 - state_, 193
 - TakeBotTurn, 192
 - Tick, 192
- game::NoBluffingRule, 193
 - ValidatePlay, 194
- game::PassHandsEffect, 195
 - Resolve, 196
- game::Player, 198
 - hand, 199
 - has_called_uno, 199
 - is_bot, 200
 - Player, 199
 - username, 200
- game::PlayerSessionStats, 200
 - color_counts, 201
 - value_counts, 201
- game::ProgressiveRule, 201
 - OnCardDrawn, 202
- game::ReverseEffect, 202
 - GetType, 203
 - Resolve, 203
- game::RuleRegistrar, 208
 - RuleRegistrar, 209
- game::RuleRegistry, 209
 - Create, 210
 - GetMap, 211
- game::SevenZeroRule, 212
 - ~SevenZeroRule, 213
 - OnCardPlayed, 213
 - ValidatePlay, 213
- game::SkipEffect, 214
 - GetType, 215
 - Resolve, 215
- game::StandardRule, 216
 - ~StandardRule, 218
 - GetNextPlayer, 218
 - OnCardPlayed, 218
 - ValidatePlay, 219
- game_started_callback_
 - LobbyController, 164
- GameController, 109
 - action_router_, 119
 - active_turn_timers_, 119
 - BroadcastGameState, 111

- ClearTurnTimer, [112](#)
- GameController, [111](#)
- HandleCallUno, [113](#)
- HandleDrawCard, [114](#)
- HandlePlayCard, [115](#)
- HandleProvideInput, [115](#)
- lobby_controller_, [119](#)
- OnTurnStarted, [116](#)
- SetTurnTimer, [117](#)
- GameStartedCallback
 - lobby_controller.hpp, [272](#)
- GenerateDeck
 - game::MatchInstance, [183](#)
- GenerateInviteCode
 - LobbyController, [144](#)
- Get
 - Database, [78](#)
 - DbRow, [84](#)
 - Env, [27](#)
 - HttpRouter, [128](#)
 - ws, [44](#)
- GetActionRouter
 - WebServer, [232](#)
- GetApp
 - WebServer, [232](#)
- GetColor
 - game, [35](#)
- GetCookieValue
 - http, [40](#)
- GetCurrentPlayerUsername
 - game::MatchInstance, [184](#)
- GetHTTPRouter
 - WebServer, [233](#)
- GetId
 - game, [36](#)
- GetLobbyByCode
 - LobbyController, [145](#)
- GetLobbyById
 - LobbyController, [145](#)
- GetMap
 - game::RuleRegistry, [211](#)
- GetMatchId
 - game::MatchInstance, [184](#)
- GetMimeType
 - WebServer, [233](#)
- GetNextPlayer
 - game::StandardRule, [218](#)
- GetOr
 - DbRow, [85](#)
 - ws, [45](#)
- GetPendingInputContext
 - game::MatchInstance, [185](#)
- GetPendingInputType
 - game::MatchInstance, [185](#)
- GetPendingPlayer
 - game::MatchInstance, [185](#)
- GetPlayer
 - game::MatchInstance, [186](#)
- GetRandomBotName
 - lobby_controller.cpp, [307](#)
- GetType
 - game::AdvanceTurnEffect, [57](#)
 - game::ChooseColorEffect, [73](#)
 - game::DecideDrawnCardEffect, [89](#)
 - game::DecideSwapTargetEffect, [93](#)
 - game::DrawEffect, [96](#)
 - game::Effect, [101](#)
 - game::ReverseEffect, [203](#)
 - game::SkipEffect, [215](#)
- GetValue
 - game, [36](#)
- GetWinner
 - game::MatchInstance, [186](#)
- Gray
 - Logger, [177](#)
- Green
 - Logger, [177](#)
- hand
 - game::Player, [199](#)
- hand_index
 - game::LastPlay, [135](#)
- HandleCallUno
 - GameController, [113](#)
- HandleCreate
 - LobbyController, [146](#)
- HandleDeleteSavedMatch
 - LobbyController, [147](#)
- HandleDrawCard
 - GameController, [114](#)
- HandleGet
 - WebServer, [234](#)
- HandleGetLeaderboard
 - StatsController, [222](#)
- HandleGetMe
 - StatsController, [222](#)
- HandleGetSavedMatchesList
 - LobbyController, [147](#)
- HandleJoin
 - LobbyController, [148](#)
- HandleKick
 - LobbyController, [149](#)
- HandleLeave
 - LobbyController, [150](#)
- HandleList
 - LobbyController, [151](#)
- HandleLogin
 - AuthController, [61](#)
- HandleLogout
 - AuthController, [61](#)
- HandleMe
 - AuthController, [62](#)
- HandlePlayCard
 - GameController, [115](#)
- HandlePost
 - WebServer, [234](#)
- HandlePromote

- LobbyController, 152
- HandleProvideInput
 - GameController, 115
- handler
 - HttpRouter::Route, 205
- HandleRegister
 - AuthController, 62
- HandleRejoin
 - LobbyController, 153
- HandleResumeSavedMatch
 - LobbyController, 154
- handlers_
 - ActionRouter, 56
- HandleStartGame
 - LobbyController, 155
- HandleUpdateSettings
 - LobbyController, 156
- Has
 - DbRow, 86
- has_called_uno
 - game::Player, 199
- HashPassword
 - AuthController, 63
- HasMod
 - game::MatchInstance, 187
- host
 - Lobby, 137
- HTTP
 - Logger, 168
- http, 39
 - GetCookieValue, 40
 - ReadBody, 40
 - TrimWhitespace, 41
- http_router.hpp
 - HttpHandler, 295
- http_router_
 - WebServer, 239
- HttpHandler
 - http_router.hpp, 295
- HttpRouter, 125
 - Attach, 127
 - Get, 128
 - HttpRouter, 127
 - OnAny, 129
 - Post, 130
 - routes_, 131
 - wildcards_, 131
 - WrapWithWildcards, 131
- HttpRouter::Route, 204
 - handler, 205
 - method, 205
 - path, 205
 - Route, 205
- HttpStatus
 - Error, 105
- id
 - Lobby, 137
- include/action_router.hpp, 243, 245
 - include/common/base64.hpp, 245, 246
 - include/common/bot_names.hpp, 247, 249
 - include/common/env.hpp, 250, 252
 - include/common/game/card_types.hpp, 253, 254
 - include/common/game/effect.hpp, 255, 256
 - include/common/game/gamerule.hpp, 257, 258
 - include/common/http.hpp, 258, 260
 - include/common/lobby.hpp, 261, 262
 - include/common/ws.hpp, 263, 265
 - include/controllers/auth_controller.hpp, 267, 269
 - include/controllers/game_controller.hpp, 269, 271
 - include/controllers/lobby_controller.hpp, 271, 273
 - include/controllers/stats_controller.hpp, 274, 276
 - include/database.hpp, 276, 278
 - include/error.hpp, 279, 280
 - include/game/effects/decide_swap_target.hpp, 280, 282
 - include/game/effects/pass_hands.hpp, 282, 283
 - include/game/effects/standard.hpp, 290, 291
 - include/game/game_state.hpp, 284, 285
 - include/game/match_instance.hpp, 286, 287
 - include/game/rule_registry.hpp, 288, 289
 - include/game/rules/standard.hpp, 292, 293
 - include/http_router.hpp, 293, 295
 - include/logger.hpp, 295, 296
 - include/result.hpp, 298, 299
 - include/router.hpp, 300
 - include/webserver.hpp, 301, 302
 - include/websocket_context.hpp, 302, 304
- Info
 - Logger, 169
- Init
 - Logger, 170
- InitDB
 - WebServer, 234
- input_context
 - game::EffectResult, 102
- input_type
 - game::EffectResult, 102
- Internal
 - Error, 105
- InvalidInput
 - Error, 105
- invite_code
 - Lobby, 137
- is_bot
 - game::Player, 200
- is_handled
 - game::CardDrawnEvent, 69
 - game::CardPlayedEvent, 70
- is_initialized
 - Logger, 177
- is_out_of_turn
 - game::CardPlayedEvent, 70
- is_valid_play
 - game::CardPlayedEvent, 70
- IsGameOver
 - game::MatchInstance, 187

- IsOpen
 - Database, 79
- IssueToken
 - AuthController, 64
- IsWaitingForInput
 - game::MatchInstance, 187
- json
 - action_router.hpp, 244
- JwtPayload, 134
 - username, 135
- k0
 - game, 35
- k1
 - game, 35
- k2
 - game, 35
- k3
 - game, 35
- k4
 - game, 35
- k5
 - game, 35
- k6
 - game, 35
- k7
 - game, 35
- k8
 - game, 35
- k9
 - game, 35
- kAdvanceTurn
 - game, 34
- kAlphabet
 - Base64, 26
- kAlphabetLen
 - lobby_controller.cpp, 308
- kBadRequest
 - Error, 104
- kBlue
 - game, 33
- kChatMessage
 - ws, 44
- kChatSend
 - ws, 43
- kChooseColor
 - game, 34
- kClientActionStr
 - ws, 50
- kCodeAlphabet
 - lobby_controller.cpp, 308
- kCodeLen
 - lobby_controller.cpp, 308
- kConflict
 - Error, 104
- kCustom
 - game, 34
- kDatabaseFailure
 - Error, 104
- kDecideDrawnCard
 - game, 34
- kDecideSwapTarget
 - game, 34
- kDecodeTable
 - Base64, 26
- kDraw
 - game, 34
- kDraw2
 - game, 35
- keep_drawing
 - game::CardDrawnEvent, 69
- kError
 - game, 34
 - ws, 44
- kFinished
 - game, 35
- kGameCallUno
 - ws, 43
- kGameDrawCard
 - ws, 43
- kGameExit
 - ws, 43
- kGameOver
 - ws, 44
- kGamePlayCard
 - ws, 43
- kGameStateUpdated
 - ws, 44
- kGameSubmitInput
 - ws, 43
- kGreen
 - game, 33
- kHashBytes
 - AuthController, 67
- kInternalError
 - Error, 104
- kInvalid
 - Base64, 26
- kInvalidInput
 - Error, 104
- kIterations
 - AuthController, 67
- kLobbyCreate
 - ws, 43
- kLobbyDeleteSavedMatch
 - ws, 43
- kLobbyEvicted
 - ws, 44
- kLobbyJoin
 - ws, 43
- kLobbyJoined
 - ws, 44
- kLobbyKick
 - ws, 43
- kLobbyLeave
 - ws, 43

- kLobbyLeft
 - ws, [44](#)
- kLobbyList
 - ws, [43](#), [44](#)
- kLobbyListSavedMatches
 - ws, [43](#)
- kLobbyPromote
 - ws, [43](#)
- kLobbyRejoin
 - ws, [43](#)
- kLobbyResumeSavedMatch
 - ws, [43](#)
- kLobbyStartMatch
 - ws, [43](#)
- kLobbyUpdated
 - ws, [44](#)
- kLobbyUpdateSettings
 - ws, [43](#)
- kMaxBodyBytes
 - AuthController, [67](#)
- kMaxMembers
 - LobbyController, [164](#)
- kMaxUsernameLen
 - AuthController, [68](#)
- kMinPasswordLen
 - AuthController, [68](#)
- kMinUsernameLen
 - AuthController, [68](#)
- kNeedsInput
 - game, [34](#)
- kNotFound
 - Error, [104](#)
- kPassHands
 - game, [34](#)
- kPlayCard
 - game, [34](#)
- kPlaying
 - game, [35](#)
- kPlayInstantly
 - lobby.hpp, [262](#)
- kReconnectGraceMs
 - LobbyController, [164](#)
- kRed
 - game, [33](#)
- kReservedBotNames
 - game, [38](#)
- kResolved
 - game, [34](#)
- kReverse
 - game, [34](#), [35](#)
- kSaltBytes
 - AuthController, [68](#)
- kServerActionStr
 - ws, [50](#)
- kSkip
 - game, [34](#), [35](#)
- kStrClientAction
 - ws, [50](#)
- kSuccess
 - ws, [44](#)
- kUnauthorised
 - Error, [104](#)
- kWaitingToStart
 - game, [34](#)
- kWaitUntilTurnEnd
 - lobby.hpp, [262](#)
- kWild
 - game, [33](#), [35](#)
- kWildDraw4
 - game, [35](#)
- kYellow
 - game, [33](#)
- last_play
 - game::GameState, [123](#)
- Load
 - Env, [27](#)
- lobbies_
 - LobbyController, [164](#)
- Lobby, [136](#)
 - host, [137](#)
 - id, [137](#)
 - invite_code, [137](#)
 - Logger, [171](#)
 - match, [137](#)
 - members, [137](#)
 - name, [138](#)
 - settings, [138](#)
- lobby.hpp
 - BotTakeoverMode, [262](#)
 - kPlayInstantly, [262](#)
 - kWaitUntilTurnEnd, [262](#)
- lobby_code
 - PerSocketData, [198](#)
- lobby_controller.cpp
 - GetRandomBotName, [307](#)
 - kAlphabetLen, [308](#)
 - kCodeAlphabet, [308](#)
 - kCodeLen, [308](#)
- lobby_controller.hpp
 - GameStartedCallback, [272](#)
 - PlayerReplacedCallback, [272](#)
- lobby_controller_
 - GameController, [119](#)
- lobby_id
 - TurnTimerData, [228](#)
- LobbyController, [138](#)
 - ~LobbyController, [142](#)
 - action_router_, [163](#)
 - app_, [163](#)
 - BroadcastUpdate, [142](#)
 - CheckMatchIntegrity, [143](#)
 - code_to_id_, [163](#)
 - eviction_timer_, [164](#)
 - FindLobbyForUser, [143](#)
 - game_started_callback_, [164](#)
 - GenerateInviteCode, [144](#)

- GetLobbyByCode, [145](#)
- GetLobbyById, [145](#)
- HandleCreate, [146](#)
- HandleDeleteSavedMatch, [147](#)
- HandleGetSavedMatchesList, [147](#)
- HandleJoin, [148](#)
- HandleKick, [149](#)
- HandleLeave, [150](#)
- HandleList, [151](#)
- HandlePromote, [152](#)
- HandleRejoin, [153](#)
- HandleResumeSavedMatch, [154](#)
- HandleStartGame, [155](#)
- HandleUpdateSettings, [156](#)
- kMaxMembers, [164](#)
- kReconnectGraceMs, [164](#)
- lobbies_, [164](#)
- LobbyController, [141](#)
- MemberListJson, [157](#)
- next_id_, [164](#)
- OnClose, [158](#)
- OnGameStarted, [159](#)
- OnOpen, [159](#)
- OnPlayerReplaced, [160](#)
- player_replaced_callback_, [165](#)
- RemoveMember, [161](#)
- SaveMatchStateToDB, [162](#)
- SyncBots, [162](#)
- LobbyMember, [165](#)
- LobbySettings, [165](#)
- Log
 - Logger, [172](#)
- Logger, [166](#)
 - Blue, [176](#)
 - Bold, [176](#)
 - ColorReset, [167](#)
 - ColorWhite, [167](#)
 - Cyan, [177](#)
 - Error, [167](#)
 - Gray, [177](#)
 - Green, [177](#)
 - HTTP, [168](#)
 - Info, [169](#)
 - Init, [170](#)
 - is_initialized, [177](#)
 - Lobby, [171](#)
 - Log, [172](#)
 - Magenta, [177](#)
 - Print, [173](#)
 - Red, [177](#)
 - Reset, [177](#)
 - timestamp, [174](#)
 - Warn, [175](#)
 - White, [177](#)
 - WS, [176](#)
 - Yellow, [177](#)
- Magenta
 - Logger, [177](#)
- main
 - main.cpp, [317](#)
- main.cpp
 - main, [317](#)
- MakeCard
 - game, [37](#)
- MakeResponse
 - ws, [47](#)
- match
 - Lobby, [137](#)
- match_id_
 - game::MatchInstance, [192](#)
- MatchInstance
 - game::MatchInstance, [180](#)
- MatchStatus
 - game, [34](#)
- MemberListJson
 - LobbyController, [157](#)
- members
 - Lobby, [137](#)
- message
 - Error, [107](#)
- method
 - HttpRouter::Route, [205](#)
- name
 - Lobby, [138](#)
- next_id_
 - LobbyController, [164](#)
- NotFound
 - Error, [106](#)
- On
 - ActionRouter, [54](#)
- on_close_hooks_
 - WebServer, [239](#)
- on_open_hooks_
 - WebServer, [239](#)
- OnAny
 - ActionRouter, [55](#)
 - HttpRouter, [129](#)
- OnCardDrawn
 - game::ForcePlayRule, [109](#)
 - game::GameRule, [121](#)
 - game::ProgressiveRule, [202](#)
- OnCardPlayed
 - game::DrawStackingRule, [98](#)
 - game::GameRule, [121](#)
 - game::JumpInRule, [133](#)
 - game::SevenZeroRule, [213](#)
 - game::StandardRule, [218](#)
- OnClose
 - LobbyController, [158](#)
- OnConnectionClose
 - WebServer, [235](#)
- OnConnectionOpen
 - WebServer, [235](#)
- OnGameStarted
 - LobbyController, [159](#)

- OnOpen
 - LobbyController, [159](#)
- OnPlayerReplaced
 - LobbyController, [160](#)
- OnSocketClosed
 - WebServer, [236](#)
- OnSocketMessage
 - WebServer, [236](#)
- OnSocketOpen
 - WebServer, [236](#)
- OnTurnStarted
 - GameController, [116](#)
- op_code
 - WsContext, [240](#)
- Open
 - Database, [79](#)
- operator=
 - Database, [80](#)
 - Router, [208](#)
 - TransactionGuard, [226](#)
 - WebServer, [237](#)
- path
 - HttpRouter::Route, [205](#)
- pending_draws
 - game::GameState, [123](#)
- pending_input_context
 - game::GameState, [123](#)
- pending_input_type
 - game::GameState, [124](#)
- pending_player
 - game::GameState, [124](#)
- PerSocketData, [197](#)
 - lobby_code, [198](#)
 - room, [198](#)
 - username, [198](#)
- play_direction
 - game::GameState, [124](#)
- PlayCard
 - game::MatchInstance, [188](#)
- played_card
 - game::CardPlayedEvent, [70](#)
- Player
 - game::Player, [199](#)
- player
 - game::LastPlay, [136](#)
- player_replaced_callback_
 - LobbyController, [165](#)
- player_username
 - game::CardDrawnEvent, [69](#)
 - game::CardPlayedEvent, [70](#)
- PlayerReplacedCallback
 - lobby_controller.hpp, [272](#)
- players
 - game::GameState, [124](#)
- port_
 - WebServer, [239](#)
- Post
 - HttpRouter, [130](#)
- Prepare
 - Database, [80](#)
- Print
 - Logger, [173](#)
- provided_input
 - game::GameState, [124](#)
- ProvideInput
 - game::MatchInstance, [189](#)
- Query
 - Database, [81](#)
- QueryOne
 - Database, [81](#)
- ReadBody
 - http, [40](#)
- ReadFile
 - WebServer, [237](#)
- ReadRow
 - Database, [82](#)
- Red
 - Logger, [177](#)
- RegisterRoutes
 - WebServer, [237](#)
- registrar
 - game, [38](#), [39](#)
- RemoveMember
 - LobbyController, [161](#)
- RemovePlayerMidGame
 - game::MatchInstance, [190](#)
- Require
 - Env, [28](#)
- Reset
 - Logger, [177](#)
- ReshuffleDiscardIntoDraw
 - game, [38](#)
- Resolve
 - game::AdvanceTurnEffect, [57](#)
 - game::ChooseColorEffect, [73](#)
 - game::DecideDrawnCardEffect, [89](#)
 - game::DecideSwapTargetEffect, [93](#)
 - game::DrawEffect, [96](#)
 - game::Effect, [101](#)
 - game::PassHandsEffect, [196](#)
 - game::ReverseEffect, [203](#)
 - game::SkipEffect, [215](#)
- Result
 - result.hpp, [299](#)
- result.hpp
 - Result, [299](#)
 - VoidResult, [299](#)
- Riferimenti per la directory include, [22](#)
- Riferimenti per la directory include/common, [15](#)
- Riferimenti per la directory include/common/game, [19](#)
- Riferimenti per la directory include/controllers, [16](#)
- Riferimenti per la directory include/game, [20](#)
- Riferimenti per la directory include/game/effects, [18](#)

- Riferimenti per la directory include/game/rules, 23
- Riferimenti per la directory src, 24
- Riferimenti per la directory src/controllers, 17
- Riferimenti per la directory src/game, 21
- Riferimenti per la directory src/game/effects, 19
- Riferimenti per la directory src/game/rules, 23
- room
 - PerSocketData, 198
- Route
 - HttpRouter::Route, 205
- Router, 206
 - ~Router, 207
 - operator=, 208
 - Router, 207
- routes__
 - HttpRouter, 131
- RuleFactory
 - game, 33
- RuleRegistrar
 - game::RuleRegistrar, 209
- Run
 - WebServer, 238
- SaveMatchStateToDB
 - LobbyController, 162
- SendError
 - ws, 48
- SerializePlayerState
 - game::MatchInstance, 190
- ServerAction
 - ws, 43
- session_stats__
 - game::MatchInstance, 193
- Set
 - DbRow, 87
- SetEnv
 - Env, 29
- SetMatchId
 - game::MatchInstance, 190
- settings
 - Lobby, 138
- settings__
 - game::MatchInstance, 193
- SetTurnEndTime
 - game::MatchInstance, 191
- SetTurnTimer
 - GameController, 117
- socket
 - WsContext, 240
- socket_data
 - WsContext, 241
- src/action_router.cpp, 305
- src/controllers/auth_controller.cpp, 305
- src/controllers/game_controller.cpp, 305
- src/controllers/lobby_controller.cpp, 306
- src/controllers/stats_controller.cpp, 308
- src/database.cpp, 309
- src/game/effects/decide_swap_target.cpp, 309
- src/game/effects/pass_hands.cpp, 310
- src/game/effects/standard.cpp, 315
- src/game/match_instance.cpp, 310
- src/game/rules/70.cpp, 311
- src/game/rules/draw_stacking.cpp, 312
- src/game/rules/force_play.cpp, 313
- src/game/rules/jump_in.cpp, 313
- src/game/rules/no_bluffing.cpp, 314
- src/game/rules/progressive.cpp, 315
- src/game/rules/standard.cpp, 316
- src/http_router.cpp, 316
- src/main.cpp, 317
- src/webserver.cpp, 318
- Start
 - game::MatchInstance, 191
- state__
 - game::MatchInstance, 193
- StatsController, 220
 - HandleGetLeaderboard, 222
 - HandleGetMe, 222
 - StatsController, 221
- status
 - game::EffectResult, 102
 - game::GameState, 124
- SyncBots
 - LobbyController, 162
- TakeBotTurn
 - game::MatchInstance, 192
- target_player
 - game::EffectResult, 102
- target_username__
 - game::ChooseColorEffect, 73
 - game::DrawEffect, 97
- Tick
 - game::MatchInstance, 192
- timestamp
 - Logger, 174
- TransactionGuard, 223
 - ~TransactionGuard, 225
 - Commit, 226
 - committed_, 227
 - db_, 227
 - operator=, 226
 - TransactionGuard, 225
- Trim
 - Env, 29
- TrimWhitespace
 - http, 41
- turn_end_time
 - game::GameState, 124
- TurnTimerData, 227
 - callback, 228
 - controller, 228
 - lobby_id, 228
- Unauthorised
 - Error, 106
- username
 - game::Player, 200

- JwtPayload, [135](#)
- PerSocketData, [198](#)
- username_
 - game::DecideDrawnCardEffect, [90](#)
 - game::DecideSwapTargetEffect, [94](#)
- valid
 - game::LastPlay, [136](#)
- ValidatePlay
 - game::DrawStackingRule, [98](#)
 - game::GameRule, [121](#)
 - game::JumpInRule, [133](#)
 - game::NoBluffingRule, [194](#)
 - game::SevenZeroRule, [213](#)
 - game::StandardRule, [219](#)
- Value
 - game, [35](#)
- value_counts
 - game::PlayerSessionStats, [201](#)
- VerifyPassword
 - AuthController, [65](#)
- VerifyToken
 - AuthController, [66](#)
- VoidResult
 - result.hpp, [299](#)
- Warn
 - Logger, [175](#)
- WebServer, [228](#)
 - ~WebServer, [231](#)
 - app_, [238](#)
 - ConnectionHandler, [230](#)
 - connections_, [238](#)
 - db_file_, [239](#)
 - GetActionRouter, [232](#)
 - GetApp, [232](#)
 - GetHTTPRouter, [233](#)
 - GetMimeType, [233](#)
 - HandleGet, [234](#)
 - HandlePost, [234](#)
 - http_router_, [239](#)
 - InitDB, [234](#)
 - on_close_hooks_, [239](#)
 - on_open_hooks_, [239](#)
 - OnConnectionClose, [235](#)
 - OnConnectionOpen, [235](#)
 - OnSocketClosed, [236](#)
 - OnSocketMessage, [236](#)
 - OnSocketOpen, [236](#)
 - operator=, [237](#)
 - port_, [239](#)
 - ReadFile, [237](#)
 - RegisterRoutes, [237](#)
 - Run, [238](#)
 - WebServer, [230](#), [231](#)
 - ws_router_, [239](#)
- websocket_context.hpp
 - AppRequest, [304](#)
 - AppResponse, [304](#)
 - AppWebSocket, [304](#)
 - White
 - Logger, [177](#)
 - wildcards_
 - ActionRouter, [56](#)
 - HttpRouter, [131](#)
 - winner
 - game::GameState, [124](#)
 - WrapWithWildcards
 - HttpRouter, [131](#)
 - WS
 - Logger, [176](#)
 - ws, [42](#)
 - ClientAction, [43](#)
 - Get, [44](#)
 - GetOr, [45](#)
 - kChatMessage, [44](#)
 - kChatSend, [43](#)
 - kClientActionStr, [50](#)
 - kError, [44](#)
 - kGameCallUno, [43](#)
 - kGameDrawCard, [43](#)
 - kGameExit, [43](#)
 - kGameOver, [44](#)
 - kGamePlayCard, [43](#)
 - kGameStateUpdated, [44](#)
 - kGameSubmitInput, [43](#)
 - kLobbyCreate, [43](#)
 - kLobbyDeleteSavedMatch, [43](#)
 - kLobbyEvicted, [44](#)
 - kLobbyJoin, [43](#)
 - kLobbyJoined, [44](#)
 - kLobbyKick, [43](#)
 - kLobbyLeave, [43](#)
 - kLobbyLeft, [44](#)
 - kLobbyList, [43](#), [44](#)
 - kLobbyListSavedMatches, [43](#)
 - kLobbyPromote, [43](#)
 - kLobbyRejoin, [43](#)
 - kLobbyResumeSavedMatch, [43](#)
 - kLobbyStartMatch, [43](#)
 - kLobbyUpdated, [44](#)
 - kLobbyUpdateSettings, [43](#)
 - kServerActionStr, [50](#)
 - kStrClientAction, [50](#)
 - kSuccess, [44](#)
 - MakeResponse, [47](#)
 - SendError, [48](#)
 - ServerAction, [43](#)
 - ws_router_
 - WebServer, [239](#)
 - WsContext, [240](#)
 - op_code, [240](#)
 - socket, [240](#)
 - socket_data, [241](#)
 - Yellow
 - Logger, [177](#)

Class: WebSocketClient

Defined in: [stores/ws.svelte.ts:149](#)

WebSocketClient

Brief

Client WebSocket Stateful e Reattivo. Gestisce riconessioni automatiche (Exponential Backoff), correlazione tra richieste e risposte, e incapsula gli stati della connessione sfruttando Svelte 5 (`$state`).

Tag

FRONT-WS-002

Constructors

Constructor

```
new WebSocketClient(): WebSocketClient
```

Returns

```
WebSocketClient
```

Properties

`_nextRequestId`

```
private _nextRequestId: number = 1
```

Defined in: [stores/ws.svelte.ts:161](#)

`closeHandlers`

```
private closeHandlers(): void | Promise < void >[] = []
```

Defined in: [stores/ws.svelte.ts:166](#)

Returns

```
void | Promise < void >
```

`connectionStatus`

```
connectionStatus: ConnectionStatus
```

Defined in: [stores/ws.svelte.ts:154](#)

Metadati e stato della connessione (reattivi per interfacciarsi con l'UI).

`intentionalClose`

```
private intentionalClose: boolean = false
```

Defined in: [stores/ws.svelte.ts:172](#)

onHandlers

```
private onHandlers: Map < string , Set < MessageHandler > >
```

Defined in: [stores/ws.svelte.ts:164](#)

openHandlers

```
private openHandlers: () => void | Promise < void >[] = []
```

Defined in: [stores/ws.svelte.ts:165](#)

Returns

```
void | Promise < void >
```

pendingRequests

```
private pendingRequests: Map < string , PendingRequest >
```

Defined in: [stores/ws.svelte.ts:162](#)

reconnectDelayMs

```
private reconnectDelayMs: number = 1000
```

Defined in: [stores/ws.svelte.ts:171](#)

reconnectTimer

```
private reconnectTimer: number | null = null
```

Defined in: [stores/ws.svelte.ts:170](#)

socket

```
socket: WebSocket | null
```

Defined in: [stores/ws.svelte.ts:151](#)

Istanza nativa del WebSocket (reattiva).

visibilityListenerAttached

```
private visibilityListenerAttached: boolean = false
```

Defined in: [stores/ws.svelte.ts:168](#)

Accessors

isConnected

Get Signature

```
get isConnected(): boolean
```

Defined in: [stores/ws.svelte.ts:177](#)

Brief

Verifica se il WebSocket è attualmente aperto e attivo.

Returns

boolean

Methods

`_connectOnce()`

```
private _connectOnce(): Promise < void >
```

Defined in: [stores/ws.svelte.ts:307](#)

Returns

Promise < void >

Brief

Routine interna per stabilire una singola connessione grezza e legare gli eventi nativi.

`_dispatch()`

```
private _dispatch( data ): void
```

Defined in: [stores/ws.svelte.ts:356](#)

Parameters **data**

Record < string , unknown >

Returns

void

Brief

Smista il messaggio in ingresso risolvendo eventuali richieste pendenti o innescando Listener globali.

`_scheduleReconnect()`

```
private _scheduleReconnect(): void
```

Defined in: [stores/ws.svelte.ts:382](#)

Returns

void

Brief

Implementa un algoritmo di Exponential Backoff per tentare le riconessioni. Raddoppia il ritardo ad ogni tentativo fallito fino a un cap massimo.

`connect()`

```
connect(): Promise < void >
```

Defined in: [stores/ws.svelte.ts:185](#)

Returns

Promise < void >

Promise risolta quando la connessione si apre con successo.

Brief

Installa e stabilisce la connessione WebSocket. Sicuro da chiamare più volte.

disconnect()

```
disconnect( code , reason ): void
```

Defined in: [stores/ws.svelte.ts:198](#)

Parameters

code

number

Il codice di chiusura WebSocket.

reason

string

La motivazione testuale per la chiusura.

Returns

void

Brief

Disconnette intenzionalmente il WebSocket bloccando il sistema di auto-riconnessione.

emit()

```
emit( action , payload? ): void
```

Defined in: [stores/ws.svelte.ts:225](#)

Parameters

action

string

L'azione da compiere (ClientAction).

payload?

```
Record < string , unknown > = {}
```

Dati opzionali da allegare in formato JSON.

Returns

void

Brief

Invia un messaggio asincrono al server senza attendere alcuna risposta (Fire and Forget). Non allega nessun `request_id` al pacchetto.

emitAndWait()

```
emitAndWait( action , payload? , timeoutMs? ): Promise < WsResponse >
```

Defined in: [stores/ws.svelte.ts:240](#)

Parameters

action

string

L'azione da compiere (ClientAction).

payload?

```
Record < string , unknown > = {}
```

Dati opzionali da allegare.

timeoutMs?

```
number = 5000
```

Tempo massimo di attesa prima di abortire (Default: 5000ms).

Returns

```
Promise < WsResponse >
```

Promise risolta col pacchetto di risposta del server.

Brief

Invia un messaggio e attende la risposta correlata dal server generando un `request_id` . Risolve in una `WsResponse` anche per errori logici generati dal server, ma rigetta in caso di fallimenti infrastrutturali (es. caduta rete o Timeout).

on()

```
on( action , handler ):()=> void
```

Defined in: [stores/ws.svelte.ts:271](#)

Parameters

action

```
string
```

L'azione del server da ascoltare (usa "*" per ascoltare tutto).

handler

```
MessageHandler
```

La funzione di callback innescata alla ricezione del messaggio.

Returns

Una lambda per rimuovere l'iscrizione all'evento.

```
()=> void
```

Brief

Registra un Listener globale per intercettare specifiche Server Action.

onClose()

```
onClose( handler ):()=> void
```

Defined in: [stores/ws.svelte.ts:295](#)

Parameters

handler

```
()=> void | Promise < void >
```

Returns

Funzione per rimuovere l'hook.

()=> void

Brief

Registra un Listener innescato alla chiusura del socket.

onOpen()

`onOpen( handler ):()=> void`

Defined in: [stores/ws.svelte.ts:282](#)

Parameters

handler

()=> void | Promise < void >

Returns

Funzione per rimuovere l'hook.

()=> void

Brief

Registra un Listener innescato immediatamente all'apertura del socket. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / WsResponse

Class: WsResponse

Defined in: [stores/ws.svelte.ts:87](#)

WsResponse

Brief

Wrapper unificato per le risposte dei messaggi WebSocket. Mima l'API HTTP Fetch per fornire un flusso di controllo lineare e prevedibile quando si attendono risposte dal server.

Tag

FRONT-WS-001

Constructors

Constructor

`new WsResponse( rawData ): WsResponse`

Defined in: [stores/ws.svelte.ts:97](#)

Parameters

rawData

Record < string , unknown >

Returns

WsResponse

Properties

action


```
readonly action: string
```

Defined in: [stores/ws.svelte.ts:91](#)

Il comando ServerAction ritornato nel pacchetto.

data

```
readonly data: Record < string , unknown >
```

Defined in: [stores/ws.svelte.ts:95](#)

Il dizionario JSON grezzo coi dati restituiti dal server.

ok

```
readonly ok: boolean
```

Defined in: [stores/ws.svelte.ts:89](#)

True se l'azione ritornata dal server NON è un errore.

request_id?

```
readonly optional request_id?: string
```

Defined in: [stores/ws.svelte.ts:93](#)

L'ID univoco della richiesta originaria inoltrata dal client, se presente.

Accessors

reason

Get Signature

```
get reason(): string
```

Defined in: [stores/ws.svelte.ts:109](#)

Brief

Estrae in sicurezza il motivo dell'errore (reason) restituito dal server.

Returns

```
string
```

Il messaggio di errore o un fallback predefinito.

Methods

get()

```
get< T >( key ): T | null
```

Defined in: [stores/ws.svelte.ts:118](#)

Type Parameters

T

```
T
```

Parameters

key

string

La chiave da cercare nel dizionario JSON.

Returns

T | null

Il valore tipizzato o null se la chiave non esiste.

Brief

Estrae un valore tipizzato dal payload.

getOr()

```
getOr< T >( key , fallback ): T
```

Defined in: stores/ws.svelte.ts:127

Type Parameters

T

T

Parameters

key

string

La chiave da cercare nel payload JSON.

fallback

T

Il valore di sicurezza da restituire.

Returns

T

Brief

Estrae un valore tipizzato dal payload fornendo un fallback in caso di assenza. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / BotTakeoverMode

Enumeration: BotTakeoverMode

Defined in: stores/lobby.svelte.ts:15

BotTakeoverMode

Brief

Modalità di comportamento quando un bot sostituisce o viene sostituito.

Enumeration Members

PlayInstantly

PlayInstantly: 0

Defined in: stores/lobby.svelte.ts:17

Il bot gioca istantaneamente la sua mossa non appena arriva il suo turno.

WaitUntilTurnEnd

`WaitUntilTurnEnd: 1`

Defined in: [stores/lobby.svelte.ts:19](#)

Il bot attende fino a 5 secondi per il suo turno, o fino alla fine del turno per altri giocatori AFK [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / fromResponse

Function: fromResponse()

`fromResponse( res ): Promise < AppError >`

Defined in: [utils/errors.ts:33](#)

Build an AppError from a failed fetch Response. Prefers the server's own `error` string if the body is JSON.

Parameters

res

Response

Returns

Promise < [AppError](#) > [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / fromThrown

Function: fromThrown()

`fromThrown( err ): AppError`

Defined in: [utils/errors.ts:55](#)

Wrap any thrown value into an AppError.

Parameters

err

unknown

Returns

[AppError](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateEmail

Function: validateEmail()

`validateEmail( email ): ValidationResult`

Defined in: [utils/validation.ts:30](#)

Validates an email string.

Parameters

email

string

Returns

[ValidationResult](#)

ValidationResult whether the email is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateLobbyName

Function: validateLobbyName()

```
validateLobbyName( name ): ValidationResult
```

Defined in: [utils/validation.ts:84](#)

Validates a lobby string.

Parameters

name

string

Returns

[ValidationResult](#)

ValidationResult whether the lobby is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validatePasswordMatch

Function: validatePasswordMatch()

```
validatePasswordMatch( password , confirm ): ValidationResult
```

Defined in: [utils/validation.ts:73](#)

Validates whether two password strings match.

Parameters

password

string

confirm

string

Returns

[ValidationResult](#)

ValidationResult whether they match, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validatePassword

Function: validatePassword()

```
validatePassword( password ): ValidationResult
```

Defined in: [utils/validation.ts:41](#)

Validates a password string.

Parameters

password

string

Returns

[ValidationResult](#)

ValidationResult whether the password is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / validateUsername

Function: validateUsername()

```
validateUsername( username ): ValidationResult
```

Defined in: [utils/validation.ts:55](#)

Validates a username string.

Parameters

username

string

Returns

[ValidationResult](#)

ValidationResult whether the username is valid, or the reason if invalid. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AppError

Interface: AppError

Defined in: [utils/errors.ts:10](#)

The backend uses HTTP status codes consistently and returns plain English strings in the `error` JSON field — it does NOT return machine-readable error code strings like "INVALID_PASSWORD".

Use HTTP status codes for routing and fallback messages, and prefer showing the server's own error string when available.

Properties

code

`code: string`

Defined in: [utils/errors.ts:11](#)

details?

`optional details?: Record < string , unknown >`

Defined in: [utils/errors.ts:14](#)

message

`message: string`

Defined in: [utils/errors.ts:12](#)

userMessage

`userMessage: string`

Defined in: [utils/errors.ts:13](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AuthFieldErrors

Interface: AuthFieldErrors

Defined in: [stores/auth.svelte.ts:21](#)

AuthFieldErrors

Brief

Mappatura degli errori per ogni singolo campo del form di registrazione/login.

Properties

confirmPassword?

`optional confirmPassword?: string`

Defined in: [stores/auth.svelte.ts:25](#)

email?

`optional email?: string`

Defined in: [stores/auth.svelte.ts:23](#)

password?

`optional password?: string`

Defined in: [stores/auth.svelte.ts:24](#)

username?

`optional username?: string`

Defined in: [stores/auth.svelte.ts:22](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Card

Interface: Card

Defined in: [stores/game.svelte.ts:37](#)

Card

Brief

Rappresentazione frontend di una singola carta.

Properties

color

```
color: "red" | "blue" | "green" | "yellow" | "wild"
```

Defined in: [stores/game.svelte.ts:41](#)

Il colore decodificato della carta.

id

```
id: number
```

Defined in: [stores/game.svelte.ts:39](#)

L'identificativo univoco della carta a 16-bit.

value

```
value: "reverse" | "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "skip" | "+2" | "colorswitch" | "+4"
```

Defined in: [stores/game.svelte.ts:43](#)

Il valore o l'azione associata alla carta. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ConnectionStatus

Interface: ConnectionStatus

Defined in: [stores/ws.svelte.ts:69](#)

ConnectionStatus

Brief

Rappresenta lo stato reattivo della connessione WebSocket utile per la UI.

Properties

lobby_code

```
lobby_code: string
```

Defined in: [stores/ws.svelte.ts:77](#)

Codice invito della lobby attuale.

room

room: *string*

Defined in: [stores/ws.svelte.ts:75](#)

Nome della stanza corrente.

status

status: *"connected" | "connecting" | "disconnected"*

Defined in: [stores/ws.svelte.ts:71](#)

Stato attuale della connessione.

username

username: *string*

Defined in: [stores/ws.svelte.ts:73](#)

Username sincronizzato dal server. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / `GamePlayer`

Interface: GamePlayer

Defined in: [stores/game.svelte.ts:51](#)

GamePlayer

Brief

Dati associati ad un giocatore in-game. Se il giocatore non è il client corrente, l'array **hand** sarà undefined e sostituito da **card_count** calcolato dal server.

Properties

card_count

card_count: *number*

Defined in: [stores/game.svelte.ts:55](#)

Numero totale di carte in mano al giocatore.

hand?

optional **hand?:** [Card](#) []

Defined in: [stores/game.svelte.ts:59](#)

Le carte fisiche in mano al giocatore (Presente solo per il Local Player).

has_called_uno

has_called_uno: *boolean*

Defined in: [stores/game.svelte.ts:57](#)

Indica se il giocatore ha cliccato correttamente su "UNO!".

is_bot

is_bot: `boolean`

Defined in: [stores/game.svelte.ts:61](#)

Indica se il giocatore è un bot controllato dal server.

username

username: `string`

Defined in: [stores/game.svelte.ts:53](#)

Username del giocatore. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / GameState

Interface: GameState

Defined in: [stores/game.svelte.ts:68](#)

GameState

Brief

Fotografia dello stato completo del tavolo in un preciso istante.

Properties

active_color

active_color: `string`

Defined in: [stores/game.svelte.ts:70](#)

Colore attualmente attivo per le giocate.

current_turn

current_turn: `string`

Defined in: [stores/game.svelte.ts:72](#)

Username del giocatore a cui spetta il turno corrente.

is_over?

optional is_over?: `boolean`

Defined in: [stores/game.svelte.ts:84](#)

Flag che indica se la partita ha raggiunto uno stato terminale.

last_play?

optional last_play?: [LastPlay](#)

Defined in: [stores/game.svelte.ts:82](#)

Provenienza dell'ultima carta giocata, usata per animarla dallo slot di origine.

pending_draws

pending_draws: `number`

Defined in: [stores/game.svelte.ts:80](#)

Carte accumulate (catena +2/+4) che il prossimo giocatore dovrà pescare.

play_direction

play_direction: `number`

Defined in: [stores/game.svelte.ts:74](#)

Direzione del gioco (1 per orario, -1 per antiorario).

players

players: [GamePlayer](#) []

Defined in: [stores/game.svelte.ts:78](#)

La lista di tutti i giocatori presenti nella partita.

top_card?

optional top_card?: [Card](#)

Defined in: [stores/game.svelte.ts:76](#)

La carta attualmente in cima alla pila degli scarti.

winner?

optional winner?: `string`

Defined in: [stores/game.svelte.ts:86](#)

Username del giocatore vincitore, qualora la partita sia finita. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / LastPlay

Interface: LastPlay

Defined in: [stores/game.svelte.ts:93](#)

LastPlay

Brief

Descrive chi ha giocato l'ultima carta e da quale slot della mano.

Properties

hand_index

hand_index: `number`

Defined in: [stores/game.svelte.ts:97](#)

Indice dello slot nella mano da cui la carta è partita.

player

player: `string`

Defined in: [stores/game.svelte.ts:95](#)

Username del giocatore che ha effettuato la giocata. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ListedLobby

Interface: ListedLobby

Defined in: [stores/lobby.svelte.ts:120](#)

ListedLobby

Brief

Dati minimizzati inviati dal server per renderizzare la lista delle lobby pubbliche. Risparmia banda non inviando la lista completa dei membri e le regole dettagliate.

Properties

bot_count

bot_count: `number`

Defined in: [stores/lobby.svelte.ts:126](#)

Numero totale di bot presenti in questa lobby.

invite_code

invite_code: `string`

Defined in: [stores/lobby.svelte.ts:128](#)

Il codice di invito per permettere al client di unirsi con un click.

member_count

member_count: `number`

Defined in: [stores/lobby.svelte.ts:124](#)

Numero totale di giocatori umani attualmente connessi.

name

name: `string`

Defined in: [stores/lobby.svelte.ts:122](#)

Il nome personalizzato della lobby. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Lobby

Interface: Lobby

Defined in: [stores/lobby.svelte.ts:100](#)

Lobby

Brief

Rappresenta una lobby di gioco completa con tutti i suoi dati di stato.

Properties

host

host: `string`

Defined in: [stores/lobby.svelte.ts:104](#)

L'username del creatore (Host) della lobby.

invite_code

invite_code: `string`

Defined in: [stores/lobby.svelte.ts:102](#)

Il codice univoco alfanumerico di 6 caratteri utilizzato per unirsi.

member_count

member_count: `number`

Defined in: [stores/lobby.svelte.ts:108](#)

Il numero attuale di membri presenti nella lobby (massimo 4).

members

members: [LobbyMember](#) []

Defined in: [stores/lobby.svelte.ts:110](#)

Lista dettagliata dei giocatori attualmente nella lobby.

name

name: `string`

Defined in: [stores/lobby.svelte.ts:106](#)

Il nome personalizzato della lobby mostrato nella lista pubblica.

settings

settings: [LobbySettings](#)

Defined in: [stores/lobby.svelte.ts:112](#)

Le impostazioni configurate per questa stanza di gioco. [UNI - Documentazione Frontend](#)

Interface: LobbyMember

Defined in: [stores/lobby.svelte.ts:85](#)

LobbyMember

Brief

Rappresenta un singolo giocatore connesso all'interno di una lobby.

Properties

is_bot

is_bot: *boolean*

Defined in: [stores/lobby.svelte.ts:93](#)

Vero se questo "membro" è in realtà un account bot gestito dal server.

is_connected

is_connected: *boolean*

Defined in: [stores/lobby.svelte.ts:89](#)

Indica se il giocatore è attualmente connesso tramite WebSocket o è in fase di riconnessione.

is_host

is_host: *boolean*

Defined in: [stores/lobby.svelte.ts:91](#)

Vero se questo giocatore è il proprietario/host della lobby e può avviare la partita.

username

username: *string*

Defined in: [stores/lobby.svelte.ts:87](#)

Il nome visualizzato del giocatore. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / LobbySettings

Interface: LobbySettings

Defined in: [stores/lobby.svelte.ts:40](#)

LobbySettings

Brief

Rappresenta la configurazione e le regole scelte per la lobby. Definisce i parametri della partita e la composizione del mazzo iniziale.

Properties

active_mods

—

active_mods: `string []`

Defined in: [stores/lobby.svelte.ts:44](#)

Array di regole speciali (Mod) attivate per questa partita.

allow_bot_replacement

allow_bot_replacement: `boolean`

Defined in: [stores/lobby.svelte.ts:63](#)

Se true, un giocatore umano che abbandona viene rimpiazzato da un bot anziché terminare il gioco.

allow_bot_takeover

allow_bot_takeover: `boolean`

Defined in: [stores/lobby.svelte.ts:61](#)

Se true, i nuovi giocatori umani possono entrare in partita sostituendo un bot.

bot_count

bot_count: `number`

Defined in: [stores/lobby.svelte.ts:57](#)

Numero di bot da inserire automaticamente all'avvio della partita.

bot_mode

bot_mode: `BotTakeoverMode`

Defined in: [stores/lobby.svelte.ts:59](#)

Comportamento del bot (es. gioca subito o attende il timer).

count_draw_two

count_draw_two: `number`

Defined in: [stores/lobby.svelte.ts:74](#)

Copie per colore della carta "Pesca Due (+2)".

count_numbered

count_numbered: `number`

Defined in: [stores/lobby.svelte.ts:68](#)

Copie per colore dei numeri da 1 a 9 nel mazzo.

count_reverses

count_reverses: `number`

Defined in: [stores/lobby.svelte.ts:72](#)

Copie per colore della carta "Inverti Giro".

count_skips

count_skips: *number*

Defined in: [stores/lobby.svelte.ts:70](#)

Copie per colore della carta "Divieto/Salta Turno".

count_wild

count_wild: *number*

Defined in: [stores/lobby.svelte.ts:76](#)

Numero totale di carte Jolly (Cambio Colore) nel mazzo.

count_wild_draw_four

count_wild_draw_four: *number*

Defined in: [stores/lobby.svelte.ts:78](#)

Numero totale di carte Jolly Pesca Quattro (+4) nel mazzo.

count_zeros

count_zeros: *number*

Defined in: [stores/lobby.svelte.ts:66](#)

Copie per colore del numero 0 nel mazzo.

is_public

is_public: *boolean*

Defined in: [stores/lobby.svelte.ts:42](#)

Se true, la lobby apparirà nella lista pubblica delle partite.

quit_deletes_match

quit_deletes_match: *boolean*

Defined in: [stores/lobby.svelte.ts:51](#)

Se true, l'abbandono volontario di un giocatore eliminerà definitivamente il salvataggio.

save_state

save_state: *boolean*

Defined in: [stores/lobby.svelte.ts:49](#)

Se true, lo stato della partita verrà salvato nel database ad ogni mossa.

starting_cards

starting_cards: *number*

Defined in: [stores/lobby.svelte.ts:54](#)

Numero di carte distribuite a ciascun giocatore all'inizio della partita.

turn_time_limit_ms

turn_time_limit_ms: *number*

Defined in: [stores/lobby.svelte.ts:46](#)

Limite di tempo in millisecondi concesso per completare un turno (es. 15000). [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / SavedMatch

Interface: SavedMatch

Defined in: [stores/lobby.svelte.ts:26](#)

SavedMatch

Brief

Dati di base di una partita salvata sul server.

Properties

match_id

match_id: *string*

Defined in: [stores/lobby.svelte.ts:28](#)

Identificativo univoco del salvataggio nel database.

players

players: *string []*

Defined in: [stores/lobby.svelte.ts:32](#)

Lista degli username dei giocatori presenti in quella partita.

saved_at

saved_at: *string*

Defined in: [stores/lobby.svelte.ts:30](#)

Data e ora del salvataggio. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / Toast

Interface: Toast

Defined in: [stores/toast.svelte.ts:18](#)

Toast

Brief

Rappresenta una singola notifica attualmente visibile a schermo.

Properties

id

id: `number`

Defined in: [stores/toast.svelte.ts:20](#)

Identificativo numerico univoco della notifica, usato per la rimozione.

message

message: `string`

Defined in: [stores/toast.svelte.ts:24](#)

Il messaggio testuale da mostrare all'utente.

type

type: [ToastType](#)

Defined in: [stores/toast.svelte.ts:22](#)

Tipo di notifica (determina il colore di sfondo/bordo). [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ValidationResult

Interface: ValidationResult

Defined in: [utils/validation.ts:13](#)

Rules here must stay in sync with the C++ backend constraints: kMinUsernameLen = 3, kMaxUsernameLen = 32 (auth_controller.hpp)
kMinPasswordLen = 8 (auth_controller.hpp) topic max 64 chars, alphanumeric/-/_ (webserver.cpp)

The backend does NOT require uppercase/lowercase/numbers in passwords — the previous validatePassword rule was stricter than the server and would reject valid passwords. Removed. If you later add server-side complexity rules, add them here at the same time.

Properties

error?

optional error?: `string`

Defined in: [utils/validation.ts:15](#)

valid

valid: `boolean`

Defined in: [utils/validation.ts:14](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / AppScreen

Type Alias: AppScreen

AppScreen = `"main" | "auth" | "lobbies" | "lobby" | "game" | "settings" | "stats" | "detailedStats"`

Defined in: [stores/navigation.svelte.ts:14](#)

Brief

Elenco delle schermate disponibili nell'applicazione frontend. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / CardColor

Type Alias: CardColor

```
CardColor = typeof COLOR_MAP [ number ]
```

Defined in: [stores/game.svelte.ts:30](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / CardValue

Type Alias: CardValue

```
CardValue = typeof VALUE_MAP [ number ]
```

Defined in: [stores/game.svelte.ts:31](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ClientActionType

Type Alias: ClientActionType

```
ClientActionType = typeof ClientAction [keyof *typeof* ClientAction ]
```

Defined in: [stores/ws.svelte.ts:54](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / MessageHandler

Type Alias: MessageHandler

```
MessageHandler = ( data ) => void
```

Defined in: [stores/ws.svelte.ts:63](#)

Parameters

data

```
Record < string , unknown >
```

Returns

```
void
```

Brief

Firma della funzione di callback per intercettare i messaggi WebSocket. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerActionDef

Type Alias: ServerActionDef

```
ServerActionDef = ServerActionType | string
```

Defined in: [stores/ws.svelte.ts:57](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerActionType

Type Alias: ServerActionType

```
ServerActionType = typeof ServerAction [keyof *typeof* ServerAction ]
```

Defined in: [stores/ws.svelte.ts:55](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ToastType

Type Alias: ToastType

```
ToastType = "success" | "error" | "info" | "warning"
```

Defined in: [stores/toast.svelte.ts:12](#)

Brief

Tipologia della notifica, utilizzata per determinare lo stile e l'icona del toast. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ClientAction

Variable: ClientAction

```
const ClientAction: object
```

Defined in: [stores/ws.svelte.ts:11](#)

Type Declaration

ChatSend

```
readonly ChatSend: "chat_send" = "chat_send"
```

GameCallUno

```
readonly GameCallUno: "game_call_uno" = "game_call_uno"
```

GameDrawCard

```
readonly GameDrawCard: "game_draw_card" = "game_draw_card"
```

GameExit

```
readonly GameExit: "game_exit" = "game_exit"
```

GamePlayCard

```
readonly GamePlayCard: "game_play_card" = "game_play_card"
```

GameSubmitInput

```
readonly GameSubmitInput: "game_submit_input" = "game_submit_input"
```

LobbyCreate

```
readonly LobbyCreate: "lobby_create" = "lobby_create"
```

LobbyDeleteSavedMatch

```
readonly LobbyDeleteSavedMatch: "lobby_delete_saved_match" = "lobby_delete_saved_match"
```

LobbyJoin

```
readonly LobbyJoin: "lobby_join" = "lobby_join"
```

LobbyKick

```
readonly LobbyKick: "lobby_kick" = "lobby_kick"
```

LobbyLeave

```
readonly LobbyLeave: "lobby_leave" = "lobby_leave"
```

LobbyList

```
readonly LobbyList: "lobby_list" = "lobby_list"
```

LobbyListSavedMatches

```
readonly LobbyListSavedMatches: "lobby_list_saved_matches" = "lobby_list_saved_matches"
```

LobbyPromote

```
readonly LobbyPromote: "lobby_promote" = "lobby_promote"
```

LobbyRejoin

```
readonly LobbyRejoin: "lobby_rejoin" = "lobby_rejoin"
```

LobbyResumeSavedMatch

```
readonly LobbyResumeSavedMatch: "lobby_resume_saved_match" = "lobby_resume_saved_match"
```

LobbyStartMatch

```
readonly LobbyStartMatch: "lobby_start_match" = "lobby_start_match"
```

LobbyUpdateSettings

```
readonly LobbyUpdateSettings: "lobby_update_settings" = "lobby_update_settings"
```

Brief

Dizionario delle costanti per le azioni inviate dal Client verso il Server: [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / ServerAction

Variable: ServerAction

```
const ServerAction: object
```

Defined in: [stores/ws.svelte.ts:38](#)

Type Declaration

ChatMessage

```
readonly ChatMessage: "chat_message" = "chat_message"
```

Error

```
readonly Error: "error" = "error"
```

GameOver

```
readonly GameOver: "game_over" = "game_over"
```

GameStateUpdated

```
readonly GameStateUpdated: "game_state_updated" = "game_state_updated"
```

LobbyEvicted

```
readonly LobbyEvicted: "lobby_evicted" = "lobby_evicted"
```

LobbyJoined

```
readonly LobbyJoined: "lobby_joined" = "lobby_joined"
```

LobbyLeft

```
readonly LobbyLeft: "lobby_left" = "lobby_left"
```

LobbyList

```
readonly LobbyList: "lobby_list" = "lobby_list"
```

LobbyUpdated

```
readonly LobbyUpdated: "lobby_updated" = "lobby_updated"
```

Success

```
readonly Success: "success" = "success"
```

Brief

Dizionario delle costanti per le azioni inviate dal Server verso il Client. [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeAuth

Variable: storeAuth

```
const storeAuth: StoreAuth
```

Defined in: [stores/auth.svelte.ts:214](#) [UNI - Documentazione Frontend](#)

[UNI - Documentazione Frontend](#) / storeGame

Variable: storeGame

```
const storeGame: StoreGame
```

Defined in: [stores/game.svelte.ts:283](#) [UNI - Documentazione Frontend](#)

Variable: storeLobby

```
const storeLobby: StoreLobby
```

Defined in: [stores/lobby.svelte.ts:411](#) [UNI - Documentazione Frontend](#)

Variable: storeNavigation

```
const storeNavigation: StoreNavigation
```

Defined in: [stores/navigation.svelte.ts:71](#) [UNI - Documentazione Frontend](#)

Variable: storeToast

```
const storeToast: StoreToast
```

Defined in: [stores/toast.svelte.ts:104](#)

Brief

Istanza singleton esportata globalmente. È il punto di accesso per tutti gli altri store. [UNI - Documentazione Frontend](#)

Variable: ws

```
const ws: WebSocketClient
```

Defined in: [stores/ws.svelte.ts:423](#)